

MATH 3150 – Applied Linear Models

Course Notes

Fall 2026

Table of Contents

Notes 1: Intro	1
Notes 2: Summary and Data Visualization	31
Notes 3: Inference Review	47
Notes 4: More ANOVA	61
Notes 5: Simple Linear Regression	76
Notes 6: Simple Linear Regression Transformations and Inference . .	94
Notes 7: Multiple Linear Regression	117
Notes 8: Model Selection and Diagnostics	136
Notes 9: Logistic Regression	155
Notes 10: Nonparametric Tests and Bootstrapping	174
R Reference Sheet	199

Notes 1: Introduction and R Basics

Introduction

What is statistics?

Statistics is the science of _____.

- **Data** is _____. This information can come from observations, counts, measurements, or responses. Typically data takes the form of observed measurements (e.g. height, temperature) or descriptions (e.g. dog, cat, blue, female).
- Any task involving collecting, processing, displaying, and interpreting data, especially for the purpose of making decisions, is part of the discipline of statistics.

Big picture: statistics is how we use data to say things about _____.

Sometimes we do this by displaying data, so that we can get a good visual idea of what is going on.

Unknowns and Uncertainty

Very often we are trying to say something about a quantity that is not only unknown, but is impossible or nearly impossible to know for sure. For instance, how many trees are there in Rocky Mountain National Park? No one knows! But maybe we can get an approximate idea.

_____ is rampant! We use statistics to answer questions about things that are unknown (such as the average height of trees in Rocky Mountain National Park), but just as importantly, we use statistics to get an idea of how sure we are of our answers. Sometimes it is useful to know that you don't know!

This is what the “margin of error” is in opinion polls. A big “margin of error” means “we don't really know”.

Statistical statements

- “I sleep seven hours per night, on average.”
- “45% of likely voters intend to vote for president Trump, with a margin of error of $\pm 3\%$.”
- “Cam Newton set NFL rookie records for passing yards in a game, yards in a season, yards in back to back games, and touchdowns in a season.”
- “Students who attend class regularly tend to get higher grades than those who do not.”

Generally, if you are taking raw information (data) and trying to make real world sense of it, you're doing statistics.

Statistics Types

We can divide statistics into two categories: _____.

_____ statistics involve describing a data set:

- We could make a graph of it.
- We can describe its average and how spread out it is.
- We could tell about any interesting features.

However, in descriptive statistics, we limit ourselves to describing the data itself. We do not generalize facts about the data set to a larger group.

If we generalize from a sample to a larger group, we are performing _____ statistics.

The larger group is called a _____. These sorts of generalizations are called inferences, and the investigator is said to “draw inference” on the population, using the **sample**. For example, if we want to learn something about the average height of trees in RMNP, we can take the average of a sample of 100 trees.

Inferential Statistics Jargon

A **population** of items is the entire collection of items of interest to the investigator in his or her study. This is the large group we want to know about.

A **sample** is a subset of the population that we actually collect information from.

We have to be careful when generalizing from a sample to a population. The descriptive characteristics of your sample may not accurately reflect the characteristics of the population you are studying.

Two very important questions in statistical inference are:

1. Is my sample large enough?
2. Is my sample representative of the population of interest?

R Guide

Math 3150 – Applied Linear Models
Southern Utah University

1

Table of contents

Introduction to R and RStudio

Data Structures

R Functions

Data Import and Export

Tidyverse

Graphing in Base R

Graphing with ggplot2

2

Introduction to R and RStudio

3

Introduction

R is a powerful and popular programming language for statistical computing and data analysis. It provides a wide range of tools and packages for data manipulation, visualization, and modeling. **R** has a rich ecosystem with a vast community contributing to its growth, making it an excellent choice for data scientists, statisticians, and researchers.

4

Intro

While **R** was developed by statisticians for statisticians for statistical analysis, it has evolved to be a full-fledged programming language comparable to C, Python, Java, etc., though it is arguably still best used for data analysis tasks since many tasks are much more efficient in other languages. In this class, we will explore the syntax of **R** and the most commonly used functionality of the language.

5

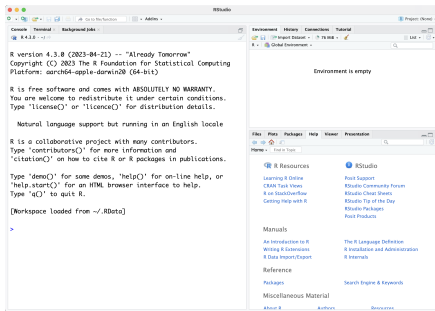
Installing R and RStudio

In the Introduction module on Canvas, there are some documents about installing **R** and **RStudio**. Be sure to follow those instructions to get the programs installed on your system. In this class, we will use **RStudio** exclusively once it is installed.

6

R First Impressions

When you first open **RStudio**, you will be presented with several windows, the main one being the **R** console, as shown below.



If yours does not look similar to this, then **something is wrong!**

7

R Console

You can interact with the console directly by typing in commands. For instance, if you type

```
1+1
```

```
into the console and it will give  
[1] 2
```

as an output. The [1] on the left indicates that the output has only 1 value. I encourage you to play around with the console for a few minutes.

8

R Operations

You can perform all of the usual arithmetic operations in **R**. This includes

- **+** for addition.
- **-** for subtraction.
- ***** for multiplication.
- **/** for division.
- **^** for exponents.

R will do the order of operations: PEMDAS, so be sure to use parentheses properly.

```
2 - 6 / 2 + 5
```

```
[1] 4
```

```
(2 - 6) / 2 + 5
```

```
[1] 3
```

9

R Operations

You can also use **<**, **>**, **<=**, **>=**, or **==** for **R** to check if one expression is less than, greater than, at most, at least, or exactly equal to another expression.

R will return either **TRUE** or **FALSE**, which is known as a Boolean data type.

```
3 > 1
```

```
[1] TRUE
```

```
-5 < -4
```

```
[1] TRUE
```

```
5 <= 5
```

```
[1] TRUE
```

```
5 == 5
```

```
[1] TRUE
```

10

R Data Types

Everything that you enter in **R** will have a data type. The primary data types in **R** are:

- **numeric**: data that are numbers. This will be the default for any number entered.
- **integer**: data that are numbers without decimal points. You can force a number to be stored as an integer by typing a capital "L" after the number. Example: 14L.
- **character**: data that are strings, which can include numbers and letters. This will be the default for anything entered in quotes (either single quotes or double quotes) like 'yellow' or "yellow".

11

R Data Types

- **factor**: data that are categorical. **R** thinks of these just in terms of categories.
- **logical**: true or false. Also known as Boolean. **R** will convert to logical if you are making a comparison like we did a couple slides ago.

The type of variable is important when working in **R**. If you have the wrong data type, then you may get errors. The code below will give an error.

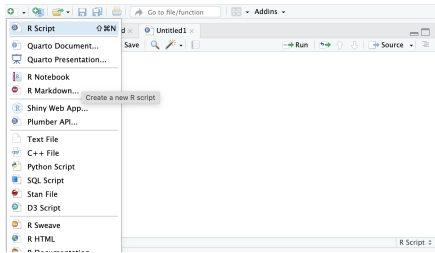
```
"1" + "1"
```

Error in "1" + "1" : non-numeric argument to binary operator

12

R Scripts

When programming in **R**, it is good practice to type all of your code in an **R** script. You can access a new script by clicking on the green plus icon at the top of the **RStudio** menu and then clicking on “R Script”.



This way, you can save all your code to easily rerun it later.

13

R Commenting

You can comment out code or other words in **R** scripts with the **#** key. Anything on the same line after **#** will not be run.

```
x <- 5
# print(x)
x + 2

[1] 7
```

In the output above, the value of **x** was not printed since the **print(x)** command was commented out.

14

Built-In R Functions

There are many functions that **R** comes pre-loaded with. To use a function in **R**, you must type the name of the function, then put the arguments of the function in parentheses. Some examples are

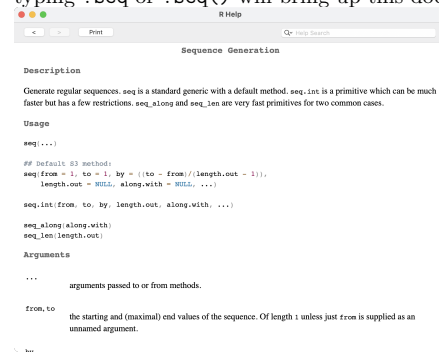
- **c()** to create a data vector.
- **mean()** to compute the average of a vector.
- **sd()** to compute the standard deviation of a vector.
- **length()** to compute the number of elements in a vector.
- **rep()** to repeat a value, string, or vector.
- **seq()** to create a sequence of values.
- **summary()** to compute some summary statistics for a vector.
- **cos()** to compute the cosine of a value.
- **exp()** to compute the number **e** raised to a power.
- **log()** to compute the logarithm (natural log is default) of a number.

There are hundreds of built-in **R** functions.

15

More on Functions

If you are ever in doubt about what a function does or what inputs it takes, you can always type a question mark in front of the function. For instance, typing **?seq** or **?seq()** will bring up this documentation:



16

Creating a Numeric Variable

It is remarkably useful to create variables for use in **R**. For example, if I have a small data set containing the values [1, 5, 3, 6], then I can input that into **R** and give that data set a name.

```
x <- c(1, 5, 3, 6)
```

The code above will create a new variable called **x** and then if I type **x**, it will display what is contained in **x**.

```
x

[1] 1 5 3 6
```

Then I can tell **R** to report how many elements are in **x** using the **length()** function, for instance.

```
length(x)

[1] 4
```

17

Creating a Character Variable

Variables do not have to be numbers. They can also be words (called “strings”). Just be sure to put strings in quotes.

```
y <- c("This", "Is", "A", "String", "Variable")
y

[1] "This"      "Is"        "A"         "String"    "Variable"
```

There are many functions that work with character variables.

```
y <- c("This", "Is", "A", "String", "Variable")
grep("A", y, ignore.case = T)

[1] 3 5
```

This **grep()** function tells you which element(s) of a vector contain a pattern that you give it.

18

A Note on Assignment

When we created the variable `x`, we used an *assignment operator*. There are two primary assignment operators in **R**:

1. The arrow: `<-`
2. The equal sign: `=`

Either one works. `x <- c(1, 5, 3, 6)` and `x = c(1, 5, 3, 6)` will both define a new vector `x` the same way.

Note: There is a somewhat fierce debate about assignment operators in **R** with some arguing the arrow is the only way it should be done to avoid confusion in functions (and for tidiness and tradition). Others argue that the arrow is outdated since it takes more key strokes and relies too much on white space since `x <- 5` and `x < -5` are different commands. The first assigns the value 5 to `x` and the second checks if `x` is less than `-5`.

19

R Libraries

Often times, **R** will not have a built-in function that does what you want to do. In that case, you can usually find a package (or library) you can install that will have other functions. You can install **R** packages by issuing the `install.packages()` function. You will need to put the name of the package in quotes. For example, to install the `knitr` package, you can type

```
install.packages("knitr")
```

It may prompt you to choose an install source. You can always select the Cloud for this. Once it is installed, you do not need to install it again. You will, however, need to load it each time you reopen **R**. You can load a package with the `library()` function. The name of the package does not need to be in quotes anymore, but it can be.

```
library(knitr)
```

20

The `knitr` library has many functions that base **R** does not. For example, the `kable()` function in the `knitr` packages will convert a data frame to a nice table:

```
kable(mtcars[1:5, 1:5], format = "latex")
```

	mpg	cyl	disp	hp	drat
Mazda RX4	21.0	6	160	110	3.90
Mazda RX4 Wag	21.0	6	160	110	3.90
Datsun 710	22.8	4	108	93	3.85
Hornet 4 Drive	21.4	6	258	110	3.08
Hornet Sportabout	18.7	8	360	175	3.15

There are many different packages that we will be installing and using this semester.

21

Data Structures

22

Intro to Data Structures

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks. Data structures in **R** programming are tools for holding multiple values.

R's base data structures are often organized by their dimensionality (1D, 2D, or nD) and whether they're homogeneous (all elements must be of the identical type) or heterogeneous (the elements are often of various types). This gives rise to the six data types which are most frequently utilized in data analysis.

23

Data Structure Types

The most essential data structures used in **R** include:

- Vectors
- Data Frames
- Factors

24

Vectors

A vector is an ordered collection of basic data types of a given length. The only key thing here is all the elements of a vector must be of the identical data type e.g homogeneous data structures. Vectors are one-dimensional data structures.

We can create a vector in **R** using the `c()` function. Separate values with commas. Note: other functions, like the `rep()` function, can be used inside the `c()` function.

```
x <- c(10, 12, 3, 4, 5, 5) # Define vector x
# Operations work in vectors
x <- c(10, 11 + cos(0), 3:4, rep(5, 2))
x # Print vector x to screen. print(x) also works

[1] 10 12 3 4 5 5
```

25

Functions with Vectors

R makes it very easy to work with vectors. If you put a vector in a function that works on a single number, it will apply that function to every element of the vector.

```
x = c(10, 12, 3, 4, 5, 5) # Define x differently
rep(x, 2) # Repeat x twice
```

```
[1] 10 12 3 4 5 5 10 12 3 4 5 5
```

Many functions expect a vector to be entered.

```
length(x) # Report how many elements are in x
```

```
[1] 6
```

```
# Takes the difference between
diff(x) # values in x that are one apart
```

```
[1] 2 -9 1 1 0
```

26

Operations with Vectors

If we perform an arithmetic operation of a number with a vector, then it will do the operation element-wise (to each element in the vector).

```
x + 2 # Add 2 to every element in x
```

```
[1] 12 14 5 6 7 7
```

```
2*x # Multiply each element in x by 2
```

```
[1] 20 24 6 8 10 10
```

```
x^2 # Square each element in x
```

```
[1] 100 144 9 16 25 25
```

```
x > 3 # Check whether each element is greater than 3
```

```
[1] TRUE TRUE FALSE TRUE TRUE TRUE
```

27

Subsetting Vectors

If we want to work with only certain elements in a vector, we can subset it using square brackets `[]`.

```
x[1:2] # Takes the first two elements in x
```

```
[1] 10 12
```

```
x[c(1,2)] # Another way to do that
```

```
[1] 10 12
```

We can exclude certain elements in `x` by using negative values in the brackets.

```
x[-c(2,5)] # Excludes the 2nd and 5th elements of x
```

```
[1] 10 3 4 5
```

28

Subsetting Vectors

We can also put a logical statement in the `[]` when subsetting.

```
x
```

```
[1] 10 12 3 4 5 5
```

```
x[x > 5]
```

```
[1] 10 12
```

```
x[x == 5 | x == 10] # The | means "or"
```

```
[1] 10 5 5
```

```
x[x > 5 & x < 11] # The & means "and"
```

```
[1] 10
```

29

Working with Multiple Vectors

We can also add, subtract, multiply, or divide one vector by another as long as the sizes are the same. These operations will be performed pairwise.

```
y = c(10, 5, 3, 2, 7, 9)
x + y
```

```
[1] 20 17 6 6 12 14
```

This next line will give an unexpected result since the sizes are not the same.

```
x + y[1:5]
```

```
Warning in x + y[1:5]: longer object length is not a
multiple of shorter object length
```

```
[1] 20 17 6 6 12 15
```

30

Data Frames

Data frames are generic data objects of **R** which are used to store the tabular data. Data frames are the foremost popular data objects in **R** programming because we are comfortable in seeing the data within the tabular form. They are two-dimensional, heterogeneous data structures. These are lists of vectors of equal lengths.

Data frames have the following constraints placed upon them:

- A data-frame must have column names and every row should have a unique name. If you don't specify the row name, **R** will give it a number by default.
- Each column must have the identical number of items.
- Each item in a single column must be of the same data type.
- Different columns may have different data types.
- To create a data frame we use the `data.frame()` function.

31

Data Frame Example

Let's load a data frame that is already in **R** using the `data()` function. We can check the object we load into memory is a data frame by using the `class()` function.

```
data(USArrests)
class(USArrests)

[1] "data.frame"
```

We can observe the first rows of a data frame using the `head()` function. By default, `head()` will display the first 6 rows, but we can adjust that.

```
head(USArrests, 3) # Displays the first three rows.
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0

32

Data Frame Example

This USArrests data frame has 4 columns (variables) and 50 rows. We can check that using the `ncol()` and `nrow()` functions or by using the `dim()` function.

```
ncol(USArrests); nrow(USArrests); dim(USArrests)
```

```
[1] 4
[1] 50
[1] 50 4
```

This data frame's columns are named Murder, Assault, UrbanPop, and Rape. The UrbanPop variable gives the percentage of the state's population that lives in urban areas. The other variables give the number of arrests for that crime per 100,000 people.

There is no state variable. Instead, the rows are named after the states. The command `rownames(USArrests)` will give the names of each row.

33

Subsetting Data Frames

VERY IMPORTANT

We can access a specific column of the data frame using the `$` operator and then typing the name of the variable (or column) that we are interested in.

```
USArrests$Murder
```

```
[1] 13.2 10.0 8.1 8.8 9.0 7.9 3.3 5.9 15.4 17.4 5.3
[12] 2.6 10.4 7.2 2.2 6.0 9.7 15.4 2.1 11.3 4.4 12.1
[23] 2.7 16.1 9.0 6.0 4.3 12.2 2.1 7.4 11.4 11.1 13.0
[34] 0.8 7.3 6.6 4.9 6.3 3.4 14.4 3.8 13.2 12.7 3.2
[45] 2.2 8.5 4.0 5.7 2.6 6.8
```

We can achieve the same thing by using square brackets after the name of the data frame. Since the murder values are in the first column, we can type

```
USArrests[,1] # Will also give the Murder values
```

34

Subsetting Data Frames

If we only want specific rows, we can put those before the comma in the square bracket

```
USArrests[c(1,5),] # Gives the 1st and 5th rows
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
California	9.0	276	91	40.6

We can also select a specific value of a specific column by putting the row number first and the column number second. Or we can use the `$` operator.

```
USArrests[10, 2] # Gives Assault rate for the 10th state
```

```
[1] 211
```

```
USArrests$Assault[10] # Another way to get the same value
```

```
[1] 211
```

35

Creating Data Frames

We can create a data frame using the `data.frame()` function. It is best to give a name to each column and then list what we want for each column.

```
df1 <- data.frame(Name=c("Juan", "Carlos"), Age = c(22, 31))
df1
```

	Name	Age
1	Juan	22
2	Carlos	31

We can add another column to the data frame using the `cbind()` function.

```
vote <- c(TRUE, FALSE) # Create a new vector
df2 <- cbind(df1, vote)
df2
```

	Name	Age	vote
1	Juan	22	TRUE
2	Carlos	31	FALSE

36

Changing Variable Names

You can also use the `rbind()` function to add more rows.

We can change the name of a variable using the `names()` function. We can change the names of the rows too. Suppose I want to change all the variable names to be lowercase and add row names. I can type the following.

```
names(df2) <- c("name", "age", "vote") # Change column names
rownames(df2) <- c("person1", "person2") # Change row names
df2
```

	name	age	vote
person1	Juan	22	TRUE
person2	Carlos	31	FALSE

37

Factors

Factors are the data objects which are used to categorize the data and store it as levels. They are useful for storing categorical data. They can store both strings and integers. They are useful to categorize unique values in columns like "TRUE" or "FALSE", or "MALE" or "FEMALE", etc.. They are useful in data analysis for statistical modeling.

To create a factor in **R** you need to use the function called `factor()`. The argument to this `factor()` is the vector.

```
fac = factor(c("Male", "Female", "Male",
               "Male", "Female", "Male", "Female"))
fac
```

[1]	Male	Female	Male	Male	Female	Male	Female
-----	------	--------	------	------	--------	------	--------

Levels: Female Male

38

Working with Factors

We can rearrange the levels of the factor (which are alphabetical by default) also using the `factor()` function.

```
fac = factor(fac, levels = c("Male", "Female"))
fac
```

[1]	Male	Female	Male	Male	Female	Male	Female
-----	------	--------	------	------	--------	------	--------

Levels: Male Female

We can also force a variable to be a factor using the `as.factor()` function.

```
department = c("Math", "CS", "Other")
class(department)
```

[1]	"character"
-----	-------------

```
dep_factor = as.factor(department)
class(dep_factor)
```

[1]	"factor"
-----	----------

39

R Functions

R Functions

We have already used functions in **R**, but this topic will expand our knowledge of functions and introduce writing our own functions.

First, consider the function `seq()`. If we look at the documentation of the sequence function (by typing `?seq`), it will tell us the syntax is

```
seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),
length.out = NULL, along.with = NULL, ...)
```

So, if we type `seq(1, 5, 2)`, then it will output the numbers from 1 to 5 counting by 2s since those are the first three arguments.

```
seq(1, 5, 2)
```

[1]	1	3	5
-----	---	---	---

41

Working with Functions

If I want my third number to instead be the length of the vector, then I need to specify that.

```
seq(1, 5, length.out = 2)
```

[1]	1	5
-----	---	---

In fact, it is not required to type the entire name of that `length.out` argument. You just need to type enough letters for **R** to know what argument you are referring to.

```
seq(1, 5, length = 2)
```

[1]	1	5
-----	---	---

```
seq(1, 5, l = 2)
```

[1]	1	5
-----	---	---

Even just typing `l` was enough since no other argument starts with that letter. It is good practice, however, to type enough for clarity.

42

Default Function Inputs

Additionally, many functions will have a default entry for an argument. For example, the `seq()` function, by default, has a lower and upper bound of 1. So, if you just type

```
seq()
```

```
[1] 1
```

It gives an output.

Another example: the `sample()` function randomly selects values of an input vector. The syntax is

```
sample(x, size, replace = FALSE, prob = NULL)
```

So, by default it will sample without replacement. Typing `replace = T` or `replace = TRUE` will have it sample with replacement.

Note: we should always put an equal sign = not an arrow <- when telling **R** what our function inputs are in the function.

43

Writing Functions

It is possible to write your own function in **R**. Writing a function begins with the `function()` function. The syntax is

```
function_name <- function(all inputs) { function_body }
```

Example: if we want to write a function that takes an input, divides it by 2, and then squares it, we will type

```
halfsquare <- function(x) {  
  result <- (x/2)^2  
  return(result)  
}  
halfsquare(10)
```

```
[1] 25
```

In this function, we specified that `x` was the input, so we can refer to `x` in the function even if `x` was never defined globally.

44

Writing Functions

It is important to make sure the variable we want the function to output is listed on the last line of the function body. Some will argue that should be put in a `return()` function, but it doesn't matter.

We can also give our function default inputs. When defining the function, just let the input equal something. That way, it will still return something even when no input is given.

```
halfsquare <- function(x = 2) {  
  result <- (x/2)^2  
  result  
}  
halfsquare(10)
```

```
[1] 25
```

```
halfsquare()
```

```
[1] 1
```

45

Writing Functions

The functions we write can have any number of inputs.

```
diff_func <- function(x, y, z = 0) {  
  d <- x - y - z  
  d  
}  
diff_func(100, 10, 1)
```

```
[1] 89
```

When writing your own functions, make sure you do not name it anything that is already a built-in **R** function. For example, there is already a `diff()` function, which is why I named this `diff_func`. This is good practice for all variable names as well.

46

Documenting Functions

When we write a function that is a bit complicated, it is good practice to document the function to explain how it works. This can be done in comments in the function body.

```
diff_func <- function(x, y, z = 0) {  
  # Takes the first entered value and subtracts the others.  
  # A third number is optional.  
  d <- x - y - z  
  d  
}
```

47

Function Code

If you are curious about the code used to define a function, you can simply type the function name without parentheses.

```
diff_func
```

```
function (x, y, z = 0)  
{  
  d <- x - y - z  
  d  
}
```

48

Function Code

Many built-in functions, however, simply call a version of the function written more efficiently in the C programming language. Therefore, we cannot see the exact code of the function. It will tell you what environment, or package, the function is in.

```
rnorm
function (n, mean = 0, sd = 1)
.Call(C_rnorm, n, mean, sd)
<bytecode: 0x92e9b35e8>
<environment: namespace:stats>
```

49

Functions with Same Name

Sometime you will encounter different functions that have the same name. This can occur when you load different **R** packages that each have a function with the same name or when you name your function a name that already exists. **R** can know which function you want to use if you type the name of the packages followed by two colons (:) with no spaces between them.

So, the syntax is

```
package_name::function_name()
```

50

Functions with Same Name

For example, suppose I wrote function to calculate the weighted average of a vector and called that function `mean`.

```
mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
mean(x, w)
```

```
[1] 2.733333
```

Of course, there already is a built-in `mean` function that now we cannot use by typing `mean(x)`, since it will only use the new function I wrote. To use the built-in `mean()` function, which is in the **base** environment, I can type

```
base::mean(x) # The syntax is package::function()
```

```
[1] 3
```

51

Removing Functions

To remove any object that you create, you can use the `rm()` function. In this case, it would be better if I remove the function called `mean()` that I wrote and name it something else so there is no confusion. It is not possible to remove a function that is built-in or loaded in with an **R** package.

```
rm(mean)
weighted_mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
weighted_mean(x, w)
```

```
[1] 2.733333
```

```
mean(x) # The unweighted, built-in mean function
```

```
[1] 3
```

52

Data Import and Export

53

Intro to Data Import and Export

When working with **R** it is not typical that we will use data sets that are loaded with packages or that we type the data in ourselves. We will often want to read in data files that are stored on our computers.

We can read in data files using a couple **R** functions. But first, it is important to know common ways that data sets are stored.

54

Types of Data Files

There are a couple very common types of data files

- `.txt` files. These are the standard text file.
- `.csv` files. These are comma separated value files. Microsoft Excel can save files this way.
- `.xlsx` files. These are Microsoft Excel files.
- `.xls` files. These are an older type of Microsoft Excel file.
- `.json` files. These are files that have data stored in key-value pairs and arrays
- `.dat` files. These are another type of file that store data.

In this class, we will primarily be using `.txt`, `.csv`, and `.xlsx` files. So we will focus on those.

55

How Data Are Stored

In a data file, we will usually want the data stored in **tidy format**. This indicates that the data file is set up similar to a data frame with one observation per row. Then each row can have the information from all of the variables.

However, it is important to know how the data values are separated from each other. One way to do this is with a space. These are called **space delimited files**. In this case, the data may look like this:

```
Age Weight
31 220
15 94
52 131
```

This is also an example of a data file that has a header. That is, the first row contains the column names, not data values.

56

How Data Are Stored

Another way is to use a comma. If commas are used, it is called a **comma separated value** or **comma delimited** file (even if the file name does not end in `.csv`).

```
Age,Weight
31,220
15, 94
52,131
```

Spaces in these files are ignored unless they are in quotes.

There are other ways to separate data values like using tab delimiters or new line delimiters. Comma and space are the most common, though.

57

Reading in Data Files

There are two primary functions we can use to read in data files.

- `read.csv()`
- `read.table()`

The `read.csv()` function is simpler, but not as powerful or adaptable as the `read.table()` function.

58

The `read.csv()` Function

This function works best for csv files or text files with comma delimiters. The syntax is

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",
         dec = ".", fill = TRUE, comment.char = "#", ...)
```

59

The `read.table()` Function

This is the more complicated, but more customizable, function. The syntax is given below, but many of these arguments will not be used.

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
           dec = ".", numerals = c("allow.loss", "warn.loss",
                                   "no.loss"),
           row.names, col.names, as.is = !stringsAsFactors,
           tryLogical = TRUE,
           na.strings = "NA", colClasses = NA, nrows = -1,
           skip = 0, check.names = TRUE,
           fill = !blank.lines.skip,
           strip.white = FALSE, blank.lines.skip = TRUE,
           comment.char = "#",
           allowEscapes = FALSE, flush = FALSE,
           stringsAsFactors = FALSE, fileEncoding = "",
           encoding = "unknown", text, skipNul = FALSE)
```

60

Using the `read.csv()` and `read.table()` Functions

To use these functions, we need to let **R** know exactly where the file is. We can do this in one of two ways.

1. Set the working directory as the folder that contains the file.
2. Put the file path in the `read.csv()` or `read.table()` function we are using.

61

Setting the Working Directory

Here are the steps to set a new working directory in **RStudio**.

1. Click on “Session” in the menu bar.
2. Go down to “Set Working Directory” and then “Choose Directory”.
3. Navigate to the folder (not file) that contains the file you want to read in.
4. Click “Open”.

Now the directory will be set and you can put the file name into the `read.csv()` or `read.table()` function.

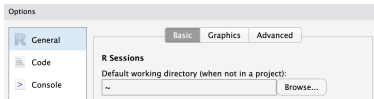
Note: you will need to set the working directory every time you quit and reopen **RStudio**. You can avoid this by changing the default directory to a folder than will contain all of your data files for this class if you'd like.

62

Set the Default Directory

The steps for changing the default directory are:

1. Open the **RStudio** settings by clicking on “Tools” from the menu bar and the going down to “Global Options”.
2. The settings will open (that look like the image below). Click on the Browse... button under the option for default directory.
3. Navigate to the folder (not file) that contains the file you want to read in and click “Open”.



4. Click “Apply” at the bottom right side of the settings pane to save and close the settings.

63

Read Data File using `read.csv()`

Once we have changed our directory, we are ready to read in a file. Suppose I want to read in the file called `USTempLocation.csv`.

Here is what the file looks like if we open it in a text editor.

```
City,JanTemp,Lat,Long
"Mobile,AL",44,31.2,88.5
"Montgomery,AL",38,32.9,86.8
"Phoenix,AZ",35,33.6,112.5
"Little_Rock,AR",31,35.4,92.8
"Los_Angeles,CA",47,34.3,118.7
"San_Francisco,CA",42,38.4,123
"Denver,CO",15,48.7,105.3
"New_Haven,CT",22,41.7,73.4
"Wilmington,DE",26,48.5,76.3
"Washington,DC",38,39.7,77.5
```

64

Read Data File using `read.csv()`

Since this is a `csv` file, we will begin using the `read.csv()` function.

```
dat <- read.csv("USTempLocation.csv")
head(dat)
```

	City	JanTemp	Lat	Long
1	Mobile,AL	44	31.2	88.5
2	Montgomery,AL	38	32.9	86.8
3	Phoenix,AZ	35	33.6	112.5
4	Little_Rock,AR	31	35.4	92.8
5	Los_Angeles,CA	47	34.3	118.7
6	San_Francisco,CA	42	38.4	123.0

Notice that by default, the `read.csv()` function will use the first row in the file as column names. We can change this by using the `header=F` argument, but we usually want the first row to be names.

65

Read Data File using `read.table()`

We can also use the `read.table()` function to read in that data file, but a few more options need to be added.

```
dat <- read.table("USTempLocation.csv", sep = ",",
                  header=T)
head(dat)
```

	City	JanTemp	Lat	Long
1	Mobile,AL	44	31.2	88.5
2	Montgomery,AL	38	32.9	86.8
3	Phoenix,AZ	35	33.6	112.5
4	Little_Rock,AR	31	35.4	92.8
5	Los_Angeles,CA	47	34.3	118.7
6	San_Francisco,CA	42	38.4	123.0

66

Read Data File using read.table()

By default, `read.table()` will use a space delimiter, not a comma delimiter, and it will not count the first row as column names. If we had used the default options, then it would have looked like this:

```
dat <- read.table("USTempLocation.csv")
head(dat)
```

	City	JanTemp	Lat	Long
Mobile,AL		44	31.2	88.5
Montgomery,AL		38	32.9	86.8
Phoenix,AZ		35	33.6	112.5
Little_Rock,AR		31	35.4	92.8
Los_Angeles,CA		47	34.3	118.7
San_Francisco,CA		42	38.4	123

Not good!

67

Read Data File using read.table()

The `read.table()` function is also useful when the data file is not comma delimited. For instance, suppose I want to read the file `turkey.txt`, which contains info on the bone lengths of turkey and whether they are wild or not. The data file looks like this:

By default, `read.table()` will assume the data is space delimited, like it is in this case.

```
id humerus tibiotarsus type
B093 151 152 0
B102 153 156 0
B103 149 151 0
L641 150 141 0
L783 154 157 0
L905 169 163 0
B710 153 151 0
B790 156 155 0
B819 158 152 0
```

So we can read it in using this code:

```
turkey <- read.table("turkey.txt", header = T)
head(turkey, 2)
```

	id	humerus	tibiotarsus	type
1	B093	151	152	0
2	B102	153	156	0

68

Read Data File Without Setting Directory

We can also read a data file without setting the directory to be where the file is beforehand as long as we put the path to the file in the function we are using.

For example, if my data file is on the desktop of my computer, then the file path would be `~/Desktop/` and then I would put the file name after that.

For Mac users, the `~` character represents the home folder of the logged-in user. You can determine the path to a file by clicking on it and typing “option-command-c”.

On Windows, if you click the Start button and then click Computer, click to open the location of the desired file, hold down the Shift key and right-click the file. Then on that menu, you can select “Copy as path”.

```
dat <- read.table("~/Desktop/USTempLocation.csv")
head(dat)
```

69

Reading Excel Files

In the `readxl` package is the function `read_excel()`, which can read in files that end in `.xls` or `.xlsx`.

```
install.packages("readxl")
library(readxl)
dat <- read_excel("~/Desktop/USTempLocation.xlsx")
head(dat)
```

If there is more than one sheet in the Excel notebook, you can specify which one using the `sheet` option in the `read_excel` function.

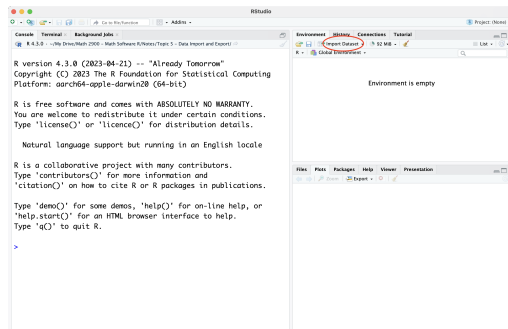
Note: the `read_excel()` function will read in the data file as a *tibble*, not as a data frame. We will discuss tibbles more in Section 5. Tibbles work just like data frames for nearly all practical things we do with them.

70

Reading Data Files Using RStudio Import Dataset

While I recommend using the `read.csv()` or `read.table()` functions because then it is easy to run the code again to read in the file, you can also read in data files using the “Import Dataset” button in **RStudio**.

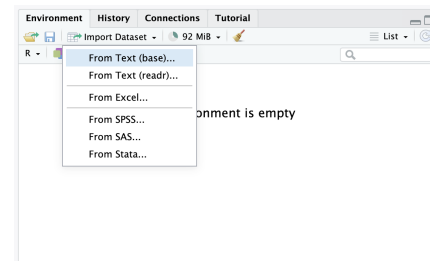
In the Environment tab on the top right, there is an Import Dataset button.



71

Reading Data Files Using RStudio Import Dataset

Clicking on the Import Dataset button will give these options:

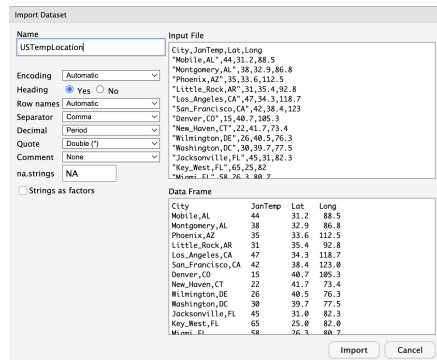


Click on “From Text (base)...”, navigate to the data file on your computer, and then click “Open”.

72

Reading Data Files Using RStudio Import Dataset

That will bring up this window. We then can change the options as needed.



73

Exporting Data

We can export a data frame into a file using the `write.csv()` or `write.table()` function. The syntax of `write.table()` is

```
write.table(x, file = "", append = FALSE, quote = TRUE,
           sep = " ", eol = "\n", na = "NA", dec = ".",
           row.names = TRUE, col.names = TRUE,
           qmethod = c("escape", "double"),
           fileEncoding = "")
```

`x` is the data frame we want to export and then we need to put the file name (and a file path if we don't want it saved in our working directory)

Like the read functions, `write.csv()` is just a simpler version of `write.table()`.

74

Exporting Data

We can make a data frame and then export it as a text file using the following code

```
Xdf = data.frame(age = c(10, 24, 51, 14),
                 state = c("UT", "UT", "CA", "CA"))
```

```
Xdf
  age state
1  10    UT
2  24    UT
3  51    CA
4  14    CA
```

```
write.table(Xdf, "age_state.txt", sep = ",")
```

We won't need to do this often, but it is good to be able to save data frames easily into data files.

75

Tidyverse

76

Intro to the Tidyverse

Modern **R** users are migrating away from many base **R** packages and functions to instead work in the **tidyverse**.

The tidyverse is both a philosophy for coding and organizing data as well as a collection of packages in **R**.

```
library(tidyverse)
```

— Attaching core tidyverse packages — tidyverse 2.0.0 —

```
✓ dplyr      1.1.4    ✓ readr      2.1.4
✓ forcats    1.0.0    ✓ stringr    1.5.1
✓ ggplot2    3.4.4    ✓ tibble     3.2.1
✓ lubridate  1.9.3    ✓ tidyr      1.3.0
✓ purrr      1.0.2
```

— Conflicts — tidyverse_conflicts() —

```
* dplyr::filter() masks stats::filter()
* dplyr::lag()    masks stats::lag()
i Use the conflicted package to force all conflicts to become errors
```

77

Tidy Format (murders data)

We say that a data table is in **tidy** format if each row represents one observation and columns represent the different variables available for each of these observations. For example, the following data set is in tidy format:

```
library(dslabs)
data(murders)
head(murders)
```

```
      state abb region population total
1  Alabama  AL  South   4779736   135
2  Alaska   AK  West    710231    19
3  Arizona  AZ  West    6392017   232
4  Arkansas AR  South   2915918    93
5 California CA  West   37253956  1257
6  Colorado CO  West    5029196    65
```

78

Not Tidy Format (fertility)

The following dataset is organized, but not tidy. Why?

```
##      country 1960 1961 1962
## 1    Germany 2.41 2.44 2.47
## 2 South Korea 6.16 5.99 5.79
```

79

Tidy Format (fertility)

Here is what the data would look like in tidy format:

```
      country year fertility
1    Germany 1960      2.41
2 South Korea 1960      6.16
3    Germany 1961      2.44
4 South Korea 1961      5.99
5    Germany 1962      2.47
6 South Korea 1962      5.79
```

The same information is provided, but there are important differences in the format. For the **tidyverse** packages to be optimally used, data need to be reshaped into 'tidy' format. The advantage of working in tidy format allows the data analyst to focus on more important aspects of the analysis rather than the format of the data.

80

Tibbles

A **tibble** is a modern version of a data.frame.

```
library(tidyverse)
dat1 <- tibble(x = 1:4, y = 5:8, z = c("A", "B", "C", "D"))
```

Or convert a data frame to a tibble

```
dat <- data.frame(x=1:4, y = 5:8, z = c("A", "B", "C", "D"))
dat1 <- as_tibble(dat)
dat1
```

```
# A tibble: 4 x 3
      x     y z
  <int> <int> <chr>
1     1     5 A
2     2     6 B
3     3     7 C
4     4     8 D
```

81

Tibbles

Important characteristics that make tibbles unique:

1. Tibbles are primary data structure for the **tidyverse**
2. Tibbles display better and printing is more readable
3. Tibbles can be grouped
4. Subsets of tibbles are tibbles
5. Tibbles can have complex entries—numbers, strings, logicals, lists, functions, other tibbles, etc.

82

Subsetting Tibbles

Note: tibbles work just like data frames in just about every way except one. With data frames, using brackets `[]` will give vectors and with tibbles, using `[[ ]]` will give tibbles.

```
class(dat[,1]) # Class of first column from a data frame
```

```
[1] "integer"
```

```
class(dat1[,1]) # Class of first column from a tibble
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

```
mean(dat[,1])
```

```
[1] 2.5
```

```
mean(dat1[,1])
```

```
Warning in mean.default(dat1[, 1]): argument is not numeric
or logical: returning NA
```

```
[1] NA
```

83

Subsetting Tibbles

We can choose the columns of a tibble as a vector using the `$` operator or by putting double brackets `[[ ]]`.

```
dat1$x # Gives the first column (whose name is x)
```

```
[1] 1 2 3 4
```

```
dat1[[1]] # Gives the first column
```

```
[1] 1 2 3 4
```

```
mean(dat1[[1]])
```

```
[1] 2.5
```

This subsetting is not a problem for rows. Rows of data frames are data frames and rows of tibbles are tibbles, so nothing of note changes there.

84

Data Import in the Tidyverse

The **tidyverse** has its own functions that will read in data sets as tibbles. They are the functions

- `read_csv()`
- `read_table()`

These work very much like the `read.csv()` and `read.table()` functions. The primary difference is that `read.csv()` and `read.table()` read in the data as a data frame whereas `read_csv()` and `read_table()` read in the data as a tibble.

The `read_excel()` function in the `readxl` library also reads in data files as tibbles even though the `readxl` library is not technically part of the tidyverse.

85

dplyr Functions

One of the most useful packages in the **tidyverse** is the **dplyr** package that is used for data wrangling. **dplyr** is called that since it is a tool (like a set of pliers) for data frames (or tibbles).

The **dplyr** package has the following useful functions:

- `mutate()` adds new variables that are functions of existing variables.
- `filter()` picks cases based on their values. Selects rows.
- `select()` picks variables based on their names. Selects columns.
- `summarize()` or `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.
- `group_by()` allows you to perform any operation “by group”

Note an important point: most dplyr functions (and most functions in the tidyverse) input a tibble and then output a modified tibble, although many can also work with data frames.

86

Mutate

The function `mutate()` takes the data frame or tibble, the instructions for the new columns in next arguments, and returns a modified data frame. For example:

```
murders <- as_tibble(murders)
head(murders)
```

```
# A tibble: 6 x 5
  state   abb region population total
  <chr>   <chr> <fct>         <dbl> <dbl>
1 Alabama AL   South    4779736  135
2 Alaska  AK   West      710231   19
3 Arizona AZ    West    6392017  232
4 Arkansas AR   South    2915918   93
5 California CA  West    37253956 1257
6 Colorado CO   West    5029196   65
```

87

Mutate

To add murder rates, we mutate as follows:

```
murdersRate <- mutate(murders,
  rate = total / population * 100000
)
head(murdersRate)
```

```
# A tibble: 6 x 6
  state   abb region population total rate
  <chr>   <chr> <fct>         <dbl> <dbl> <dbl>
1 Alabama AL   South    4779736  135 2.82
2 Alaska  AK   West      710231   19 2.68
3 Arizona AZ    West    6392017  232 3.63
4 Arkansas AR   South    2915918   93 3.19
5 California CA  West    37253956 1257 3.37
6 Colorado CO   West    5029196   65 1.29
```

88

Filter

Now suppose that we want to filter the data table to only show the entries for which the murder rate is lower than 0.71. We do this as follows:

```
filter(murdersRate, rate <= 0.71)
murdersRate[murdersRate$rate <= 0.71,] # How do get the same
# result without
# filter function
```

```
# A tibble: 5 x 6
  state   abb region population total rate
  <chr>   <chr> <fct>         <dbl> <dbl> <dbl>
1 Hawaii  HI   West    1360301   7 0.515
2 Iowa    IA   North Central 3046355  21 0.689
3 New Hampshire NH Northeast 1316470   5 0.380
4 North Dakota ND North Central 672591   4 0.595
5 Vermont VT   Northeast 625741   2 0.320
```

89

Select

If we want to view just a few of our columns, we can use the following:

```
murdersRate <- mutate(murders, rate = total / population * 100000)
murdersRateSelect <- select(murdersRate, state, rate)
filter(murdersRateSelect, rate <= 0.71)
# The above is the same as the following
murdersRate[murdersRate$rate <= 0.71, c("state", "rate")]
```

```
# A tibble: 5 x 2
  state   rate
  <chr>   <dbl>
1 Hawaii 0.515
2 Iowa 0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont 0.320
```

90

Nesting Functions

Instead of defining new objects along the way, we could do everything in one complex nested function:

```
filter(select(mutate(murders, rate = total / population * 100000),
  state, rate), rate <= 0.71)
```

```
# A tibble: 5 x 2
  state      rate
  <chr>    <dbl>
1 Hawaii  0.515
2 Iowa    0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont  0.320
```

This is fairly concise but a little confusing. Is there a better, clearer way?

91

Pipes

In the previous example, we performed the following wrangling operations:

original data → mutate → select → filter

We can perform a series of operations in **R** by sending the results of one function to another using the **pipe operator**: `|>` that was added in **R** version 4.1.

There is also a pipe (that was actually added first) in the **magrittr** package that is loaded with the tidyverse with the syntax `%>%`. This **magrittr** pipe can do a few things the native pipe cannot¹, but for the vast majority of cases they work the same, so it is recommended to use the native pipe since it is built-in and runs slightly faster.

92

Pipes

The pipe is a combination of characters that when used properly does two things: *It shortens and simplifies the code* and it makes the code more intuitive to read.

There is a keyboard shortcut in RStudio for inserting the pipe. While you can always just type `|>`, you can also type:

Mac: Command-Shift-M
Windows: Control-Shift-M

However, this will not give you the default pipe unless you change a setting in RStudio. Go to
Tools → Global Options → Code → Select “Use native pipe operator”.

93

Pipes

All the pipe does is provide **forward application** of an object to the first argument of a function. The pipe sends left side of the input to the function to the right of the pipe. For example, if we wanted to calculate

$$\log_2(\sqrt{16})$$

We could use:

```
16 |> sqrt() |> log2()
```

```
[1] 2
```

Since the pipe sends values to the first argument, we can define other arguments as follows:

```
16 |> sqrt() |> log(base = 2)
```

```
[1] 2
```

While piping works the way it is formatted above, it is better practice to use a new line after each pipe.

94

Pipes (murders)

Creating the prior tibble operation using pipes:

```
murders |>
  mutate(rate = total / population * 100000) |>
  select(state, rate) |>
  filter(rate <= 0.71)
```

```
# A tibble: 5 x 2
  state      rate
  <chr>    <dbl>
1 Hawaii  0.515
2 Iowa    0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont  0.320
```

95

Piping into Other Arguments

By default, pipes will send the object being piped to the first argument of the next command, but we can send it to another argument by using the underscore (`_`) placeholder and specifying the argument.

```
murdersRate |>
  lm(rate ~ population, data = _)
```

Call:

```
lm(formula = rate ~ population, data = murdersRate)
```

Coefficients:

```
(Intercept)  population
2.575e+00    3.363e-08
```

We can also use the pipe placeholder along with `$`, `[]`, and `[[[]]`:

```
murdersRate |> _$rate |> head(5)
```

```
[1] 2.824424 2.675186 3.629527 3.189390 3.374138
```

96

Arrange

We know about the **order** and **sort** functions, but for ordering entire tables, the **arrange** function is much more useful. For example, here we order the tibble by the state's murder rate:

```
murdersRate |>
  arrange(rate) |>
  head()
```

```
# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>          <dbl> <dbl> <dbl>
1 Vermont    VT  Northeast      625741     2 0.320
2 New Hampshire NH Northeast     1316470     5 0.380
3 Hawaii     HI   West         1360301     7 0.515
4 North Dakota ND North Central 672591     4 0.595
5 Iowa       IA  North Central 3046355    21 0.689
6 Idaho      ID   West         1567582    12 0.766
```

97

Arrange (descending order)

Note that the default behavior is to order in ascending order. The function **desc** transforms a vector so that it is in descending order. To sort the table in descending order, we can type:

```
murdersRate |>
  arrange(desc(rate)) |>
  head()
```

```
# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>          <dbl> <dbl> <dbl>
1 District of Columbia DC South         601723     99 16.5
2 Louisiana      LA South         4533372    351  7.74
3 Missouri       MO North Central 5988927    321  5.36
4 Maryland       MD South         5773552    293  5.07
5 South Carolina SC South         4625364    207  4.48
6 Delaware       DE South          897934     38  4.23
```

98

Nested sorting

If we are ordering by a column with ties, we can use a second (or third) column to break the tie. for example:

```
murdersRate |>
  arrange(region, rate) |>
  head()
```

```
# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>          <dbl> <dbl> <dbl>
1 Vermont    VT  Northeast      625741     2 0.320
2 New Hampshire NH Northeast     1316470     5 0.380
3 Maine      ME  Northeast     1328361    11 0.828
4 Rhode Island RI Northeast     1052567    16 1.52
5 Massachusetts MA Northeast     6547629   118 1.80
6 New York   NY  Northeast     19378102   517 2.67
```

99

Summarize

The **summarize()** function computes summary statistics in an intuitive way. The 'heights' dataset includes heights and sex reported by students in an in-class survey.

```
data(heights)
heights |>
  filter(sex == "Female") |>
  summarize(
    avg = mean(height),
    std_dev = sd(height)
  )
```

```
      avg std_dev
1 64.93942 3.760656
```

100

Group then summarize with group_by()

A common operation in data exploration is to first split data into groups and then compute summaries for each group. For example, we may want to compute the average and standard deviation for men's and women's heights separately. We can do the following

```
heights |>
  group_by(sex) |>
  summarize(
    average = mean(height),
    standard_deviation = sd(height)
  )
```

```
# A tibble: 2 x 3
  sex      average standard_deviation
  <fct>      <dbl>          <dbl>
1 Female    64.9            3.76
2 Male     69.3            3.61
```

101

The pivot_longer() Function

Sometimes it is the case that the data need to be manually put into tidy format. This is where the **pivot_longer()** function can help.

```
prices <- read.csv("data/houseprice.txt")
head(prices, 10)
```

```
      gainesville orlando tampa
1      173.0      243.9 230.7
2      145.5      201.1 115.7
3      190.6      185.3 211.0
4      186.3      187.5 203.5
5      248.7      207.9 149.9
6      206.4      234.8 166.8
7       86.8      253.2 134.1
8      204.6      144.7 214.2
9      174.5       NA 105.5
10     220.0       NA 216.2
```

102

The pivot_longer() Function

```
new_prices <- pivot_longer(prices, cols = everything())
head(new_prices)
```

```
# A tibble: 6 x 2
  name      value
  <chr>    <dbl>
1 gainesville 173
2 orlando    244.
3 tampa      231.
4 gainesville 146.
5 orlando    201.
6 tampa      116.
```

This looks much better! Except the column names are just set to the default of `name` and `value`. Also, there are some NA values as we saw in the original data set.

103

The pivot_longer() Function

```
house_prices <- pivot_longer(prices, cols = everything()) |>
  na.omit() |>
  rename(city = name, price = value) |>
  arrange(city)
head(house_prices)
```

```
# A tibble: 6 x 2
  city      price
  <chr>    <dbl>
1 gainesville 173
2 gainesville 146.
3 gainesville 191.
4 gainesville 186.
5 gainesville 249.
6 gainesville 206.
```

104

The pivot_wider() Function

It's relatively rare to need `pivot_wider()` to make tidy data, but it can be useful for creating summary tables for presentation, or data in a format needed by other tools. We won't focus too much on `pivot_wider()`, but here is an example. We do need an "id" or a "row number" variable to make this work.

```
price_wide <- house_prices |>
  mutate(row_num = c(1:11, 1:8, 1:10)) |>
  pivot_wider(names_from = city, values_from = price)
head(price_wide, 3)
```

```
# A tibble: 3 x 4
  row_num gainesville orlando tampa
  <int>    <dbl>    <dbl> <dbl>
1     1      173      244.  231.
2     2      146.     201.  116.
3     3      191.     185.  211
```

105

More on the tidyverse

There are some other tidyverse operations, including the `inner_join()`, `left_join()`, `right_join()`, `full_join()`, `pull()`, `dot()`, `reframe()`, `nest_by()`, and `pick()` functions. We will work with a few of these throughout the course.

106

Graphing in Base R

107

Introduction

One of the primary reasons **R** became so popular is its graphics capabilities. In this topic, we discuss how to create and customize plots.

R has many built-in functions for graph creation:

- `plot()`: used for making scatter plots for two quantitative variables.
- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

There are many other plots, but we will start here.

108

The plot()

Function}

The `plot()` function is probably the most versatile and complicated of all the functions listed on the previous slide. The syntax is

```
plot(x, y = NULL, type = "p", xlim = NULL, ylim = NULL,
     log = "", main = NULL, sub = NULL, xlab = NULL,
     ylab = NULL, ann = par("ann"), axes = TRUE,
     frame.plot = axes, panel.first = NULL,
     panel.last = NULL, asp = NA,
     xgap.axis = NA, ygap.axis = NA,
     ...)
```

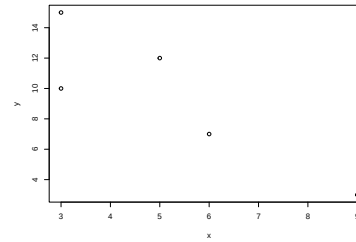
There are many arguments we can use to adjust and customize the output of the plot.

109

The plot() Function

At its most basic, plotting in **R** is very easy.

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y) # You can also use plot(y~x)
```



This plot, however, is very boring and possibly difficult to read.

110

Customizing Plots

We can customize plots by using the arguments of the plot function. Here are some commonly arguments and what they do:

- **type**: Changes the type of graph. By default, the graph will put points for each (x,y) pair. This can be changed to "l" for lines, "b" for both lines and points, "h" for vertical lines up to the height of the point, "n" for no output, etc.
- **xlim**: Adjusts the bounds on the x axis. Requires a vector `c(lower, upper)`.
- **ylim**: Adjusts the bounds on the y axis. Requires a vector `c(lower, upper)`.
- **xlab**: Gives the label on the x axis. Requires a string in quotes.
- **ylab**: Gives the label on the y axis. Requires a string in quotes.
- **main**: Gives the title of the plot. Requires a string in quotes.

111

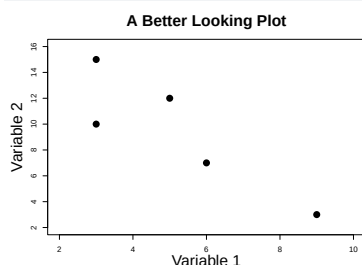
Customizing Plots (Continued)

- **cex**: Adjusts the size of what is plotted. Requires a number. The default size is 1.
- **cex.lab**: Adjusts the font size of the axis labels. Requires a number. The default size is 1.
- **cex.main**: Adjusts the font size of the main title. Requires a number. The default size is 1.
- **pch**: Changes the look of what is plotted. Requires a number or a symbol in quotes.
- **lty**: This changes the type of the line that is plotted if **type** = "l". Requires a number. The default type is 1 (solid).
- **lwd**: This changes the width of the line that is plotted if **type** = "l". Requires a number. The default width is 1.
- **col**: Adjusts the color of the plot. Requires a number or a string with the color name in quotes.
- **mgp**: Adjusts where the labels, axis numbers, and axes are on the plot. Requires a vector of length 3. The default is `c(3,1,0)`.

112

The plot() Function Customized

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
```



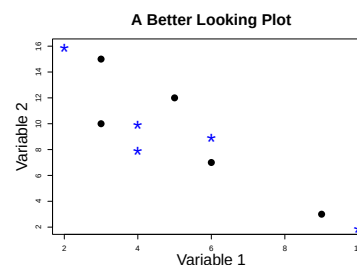
113

The points() and lines() Functions

If we ever want to add more points or additional lines to our graph, we can do so using the `points()` and `lines()` functions, respectively.

These functions do not create a new plot. They take the existing plot and simply add to it. For example, adding on to the previous graph.

```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
```



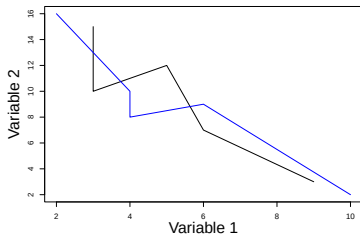
114

The points() and lines() Functions

If we had done a line plot and used the `lines()` function, it would look like this:

```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
lines(xnew, ynew, pch = "*", cex = 3, col = "blue")
```

A Better Looking Plot



115

Creating a Legend

When making a more complicated plot with multiple lines or types of points, it is good practice to include a legend. We can do this with the `legend()` function.

There are lots of arguments for the legend function, but at its most basic, we should just match all the options we used when plotting. We can specify an exact position of the legend using x and y values, or you can just type a position like "topright" or "bottom". Then you need some text as the next argument. An example is on the next slide.

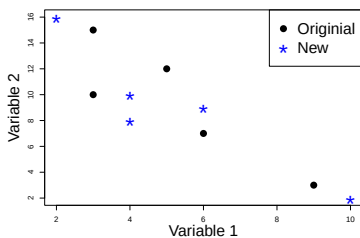
116

Creating a Legend Example

Again, adding to the last plot, we get:

```
legend("topright", c("Original", "New"), pch = c(16, 42),
      cex = c(2, 2), col = c("black", "blue"),
      pt.cex = c(2, 3))
# In the legend, cex affects text size
# and pt.cex affects symbol size.
```

A Better Looking Plot



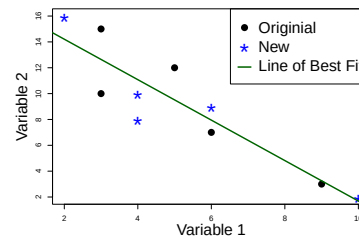
117

Adding a Line to the Plot

We can add a line of best fit through those data using the `abline()` function.

```
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Original", "New", "Line of Best Fit"),
      pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
      col = c("black", "blue", "darkgreen"))
```

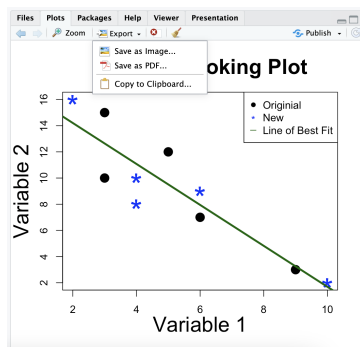
A Better Looking Plot



118

Saving Plots

You can save a plot by clicking on the Export button in the Plots viewer in **RStudio**.



119

Saving Plots

You can also save plots by using the `pdf()`, `png()`, or `jpeg()` functions. For any of those functions, we can use the following syntax (shown with the `pdf()` function):

```
pdf("file_path/file_name.pdf", width, height, ...)
All plotting code should go here
dev.off()
```

We end with the `dev.off()` function so **R** knows when the plotting is done.

120

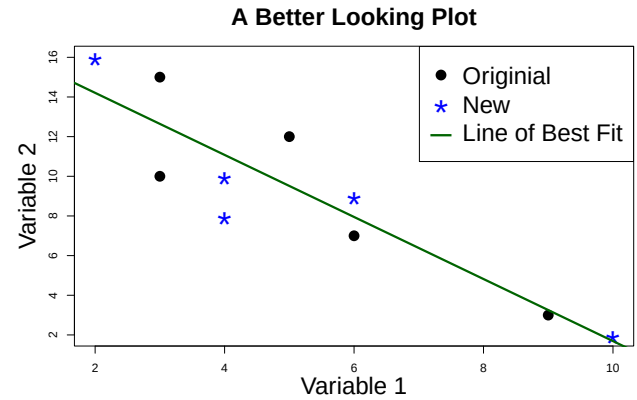
Saving Plots Example

```
x <- c(3, 3, 5, 6, 9); y <- c(15, 10, 12, 7, 3)
pdf("better_plot.pdf", width=9, height=6)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Original", "New", "Line of Best Fit"),
     pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
     col = c("black", "blue", "darkgreen"))
dev.off()

pdf
2
```

121

Saving Plots Example



122

Other Plots

We mentioned that there are lots of other types of plots:

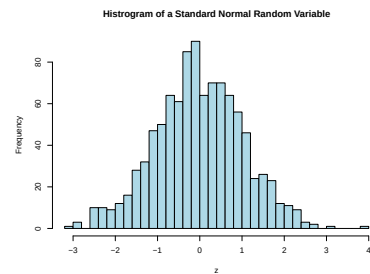
- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

Many of these have similar syntax to the `plot()` function. As always, you can type “?” before the name of the function to pull up documentation on these functions.

123

Histogram Example

```
set.seed(1) # Sets the random seed so
             # results are reproducible
z <- rnorm(1000) # Generates random values
hist(z, breaks = 30, col = "lightblue",
     main = "Histogram of a Standard Normal Random Variable")
```

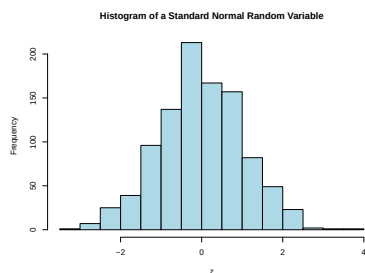


124

Histogram Example

This is a histogram of the same data, but with a different binwidth.

```
hist(z, breaks = 15, col = "lightblue",
     main = "Histogram of a Standard Normal Random Variable")
```

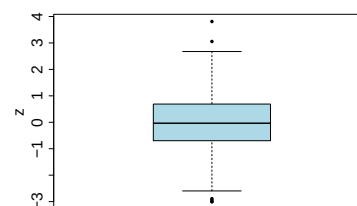


125

Boxplot Example

```
set.seed(1)
z <- rnorm(1000)

boxplot(z, cex.lab = 2, cex.axis = 2, ylab = "z",
        col = "lightblue", pch = 16)
```



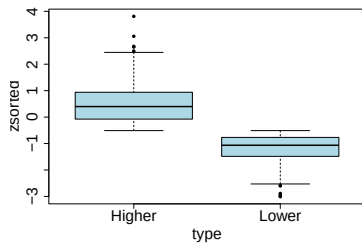
126

Boxplot Example

Boxplots are most useful for making comparisons.

```
zsorted <- sort(z) # Arrange from least to greatest
type <- c(rep("Lower", 300), rep("Higher", 700))

# Splits z by type and make two boxplots
boxplot(zsorted ~ type, cex.lab = 2, cex.axis = 2,
        col = "lightblue", pch = 16)
```

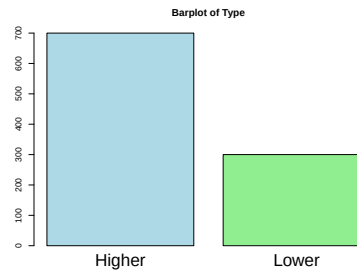


127

Barplot Example

For barplots, we need to use the `table()` function to count the frequencies of each category for a categorical variable.

```
type <- c(rep("Lower", 300), rep("Higher", 700))
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        col = c("lightblue", "lightgreen"))
```

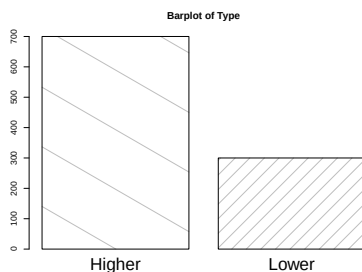


128

Barplot Example

There is lots of customization you can do with barplots. For example the `density` option determines how close the lines in the barplot are to each other while the `angle` option will determine the angle of those lines.

```
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        density = c(1, 5), angle = c(-30, 45))
```



129

Graphing with ggplot2

ggplot2 Introduction

While knowing how to plot using the base **R** packages is important, many **R** users are using the `ggplot2` package (which is part of the tidyverse) more and more for making better-looking plots.

Advantages of ggplot2

- It's consistent! gg = "grammar of graphics"; easy base system for adding/removing plot elements, with room for being fancy too
- Very flexible
- Themes available to polish plot appearance
- Active maintenance/development = getting better all the time!
- It can do quick-and-dirty and complex, so you only need one system
- Plots, or whole parts of plots, can be saved as objects
- Easy to add complexity or revert to earlier plot

131

ggplot2 Introduction

Disadvantages of ggplot2

- Sometimes more complicated than base **R** plotting
- Difficult to work with in iterated functions
- No 3-D graphics
- ggplot is often slower than base graphics
- The default colors can be difficult to change
- You might need to change the structure of your data frame to make certain plots (use `tidyr::pivot_longer()`)

132

ggplot Basics

There are three primary components to plotting with `ggplot2`:

- The **data** component. This is what data set and variables we are actually plotting.
- The **geometry** component. This describes what it is we are plotting. Examples include barplots, scatter plots, histograms, smooth densities, qqplots, boxplots, etc.
- The **aesthetic mapping** or just the **mapping**. The two most important cues in this plot are the point positions on the x-axis and y-axis. Each point represents a different observation, and we map data about these observations to visual cues like x- and y-scale. Color is another visual cue that we map to region. How this is defined depends on what type of geometry we are using.

133

Example dataset: Diamonds

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

Variable	Description	Values
price	price in US dollars	\$326-\$18,823
carat	weight of the diamond	0.2-5.01
cut	quality of the cut	Fair, Good, Very Good, Premium, Ideal
color	diamond color	J (worst) to D (best)
clarity	measurement of how clear the diamond is	I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best)
x	length in mm	0-10.74
y	width in mm	0-58.9
z	depth in mm	0-31.8
depth	total depth percentage	43-79
table	width of top of diamond relative to widest point	43-95

Figure 1: Diamonds

134

Example dataset: Diamonds

```
head(diamonds)
```

```
# A tibble: 6 x 10
```

```
  carat cut      color clarity depth table price    x
<dbl> <ord>    <ord>    <ord>    <dbl> <dbl> <int> <dbl>
1  0.23 Ideal    E     SI2     61.5    55   326  3.95
2  0.21 Premium E     SI1     59.8    61   326  3.89
3  0.23 Good    E     VS1     56.9    65   327  4.05
4  0.29 Premium I     VS2     62.4    58   334  4.2
5  0.31 Good    J     SI2     63.3    58   335  4.34
6  0.24 Very Good J     VVS2     62.8    57   336  3.94
```

```
# i 2 more variables: y <dbl>, z <dbl>
```

135

Example dataset: Diamonds

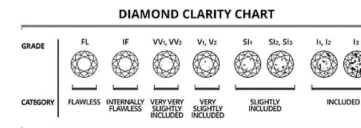


Figure 2: Diamond clarity is a measure of the purity and rarity of the stone, graded by the visibility of these characteristics under 10-power magnification. A stone is graded as flawless if, under 10-power magnification, no inclusions (internal flaws) and no blemishes (external imperfections) are visible.

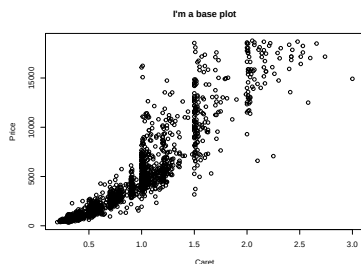
- The dataset contains information about 53,940 round-cut diamonds
- There are 10 variables measuring various pieces of information about the diamonds.
- There are 3 variables with an ordered factor structure: cut, color, & clarity

136

Example in Base Plotting

There is essentially just one primary function to know: `ggplot()`. However, `ggplot()` needs lots of other basic functions.

```
# load the diamonds dataset and take a sample of 2000
data(diamonds); set.seed(1190)
diam <- diamonds[sample(1:53940,2000),]
plot(diam$carat, diam$price, main = "I'm a base plot",
     xlab = "Carat", ylab = "Price")
```

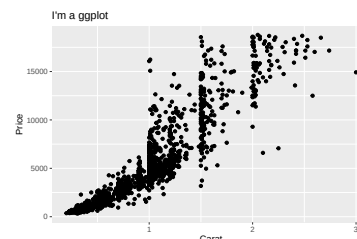


137

Example in ggplot2

In a `ggplot`, we need to begin with the `ggplot()` function and then add on (literally with a `+` sign) to that plot using other commands. In this case, I put `geom_point()` to add those solid dots.

```
ggplot(data = diam) +
  geom_point(aes(x = carat, y = price)) +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```

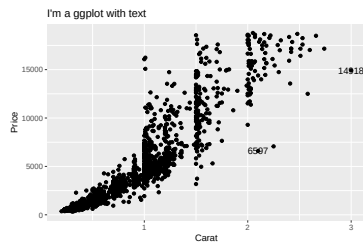


138

Example in ggplot2

Here is an example adding text to the plot.

```
ggplot(data = diam) +  
  geom_point(aes(x = carat, y = price)) +  
  geom_text(data = diam[c(954, 1565),],  
            aes(x = carat, y = price, label = price)) +  
  labs(x = "Carat", y = "Price",  
        title = "I'm a ggplot with text")
```



139

Global vs Local Aesthetic Mapping

Instead of putting the x and y in the `geom_()` function, we can put it in the `ggplot()` and it will apply everywhere.

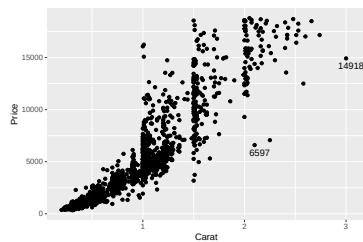
Anything put into the `ggplot()` function will apply globally to the entire plot whereas anything put into the geometry will only apply to that geometry. Some options, like size, can only be put into the geometry.

140

Example in ggplot2

Now the `aes(x = carat, y = price)` applies to both the `geom_point()` and the `geom_text()`. The `nudge_x` option will move the text to the right 0.1 units.

```
ggplot(data = diam, aes(x = carat, y = price)) +  
  geom_point() +  
  geom_text(data = diam[c(954, 1565),], aes(label = price),  
            nudge_x = 0.05, nudge_y = -700) +  
  labs(x = "Carat", y = "Price")
```

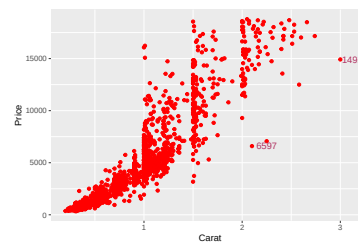


141

Example in ggplot2

This will make the points red, but the text maroon

```
ggplot(data = diam, aes(x = carat, y = price)) +  
  geom_point(color = "red") +  
  geom_text(data = diam[c(954, 1565),], aes(label = price),  
            nudge_x = 0.15, color = "maroon") +  
  labs(x = "Carat", y = "Price")
```

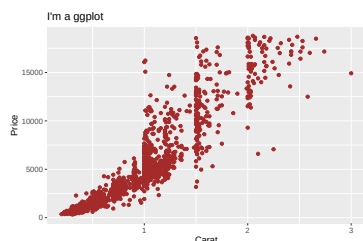


142

Piping in ggplot2

Pipes work very well with `ggplot` also. Remember that pipes are a part of the tidyverse (in the `dplyr` package) and by default, piping puts the thing being piped into the first argument of the function.

```
library(tidyverse)  
diam |>  
  ggplot(aes(carat, price)) + geom_point(col = "brown") +  
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```

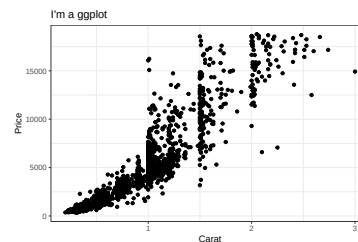


143

Example in ggplot2

We can change the background using `theme_...()`. There are `theme_bw()`, `theme_dark()`, `theme_classic()`, and more.

```
ggplot(data = diam, aes(x = carat, y = price)) +  
  geom_point() +  
  labs(x = "Carat", y = "Price", title = "I'm a ggplot") +  
  theme_bw()
```

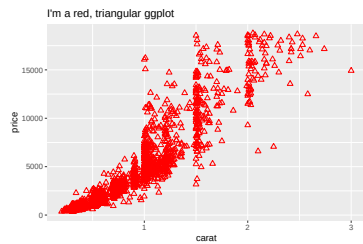


144

Changing the Graph Options in ggplot2

We can change the type of points added in the `geom_point()` function.

```
ggplot(data = diam, aes(x = carat, y = price)) +  
  geom_point(size = 2, color = "red", shape = 2) +  
  labs(title = "I'm a red, triangular ggplot")
```



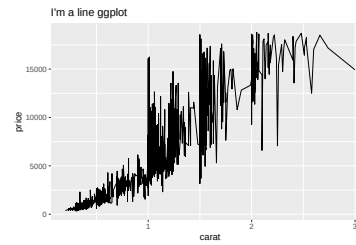
The `shape` argument works just like `pch` in base plotting.

145

Changing the Graph Type in ggplot2

Instead of adding `geom_point()`, we can add something else, like `geom_line()`

```
ggplot(data = diam, aes(x = carat, y = price)) +  
  geom_line() + labs(title = "I'm a line ggplot")
```



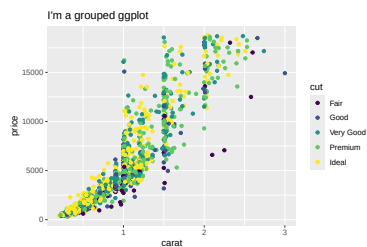
This looks strange for this plot, though.

146

Grouping by Another Variable

ggplot makes it easy to split the data using another variable. Simply put the `col` argument in the `aes()` function in `ggplot`. This will automatically add a legend as well. We can change the shape based on another variable too.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +  
  geom_point() + labs(title = "I'm a grouped ggplot")
```

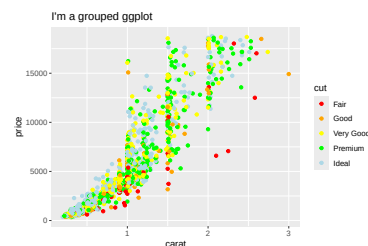


147

Grouping by Another Variable

We can manually change the colors using the `values` option in the `scale_color_manual()` add on function.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +  
  geom_point() + labs(title = "I'm a grouped ggplot") +  
  scale_color_manual(values = c("red", "orange", "yellow",  
                                "green", "lightblue"))
```

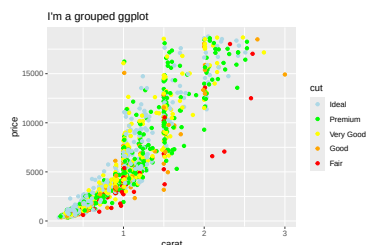


148

Grouping by Another Variable

We can manually change the order of the categories in the legend using the `breaks` option in the `scale_color_manual()` add on function.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +  
  geom_point() + labs(title = "I'm a grouped ggplot") +  
  scale_color_manual(  
    breaks = c("Ideal", "Premium", "Very Good", "Good", "Fair"),  
    values = c("lightblue", "green", "yellow", "orange", "red"))
```

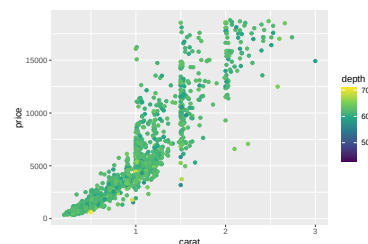


149

Grouping by Another Variable

We can also group by a continuous variable. In this case, we can change the color based on the `depth` variable.

```
# color by a continuous variable  
ggplot(data = diam, aes(x = carat,  
  y = price, color = depth)) + geom_point() +  
  scale_colour_continuous(type = "viridis")
```



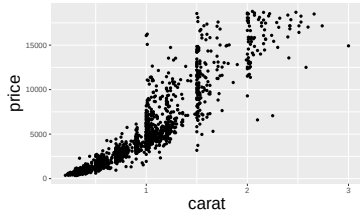
150

Changing Font Size and Type

We can change font size and type in the `theme()` function:

```
# color by a continuous variable
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(shape = 16, size = 1.5) +
  labs(title = "Check out this Font!") +
  theme(axis.title = element_text(size = 20),
        plot.title = element_text(size = 24, face = "bold"))
```

Check out this Font!

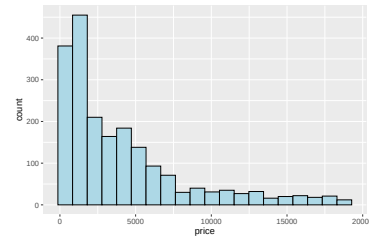


151

Histogram

Of course, we can also use `ggplot()` for plotting a single variable. We can make histograms, boxplots, dotplots, etc.

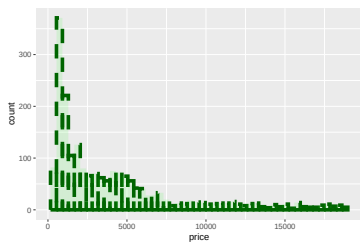
```
# Save the base plot as an object p. Then add to p.
p <- ggplot(data = diam, aes(x = price))
p + geom_histogram(color = "black", fill = "lightblue",
                  bins = 20)
```



152

Histogram

```
# We already created p, so we can add other options to it.
# Smaller alpha makes the plot more transparent.
p + geom_histogram(color = "darkgreen", fill = "green",
                  bins = 50, alpha = 0.1, lty = 2, lwd = 2)
```

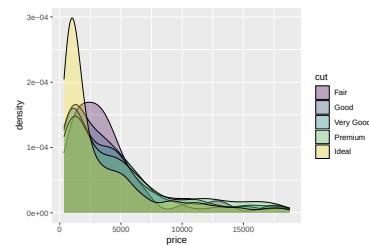


153

Layered Histograms

A layered histogram is a good way to compare the distribution of a variable across groups. `geom_histogram` works well for two groups, but `geom_density` is easier to look at for several groups.

```
ggplot(data = diam, aes(x = price, fill = cut)) +
  geom_density(alpha = 0.3)
```

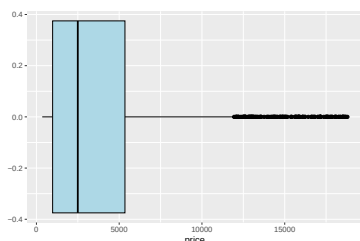


154

Boxplot

Creating a basic boxplot. We can also make it vertical by putting `y = price` in the `aes()` function instead of `x = price`.

```
ggplot(data = diam, aes(x = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```

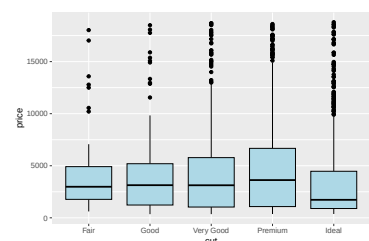


155

Side-by-Side Boxplots

We can make side-by-side boxplots grouped by a categorical variable as `x` (or as `y` if you want side-by-side horizontal boxplots).

```
ggplot(data = diam, aes(x = cut, y = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```

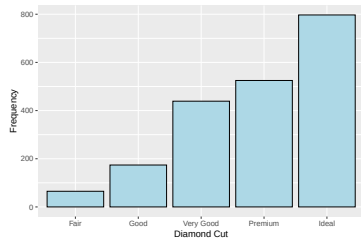


156

Barplot

We can make plots for categorical variables as well.

```
ggplot(data = diam, aes(x = cut)) +  
  geom_bar(color = "black", fill = "lightblue") +  
  labs(x = "Diamond Cut", y = "Frequency")
```

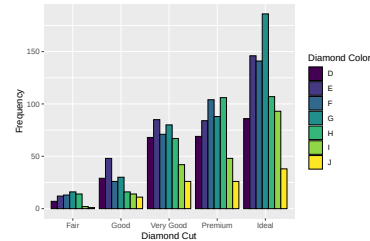


157

Side-by-Side Barplots

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and put the bars side by side.

```
ggplot(data = diam, aes(x = cut, fill = color)) +  
  geom_bar(color = "black", position = "dodge") +  
  labs(x = "Diamond Cut", y = "Frequency",  
       fill = "Diamond Color")
```

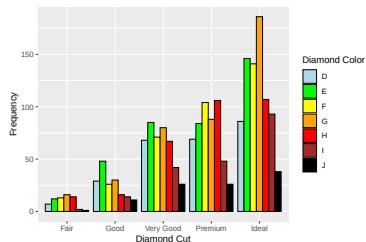


158

Side-by-Side Barplots

We can change the fill colors using `scale_fill_manual()`.

```
ggplot(data = diam, aes(x = cut, fill = color)) +  
  geom_bar(color = "black", position = "dodge") +  
  labs(x = "Diamond Cut", y = "Frequency",  
       fill = "Diamond Color") +  
  scale_fill_manual(values = c("lightblue", "green", "yellow",  
                               "orange", "red", "brown", "black"))
```

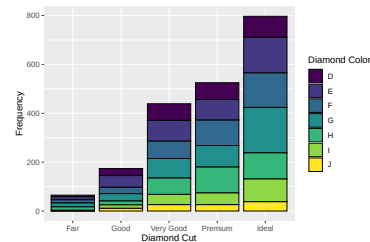


159

Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and stack the bars.

```
ggplot(data = diam, aes(x = cut, fill = color)) +  
  geom_bar(color = "black", position = "stack") +  
  labs(x = "Diamond Cut", y = "Frequency",  
       fill = "Diamond Color")
```

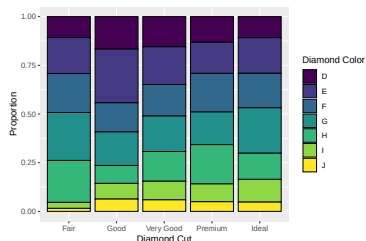


160

Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color, stack the bars, and adjust them so each bar totals 100%.

```
ggplot(data = diam, aes(x = cut, fill = color)) +  
  geom_bar(color = "black", position = "fill") +  
  labs(x = "Diamond Cut", y = "Proportion",  
       fill = "Diamond Color")
```

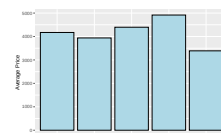


161

Barplot Identity

We often want to use a column of a data frame or tibble as the heights of our bar plot instead of having ggplot tabulate them for us. For this, we need to put `stat = identity` in the `geom_bar()` function. Alternatively, you can use `geom_col()` and then not put `stat = identity`.

```
diam |> group_by(cut) |>  
  summarize(avg_price = mean(price)) |>  
  ggplot(aes(x = cut, y = avg_price)) +  
  geom_bar(color = "black", stat = "identity",  
           fill = "lightblue") +  
  labs(x = "Diamond Cut", y = "Average Price")
```



162

Using ggplot2 for more complex plots

Use `facet_grid()` or `facet_wrap()` to create a separate plot for each value of a factor variable. We don't have to change any of the original plotting code, just add the facet command to it. Faceting can also be done on more than one categorical variable to create a grid of plots.

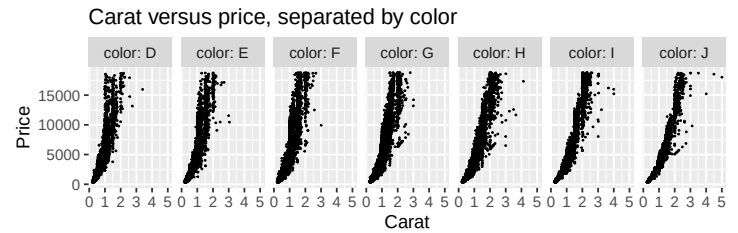
Additionally, it is sometimes helpful to save a simpler version of a plot, and then add onto it later with additional layers (for example, an if/else statement that plots different layers dependent on if a criterion is met or not).

We might want to summarize the data in the previous plot with a smoother on top of the points. With ggplot, we can simply add the `geom_smooth()` command. Each geom just adds another layer to the plot.

163

Facet Grid

```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diamonds, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(cols = vars(color),
  labeller = label_both) + labs(x = "Carat", y = "Price",
  title = "Carat versus price, separated by color")
```



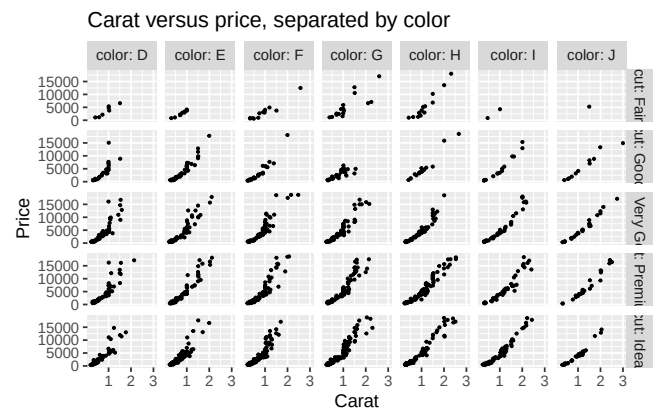
164

Facet Grid

```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diam, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(rows = vars(cut),
  cols = vars(color), labeller = label_both) +
  labs(x = "Carat", y = "Price",
  title = "Carat versus price, separated by color")
```

165

Facet Grid



166

Summary

The syntax of a ggplot is `ggplot(data, aes(x, y))` and you add on to the plot with `+` at the end of each line.

The most useful functions to add onto a ggplot are:

- `geom_point()`, `geom_line()`, `geom_histogram()`, `geom_boxplot()`, `geom_text()`, etc.
- `labs()` for labels including a plot title.
- `lims()` for limits.
- `theme()` for text size and visually changing other things.
- `scale_color_manual()` or `scale_fill_manual()` for changing the color or fill of the plot manually.

167

Further Resources & Assistance

- Cheat sheet for data visualization with ggplot2 (accessible in Rstudio by going to Help → Cheat Sheets → Data visualization with ggplot2)
- ggplot2 documentation
- Google
- Stack overflow
- Hadley Wickham's book: <https://ggplot2-book.org/>
- Rafael Irizarry's book: <http://rafalab.dfci.harvard.edu/dsbook-part-1/dataviz/ggplot2.html>
- Note: these slides were adapted from slides created by Aubrey Odom.

168

Notes 2: Exploratory Data Analysis and Data Visualization

Table of contents

Exploratory Data Analysis (EDA)	32
Graphical Descriptions of Data	34
Bar Plots	34
Histograms	37
Kernel Density Plots	40
Box Plots	42
Scatterplots	44

Goal of Statistics: The underlying goal of statistical methods is to summarize and gain information from data. We will explore two important roles of statistics as they are typically used:

1. **Exploratory Data Analysis (EDA)**: EDA deals with graphical and numerical summaries of data to acquire information about the center, spread, shape, and any unusual values in those data.
2. **Statistical Inference**: Statistical inference concerns the estimation of unknown population values (such as means or variances) typically with confidence intervals, and the formal testing of statistical hypotheses. We will consider this in the next set of notes.

Exploratory Data Analysis (EDA)

- EDA is also sometimes referred to as “descriptive statistics.”

Goal of EDA: To reduce a “large” set of data to a few numerical and graphical “measures” which provide a rough description of the entire dataset.

Before looking at methods of EDA, definitions of some important terms are given.

- **Observational Units**: The objects on which data are collected. These might be trees in a forest, areas of land, or people, among others. If the observational units are people, they are generally referred to as subjects.
- **Variable**: A characteristic of an experimental unit that can be expressed as a number or category (ex: height, age, gender, growth rate, distance run/walked in one day, etc.)
- **Values**: Specific numbers or categories for the individual observational units.

There are two basic types of variables:

1. _____ (numeric in **R**): These take numerical values for which arithmetic comparisons make sense.
2. _____ (factor in **R**): These record into which of several categories an observational unit falls.

Definition: The pattern of variation of a variable is its distribution. A distribution records numerical values and how often they occur.

- There are two basic ways to describe the distribution of some data:
 1. Graphical Descriptions
 2. Numerical Descriptions

Example: To illustrate some basic numerical statistical summaries, we will use the data collected from one of my classes in 2019 on the first day. These data were read into **R** and summarized using the `summary()` function:

```
class19 <- read_csv("class_data_F2019.txt") |>
  mutate(level = factor(level), major = factor(major),
         sex = factor(sex), ski = factor(ski), penny = factor(penny))
summary(class19) # Data summary
```

level	major	sex	ski
Freshman : 1	Computer Science :21	F:16	Neither :15
Graduate : 1	Mathematics :10	M:26	Ski :24
Junior :14	Physics : 2		Snowboard: 3
Senior :22	Business Analytics : 1		
Sophomore: 4	Computer Science & Math: 1		
	Ecology & Organic Bio : 1		
	(Other) : 6		

penny	speed	sleep
Abolish :21	Min. : 56	Min. : 3.500
No Answer: 2	1st Qu.: 90	1st Qu.: 6.000
Retain :19	Median :100	Median : 7.000
	Mean :108	Mean : 6.917
	3rd Qu.:114	3rd Qu.: 8.000
	Max. :188	Max. :10.000
	NAs :1	

In **R**, quantitative variables are referred to as **numeric** and categorical variables as **factor**. To check that **penny** is a categorical variable, we can type:

```
is.factor(class19$penny) # Checks whether the penny variable is a factor
[1] TRUE
```

If the answer had been false, we could coerce it to be a factor variable by typing:

```
as.factor(class19$penny) # Coerces the penny variable to be a factor
```

If we wanted to check whether the penny variable is quantitative, we can type:

```
is.numeric(class19$penny) # Checks whether the penny variable is numeric
[1] FALSE
```

Graphical Descriptions of Data

There are many graphical descriptions of data used in practice; six of the more commonly used displays are considered here.

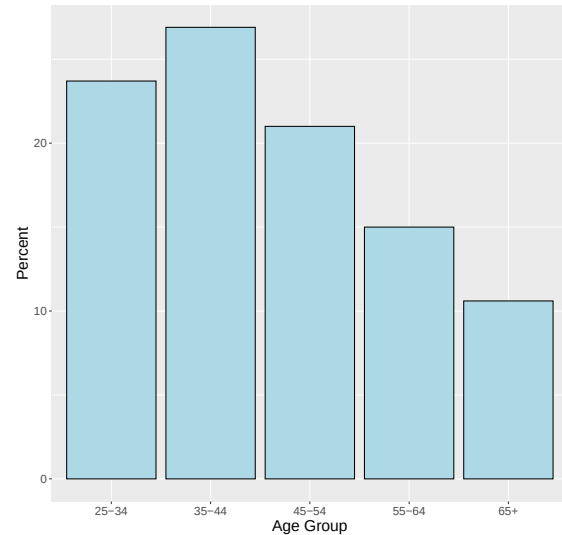
Bar Plots

Bar plots or bar charts are used for **categorical variables** only. To illustrate the use of a bar chart, consider the following example.

Example: Suppose we are interested in the association between age and level of college education among adult Americans. The table below represents the percentage of adults in different age categories with 4 years of a college education, among 151,316 sampled adults. [Data from the Census Bureau, 1987]. Also shown is a bar chart which graphically represents these data.

Age Group	% with 4 yrs. of college
25-34	23.7
35-44	26.9
45-54	21.0
55-64	15.0
>64	10.6

Barplot of 4 Years of College by Age Group



What can we say from these data?

Notes:

1. The y-axis is labeled by the frequency (or relative frequency) of occurrence of the specific categories. The x-axis is labeled by the categories of the variable of interest.
2. If the categories are not naturally ordered, it is recommended that they be ordered from largest to smallest frequency.
3. The **R** code for creating the barplots shown above is listed here and can be found on the course Canvas page in the script **Notes 2 Script.R**.

```
age_group <- as.factor(c("25-34", "35-44", "45-54", "55-64", "65+"))
perc <- c(23.7, 26.9, 21.0, 15.0, 10.6)
perc_df <- data.frame(age_group = age_group, perc = perc)
ggplot(perc_df, aes(x = age_group, y = perc)) +
  geom_bar(stat = "identity", color = "black", fill = "lightblue") +
  labs(x = "Age Group", y = "Percent",
       title = "Barplot of 4 Years of College by Age Group") +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"))
```

Now consider the following table representing the relationship between age and educational attainment from these same data.

1987 Data Education	Age Group					Total
	25-34	35-44	45-54	55-64	>64	
Didn't complete high school	5836	4841	5230	7024	13183	36114
Completed high school	17889	13200	9860	8580	9412	58921
College: 1-3 years	9069	7309	3698	2793	2915	25784
College, 4 or more years	10174	9332	5008	3246	3018	30778
Total	42968	34682	23796	21643	28528	151617

Considering only one variable gives us the **marginal distribution** of that variable. For example, the marginal distribution of the education variable is shown in this table:

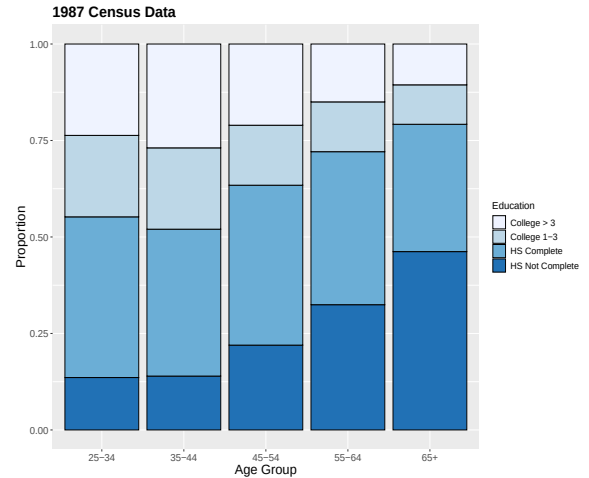
1987 Data		
Education	Number	Percentage
Didn't complete high school	36114	23.8%
Completed high school	58921	38.9%
College: 1-3 years	25784	17.0%
College, 4 or more years	30778	20.3%
Total	151617	100%

Computing the percentages within each column gives the **conditional distribution** of the education variable for each age category shown in the following table:

1987 Data Education	Age Group				
	25-34	35-44	45-54	55-64	>64
Didn't complete high school	0.136	0.140	0.220	0.324	0.462
Completed high school	0.416	0.381	0.414	0.396	0.330
College: 1-3 years	0.211	0.211	0.155	0.129	0.102
College, 4 or more years	0.237	0.269	0.210	0.150	0.106

This conditional relationship can be represented in what is known as a **segmented bar graph**, as shown here:

How should we interpret this graph? What does this say about the association between educational attainment and age?



Here is the code that generated this image:

```
census87 <- tibble(
  counts = c(5836, 4841, 5230, 7024, 13183,
             17889, 13200, 9860, 8580, 9412,
             9069, 7309, 3698, 2793, 2915,
             10174, 9332, 5008, 3246, 3018),
  Education = c(rep("HS Not Complete", 5),
                rep("HS Complete", 5),
                rep("College 1-3", 5),
                rep("College > 3", 5)),
  age_group = rep(c("25-34", "35-44", "45-54", "55-64", "65+"), 4)
)

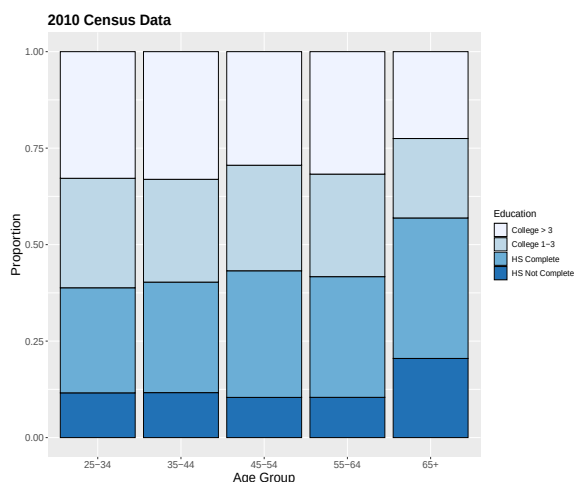
ggplot(census87, aes(x = age_group, y = counts, fill = Education)) +
  geom_bar(stat = "identity", position = "fill", color = "black") +
  labs(x = "Age Group", y = "Proportion", title = "1987 Census Data") +
  scale_fill_brewer() +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"),
        legend.title = element_text(size = 12),
        legend.text = element_text(size = 11))
```

What proportion of study participants only completed high school?

Among those aged 25-34 years, what proportion completed more than 3 years of college?

These same data were compiled by the Census Bureau as part of the 2010 Census and appear below along with a segmented bar graph.

2010 Data Education	Age Group					Total
	25-34	35-44	45-54	55-64	≥ 65	
Didn't complete high school	4764	4714	4623	3692	7919	25712
Completed high school	11186	11581	14563	11068	14056	62454
College: 1-3 years	11655	10772	12138	9405	7948	51918
College: 4 or more years	13481	13378	13062	11230	8690	59841
Total	41086	40445	44386	35395	38613	199925



Has the relationship between educational attainment and age changed over the past 25 years?

Are these variables related or do they seem to be independent of one another?

Histograms

A histogram is a plot for **quantitative variables** only, which breaks the range of data values of a variable into intervals and displays only the frequency (or relative frequency (%)) of the observations that fall in that interval.

- When are histograms most useful?

Method

1. Break the data into intervals of equal width.
2. Count the number (or compute the percentage) of observations in each interval.
3. Draw the histogram, as illustrated below.

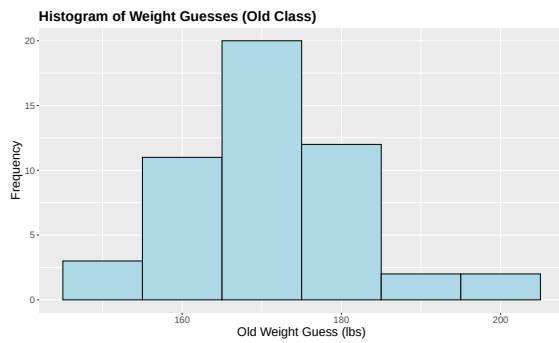
Example 1: A Professor asks his students to guess his weight in two different semesters. Consider the following guesses of his weight by a previous class, given in the table below, and plotted in the corresponding histogram. Note that the intervals in the histogram have equal width and are ordered.

Interval	146-155	156-165	166-175	176-185	186-195	196-205	206-215	Other
Frequency (Semester 1)	3	11	20	12	2	2	0	0
Relative Frequency	0.060	0.220	0.400	0.240	0.040	0.040	0	0
Frequency (Semester 2)	2	6	4	5	6	3	0	2 (10 & 240)
Relative Frequency	0.077	0.231	0.154	0.192	0.231	0.115	0	0.077

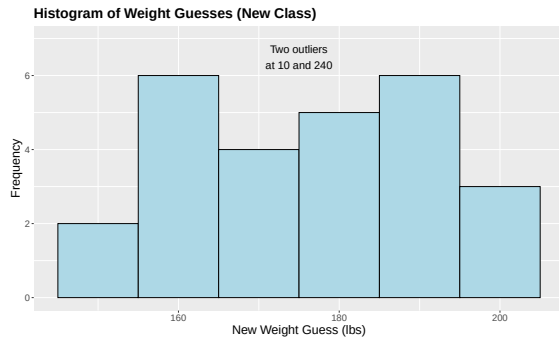
The following **R** code was used to create these histograms.

```
weight <- c(190, 195, 180, 10, 185, 205, 240,
            180, 165, 165, 165, 170, 200, 175,
            160, 190, 198, 150, 173, 190, 175,
            160, 187, 180, 155, 165, 187, 180) # New weight guess vector
oldwgt <- c(145, rep(150,2), rep(160,7), 163, # Old weight guess vector
            rep(165,3), 168, rep(170,7), rep(172,3),
            rep(175,9), 176, 177, rep(178,2), rep(180,4),
            183, rep(185,3), rep(190,2), 197, 205)
ggplot(data.frame(weight = weight), aes(x = weight)) +
  geom_histogram(color = "black", fill = "lightblue", binwidth = 10) +
  lims(x = c(145, 205), y = c(0, 7)) +
  labs(x = "New Weight Guess (lbs)", y = "Frequency",
       title = "Histogram of Weight Guesses (New Class)") +
  annotate(geom = "text", x = 175, y = 6.5, size = 5,
          label = "Two outliers\nat 10 and 240") +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"))

ggplot(data.frame(oldweight = oldwgt), aes(x = oldweight)) +
  geom_histogram(color = "black", fill = "lightblue", binwidth = 10) +
  lims(x = c(145, 205)) +
  labs(x = "Old Weight Guess (lbs)", y = "Frequency",
       title = "Histogram of Weight Guesses (Old Class)") +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"))
```



What do these histograms tell us about the distribution of guesses of his weight?



Are there any differences between the guesses for the two classes?

Primary Uses of Histograms: Histograms are used primarily to examine the shape of distribution, which is one of several key characteristics of a data set.

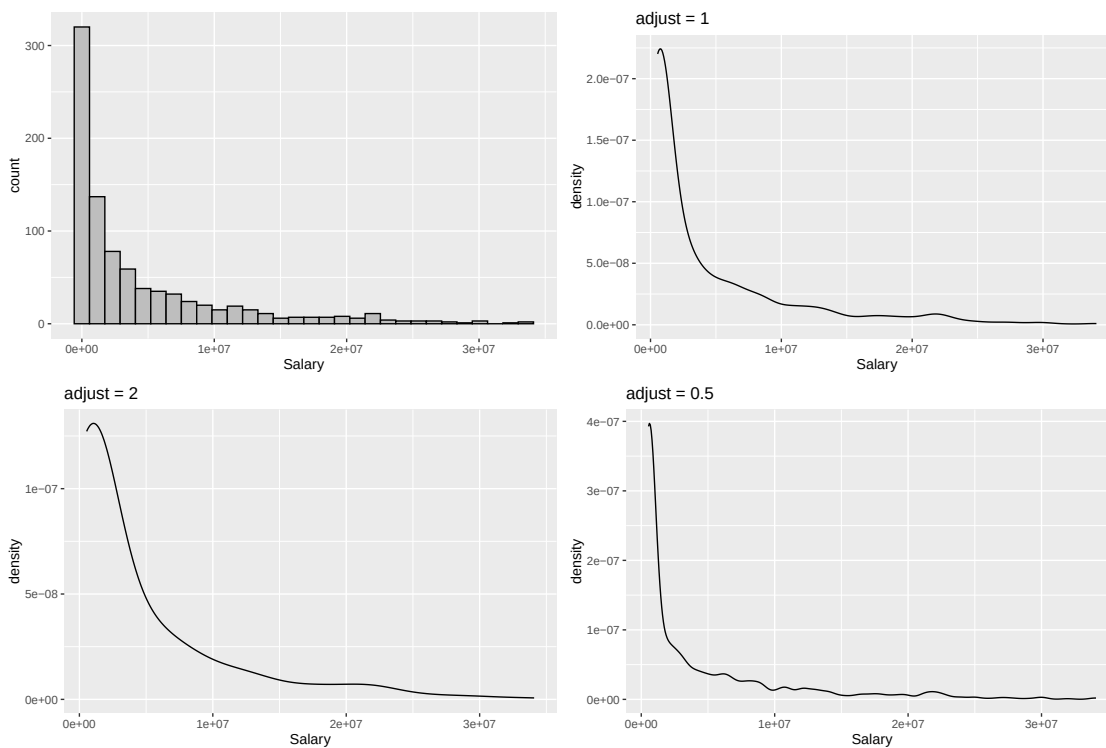
Describing Data Sets with One Quantitative Variable

1. Center: The median is the point at which half the observations are above, and half below. The mean is the average of the observed values.
2. Spread: Common measures of spread include:
3. Shape: Some common distribution shapes are indicated below.
4. Number of Peaks: Most distributions are unimodal (single peaked). When might a distribution of values be bimodal?
5. Deviations from Overall Shape (Unusual Features): Histograms are decent for identifying outlying values or aspects of the distribution which seem to deviate from the overall pattern of values. Box plots are better for identifying outliers.

Kernel Density Plots

A histogram is based on the number of observations in discrete intervals of the same width. A smoother alternative is to use a moving window of the same fixed width where, at each value on the x-axis, the number of observations in the interval centered at that value is recorded. A further refinement of this is called a kernel density estimate which weights the observations in the window, giving greater weight to observations nearer the center of the window.

- The command `geom_density()` function in `ggplot2` generates a kernel density estimate.
- The option `adjust` controls the window size relative to the default size calculated by **R**. A value smaller than 1 gives a smaller window and less smooth density estimate, while a value bigger than 1 gives a bigger window and smoother estimate.
- To see how this works, consider the three kernel density estimates for 2018 MLB salaries shown below.



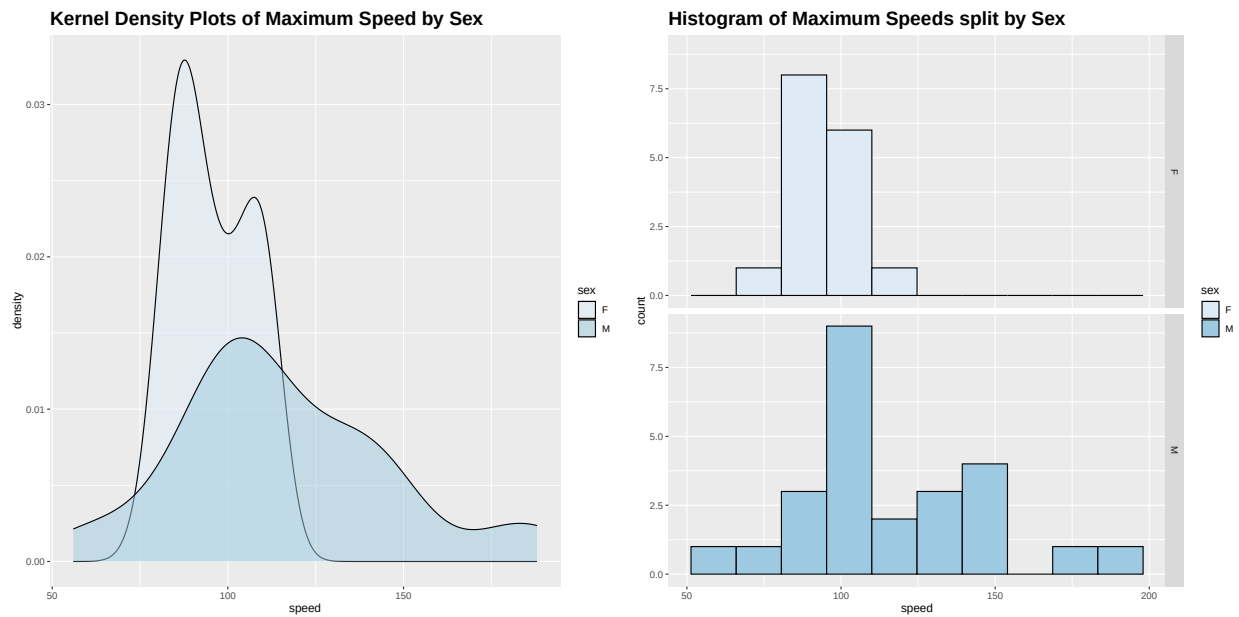
```
library(patchwork)
mlb <- read_csv("mlbsalaries.txt")
mlb_hist <- ggplot(mlb, aes(x = Salary)) +
  geom_histogram(color = "black", fill = "gray", bins = 30)
d1 <- ggplot(mlb, aes(x = Salary)) +
  geom_density(adjust = 1) +
  labs(title = "adjust = 1")
(mlb_hist | d1) / (d2 | d3)
```

Note that `d2` and `d3` are defined just like `d1` except the `adjust` term is changed.

- Kernel density plots are also useful for comparing distributions. For example, if we want to compare the distributions of the maximum speed driven between females and males, we can plot kernel density functions for both sexes on the same plot as follows.

```
ggplot(class19, aes(x = speed, fill = sex)) +
  geom_density(alpha = 0.5) +
  scale_fill_brewer() +
  labs(title = "Kernel Density Plots of Maximum Speed by Sex") +
  theme(plot.title = element_text(size = 18, face = "bold"))

ggplot(class19, aes(x = speed, fill = sex)) +
  geom_histogram(color = "black", bins = 10) +
  labs(title = "Histogram of Maximum Speeds split by Sex") +
  facet_grid(vars(sex)) +
  scale_fill_brewer() +
  theme(plot.title = element_text(size = 18, face = "bold"))
```



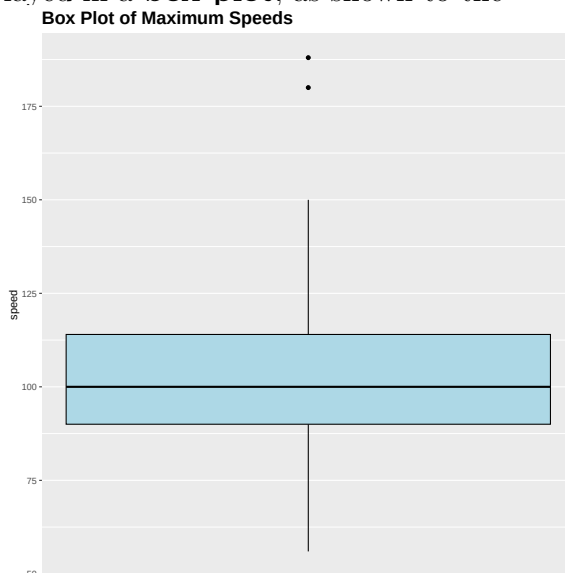
What do these plot indicate?

Box Plots

- The 5-number summary is often used to summarize a dataset.
- This group consists of the minimum, Q_1 , median, Q_3 , and maximum

The information in the 5-number summary is often displayed in a **box plot**, as shown to the right.

- Like histograms, box plots are used to display the distribution of a quantitative variable.
- Can see the center, spread, overall range, and degree of symmetry.
- Box plots are also great for making multiple comparisons
- The box plot is sometimes called the box and whisker plot.
- In a box plot, half the data lies above the thick black line and half lies below it.
- Half the data lies inside the box, and half lies outside
- The points are _____.



```
ggplot(class19, aes(y = speed)) +
  geom_boxplot(color = "black", fill = "lightblue") +
  labs(title = "Box Plot of Maximum Speeds") +
  theme(plot.title = element_text(size = 18, face = "bold")) +
  scale_y_continuous(breaks = seq(50, 200, 25)) +
  scale_x_continuous(breaks = NULL)
```

Describe the distribution of max speed.

Discerning the shape of a distribution with a box plot

- If the median is closer to Q_1 than Q_3 , then the distribution is skewed right.
- If the median is closer to Q_3 than Q_1 , then the distribution is skewed left.
- If the median is near the center of the box and the upper whisker extends farther than the lower whisker, then the distribution is skewed right.
- If the median is near the center of the box and the lower whisker extends farther than the upper whisker, then the distribution is skewed left.
- If the median is near the center of the box and whiskers are of similar length, then the distribution is approximately symmetric.

Box plots have two primary uses:

1. Comparing two or more distributions
2. Detecting & Displaying Outliers

Example: Consider the speed data from the previous example now split between females and males. First, the 5-number summaries are:

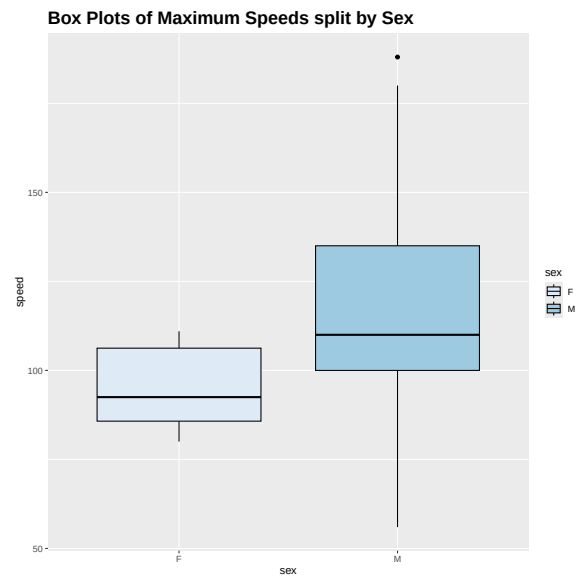
5-Number Summary for Females

Min	Q_1	Median	Q_3	Max
80.00	85.75	92.50	106.25	111.00

5-Number Summary for Males

Min	Q_1	Median	Q_3	Max
56	100	110	135	188

- Side-to-side box plots for these two speed distributions are drawn to the right.
- What comparisons can be made?



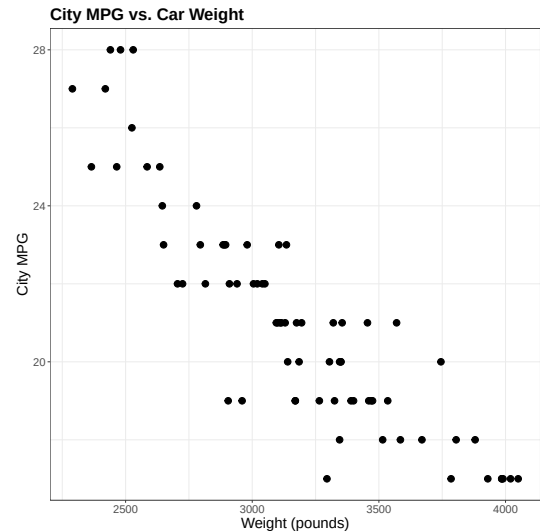
```
ggplot(class19, aes(x = sex, y = speed, fill = sex)) +
  geom_boxplot(color = "black") +
  labs(title = "Box Plots of Maximum Speeds split by Sex") +
  scale_fill_brewer() +
  theme(plot.title = element_text(size = 18, face = "bold"))
```

Scatterplots

The most common display for examining the **association** (or relationship) between two *quantitative* variables is the **scatterplot**. A scatterplot simply assigns one axis to one variable, and the second axis to the other, and plots the values against each other.

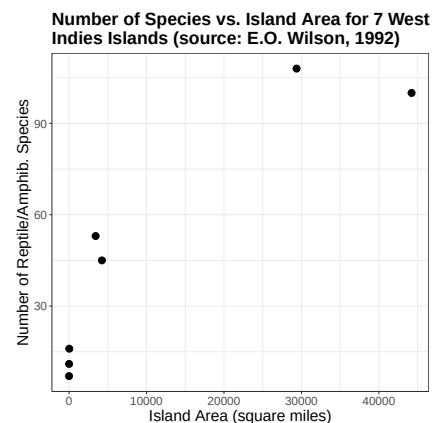
Examples:

1. Is there a relationship between the mileage (MPG) the nature of the relationship?



```
cars99 <- read_csv("cars99.txt") |>
  na.omit() # Reads in the cars99 data and removes rows with missing values
ggplot(cars99, aes(x = Weight, y = CityMPG)) +
  geom_point(size = 3) +
  labs(x = "Weight (pounds)", y = "City MPG",
       title = "City MPG vs. Car Weight") +
  theme_bw() +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"))
```

2. Is there an association between island size and the number of reptile or amphibian species an island can support? How would you describe this relationship?



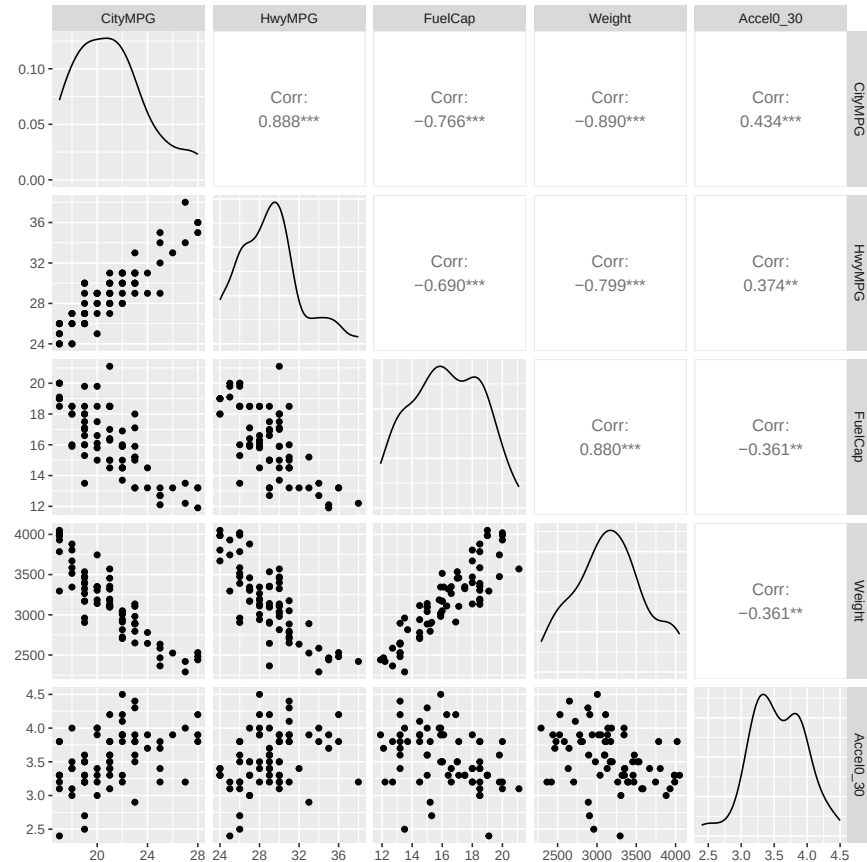
Describing Association: 4 Things to Look For

1. **Direction**: In what direction do the values of one variable change with increases in the values of the other variable?
 - (a) **Positive Association**: Two variables are said to have positive association if the values of one variable tend to *increase* with the values of the second variable. So larger values of one variable generally correspond to larger values of the second variable. Examples?
 - (b) **Negative Association**: Two variables are said to have negative association if the values of one variable tend to increase as the values of the second variable tend to *decrease*. So larger values of one variable generally correspond to smaller values of the second variable. Examples?
 - (c) **Other Types of Association**: Two variables may be associated in both a positive and negative fashion over different ranges of values for the two variables. Examples?
 - (d) **No Association**: Two variables are not associated if there is no apparent association (positive, negative, or otherwise) present in the scatterplot. Examples?
2. **Form**: In what form, or how quickly do the values of one variable change with the values of the other variable?
 - (a) **Linear**: The relationship between two quantitative variables is said to be *linear* if they exhibit a straight-line pattern in their scatterplot.
 - (b) **Nonlinear**: If the relationship is not linear, we say it is nonlinear. Common forms of nonlinear relationship are *exponential* or *quadratic*.
3. **Strength**: How much variability or spread is present in the scatterplot?
 - Lower scatter (stronger association) indicates clearer trends or patterns from which estimation is more precise. If there is too much scatter, we may not be able to discern any relationship between the two variables.
4. **Outliers**: You should note any points which do not fit the overall relationship.

Matrix of Scatterplots

In addition to looking at a single scatterplot describing the relationship between two variables, we can look at the relationships between several variables using a matrix of scatterplots with the `ggpairs()` function in the `GGally` library in **R**:

```
library(GGally) # Matrix of scatterplots for
ggpairs(cars99[,c(3, 4, 5, 6, 8)]) # variables 3, 4, 5, 6, 8
```



The variables here are:

1. City MPG
2. Highway MPG
3. Fuel Capacity (in gallons)
4. Car Weight (in pounds)
5. Acceleration time (0-30 mph)

Notes 3: Statistical Inference Review

Table of contents

Important Probability Distributions for Inference	48
The Normal Distribution	48
The t -distribution	49
The χ^2 distribution	49
The F Distribution	50
The Central Limit Theorem	50
Confidence Intervals	51
Inferential Statistics Overview Review	51
Confidence Interval Formulas	52
Confidence interval interpretations	52
Hypothesis Tests	53
Test Statistic Formulas	54
Confidence Intervals and Tests in R	54
ANOVA	56
Misuse and Abuse of Tests	59

In MATH 1040 (or the stats class you took), you were introduced to statistical inference. There are two primary types of inference procedures:

1. **Confidence Intervals.**

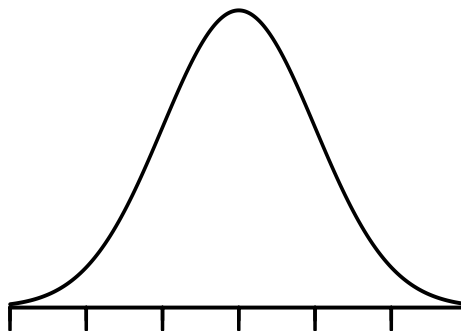
2. **Hypothesis Tests.**

For parametric inference, these are based on probabilities from known probability distributions.

Important Probability Distributions for Inference

The Normal Distribution

Normal distributions all have the same bell-shape, as shown to the right. A particular normal pdf is specified by giving its mean μ and standard deviation σ . The mean occurs at the peak of the curve. The standard deviation is the distance between the mean and the first tick mark next to it.



Definition: A random variable X has a **normal (Gaussian) distribution** with parameters μ, σ^2 if X has probability density function (pdf):

$$f(x) = \frac{1}{\sqrt{2\pi} \cdot \sigma} \exp \left\{ -\frac{(x - \mu)^2}{2\sigma^2} \right\}, \quad -\infty < x < \infty, \quad -\infty < \mu < \infty, \quad \sigma^2 > 0.$$

- In this case, we say that X is normally distributed with mean μ and variance σ^2 , and write $X \sim N(\mu, \sigma^2)$.

Properties of a Normal Distribution

1. Normal densities are always bell-shaped.
2. Normal densities are symmetric about the mean μ , which is also the median and mode.
3. It follows the empirical rule.
 - Approximately 68% of the data fall within 1 SD of the mean: $(\mu - \sigma, \mu + \sigma)$.
 - Approximately 95% of the data fall within 2 SDs of the mean: $(\mu - 2\sigma, \mu + 2\sigma)$.
 - Approximately 99.7% of the data fall within 3 SDs of the mean: $(\mu - 3\sigma, \mu + 3\sigma)$.

The normal distribution is used when certain conditions are met when performing inference on:

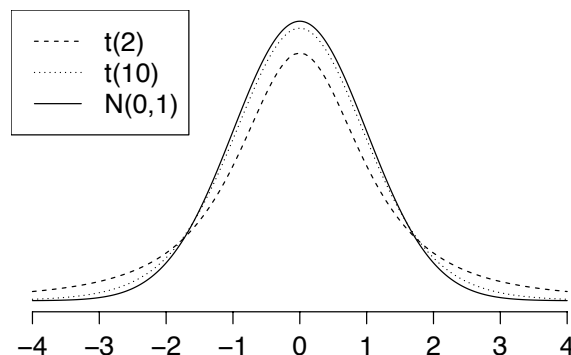
- The population proportion (p).
- The difference in population proportions ($p_1 - p_2$).
- The population mean (μ) when the population variance (σ^2) is known.
- The difference in population means ($\mu_1 - \mu_2$) when each variance (σ_1^2 and σ_2^2) is known.

The t -distribution

The t -distribution is actually an entire *family* of distributions with a single parameter called the degrees of freedom (d.f.).

This family of distributions has the following properties:

1. They are symmetric about zero and mound-shaped (like the standard normal distribution).
2. The t -distribution has wider tails than the normal distribution to account for the extra variability in estimating σ .
3. The degrees of freedom completely specify which t -distribution to use.
4. As the degrees of freedom increase, the t -distribution gets closer and closer to the standard normal distribution.



The t -distribution allows us to account for the fact that we also have to estimate multiple parameters.

The t distribution is used when performing inference on (among other things):

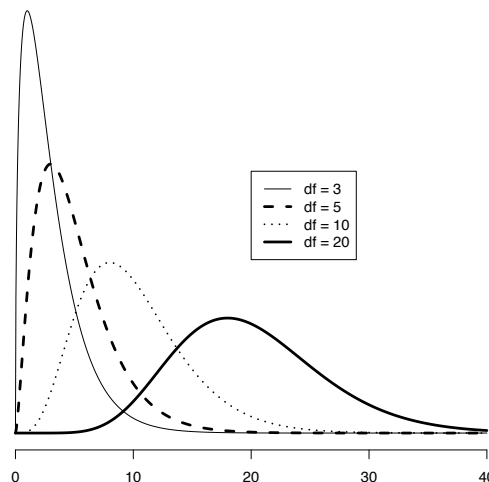
- The population mean (μ) when the population standard deviation (σ) is unknown.
- The difference in population means (independent: $\mu_1 - \mu_2$, dependent: μ_d) when the population variances are unknown.
- The population correlation coefficient (ρ).
- The population intercept (β_0).
- The population slope (β_1).

The χ^2 distribution

Like a t -distribution, a χ^2 distribution is a family of distributions that are characterized by their degrees of freedom.

This family of distributions has the following properties:

1. They are not symmetric, they are skewed right.
2. The degrees of freedom completely specify which χ^2 -distribution to use since the mean is equal to the df and the standard deviation is equal to $\sqrt{2 \cdot df}$.
3. All χ^2 values are greater than 0.
4. As the degrees of freedom increase, the χ^2 -distribution gets more and more symmetrical.



The χ^2 distribution is used when performing inference on:

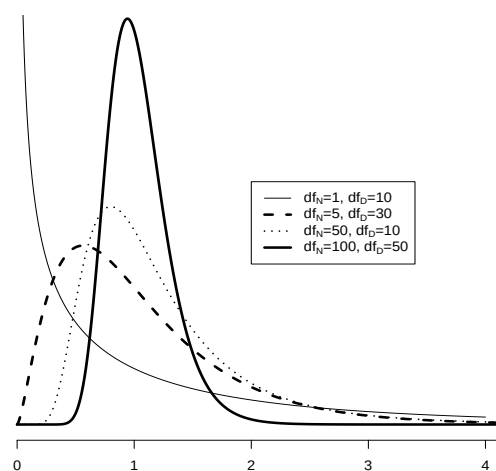
- The population standard deviation or variance (σ or σ^2).
- The distribution of a single categorical variable in a goodness-of-fit test.
- Two categorical variables to test for independence.

The F Distribution

Like a t or χ^2 distribution, an F distribution is a family of distributions that are characterized by their degrees of freedom.

This family of distributions has the following properties:

1. Like χ^2 , the F distributions are not symmetric, they are skewed right.
2. An F variable is the ratio of two χ^2 variables and each χ^2 variable has its own degrees of freedom, an F distribution will have degrees of freedom that correspond to the numerator, df_N , and degrees of freedom that correspond to the denominator, df_D .
3. All F values are at least 0.
4. As both degrees of freedom increase, the F distribution gets more and more symmetric.
5. The mean of all F distributions is approximately equal to 1.
6. If the numerator degrees of freedom equals 1, then the square root of the F distribution is identical to the t distribution.



The F distribution is used when performing inference on:

- The ratio of population standard deviations or variances (σ_1/σ_2 or σ_1^2/σ_2^2).
- The difference in more than two population means in an ANOVA test.
- The overall significance of a regression model.

The Central Limit Theorem

Many inference procedures are based on the Central Limit Theorem (CLT). This theorem has two very important statements:

1. The sample mean ($\bar{x}$) is approximately normally distributed with $\mu_{\bar{x}} = \mu$ and $\sigma_{\bar{x}} = \sigma/\sqrt{n}$ when the sample values are independent and the original data are normal or $n \geq 30$.
2. The sample proportion ($\hat{p}$) is approximately normally distributed with $\mu_{\hat{p}} = p$ and $\sigma_{\hat{p}} = \sqrt{\frac{p(1-p)}{n}}$ when the sample values are independent and $np \geq 5$ and $n(1-p) \geq 5$.

Confidence Intervals

Inferential Statistics Overview Review

A **point estimate** is a single value estimate for a population parameter.

- The point estimate for the population mean μ is
- The point estimate for the population standard deviation σ is
- The point estimate for the population proportion p is

One issue we face is that our estimates are nearly always wrong! They are wrong not because we make mistakes (though this is possible), but because they are based on limited, random samples. A sample mean, for instance, is random due to the sample itself being random. If a new sample is taken, a different mean will be calculated. This is known as **sampling variability**.

We deal with the fact that we are wrong by determining how wrong we may be. We quantify this by calculating a **margin of error**, which we add and subtract from our point estimate in order to form a **confidence interval** (CI).

This way, we have not only our “best guess” for the value of the unknown parameter, but we have an idea of the extent to which our best guess may be in error.

Confidence Interval Formulas

This gives us the formula for a confidence interval for a **population proportion** (p):

$$\hat{p} \pm z_{\alpha/2} \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

where $z_{\alpha/2}$ is known as the **critical value** that corresponds to a certain **confidence level**. The **margin of error** is the part of the formula after the “ $\pm$ ”. This confidence interval is only valid if our sample size is large enough. **Only valid** if $n\hat{p} \geq 5$ and $n\hat{q} \geq 5$. In **R**, to get the critical value you can type `qnorm(1-alpha/2)` where $\alpha = 1 - \text{confidence level}$.

The formula for a confidence interval for a **population mean** (μ) is:

$$\bar{x} \pm t_{\alpha/2} \cdot s/\sqrt{n}; \quad df = n - 1$$

where $t_{\alpha/2}$ is known as the **critical value** that corresponds to a certain **confidence level**. Each $t_{\alpha/2}$ value will have associated degrees of freedom, $df = n - 1$. The **margin of error** is the part of the formula that follows the “ $\pm$ ”. This confidence interval is only valid if our sample size is large enough. **Only valid** if the sample is random and $n \geq 30$ or data come from a normal distribution. In **R**, to get the critical value, you can type `qt(1-alpha/2,df)` where $\alpha = 1 - \text{confidence level}$.

The confidence intervals for the **population variance** (σ^2) and the **population standard deviation** (σ) are, respectively,

$$\frac{(n-1)s^2}{\chi_R^2} < \sigma^2 < \frac{(n-1)s^2}{\chi_L^2}; \quad df = n - 1$$

and

$$\sqrt{\frac{(n-1)s^2}{\chi_R^2}} < \sigma < \sqrt{\frac{(n-1)s^2}{\chi_L^2}}; \quad df = n - 1$$

These χ_L^2 and χ_R^2 values are critical values. We need two different critical values since the χ^2 distribution is not symmetric. Original data must be normal. In **R**, to get the critical values, you can type `qchisq(alpha/2,df)` for χ_L^2 and `qchisq(1-alpha/2,df)` for χ_R^2 .

Confidence interval interpretations

Once the CI is constructed, we need to give an English interpretation. The most common interpretation follows this template:

“We are (confidence level)% confident that the true (parameter) lies between (lower bound) and (upper bound).”

Hypothesis Tests

Hypothesis testing is another statistical inference procedure. In each hypothesis test, we assume the null hypothesis is true and then gather evidence based on a sample that it is not. This evidence is found in calculating a test statistic and a corresponding p -value.

Definition: A **p -value** is the probability of obtaining the sample data or something more extreme simply by chance, assuming the null hypothesis H_0 is true.

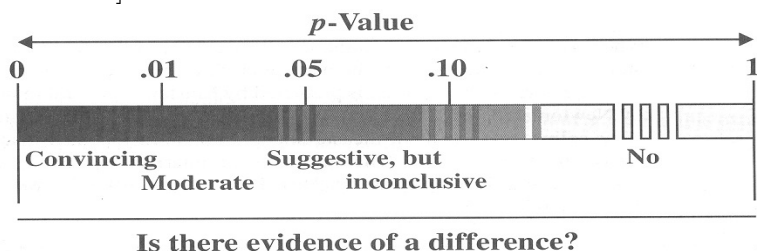
- If the p -value is small (close to 0), then we have
- If the p -value is large (close to 1), then we have

Every hypothesis test will be a left-tailed, right-tailed, or two-tailed test. We can tell which test we should use based on the sign in the alternative hypothesis, although some tests, like the χ^2 test for independence or ANOVA, are always right tailed. This will also let us know how to calculate the p -value.

Test Type	Sign in H_a	p -value
Left-tailed		$P(\text{variable} < \text{test stat})$
Right-tailed		$P(\text{variable} > \text{test stat})$
Two-tailed		$2 \cdot P(\text{variable} > \text{test stat})$

In **R**, p -values can be computed with the `pnorm()`, `pt()`, `pchisq()`, or `pf()` functions.

In MATH 1040 or a similar class, you may remember making statistical decisions by comparing the p -value of your test to the significance level (α) to make the statistical decision. In this class, we will interpret tests a bit differently. The current and probably most accepted view toward the subjectivity inherent in the choice of α is simply to report the p -value for a given test and let the reader decide on how much evidence is necessary to reject the null hypothesis. The following diagram represents current interpretations of the p -value. [Diagram taken from the *Statistical Sleuth* by Ramsey & Schafer.]



This is saying, as a guideline, that

- If $p\text{-value} > 0.10$, we have little or no evidence in favor of H_a .
- If $0.05 < p\text{-value} < 0.10$, we have suggestive evidence in favor of H_a .
- If $0.01 < p\text{-value} < 0.05$, we have moderate evidence in favor of H_a .
- If $0.001 < p\text{-value} < 0.01$, we have strong evidence in favor of H_a .
- If $p\text{-value} < 0.001$, we have very strong evidence in favor of H_a .

Test Statistic Formulas

In general, we usually obtain the test statistic by dividing the difference between the statistic and hypothesized parameter by the standard error of the statistic assuming H_0 is true.

The formula for the test statistic when testing a **population proportion** (p) is

$$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$$

The formula for the test statistic when testing a **population mean** (μ) is

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}; \quad df = n - 1$$

The formula for the test statistic when testing a **population variance** (σ^2) or **population standard deviation** (σ) is

$$\chi^2 = \frac{(n-1)s^2}{\sigma_0^2}; \quad df = n - 1$$

Confidence Intervals and Tests in R

R has many built-in functions for doing statistical inference, but you can also program your own functions or install a package to do a test that **R** does not have by default.

Parameter Testing	R Function	Special Instructions
Single Proportion (p)	<code>prop.test()</code> *	None
Difference in Proportions ($p_1 - p_2$)	<code>prop.test()</code> *	Put in vectors of successes and samples.
Single Mean (μ)	<code>t.test()</code>	None
Difference in Independent Means ($\mu_1 - \mu_2$)	<code>t.test()</code>	Put in two vectors of data.
Difference in Dependent Means (μ_d)	<code>t.test()</code>	Put <code>paired = T</code>
Single Variance or SD (σ^2 or σ)	No Function	None
Ratio of Variances or SDs (σ_1^2/σ_2^2 or σ_1/σ_2)	<code>var.test()</code>	None
Population Correlation (ρ)	<code>cor.test()</code>	Need two samples

For all of these functions, they will return a confidence interval, a test statistic and a p-value and you can set the confidence level with the `conf.level` argument and the type of test with `alternative = "two.sided"`, `alternative = "less"`, or `alternative = "greater"`.

* The `prop.test()` functions actually use a different formula than the typical ones that we use for proportions, so the results will not match exactly. That is okay.

Example: Suppose you are curious if the average speed on North Main Street is higher at night than during the day. You take a random sample of 16 vehicles at night and 20 during the day. Assume the populations are normal. The data are found in the table on the following page.

Night	48	42	40	43	45	51	53	50	46	40
	38	55	58	43	40	45				
Day	42	45	38	46	45	47	45	48	50	40
	42	38	46	40	45	46	43	48	45	46

(a) Is there enough evidence to conclude that the average speeds for day and night differ?

We can conduct this test in **R**.

```
night <- c(48, 42, 40, 43, 45, 51, 53, 50, 46, 40, 38, 55, 58, 43, 40, 45)
day <- c(42, 45, 38, 46, 45, 47, 45, 48, 50, 40, 42, 38, 46, 40,
        45, 46, 43, 48, 45, 46)
t.test(night, day) # This has the default 95% CI and two-tailed test
```

Welch Two Sample t-test

```
data: night and day
t = 1.0935, df = 22.374, p-value = 0.2858
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -1.621769  5.246769
sample estimates:
mean of x mean of y
 46.0625  44.2500

2*(1 - pt(1.0935, 22.374)) # Show p-value matches
[1] 0.2858082
```

(b) Is there evidence that time of day is related to the proportion of vehicles that travel over 50 mph?

```
sum(night > 50) # Count the number of speeders at night
```

```
[1] 4
```

```
sum(day > 50) # Count the number of speeders during the day
```

```
[1] 0
```

```
prop.test(c(4, 0), c(16, 20)) # Put in vectors of successes and sample sizes
```

```
Warning in prop.test(c(4, 0), c(16, 20)): Chi-squared approximation may be incorrect
```

2-sample test for equality of proportions with continuity correction

```

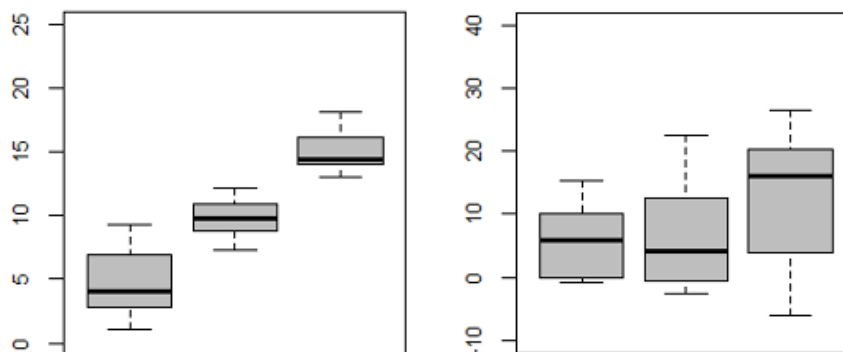
data:  c(4, 0) out of c(16, 20)
X-squared = 3.3785, df = 1, p-value = 0.06605
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.01842233  0.51842233
sample estimates:
prop 1 prop 2
 0.25   0.00

```

ANOVA

The procedure for testing if there is at least one difference in k population means is known as the **analysis of variance** or ANOVA. Of course, we know that if we are testing for a difference in two independent means, we can use a two-sample t-test. With ANOVA, we test three or more groups to see if any of groups' means differ significantly from the others.

It may seem strange to call the test that compares *means* an analysis of *variance*, but the variance is a crucial part of the process. Consider the following boxplots. In both of these cases, the sample means for the three boxplots are about 5, 10, and 15, respectively. In the first case, it seems clear that the true means must also differ. In the second case, this isn't so clear.



The thing that makes these two cases so different is that in the first, the boxplots are not very spread out, while in the second they are. More variation reflects greater _____ regarding the values of the true unknown means. %uncertainty

With ANOVA, we compare the average **between group** variance to the average **within group** variance.

Between group variance refers to how much the means of the groups vary from one another. **Within group** variance refers to how spread out the data are in each group.

The basic idea is this: if the average amount of variation _____ the groups is substantially larger than the average amount of variation _____ the groups, then the true group means likely differ. If not, then we cannot rule out that they are equal.

As you might guess, we will make this decision using a hypothesis test. For ANOVA, we have:

H_0 : The group means are all equal or $\mu_1 = \mu_2 = \cdots = \mu_k$

H_a : At least one group mean differs from the others.

All of the information regarding this ANOVA procedure can be summarized in an ANOVA table:

Source of Variation	df	Sum of Squares	Mean Square	F Statistic
Between	$k - 1$	SS_B	$MS_B = \frac{SS_B}{k-1}$	$F = \frac{MS_B}{MS_W}$
Within	$n - k$	SS_W	$MS_W = \frac{SS_W}{n-k}$	
Total	$n - 1$	SS_T		

where k is the number of groups, n is the overall sample size,

$$SS_B = n_1(\bar{y}_1 - \bar{y})^2 + \cdots + n_k(\bar{y}_k - \bar{y})^2 = \sum_{i=1}^k n_i(\bar{y}_i - \bar{y})^2,$$

and

$$SS_W = (n_1 - 1)s_1^2 + \cdots + (n_k - 1)s_k^2 = \sum_{i=1}^k (n_i - 1)s_i^2.$$

It turns out that when we divide these mean squares, we obtain a value that follows an F distribution. So, we can then compute our test statistic by calculating:

$$F = \frac{MS_B}{MS_W}; \quad df_N = k - 1; \quad df_D = n - k$$

If the F test statistic is large, that means there is much more variation _____ samples than there is _____ them. If that is the case, we should _____ the null hypothesis that all of the means are equal.

There are some necessary conditions to use ANOVA:

1. The population of each group is normal.
2. Each population has the same variance (or standard deviation).
3. Each sample is random and independent.

Example: In 1969, a study by Darley and Latan'e focused on whether the presence of other people has an influence on whether a person will help someone in distress. In this classic study, the experimenter (a female graduate student) had the subject wait in a room with 0, 2, or 4 other people. The experimenter announces that the study will begin shortly and walks into an adjacent room. In a few moments the person(s) in the waiting room hear her fall and complain of ankle pain. The dependent measure is the number of seconds it takes the subject to help the experimenter. The data are given below:

	Group 1 (0 people present)	Group 2 (2 people present)	Group 3 (4 people present)
	25	30	32
	30	33	39
	20	29	35
	32	40	41
		36	44
Sample Mean	$\bar{y}_1 = 26.75$	$\bar{y}_2 = 33.6$	$\bar{y}_3 = 38.2$
Sample Std. Dev.	$s_1 = 5.38$	$s_2 = 4.51$	$s_3 = 4.76$
Sample Size	$n_1 = 4$	$n_2 = 5$	$n_3 = 5$
True Mean			
True Variance			

In R:

```
times <- c(25, 30, 20, 32, 30, 33, 29, 40, 36, 32, 39, 35, 41, 44)
group <- c(1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3)
group <- as.factor(group) # Define group as a categorical variable
anova(aov(times ~ group)) # Use anova function to do the anova analysis
```

Analysis of Variance Table

Response: times

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
group	2	292.11	146.054	6.209	0.01567 *
Residuals	11	258.75	23.523		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Misuse and Abuse of Tests

1. Making Decisions and Interpreting

- When conducting a hypothesis test, we either reject H_0 or we fail to reject H_0 . We never accept H_0 or H_A (although it is not *always* wrong to say we accept H_A).
- When interpreting a hypothesis test result, we interpret as either having or not having enough evidence for H_A . We will **never** interpret in terms of H_0 . Failing to have evidence for H_A is *NOT* the same as having evidence for H_0 . Example:

$$H_0 : p \leq 0.5$$

$$H_A : p > 0.5$$

Suppose $\hat{p} = 0.55$ and we get a p -value of 0.12. We would have have little to no evidence that the true proportion is greater than 0.5. However, this does not mean we have evidence that the true proportion is less than or equal to 0.5. We certainly do not since our sample proportion was bigger than 0.5. So while we do not have enough evidence for the alternative to suggest the alternative is true, we have *even less* evidence for the null hypothesis.

2. Choosing a level of significance

- Levels of significance are usually chosen to make a decision (either reject H_0 , or fail to reject H_0)
- There is no sharp border between significance & insignificance, only increasingly strong evidence against H_0 as the p -value decreases.

3. Statistical significance is not the same as practical significance

- Suppose we are testing the mean pH level: $H_0: \mu \leq 5.2$ vs. $H_a: \mu > 5.2$. Suppose also now that $n = 1442$, $\bar{x} = 5.25$, $\sigma = 0.9$, and $\alpha = 0.05$. Then the standard deviation of the sample mean is:

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}} = \frac{0.9}{\sqrt{1442}} = 0.0237, \text{ with test statistic:}$$

$$z = \frac{\bar{x} - \mu}{\sigma/\sqrt{n}} = \frac{5.25 - 5.2}{0.0237} = 2.11,$$

which implies: $p\text{-value} = P(Z > 2.11) = \underline{0.0174}$ (statistically significant).

- But practically, the difference between 5.25 and 5.2 may be insignificant to the researcher.
- What causes this seemingly small difference to be statistically significant? Answer: The large sample size. Large samples ($n = 1442$) are not bad; they just detect small differences very accurately.

4. Don't ignore lack of significance

- It is a sad but true fact, that most journals only report the results of studies where the $p\text{-value} < 0.05$. A $p\text{-value}$ of 0.10, 0.15, etc. may still be worth knowing.
- Not finding an effect (not rejecting H_0) can also have scientific implications.

5. Test of significance does NOT check the design of a study

Example: In a certain company, the absentee rate was found to be 6.3 days/year, with a standard deviation of 3 days/year. Suppose flex-time is instituted and we want to see if the absentee rate decreases.

- In a simple random sample of $n = 100$ people under the new flex-time program, the average absentee rate was $\bar{x} = 5.4$ days/year.
- Test $H_0 : \mu = 6.3$ vs. $H_a : \mu < 6.3$.

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} = \frac{5.4 - 6.3}{3/\sqrt{100}} = -3 \implies p\text{-value} = P(Z < -3) = 0.0013.$$

- The small p -value gives strong evidence that absenteeism went down, not that flex-time caused the reduction.

6. p -values do not hold for multiple tests simultaneously

- If you are trying to figure out which of three groups averages differ after doing an ANOVA, we need to take into account that there are $\binom{3}{2} = 3$ different pairs that need to be tested. The p -values should be adjusted accordingly (we will see this in the next set of notes).

7. Hypothesis tests are not appropriate for Exploratory Data Analysis (EDA)

- Hypotheses should always be stated before the data are collected. The rationale behind hypothesis testing methods is to first decide what it is you want to test, set up an experiment to collect data, and use a test of significance to test your hypothesis.
- In EDA, we look for patterns (stemplots, histograms, etc.). It is not legitimate to construct hypotheses which fit the pattern of the data.
- Note that this does not say it is wrong to “look” for patterns or relationships in data that have been collected. In fact, this is how many discoveries in science are made. It is, however, wrong to then use these hypothesis testing techniques on the data where some effect was observed to draw conclusions about statistical significance of that effect. This is known as **data snooping** or **p -Hacking** and is considered to be bad statistical practice by the scientific community.
- If you discover some effect in data that were not collected specifically to examine that effect, the next step is to design a new experiment with the sole purpose of studying your now hypothesized effect.

8. Major Criticism: In recent years, the entire hypothesis testing framework has been criticized by its practitioners both in the statistics and scientific community at large. The major criticism stems from the view that the null hypothesis is never really true. In other words, do we really believe that the true mean pH could ever be *exactly* 5.2, as stated by the null hypothesis? Keep in mind that we are not testing whether the mean is *significantly different* from 5.2; we are testing whether it is *different* from 5.2. This has led to more widespread use of Bayesian inference and model selection criteria such as AIC in a model framework. We will explore some of these ideas later in the course.

Notes 4: More ANOVA

Table of contents

Checking Assumptions	62
Assumption 1: The population of each group is normal	62
Assumption 2: The variances of each group are equal	65
Multiple Comparisons	66
Tukey-Kramer Procedure	66
Two-Way ANOVA	70
Two-Way ANOVA Hypotheses	71
Two-Way ANOVA conditions	71
Two-way ANOVA in R	71
Interaction term	72
Math of Two-Way ANOVA	74

We discussed in Notes 3 that the assumptions of the ANOVA are:

1. The population of each group is normal.
2. Each population has the same variance (or standard deviation).
3. Each sample is random and independent.

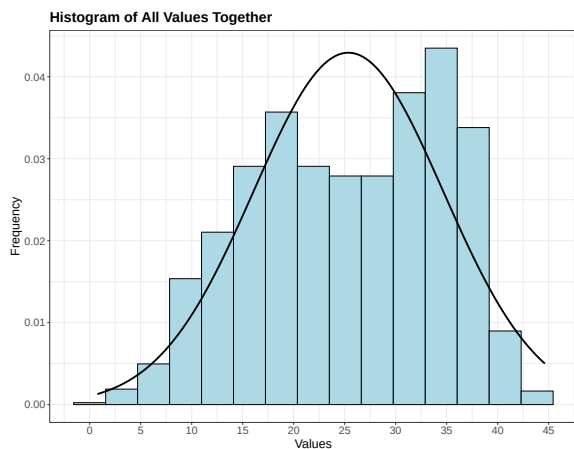
In practice, unless we have information about the population (which is unlikely), we can never know for sure that those first two assumptions actually hold. But good news: they don't have to hold exactly. When doing statistical tests and analysis, there is some **robustness** to violations of assumptions. Some tests are more robust to violation of some assumptions than others. We just need to have evidence that these assumptions are reasonably met. Or, in other words, as long as the populations are *close enough* to normal and the variances are *close enough* to equal, the ANOVA will work just fine.

Checking Assumptions

With that in mind, we need to discuss some tools for checking those assumptions. We will focus on the first two, since the third one is based on sampling technique, which occurs before and as we collect the data.

Assumption 1: The population of each group is normal

First, be aware that this does not say that our data is normal. We do not need the sample, when mixed all together, to follow a normal distribution. We need each individual group to follow a normal distribution. Consider this example:



\includegraphics[width=3in,alt={Image showing histograms of each group separately. Each histogram looks normal.}]{../Postscript/notes2_normhist2.p

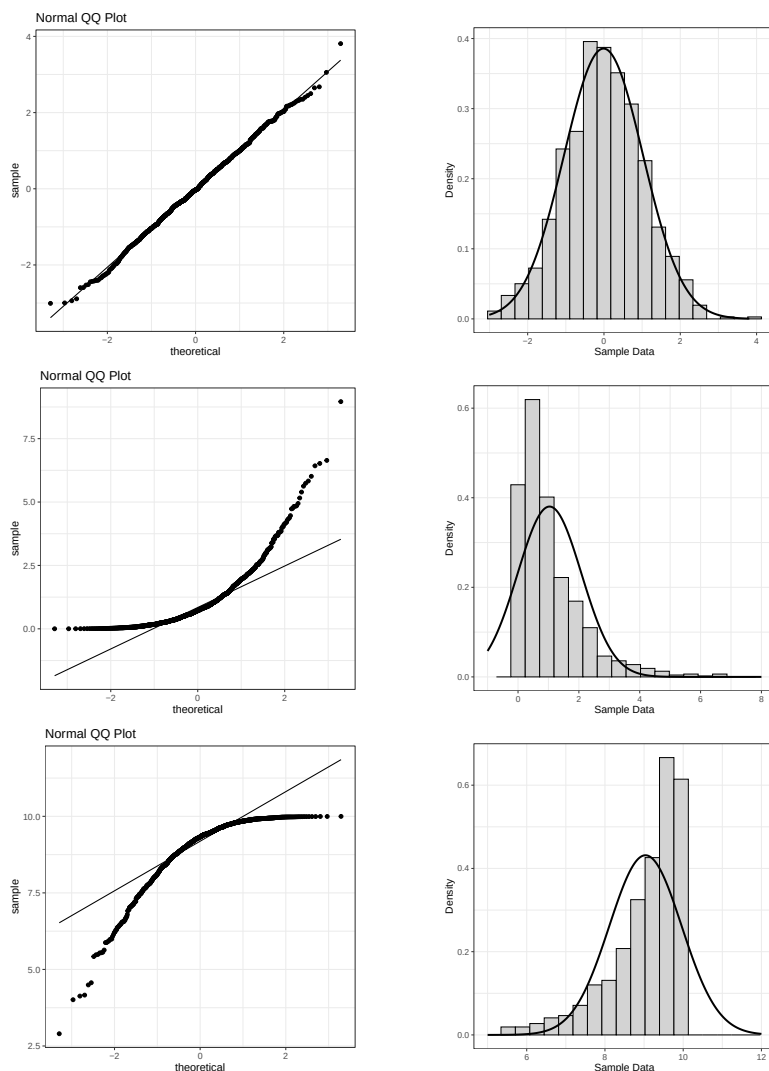
Making a histogram for each group and checking if they look approximately normal is one

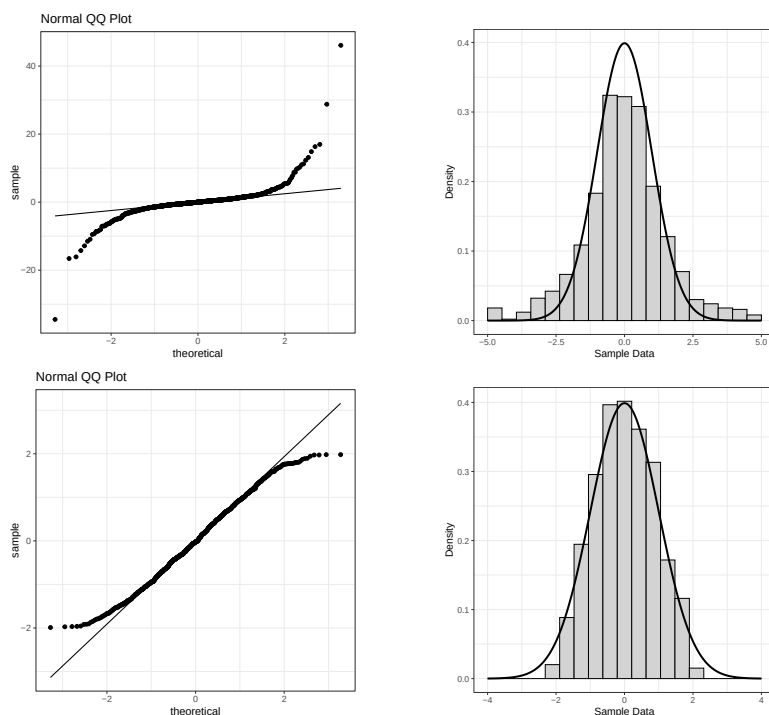
way to check this assumption. However, that is not the most common way to check normality. Instead, it is common to use a **QQ plot**.

QQ Plots

QQ plots stand for Quantile-Quantile plots and are used for checking if the residuals are approximately normally distributed. We plot the residuals against values that would be expected from a normal distribution. The closer to a straight line these points are, the closer to normality the residuals are.

Quantiles are cut points dividing the range of a probability distribution into continuous intervals with equal probabilities, or dividing the observations in a sample in the same way. Quartiles and percentiles are special quantiles. Quartiles divide the observations into quarters, percentiles divide them into hundreds.



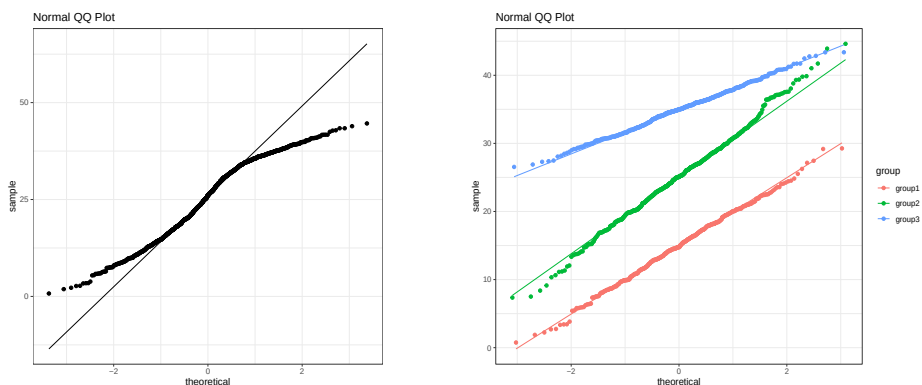


The code for creating a QQ plot in **R** is:

```
ggplot(data = df, aes(sample = values)) +
  geom_qq_line() + # Put the geom_qq_line before geom_qq to keep
  geom_qq() +      # the x and y labels
  labs(title = "Normal QQ Plot") +
  theme_bw()
```

To split it up by a categorical variable, put that variable as the color option in the `aes()` function.

```
ggplot(data = df, aes(sample = values, color = group)) +
  geom_qq_line() +
  geom_qq() +
  labs(title = "Normal QQ Plot") +
  theme_bw()
```



Assumption 2: The variances of each group are equal

The ANOVA is fairly robust when it comes to equal variances. That is, if this assumption is not met, the test still performs quite well unless the variances are *very* unequal.

As long as the largest group sample standard deviation is less than twice as large as the smallest group sample standard deviation, then the ANOVA typically still performs well.

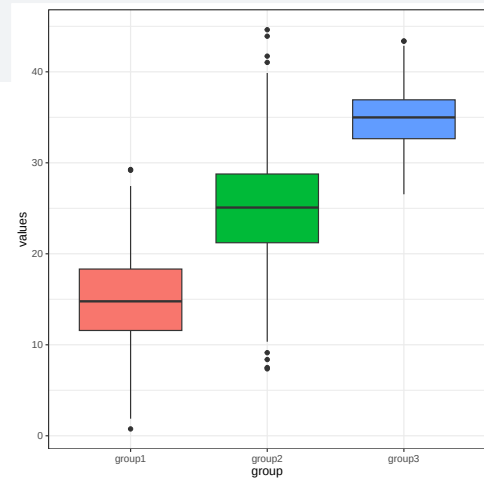
We can find each group standard deviation using the **R** code:

```
df |>
  group_by(group) |>
  summarize(st.dev = sd(values))

# A tibble: 3 x 2
  group st.dev
  <chr>   <dbl>
1 group1  4.94
2 group2  6.04
3 group3  3.08
```

It can also be helpful to make side-by-side boxplots:

```
ggplot(df, aes(x = group, y = values, fill = group)) +
  geom_boxplot() +
  theme_bw() +
  theme(legend.position = "none")
```



What if one of the assumptions is not satisfied?

Note that if the normality assumption seems to be reasonably satisfied, but the equal variances assumption does not, there is a test called the Welch's ANOVA. In **R**, this can be done using the `oneway.test()` function.

If the normality assumption is not met (whether the equal variance assumption is met or not), then ANOVA should not be used. Instead, the Kruskal-Wallis test should be used. We'll discuss this later in the course.

Multiple Comparisons

The ANOVA procedure is nice for telling if at least one population is likely to differ, but this overall ANOVA test does not identify which means differ from which other means or how many differ. This is where the idea of multiple comparisons comes into play.

There are several multiple comparisons methods, but we will concentrate on just one: the **Tukey-Kramer Procedure**. This is used for multiple comparisons between pairs of treatment means and allows for sample sizes to differ.

Tukey-Kramer Procedure

The Tukey-Kramer procedure for multiple comparisons is specifically designed to test all pairwise comparisons between the treatment means. Tukey's procedure does a **simultaneous** comparison of all pairs of treatment means. It is for this reason that Tukey's procedure is generally the recommended procedure for examining all pairwise comparisons.

We can do these pairwise comparisons using either hypothesis tests or confidence intervals. We will focus on the intervals. Like most confidence intervals, the general form of the interval is

$$\text{estimated difference} \pm \text{margin of error}$$

A t interval for the difference in means for groups i and j if we assume the population variances are the same would look like this:

$$\bar{y}_i - \bar{y}_j \pm t_{\alpha/2} \sqrt{s_p^2 \left(\frac{1}{n_i} + \frac{1}{n_j} \right)}$$

where $s_p^2 = \frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_i + n_j - 2}$. This is good for testing the difference for any one pair. For the Tukey HSD intervals, which is good for testing the difference in *every* pair, the form is

$$\bar{y}_i - \bar{y}_j \pm q_{\alpha}(k, n - k) \sqrt{\frac{MS_W}{2} \left(\frac{1}{n_i} + \frac{1}{n_j} \right)}.$$

Tukey found that this critical value $q_{\alpha}(k, n - k)$ comes from what he called a **Studentized range distribution** with k groups and $n - k$ degrees of freedom. We will rely on **R** to perform tests and compute confidence intervals for differences in group means.

Tukey-Kramer Procedure Steps

1. Define W_{ij} to be the margin of error for the difference between means for groups i and

$$j, \text{ computed as: } W_{ij} = q_{\alpha}(k, n - k) \sqrt{\frac{MS_W}{2} \left(\frac{1}{n_i} + \frac{1}{n_j} \right)}, \text{ where } MS_W \text{ is the mean}$$

square within samples. If the sample sizes are the same for each group, this only needs to be computed once.

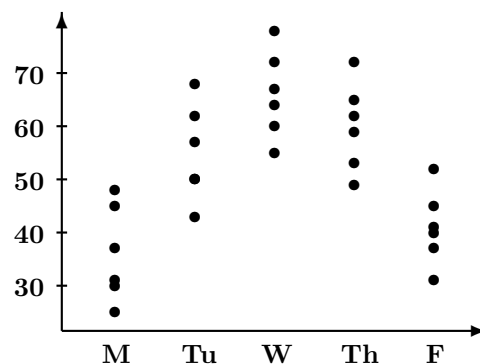
2. Compute the $100(1 - \alpha)\%$ confidence interval for the difference in any pair of treatment means $\mu_i - \mu_j$ as $(\bar{y}_i - \bar{y}_j) \pm W_{ij}$.

Some additional comments on the Tukey-Kramer Procedure:

- The significance level α associated with the Tukey-Kramer procedure is an experimentwise (family) error rate, as opposed to the per-comparison error rate of a t -procedure on a single pairwise difference. This means that the α -level used holds simultaneously for all pairwise comparisons, making it a more valid test.
- Because the error rate is experimentwise, the Tukey-Kramer procedure tends to be more conservative than using individual t -procedures. I consider this a good quality because we avoid saying that two treatment means are different unless they really are, at some α -level for all tests. In other words, the α -level applies simultaneously to all comparisons. Since the α -level applies to all comparisons made, then the probability that **ALL** of the confidence intervals for the pairwise differences include their corresponding treatment mean differences is $1 - \alpha$.

Example: A company conducts an experiment to examine differences between the mean number of visits or “hits” to its web site for the five working days (Monday through Friday). Data on the number of hits between 9:00am and 3:00pm over a period of six weeks were collected, as shown below. Only the hours from 9:00am to 3:00pm were used in the study to avoid within-day differences in web activity. Previous data have demonstrated that between 9:00am and 3:00pm, the number of hits is relatively constant. Do the data within each weekday constitute a random sample from the population of the number of hits within that weekday? Can you suggest a better design?

Populations	Data						$\bar{y}_i$	s_i^2	n_i
Monday	37	48	25	45	30	31	36	81.6	6
Tuesday	62	43	50	68	50	57	55	83.2	6
Wednesday	64	78	55	72	67	60	66	68.4	6
Thursday	49	72	65	53	59	62	60	68.8	6
Friday	52	41	45	31	37	40	41	50.8	6



Assuming the data are approximately normal, the population variances are homogeneous (seems reasonable by inspection), and the samples within each weekday are random and independent (Is this reasonable?), the following ANOVA table was obtained:

Since the p-value for the F-test of any treatment differences is small, we conclude that there are differences in the mean number of hits for the different

Source of Var.	d.f.	SS	MS	F-stat.	p-value
Treatment	4	3871.2	967.80	13.72	< 0.0001
Error	25	1764.0	70.56		
Total	29	5635.2			

days of the week. To compare the population means for the five days, Tukey’s W procedure will be used to investigate all pairwise comparisons. An α -level of 0.05 will be used throughout this example.

Since the sample sizes for all five samples are the same ($n = 6$), then the margin of error for all pairwise comparisons ($\alpha = 0.05$) will be the same. It is:

$$\underline{W} = q_{\alpha}(5, 25) \sqrt{\frac{MS_W}{2} \left(\frac{1}{n} + \frac{1}{n} \right)} = (4.153) \sqrt{\frac{(70.56)}{6}} = \underline{14.24},$$

where the Studentized range critical value is computed in **R** as:

$$q_{\alpha}(5, 25) = \text{qtukey}(.95, 5, 25) = 4.153.$$

The sample means for the web data are given below:

Weekday	Monday	Tuesday	Wednesday	Thursday	Friday
Sample Mean	36	55	66	60	41

Give simultaneous 95% confidence intervals for all treatment mean differences. With W as the margin of error and letting $\mu_1, \dots, \mu_5$ represent the mean number of web hits for Monday through Friday, these confidence intervals are:

$$\begin{aligned} \text{Tue - Mon} &\rightarrow \mu_1 \text{ vs. } \mu_2 : (\bar{y}_2 - \bar{y}_1) \pm W \Rightarrow 19 \pm 14.24 \Rightarrow (4.76, 33.24), \\ \text{Wed - Mon} &\rightarrow \mu_1 \text{ vs. } \mu_3 : (\bar{y}_3 - \bar{y}_1) \pm W \Rightarrow 30 \pm 14.24 \Rightarrow (15.76, 44.24), \\ \text{Thu - Mon} &\rightarrow \mu_1 \text{ vs. } \mu_4 : (\bar{y}_4 - \bar{y}_1) \pm W \Rightarrow 24 \pm 14.24 \Rightarrow (9.76, 38.24), \\ \text{Fri - Mon} &\rightarrow \mu_1 \text{ vs. } \mu_5 : (\bar{y}_5 - \bar{y}_1) \pm W \Rightarrow 5 \pm 14.24 \Rightarrow (-9.24, 19.24), \\ \text{Wed - Tue} &\rightarrow \mu_2 \text{ vs. } \mu_3 : (\bar{y}_3 - \bar{y}_2) \pm W \Rightarrow 11 \pm 14.24 \Rightarrow (-3.24, 25.24), \\ \text{Thu - Tue} &\rightarrow \mu_2 \text{ vs. } \mu_4 : (\bar{y}_4 - \bar{y}_2) \pm W \Rightarrow 5 \pm 14.24 \Rightarrow (-9.24, 19.24), \\ \text{Fri - Tue} &\rightarrow \mu_2 \text{ vs. } \mu_5 : (\bar{y}_5 - \bar{y}_2) \pm W \Rightarrow -14 \pm 14.24 \Rightarrow (-28.24, 0.24), \\ \text{Thu - Wed} &\rightarrow \mu_3 \text{ vs. } \mu_4 : (\bar{y}_4 - \bar{y}_3) \pm W \Rightarrow -6 \pm 14.24 \Rightarrow (-20.24, 8.24), \\ \text{Fri - Wed} &\rightarrow \mu_3 \text{ vs. } \mu_5 : (\bar{y}_5 - \bar{y}_3) \pm W \Rightarrow -25 \pm 14.24 \Rightarrow (-39.24, -9.76), \\ \text{Fri - Thu} &\rightarrow \mu_4 \text{ vs. } \mu_5 : (\bar{y}_5 - \bar{y}_4) \pm W \Rightarrow -19 \pm 14.24 \Rightarrow (-33.24, -4.76). \end{aligned}$$

- To run these procedures in **R** on the web hit data, you can use the following code.

```
# Define the web hits data
hits <- c(37, 48, 25, 45, 30, 31, 62, 43, 50, 68, 50, 57, 64, 78, 55, 72, 67,
         60, 49, 72, 65, 53, 59, 62, 52, 41, 45, 31, 37, 40)
day <- c(rep("Mon",6), rep("Tue",6), rep("Wed",6), rep("Thu",6), rep("Fri",6))
# Converts "day" to a factor with the levels in order of day of week
day <- factor(day, levels = c("Mon", "Tue", "Wed", "Thu", "Fri"))
summary(aov(hits ~ day)) # summary of ANOVA of hits on day

      Df Sum Sq Mean Sq F value    Pr(>F)
day      4   3871    967.8    13.72 4.75e-06 ***
Residuals 25   1764     70.6
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
TukeyHSD(aov(hits ~ day))
```

```
# Tukey's HSD conf.intervals
```

```
Tukey multiple comparisons of means
95% family-wise confidence level
```

```
Fit: aov(formula = hits ~ day)
```

```
$day
```

	diff	lwr	upr	p adj
Tue-Mon	19	4.756932	33.2430684	0.0050536
Wed-Mon	30	15.756932	44.2430684	0.0000168
Thu-Mon	24	9.756932	38.2430684	0.0003781
Fri-Mon	5	-9.243068	19.2430684	0.8386545
Wed-Tue	11	-3.243068	25.2430684	0.1885900
Thu-Tue	5	-9.243068	19.2430684	0.8386545
Fri-Tue	-14	-28.243068	0.2430684	0.0557029
Thu-Wed	-6	-20.243068	8.2430684	0.7302778
Fri-Wed	-25	-39.243068	-10.7569316	0.0002240
Fri-Thu	-19	-33.243068	-4.7569316	0.0050536

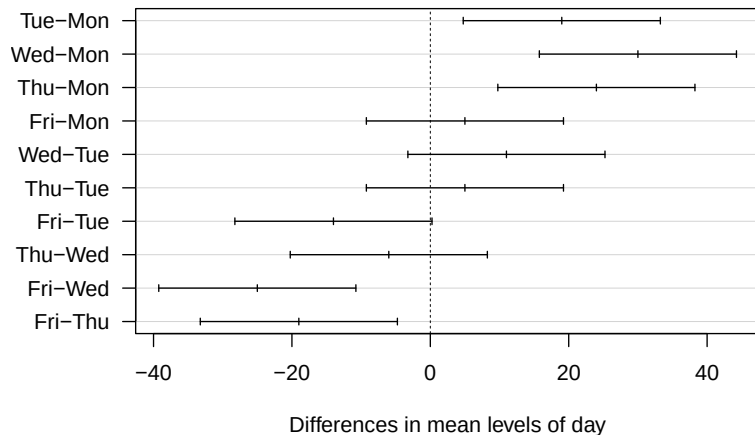
```
par(mai = c(1, 1, 0.5, 0.2))
```

```
# Specifies the 4 outer margins
```

```
plot(TukeyHSD(aov(hits ~ day)), las = 1)
```

```
# Plot of Tukey conf. intervals
```

95% family-wise confidence level



It is worth noting that the Tukey-Kramer adjustments are not the only way to account for multiple comparisons. There is also the **Bonferroni** (the simplest and most conservative one that you will explore on your homework), the **Scheffé** (used for other types of comparisons besides just pairwise), the **Holm-Bonferroni** (used for sequential tests and is less conservative), the **Dunnett** (used for comparing to a control group), and more.

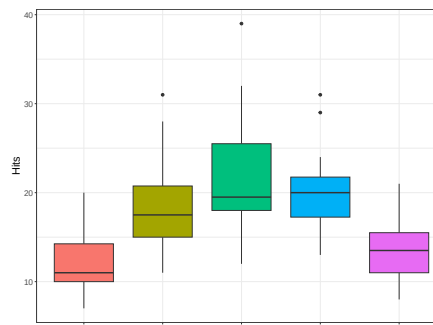
Two-Way ANOVA

When dealing with a one-way ANOVA, we only consider two variables: the response variable is quantitative (e.g. time it takes to help, corn yield, web hits) and the explanatory variable is categorical (e.g. number of people present, type of corn, day of the week). If we want to add another categorical response variable, then we cannot perform a one-way ANOVA. We can, however, deal with that by performing a two-way ANOVA.

The setup for the two-way ANOVA is an extension of a one-way ANOVA. Take, for example, the web hit data again. In addition to breaking it up by day, we will also break it up by region of the country.

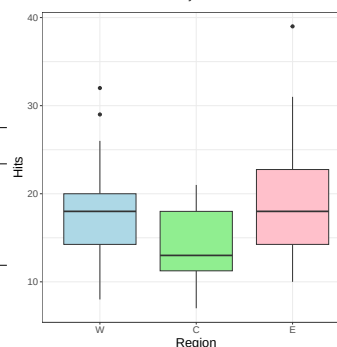
Breaking up by day:

Populations	Data						$\bar{y}_i$	s_i^2	n_i
Monday	37	48	25	45	30	31	36	81.6	6
Tuesday	62	43	50	68	50	57	55	83.2	6
Wednesday	64	78	55	72	67	60	66	68.4	6
Thursday	49	72	65	53	59	62	60	68.8	6
Friday	52	41	45	31	37	40	41	50.8	6



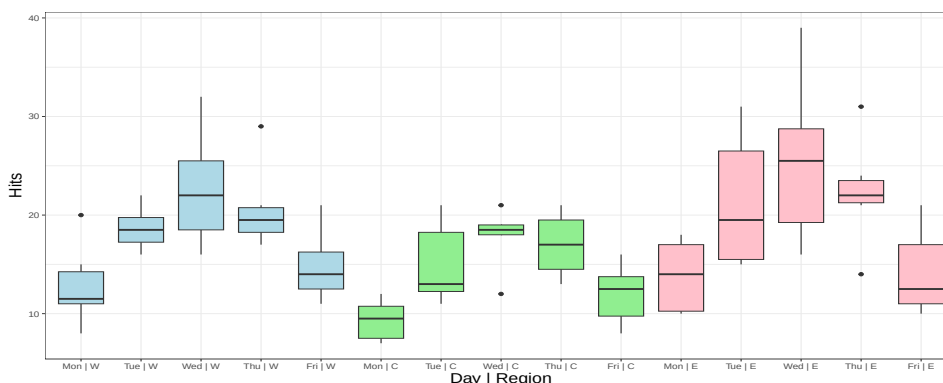
Breaking up by region:

Populations	Data						$\bar{y}_i$	s_i^2	n_i
West	87	111	80	89	83	88	89.7	120.7	6
Central	74	78	65	74	60	76	71.2	49.8	6
East	103	93	95	106	100	86	97.2	53.4	6



Breaking up by day and region:

Populations	West	Central	East
Monday	11, 20, 8, 15, 12, 11	9, 11, 7, 12, 7, 10	17, 17, 10, 18, 11, 10
Tuesday	18, 16, 17, 19, 20, 22	13, 12, 11, 21, 13, 20	31, 15, 22, 28, 17, 15
Wednesday	20, 32, 18, 24, 16, 26	21, 18, 19, 19, 12, 18	23, 28, 18, 29, 39, 16
Thursday	17, 29, 20, 19, 21, 18	18, 21, 14, 13, 16, 20	14, 22, 31, 21, 22, 24
Friday	21, 14, 17, 12, 14, 11	13, 16, 14, 9, 12, 8	18, 11, 14, 10, 11, 21



Two-Way ANOVA Hypotheses

Now we can test three hypotheses at once:

1. The population means when grouping by the first explanatory variable are equal. This is like the one-way ANOVA for the row variable.
2. The population means when grouping by the second explanatory variable are equal. This is like the one-way ANOVA for the column variable.
3. There is no interaction between the two factors. This is similar to performing a test for independence with contingency tables.

In our case, we have the hypotheses

1. H_0 : Day of the week has no effect on the mean number of web hits.
 H_a : Day of the week has an effect on the mean number of web hits.
2. H_0 : Region of the country has no effect on the mean the number of web hits.
 H_a : Region of the country has an effect on the mean the number of web hits.
3. H_0 : There is no interaction effect between day of the week and region of the country on the mean number of web hits.
 H_a : There is an interaction effect between day of the week and region of the country on the mean number of web hits.

Two-Way ANOVA conditions

1. Populations must be normal (or approximately normal).
2. The variances of the populations must be equal (or not too different).
3. The samples must be independent.
4. The groups must all have the same sample size.

Two-way ANOVA in R

```
hits <- c(11,20,8,15,12,11,    9,11,7,12,7,10,    17,17,10,18,11,10,
         18,16,17,19,20,22,    13,12,11,21,13,20,    31,15,22,28,17,15,
         20,32,18,24,16,26,    21,18,19,19,12,18,    23,28,18,29,39,16,
         17,29,20,19,21,18,    18,21,14,13,16,20,    14,22,31,21,22,24,
         21,14,17,12,14,11,    13,16,14,9,12,8,     18,11,14,10,11,21)
day <- c(rep("Mon", 18), rep("Tue", 18), rep("Wed", 18),
        rep("Thu", 18), rep("Fri", 18))
day <- factor(day, levels = c("Mon", "Tue", "Wed", "Thu", "Fri"))
region <- rep(c(rep("W", 6), rep("C", 6), rep("E", 6)), 5)
region <- factor(region, levels = c("W", "C", "E"))
anova(aov(hits ~ day * region))
```

Output is on the next page.

Analysis of Variance Table

Response: hits

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
day	4	1290.40	322.60	14.9567	4.951e-09	***
region	2	429.80	214.90	9.9634	0.0001454	***
day:region	8	54.53	6.82	0.3160	0.9576602	
Residuals	75	1617.67	21.57			

Removing the interaction term:

```
anova(aov(hits ~ day + region))
```

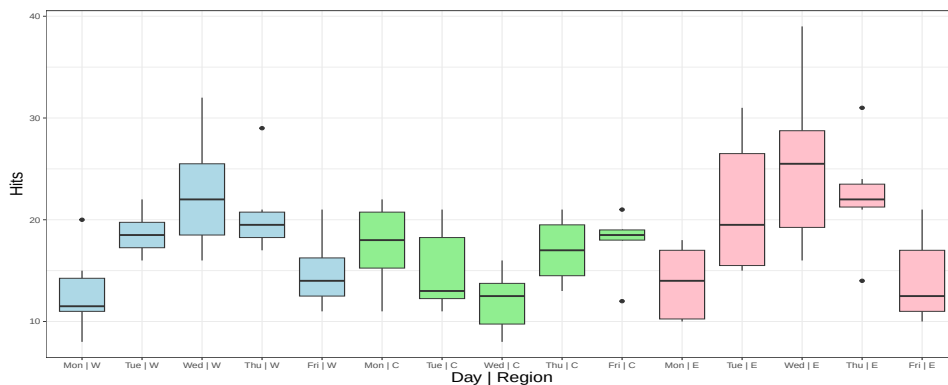
Analysis of Variance Table

Response: hits

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
day	4	1290.4	322.60	16.012	9.387e-10	***
region	2	429.8	214.90	10.667	7.537e-05	***
Residuals	83	1672.2	20.15			

Interaction term

The interaction term in a model is essential measuring not only if the explanatory variables have an effect on the response variable, but also if that effect is different for different levels of the variables. An example of what this could look like is given in the following boxplot: ->



Analysis of Variance Table

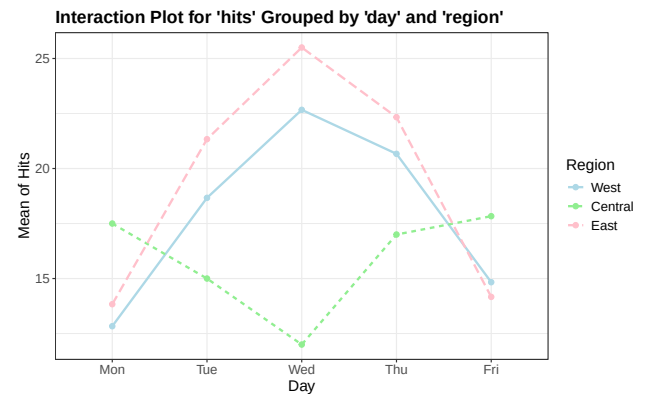
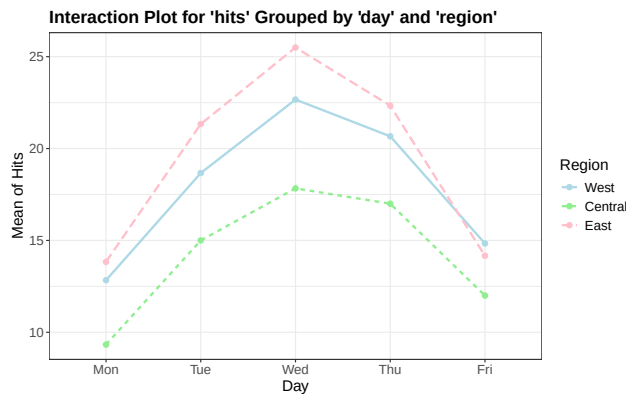
Response: hits

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
day	4	440.29	110.072	4.8969	0.0014581	**
region	2	192.42	96.211	4.2803	0.0173679	*
day:region	8	744.58	93.072	4.1406	0.0003899	***
Residuals	75	1685.83	22.478			

We can visually inspect for an interaction between variables using an **interaction plot**. In the `math3150package`, there is an `interaction_plot()` function.

- If all curves on the plot have approximately the same shape, there is no interaction.
- If the curves have different shapes and/or there are some intersections, then that is evidence of an interaction.

```
interaction_plot(hits_df, hits, day, region)
```



Math of Two-Way ANOVA

With a two-way ANOVA, we have more than one SSB because we have more than one variable. We will call the sums of squares SSB_1 , SSB_2 , SSI (for interaction), and SSW (also called SSE or sum of squares of residuals). We will say variable 1 has k categories, variable 2 has ℓ categories and then the interaction has $k \cdot \ell$ categories. We can compute SSB_1 by ignoring the second variable, SSB_2 by ignoring the first variable, and SSI by breaking the data set up into more categories for each category of the 1st and 2nd.

$$SSB_1 = \sum_{i=1}^k n_i (\bar{y}_i - \bar{\bar{y}})^2 \text{ where } \bar{y}_i \text{ is the mean of the } i\text{th category of variable 1.}$$

$$SSB_2 = \sum_{j=1}^{\ell} n_j (\bar{y}_j - \bar{\bar{y}})^2 \text{ where } \bar{y}_j \text{ is the mean of the } j\text{th category of variable 2.}$$

$$SSI = \left(\sum_{i=1}^k \sum_{j=1}^{\ell} n_{ij} (\bar{y}_{ij} - \bar{\bar{y}})^2 \right) - SSB_1 - SSB_2 \text{ where } \bar{y}_{ij} \text{ is the mean of the intersection between the } i\text{th category of variable 1 and the } j\text{th category of variable 2.}$$

$$SSE = \sum_{i=1}^n (y_i - \bar{\bar{y}})^2 - SSB_1 - SSB_2 - SSI = \sum_{i=1}^k \sum_{j=1}^{\ell} (n_{ij} - 1) s_{ij}^2$$

Then the ANOVA table will look like:

Source of Variation	df	Sum of Squares	Mean Square	F Statistic
Between Variable 1	$k - 1$	SSB_1	$MSB_1 = \frac{SSB_1}{k-1}$	$F_1 = \frac{MSB_1}{MSE}$
Between Variable 2	$\ell - 1$	SSB_2	$MSB_2 = \frac{SSB_2}{\ell-1}$	$F_2 = \frac{MSB_2}{MSE}$
Interaction	$(k - 1)(\ell - 1)$	SSI	$MSI = \frac{SSI}{(k-1)(\ell-1)}$	$F_3 = \frac{MSI}{MSE}$
Error (Within)	$n - (k \cdot \ell)$	SSE	$MSE = \frac{SSE}{n-k \cdot \ell}$	
Total	$n - 1$	SST		

where k is the number of groups for explanatory variable 1, ℓ is the number of groups for explanatory variable 2, and n is the overall sample size.

Example: In our example with the hits (where the interaction was not significant):

```
hitsdf <- data.frame(hits = hits, day = day, region = region,
                    interaction = day:region) # Put data into a data frame
ybarbar <- mean(hitsdf$hits) # Compute the grand mean

# Find SSB1: the sum of squares between categories for the days
day_means <- hitsdf |> # Find the mean hits per day
  group_by(day) |>
  summarize(means = mean(hits))
SSB1 <- sum(18*(day_means[,2] - ybarbar)^2) # Each day has 18 observations

# Find SSB2: the sum of squares between categories for the regions
```

```

region_means <- hitsdf |> # Find the mean hits per region
  group_by(region) |>
  summarize(means = mean(hits))
SSB2 <- sum(30*(region_means[,2] - ybarbar)^2) # Each region has 30 obs

# Find SSI: the sum of squares for the interaction
intera_means <- hitsdf |> # Find the mean hits per interaction
  group_by(interaction) |>
  summarize(means = mean(hits))
# Each interaction has 6 observations
SSI <- sum(6*(intera_means[,2] - ybarbar)^2) - SSB1 - SSB2

# Find SSE: the sum of squares due to error.
# Also known as SSW or sum of squares of residuals.
# First way
SSE <- sum((hitsdf$hits - ybarbar)^2) - SSB1 - SSB2 - SSI
# Second way
intera_variances <- hitsdf |> # Find the variance of hits per interaction
  group_by(interaction) |>
  summarize(vars = var(hits))
SSE <- sum((6-1)* intera_variances[,2])

# Compare output
anova(aov(hits ~ day * region))

```

Analysis of Variance Table

Response: hits

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
day	4	1290.40	322.60	14.9567	4.951e-09 ***
region	2	429.80	214.90	9.9634	0.0001454 ***
day:region	8	54.53	6.82	0.3160	0.9576602
Residuals	75	1617.67	21.57		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

SS <- data.frame(SSB1 = SSB1, SSB2 = SSB2, SSI = SSI, SSE = SSE)
SS

```

	SSB1	SSB2	SSI	SSE
1	1290.4	429.8	54.53333	1617.667

The numbers match!

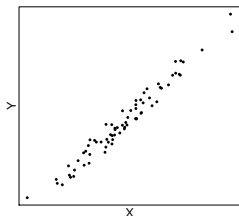
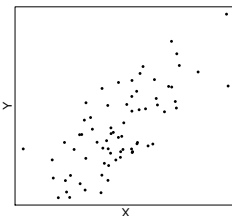
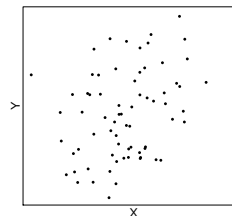
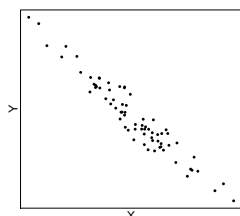
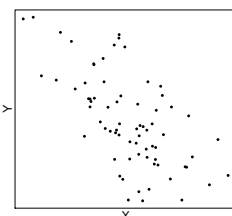
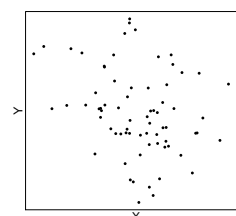
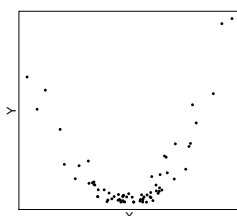
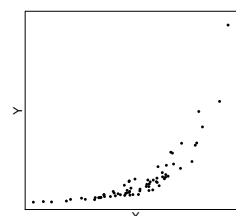
Notes 5: Correlation and Simple Linear Regression

Table of contents

Correlation	77
Correlation interpretation	77
Simple Linear Regression	78
The Simple Linear Regression Model	79
Regression Model Assumptions	79
Least Squares Method	81
Interpreting the Slope and Intercept and Other Important Points	84
Assessing Regression Fit	85
Lack of Fit	88
Visual Methods of Assessing Lack of Fit	88
Types of Residuals	88
Residual Plots	89
QQ Plots	90
Numerical Methods of Examining Lack of Fit in Regression	91

Correlation

Correlation describes the *linear* relationship between two quantitative variables. There could be an association between variables that is not linear, and we would say there is not a strong correlation.

Strong Positive ($r = 0.982$)Moderate Positive ($r = 0.724$)Weak Positive ($r = 0.343$)Strong Negative
($r = -0.963$)Moderate Negative
($r = -0.639$)Weak Negative ($r = -0.267$)Strong Quadratic ($r = 0.209$)Strong Exponential ($r = 0.770$)

Simply looking at a scatterplot to obtain correlation can be subjective, however. There is an objective way to measure correlation by using the **correlation coefficient**. This is also known as the Pearson correlation coefficient.

$$r = \frac{1}{n-1} \sum_{i=1}^n \left[\left(\frac{x_i - \bar{x}}{s_x} \right) \cdot \left(\frac{y_i - \bar{y}}{s_y} \right) \right]$$

Correlation interpretation

A very general rule of thumb for interpreting the correlation coefficient is:

If $|r| = 1$, then the correlation is “perfect”.

If $0.9 \leq |r| < 1$, then the correlation is “strong”.

If $0.6 \leq |r| < 0.9$, then the correlation is “moderate”.

If $0 < |r| < 0.6$, then the correlation is “weak”.

If $|r| = 0$, then there is no correlation.

There are other correlation coefficients, like the Kendall and Spearman correlation. In **R**, you can specify the method in the `cor()` function, but we will stick with the Pearson correlation coefficient in this class since it is widely used and the results from all are usually similar.

Simple Linear Regression

Often, we have two variables and believe that one affects or *explains* the other (e.g.: smoking & lung cancer, height & weight, life expectancy & income, etc.) These two variables are referred to as the:

1. **Response (Dependent) Variable**: the variable of primary interest in a study (denoted by Y). It is what you are trying to explain or predict.
 - Examples?
2. **Explanatory (Independent) Variable**: the variable that attempts to explain the observed outcomes of the response variable (denoted by X).
 - Examples?

What is linear regression? Linear regression is a method of developing a linear statistical model for explaining the outcomes of some continuous response variable with one (simple linear regression) or more (multiple linear regression) explanatory (or predictor) variables. What is a statistical model?

Two Types of Models

1. **Deterministic**: models with no error (i.e.: we can solve for the value of one variable given the value of another).
 - A deterministic linear model has form: $y = \beta_0 + \beta_1 x$ where β_0 = the y -intercept and β_1 = the slope of the line. What exactly is linear here?
 - Examples?
2. **Statistical (Stochastic)**: models which incorporate error.
 - In the simple linear regression case, the model represents our belief that the relationship between X and Y is close to linear, just as the normal model was appropriate only when the distribution was close to normal.
 - The simple linear model has 2 parameters, the intercept and slope.
 - In general, parameters are values which specify a particular model and give a model the flexibility to be used for different kinds of data.

Purposes of Regression:

1. Prediction of y (a value of Y) given x (some value of X).
2. To quantify the nature and magnitude of relationships between multiple variables.

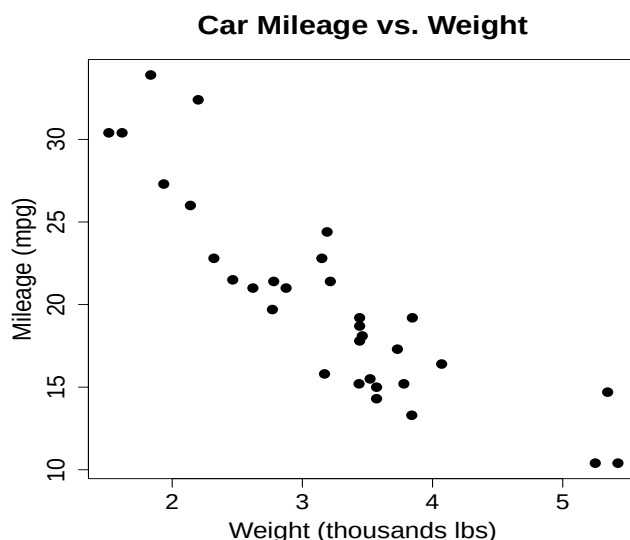
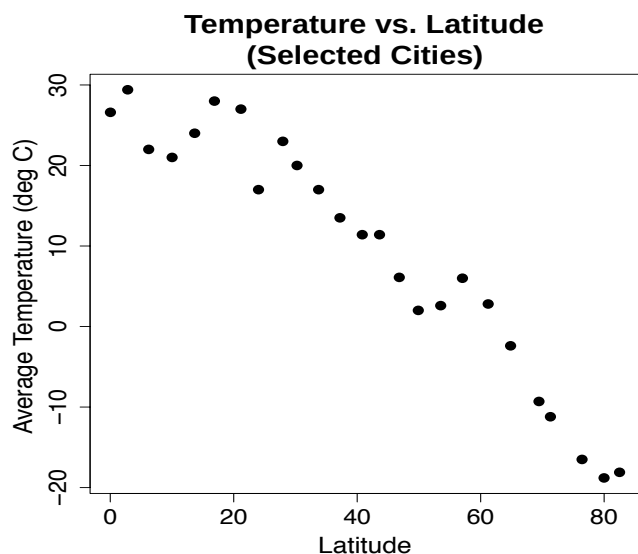
The Simple Linear Regression Model

The simple linear regression model is a model for predicting a **response** variable (y) from an **explanatory** variable (x).

- Any statistical model for predicting a response variable uses the following notation:

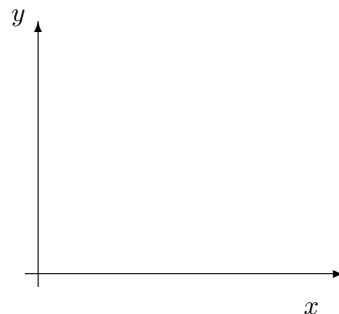
$$\begin{aligned} y_i &= \text{the } i^{\text{th}} \text{ observed or actual value of the response variable,} \\ \hat{y}_i &= \text{the } i^{\text{th}} \text{ predicted or fitted value of the response variable} \\ &\quad \text{(pronounced “y-hat”),} \\ e_i = y_i - \hat{y}_i &= i^{\text{th}} \text{ observed value} - i^{\text{th}} \text{ predicted value} = i^{\text{th}} \text{ residual or error.} \end{aligned}$$

- The linear model is: $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ OR $E(y_i) = \beta_0 + \beta_1 x_i$, where β_0 is the **intercept** of the line and β_1 is the **slope**.
- The notation $E(\bullet)$ or $E[\bullet]$ is called the **expected value** or *mean* function.
- The intercept represents the predicted value of Y ($\hat{y}$) when $x = 0$.
- The slope represents how much the predicted value of Y changes when X increases by 1 unit.



Regression Model Assumptions

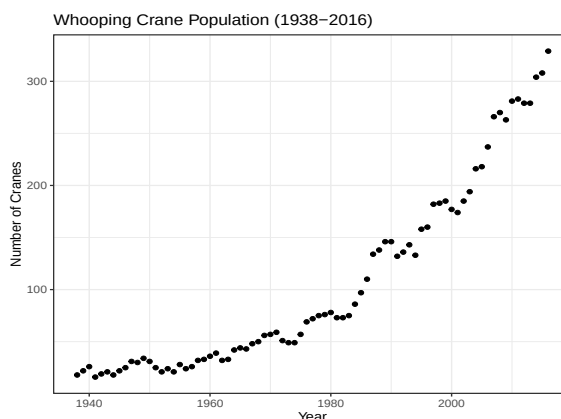
- $E(\varepsilon|x_i) = 0$ (same idea as saying the relationship between Y and X is linear).
- The errors have the same variance across all values of X .
 $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$. What if they don't?
- The errors are normal ($\varepsilon \sim N$).
- The errors are independent of one another.
What if they're not?



- Considering the first assumption above, the simple linear regression model can also be written in the form: $E(y_i|x_i) = \beta_0 + \beta_1 x_i, i = 1, 2, \dots, n$.
- When we “fit” a regression line to some data, we find a **prediction** (or **fitted**, or **regression**) **equation**, given by: $\hat{y}_i = b_0 + b_1 x_i$. This prediction line estimates the theoretical regression line $E(y_i|x_i) = \beta_0 + \beta_1 x_i$.
- The hats on the β -parameters indicate that they are *estimated* from the data.
- The Big Question: How do we decide exactly which line gives the “best” model for a set of data? What do we mean by “best”?

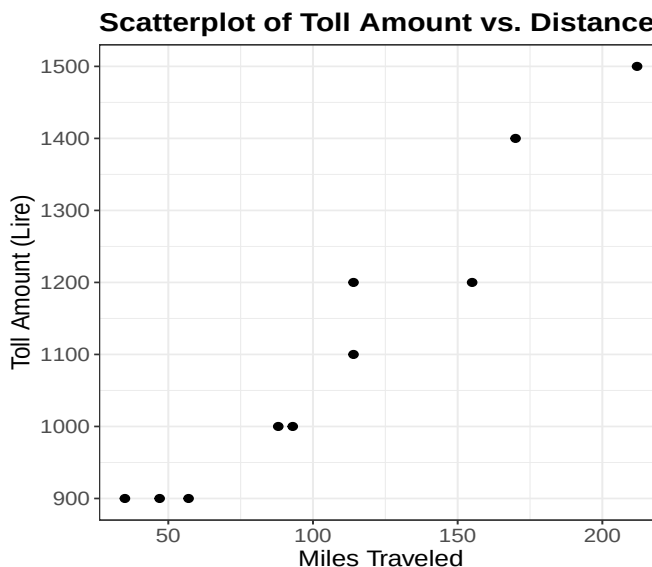
Nonlinear Relationships: Consider the following time plot of the estimated number of whooping cranes wintering at Aransas National Wildlife Refuge from 1938 through 2016 (shown below). Suppose we wish to model population size as a function of year in order to project future population sizes.

- We will return to this later, but fitting a linear model in this circumstance would be inappropriate.



Example: Suppose in traveling through Italy, data were collected from 10 trips on the Italian Autostrada toll roads as shown in the scatterplot below.

- Suppose we have 175 miles until the Austrian border and only 1400 Lire. Will we make it out of Italy without stopping for more money?
- What are X and Y here?



Least Squares Method

The most common method for finding a “best-fitting” line is **least squares**. The least squares line is the line which minimizes the sum of the squared residuals; that is, it minimizes:

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

- Graphically, the least squares line is that line which minimizes the sum of the squared vertical deviations from the line.
- It turns out that finding the least squares line is easy (that is a relative term) to do analytically using calculus. The resulting estimators for β_0 and β_1 are:

$$b_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = r \frac{s_y}{s_x} \text{ and } b_0 = \bar{y} - b_1 \bar{x}$$

Aside: (You don’t need to know all this below.)

Our goal is to minimize $Q = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$. To find the minimum of an expression, take derivative and set equal to zero. However, there are two unknowns (β_0 and β_1), so we must do that for each.

Start in this column.

Using the chain rule

$$\frac{dQ}{d\beta_0} = -2 \sum (y_i - \beta_0 - \beta_1 x_i) \stackrel{\text{set}}{=} 0$$

Solve for β_0 :

$$-2 \sum (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\sum (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\sum y_i - \sum \beta_0 - \sum \beta_1 x_i = 0$$

$$\sum y_i - \beta_0 \sum 1 - \beta_1 \sum x_i = 0$$

Since $\sum 1 = n$, we can write

$$\sum y_i - \beta_0 \cdot n - \beta_1 \sum x_i = 0$$

$$-\beta_0 \cdot n = -\sum y_i + \beta_1 \sum x_i$$

$$\beta_0 \cdot n = \sum y_i - \beta_1 \sum x_i$$

$$\beta_0 = \frac{\sum y_i}{n} - \frac{\beta_1 \sum x_i}{n}$$

$$\hat{\beta}_0 = b_0 = \bar{y} - \beta_1 \bar{x}$$

We need to estimate β_1 with $\hat{\beta}_1 = b_1$:

This gives us: $b_0 = \bar{y} - b_1 \bar{x}$

Now we need to know what b_1 is.

Using the chain rule

$$\frac{dQ}{d\beta_1} = -2 \sum x_i (y_i - \beta_0 - \beta_1 x_i) \stackrel{\text{set}}{=} 0$$

Solve for β_1 :

$$-2 \sum (x_i y_i - \beta_0 x_i - \beta_1 x_i^2) = 0$$

$$\sum (x_i y_i - \beta_0 x_i - \beta_1 x_i^2) = 0$$

$$\sum x_i y_i - \sum \beta_0 x_i - \sum \beta_1 x_i^2 = 0$$

$$\sum x_i y_i - \beta_0 \sum x_i - \beta_1 \sum x_i^2 = 0$$

Our best guess for β_0 is $\bar{y} - \beta_1 \bar{x}$, so plug that in:

$$\sum x_i y_i - (\bar{y} - \beta_1 \bar{x}) \sum x_i - \beta_1 \sum x_i^2 = 0$$

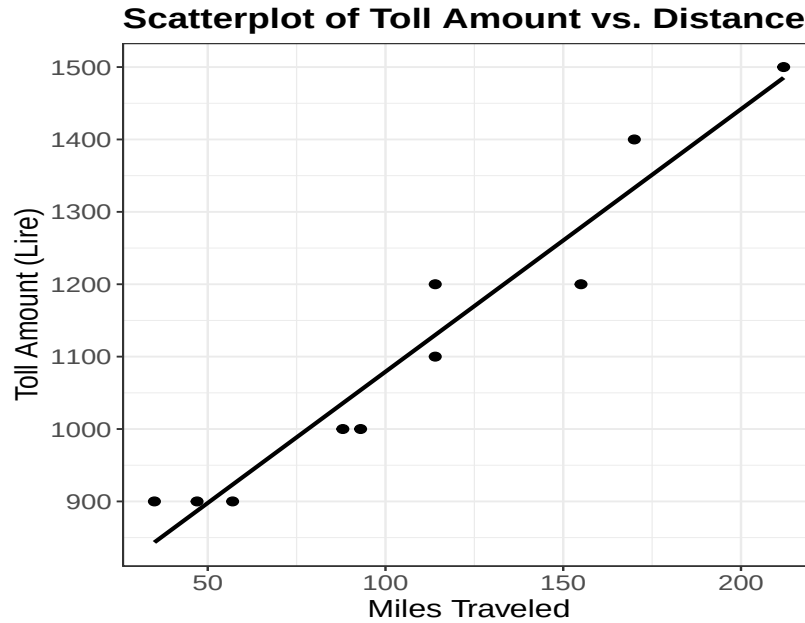
$$\sum x_i y_i - \bar{y} \sum x_i + \beta_1 \bar{x} \sum x_i - \beta_1 \sum x_i^2 = 0$$

$$\sum x_i y_i - \bar{y} \sum x_i + \beta_1 (\bar{x} \sum x_i - \sum x_i^2) = 0$$

$$\beta_1 (\bar{x} \sum x_i - \sum x_i^2) = -\sum x_i y_i + \bar{y} \sum x_i$$

$$\beta_1 (\bar{x} \sum x_i - \sum x_i^2) = -\sum x_i y_i + \bar{y} \sum x_i$$

$$\hat{\beta}_1 = b_1 = \frac{-\sum x_i y_i + \bar{y} \sum x_i}{\bar{x} \sum x_i - \sum x_i^2} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$



Back to the Toll Road Example: To find the regression line $\hat{y} = b_0 + b_1x$, we can compute the necessary summary statistics (shown on the next page) or use **R**, as given below.

```
toll <- 100*c(10,15,12,9,11,9,10,9,14,12) # Vector of toll amounts
dist <- c(93,212,155,57,114,35,88,47,170,114) # Vector of distances traveled
toll_df <- data.frame(toll, dist) # Create a data frame with those vectors
reg_out <- lm(toll ~ dist, data = toll_df) # LS fit of toll amount on distance
summary(reg_out) # Regression summary
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 716.5113    39.4539   18.16 8.68e-08 ***
dist         3.6266     0.3255   11.14 3.76e-06 ***
```

Computing using summary statistics is illustrated using the table below.

Trip	x_i	y_i	x_i^2	y_i^2	$x_i y_i$	$\hat{y}_i$	e_i
1	93	1000	8649	1000000	93000	1053.79	-53.79
2	212	1500	44944	2250000	318000	1485.36	14.64
3	155	1200	24025	1440000	186000	1278.64	-78.64
4	57	900	3249	810000	51300	923.23	-23.23
5	114	1100	12996	1210000	125400	1129.95	-29.95
6	35	900	1225	810000	31500	843.44	56.56
7	88	1000	7744	1000000	88000	1035.65	-35.65
8	47	900	2209	810000	42300	886.96	13.04
9	170	1400	28900	1960000	238000	1333.04	66.96
10	114	1200	12996	1440000	136800	1129.95	70.05
Totals	1085	11100	146937	12730000	1310300	11100	0
	$\sum x_i$	$\sum y_i$	$\sum x_i^2$	$\sum y_i^2$	$\sum x_i y_i$	$\sum \hat{y}_i$	$\sum e_i$

Note:

- The estimated residuals sum to zero. This is always true because the method of least squares always finds that line among those for which the average of the residuals (and hence the sum) is zero which minimizes the sum of the squared residuals.
- The sum of the observed toll amounts (y_i) is equal to the sum of the predicted toll amounts ($\hat{y}_i$). Again, this will always be true.

With these values then, we can compute the slope and intercept terms.

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{\sum (x_i y_i) - n\bar{x}\bar{y}}{\sum (x_i^2) - n\bar{x}^2} = r \frac{s_y}{s_x} \quad \text{and} \quad b_0 = \bar{y} - b_1 \bar{x}$$

$$\begin{aligned} \bar{x} &= \frac{1}{n} \sum x_i = \frac{1}{10}(1085) = \underline{108.5}, \quad \bar{y} = \frac{1}{n} \sum y_i = \frac{1}{10}(11100) = \underline{1110}. \\ \sum (x_i - \bar{x})^2 &= \sum x_i^2 - n\bar{x}^2 = 146937 - 10(108.5)^2 = 146937 - 117722.5 = \underline{29214.5}, \\ \sum (x_i - \bar{x})(y_i - \bar{y}) &= \sum x_i y_i - n\bar{x}\bar{y} = 1310300 - 10(108.5)(1110) \\ &= 1310300 - 1204350 = \underline{105950}, \end{aligned}$$

This gives the least squares estimates of β_0, β_1 as:

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{105950}{29214.5} = \underline{3.627}, \quad \& \quad b_0 = \bar{y} - b_1 \bar{x} = 1110 - (3.627)(108.5) = \underline{716.51},$$

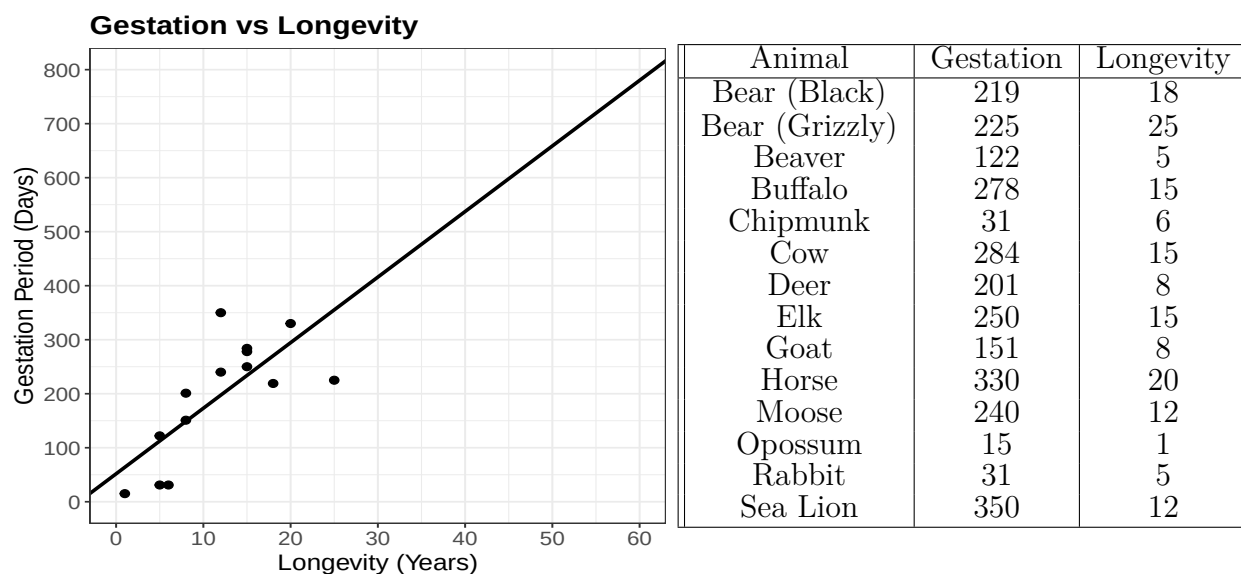
This agrees with what was found using the `lm()` function in **R** on the previous page. Hence, the predicted regression line is given by: $\hat{y} = 716.51 + 3.627x$. The resulting “fitted” or predicted values $\hat{y}_i$ are also given in the table above, as well as the estimated residuals e_i .

Will we make it out of Italy?

Interpreting the Slope and Intercept and Other Important Points

1. The slope coefficient of 3.627 Lire/mile means that the predicted toll amount increases by 3.627 Lire for every additional mile driven. Equivalently, the predicted toll amount increases by 36.27 Lire for every additional 10 miles.
2. The intercept of 716.51 Lire represents the predicted toll amount for a distance traveled of 0 miles. You might think of this as the “*entrance fee*” to get on the Autostrada. The intercept doesn’t have a meaningful interpretation in most problems.

Example 2: Gestation vs. Longevity for Mammals: Consider the data below on the average longevity (in years) and gestation period (in days) for a sample of 14 mammals, as reported in the World Almanac.



- Since the underlying relationship between gestation and longevity is fairly linear, it seems reasonable to model the relationship with a simple linear regression model. Letting y = gestation and x = longevity, the resulting least squares regression line (plotted above) is given by: $\hat{y} = 51.72 + 12.14x$.
- Which values does the regression line have trouble predicting?
What does a positive residual mean? What does a negative residual mean?
- What is the predicted gestation period for a cat (which has a longevity of 12 years)?

How well did we do? The actual gestation period for a cat is 63 days. Why is our prediction so poor?

- Because the sea lion value falls somewhat far from the regression line, it might be considered an **outlier**. Suppose we drop the sea lion value from these data and refit the regression line. The resulting line is now given by: $\hat{y} = 40.69 + 12.08x$. Removal of this value did not affect the slope of the line very much, but decreased the y-intercept by roughly 11. In general, the removal of an outlier (in its y -value) tends to raise or lower the regression line vertically, without affecting the slope a great deal.
- What is the predicted gestation period for a blue whale (assume an average longevity of 100 years)? According to the regression equation:

$$\hat{y} = 51.72 + 12.14(100) = 1265.72 \text{ days (or roughly 3.5 years!)}$$

The actual gestation period for a blue whale is about 11 months. Why is this prediction so poor? One reason is that we are trying to predict outside the range of our data - called **extrapolation**.

- The regression line is only as good as the population it represents and its range of explanatory values. In this case, the line describes the relationship between gestation and longevity, but only for longevity between 1 and 25 years.
- One might extrapolate a few years beyond the range of longevity values (i.e.: to perhaps 30 years), but extrapolating too far is a very dangerous and unreliable practice.
- The gestation period of a blue whale is actually about 330 days. Plot this point on the graph. What effect do you think this point will have on the regression line?

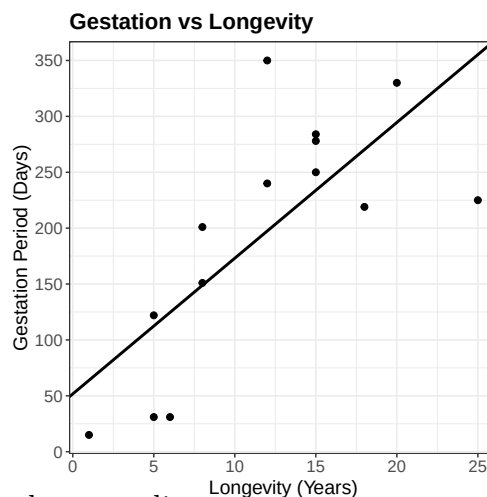
Because such a point greatly affects the *slope* of the regression line, it is said to be an **influential point**.

- The Asian elephant has an average longevity of 40 years and a gestation period of 645 days. Is this point *influential*? Any point which differs greatly in its explanatory (x) variable value from the bulk of the data is known as a **high leverage** point. In this case, the elephant is certainly a leverage point, but is not influential because it agrees with the general pattern in the data.

Assessing Regression Fit

Consider again the gestation-longevity data:

- Recall for these data, the regression line was: $\hat{y} = 51.72 + 12.14x$, and $\bar{y} = 194.79$.
- Note that for any point on this plot (I chose the point (20,330) corresponding to the horse for illustrative purposes), we can partition the vertical distance from the point to the mean line into two pieces:
 - the distance from the point to the prediction line, and
 - the distance from the prediction line to the mean line.



- Mathematically, for each data value y_i , we obtain the following relationship:

$$y_i - \bar{y} = (y_i - \hat{y}_i) + (\hat{y}_i - \bar{y}).$$

- And with a considerable amount of algebra, it can also be shown that:

$$\begin{aligned} \underbrace{\sum_{i=1}^n (y_i - \bar{y})^2}_{\substack{\text{total variability} \\ \text{in y-values} \\ \text{(from the mean)}}} &= \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\substack{\text{unexplained} \\ \text{variability} \\ \text{(error)}}} + \underbrace{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}_{\substack{\text{variability} \\ \text{explained by} \\ \text{regression}}} \\ \Leftrightarrow \text{Sum of Squares About the Mean (Total SS)} &= \text{Sum of Squares for Error} + \text{Sum of Squares Due to Regression} \\ \Leftrightarrow \boxed{\text{SST} = \text{SSE} + \text{SSR}}. \end{aligned}$$

- What does this mean? This shows precisely how the regression line partitions the total variability in the y-values (SST) into that part which is explained by the regression (SSR) and that part which remains unexplained (SSE).
- Hence, one goal of least squares regression is to find a set of explanatory variables which make the SSE (unexplained error) as small as possible, while keeping the model as simple as possible.
- Sums of squares from linear models are often organized into analysis of variance (ANOVA) tables to examine the relative contributions of the sources of variation in a model. To see this in this gestation-longevity case in **R**, we can type:

```
gestat <- c(219,225,122,278,31,284,201,250,151,330,240,15,31,350)
longev <- c(18,25,5,15,6,15,8,15,8,20,12,1,5,12)
gest_mod <- lm(gestat ~ longev)      # LS fit of gestation vs. longevity
anova(gest_mod)                     # Constructs an ANOVA table of the fit
```

Analysis of Variance Table

Response: gestat

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
longev	1	85816	85816	14.451	0.002523 **
Residuals	12	71262	5938		

Computing SS's for the Example:

$$\begin{aligned} \text{SST} &= \sum (y_i - \bar{y})^2 = \sum y_i^2 - n\bar{y}^2 \quad (\text{Computing Formula}) \\ &= 688259 - (14)(194.79)^2 = 688259 - 531180.6 = \underline{157078.4}, \\ \text{SSE} &= \sum (y_i - \hat{y}_i)^2 = \sum e_i^2 = (-51.22)^2 + (-130.20)^2 + \cdots + (152.61)^2 \\ &= \underline{71261.9}, \\ \text{SSR} &= \sum (\hat{y}_i - \bar{y})^2 = \sum \hat{y}_i^2 - n\bar{y}^2 \quad (\text{Computing Formula}) \\ &= 616997.1 - (14)(194.79)^2 = 616997.1 - 531180.6 = \underline{85816.4}. \end{aligned}$$

Within rounding error, it is simple to verify that these agree with those reported in the ANOVA table from the previous page (shown to the right) and that $SST = SSE + SSR$.

Source of Variance	d.f.	Sum of Squares
Regression	1	85816
Error	12	71262
Total	13	157078

How can these sums of squares be used to assess regression fit?

- Since SSR measures the variability explained by the linear regression of y on x , and SST measures the total variability in y , then the statistic:

$$R^2 = \frac{SSR}{SST}$$

is the percentage of the variation in y explained by the regression of y on x .

- The R^2 -statistic is sometimes referred to as the **coefficient of determination**.
- In the gestation-longevity example, $R^2 = \frac{85812.00}{157078.36} = 0.546$, which says that 54.6% of the variation in gestation periods is explained by the linear regression of gestation on longevity.
- The statistic R^2 is by far the most common measure of goodness of fit used in linear regression, and is interpreted differently in different fields. As a general guideline for assessing the quality of the regression fit, the following table reflects my interpretation (although really it depends on the type of data you are working with and your field of study):

R^2	Interpretation
≥ 0.80	Good regression fit
$0.60 < R^2 < 0.80$	Moderate regression fit
≤ 0.60	Weak or poor regression fit

- The R^2 -statistic is also reported using the **summary()** command on a regression object in **R**. In the gestation-longevity example, it can be found as shown below.

```
# LS fit of gestation vs. longevity
gest_mod <- lm(gestation ~ longevity, data = gestation)
summary(gest_mod) # Regression summary
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept)  51.716      42.902    1.205  0.25127
longev       12.139       3.193    3.801  0.00252 **
```

Residual standard error: 77.06 on 12 degrees of freedom

Multiple R-squared: 0.5463, Adjusted R-squared: 0.5085

F-statistic: 14.45 on 1 and 12 DF, p-value: 0.002523

- What effect do you think an outlier will have on the value of the R^2 statistic?

Lack of Fit

In order to think about statistical inference in the regression context, we will want to assess whether our model is a good fit and that our key assumptions seem appropriate. In other words, we want to assess any lack of fit.

Recall the assumptions for a linear model are

1. $E(\varepsilon|x_i) = 0$ (the relationship between Y and X is linear).
2. The errors have the same variance across all values of X : $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$.
3. The errors are independent of one another.
4. The errors are normal ($\varepsilon \sim N$).

Visual Methods of Assessing Lack of Fit

1. **Scatterplot** with a regression line plotted.
 - This can be used to check if the relationship between y and x is linear.
2. **Residual Plot**: A residual plot is a plot of the residuals ($e_i = y_i - \hat{y}_i$) vs. the predicted values ($\hat{y}_i$'s), or vs. the explanatory values (x_i 's) as discussed later.
 - This is primarily used to test the homogeneity of variance assumption. Residual plots do NOT address normality of the residuals.
 - Residual plots are also useful for detecting outliers and checking model adequacy. They are generally better visual tools for identifying model lack of fit than scatterplots.
3. **Normal Quantile Plot of Residuals (QQ Plot)**: plot of the residuals in a regression model vs. the normal quantiles.
 - This tests the normality assumption of the errors, where deviations from a linear pattern indicate nonnormality.

Types of Residuals

Since some of these plots involve the residuals, let's discuss different types of residuals.

If the errors are approximately normal: $\varepsilon|x_i \stackrel{\bullet}{\sim} N(0, \sigma_\varepsilon^2)$, then we would expect the unstandardized or raw residuals, $e_i = y_i - \hat{y}_i$, to be within 3 SDs of 0 (by the Empirical Rule).

Problem? σ_ε (the true SD of the residuals at x_i) is estimated by s_e , so we should use:

$$\Rightarrow \frac{e_i}{s_e} \sim \quad \text{These are called } \underline{\text{standardized residuals}}.$$

Problem? s_e uses e_i in its computation, so e_i and s_e are not independent $\Rightarrow e_i/s_e \sim t$.

Solution: Estimate σ_ε by omitting the i^{th} residual $\Rightarrow s_{(e_i)}$

$$\Rightarrow e_i \text{ and } s_{(e_i)} \text{ are independent}$$

$$\Rightarrow \frac{e_i}{s_{(e_i)}} \sim t(n-3).$$

The $e_i/s_{(e_i)}$ are known as the **studentized residuals**.

Problem: $\frac{e_i}{s_{(e_i)}}$ and $\frac{e_j}{s_{(e_j)}}$ are not independent, so we cannot do tests on all of the residuals simultaneously.

Solution: Suppose we fit n separate regression lines, omitting the i^{th} observation for the i^{th} fitted line. If we estimate the i^{th} residual using the i^{th} regression line to get $e_{(i)}$, then the values $\hat{r}_i = e_{(i)}/s_{(e_i)}$ are independent of one another. These “standardized residuals” are known as the **studentized deleted residuals**, and are also known as the **jackknife residuals**.

- Sometimes, residual plots will place “normal confidence bands” on the plot to help identify outliers.

R functions for the residuals. For each of these functions, the input is a linear model.

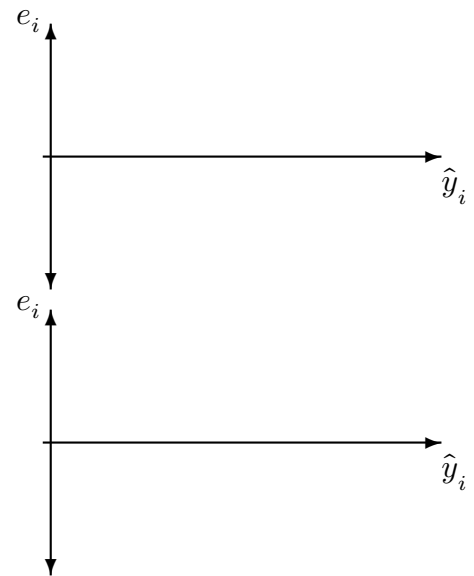
- Raw residuals: **residuals** function.
- Standardized residuals: **rstandard** function.
- Studentized residuals: no built-in function since these are not often used.
- Studentized deleted residuals: **rstudent** function.

We will most often use the **studentized deleted residuals** AKA the **jackknife residuals**, $\hat{r}_i$, and will consider it to be an outlier (for now) if $|\hat{r}_i| > 3$.

Residual Plots

Typical Residual Plot Patterns

1. Random scatter: This indicates no evidence that the homogeneity of variance assumption has been violated.
2. Fan-shaped: Here, the residuals increase with the size of the predicted values $\hat{y}_i$.
 - $\Rightarrow$ Variance heterogeneity in ε_i
 - What is the remedy?

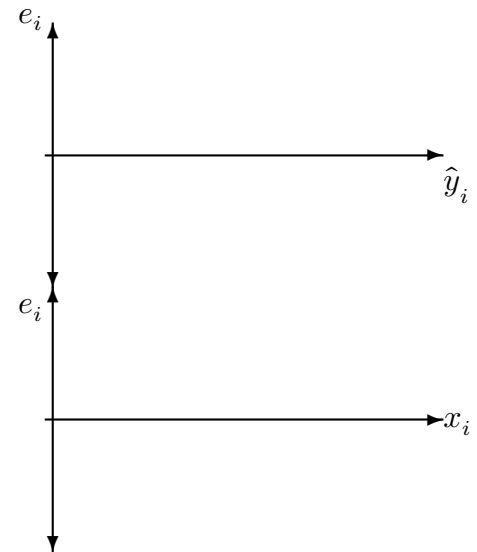


3. Trend: Sometimes a residual plot will exhibit a clear trend, as shown to the right. How could this happen?

- If the relationship between y and x were curved (quadratic), we often get alternating groups of positive and negative residuals.
- This indicates the need for a higher order term in one of the explanatory variables or some other systematic error in the data.

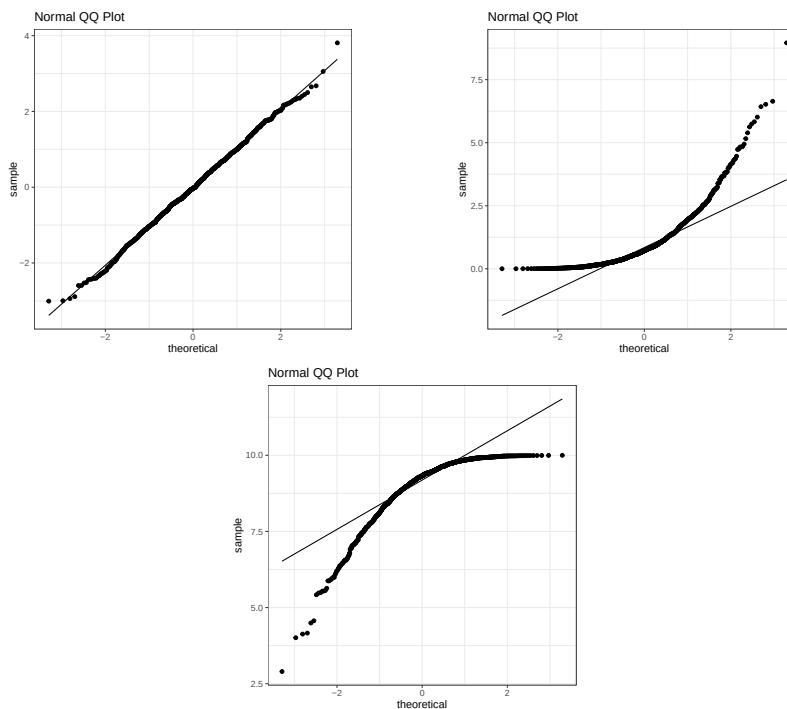
4. Random pattern with an outlier: Outliers are easy to detect on a residual plot, as points which fall far from the 0-line.

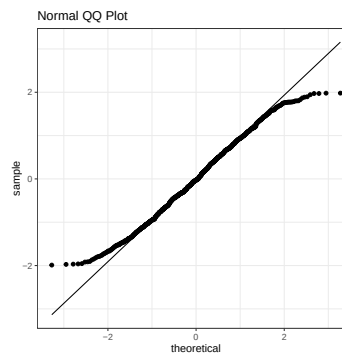
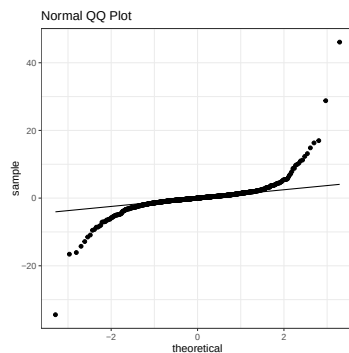
- What does a plot of e_i vs. x_i tell us?



QQ Plots

Recall QQ plots from Notes 4. We can use these to check the normality of the residuals. We plot the residuals against values that would be expected from a normal distribution. The closer to a straight line these points are, the closer to normality the residuals are.





Numerical Methods of Examining Lack of Fit in Regression

1. R^2 : The coefficient of determination R^2 gives the percentage variation in the response variable y explained by the regression on the explanatory variable x .

- Major drawback?

2. **F-Test of Model Significance:** To test:

H_0 : The regression model is not significant ($\beta_1 = 0$ in SLR)

vs. H_a : The regression model is significant ($\beta_1 \neq 0$ in SLR),

the statistic: $F^* = \frac{SSR/1}{SSE/(n-2)} = \frac{MSR}{MSE} \sim F(1, n-2)$ under H_0 .

- So large values of F^* (i.e.: $\geq F_\alpha(1, n-2)$) indicate model significance. They do NOT indicate the model is correct or even that the model is good.

3. **Tests for Model Lack of Fit:** There are numerous other tests and statistics that have been developed to examine model adequacy and to determine the “best” model by some criterion. These will not be a focus for us in this class.

Example: Back to the gestation example.

```

Coefficients:  Estimate Std. Error t value Pr(>|t|)
(Intercept)   51.716     42.902    1.205  0.25127
longev        12.139      3.193    3.801  0.00252 **

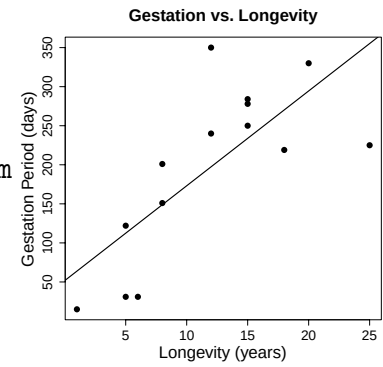
```

```

Residual standard error: 77.06 on 12 degrees of freedom
Multiple R-squared:  0.5463, Adjusted R-squared:  0.5085
F-statistic: 14.45 on 1 and 12 DF,  p-value: 0.002523

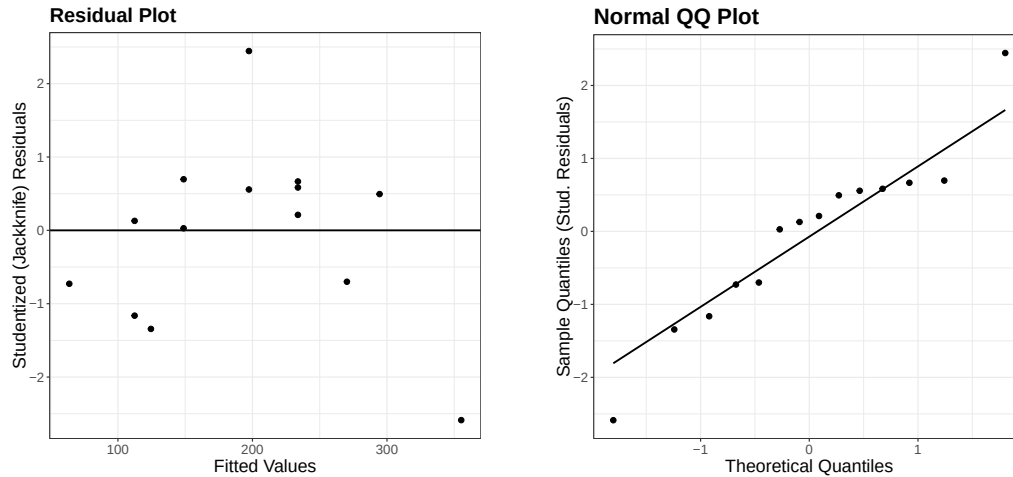
```

What does the F test tell us?



Now we can make some plots.

What can we say about the model assumptions?



```

gest_fit <- fitted.values(gest_mod) # Get the fitted values (y hats)
gest_res <- rstudent(gest_mod) # Get the studentized residuals
resid_df <- data.frame(gest_fit, gest_res) # Create data frame
ggplot(resid_df, aes(x = gest_fit, y = gest_res)) +
  geom_point(size = 2) +
  geom_hline(yintercept = 0, linewidth = 0.8) + # Adds a line at 0
  labs(x = "Fitted Values", y = "Studentized (Jackknife) Residuals",
       title = "Residual Plot") +
  theme_bw() +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"))
ggplot(resid_df, aes(sample = gest_res)) +
  geom_qq(size = 2) +
  geom_qq_line(linewidth = 0.8) +
  labs(x = "Theoretical Quantiles", y = "Sample Quantiles (Stud. Residuals)",
       title = "Normal QQ Plot") +
  theme_bw() +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 20, face = "bold"))

```

Or just use the `residual_plots()` function from the `math3150package` library!

Notes 6: Simple Linear Regression Transformations and Inference

Table of contents

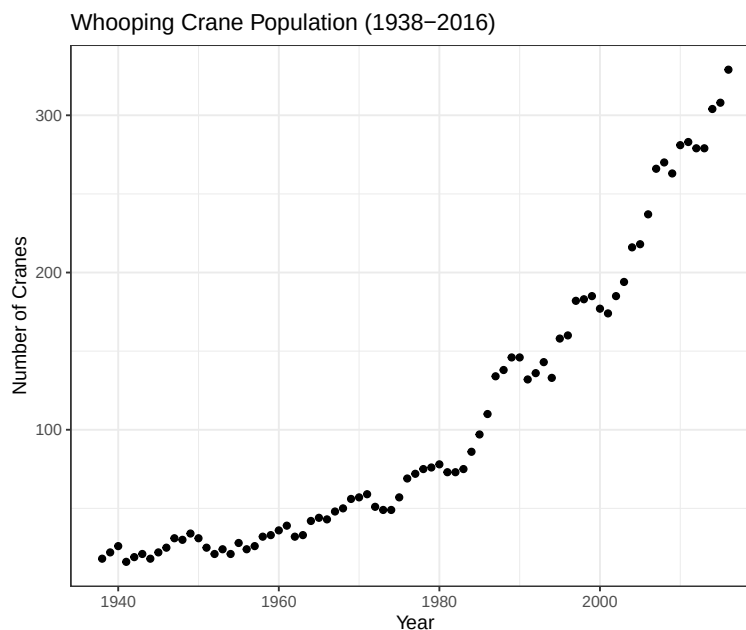
Regression Transformations	95
Interpretation when Log Transforming	99
Case 1: No transformation (linear-linear model)	100
Case 2: Y is transformed, X is not transformed (log-linear model)	100
Case 3: Y is not transformed, X is transformed (linear-log model)	101
Case 4: Y is transformed, X is transformed (log-log model)	101
Heart Wine Example	103
Inference – Simple Linear Regression	106
Assessing Quality of Fit of the Regression Model	109
Confidence Intervals (CIs) for β_0 and β_1	109
Hypothesis Tests for β_0, β_1	110
Confidence and Prediction Intervals	112
Inference on $\mathbf{E}(y x)$ (Confidence Intervals & Bands)	112
Inference on $y x$ (Prediction Intervals & Bands)	114

Regression Transformations

Recall the time plot of the estimated number of whooping cranes wintering at Aransas National Wildlife Refuge from 1938 through 2016 (shown below). Suppose we wish to model population size as a function of year in order to project future population sizes.

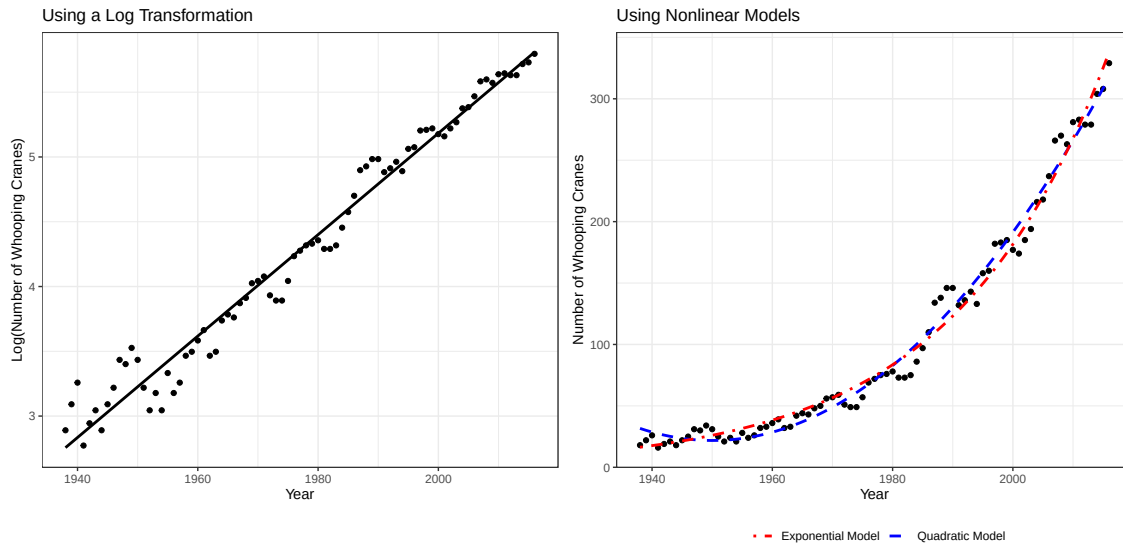
- Keep in mind that this type of **extrapolation** can be dangerous as it assumes that the relationship between population and time will stay the same in the future.

```
cranes <- math3150package::cranes |>
  mutate(logc = log(cranes)) # Take log of crane counts
ggplot(cranes, aes(x = year, y = cranes)) +
  geom_point() +
  labs(x = "Year", y = "Number of Cranes",
       title = "Whooping Crane Population (1938-2016)") +
  theme_bw()
```



- Would a simple linear regression model make sense here? How could we model this relationship? There are two primary choices:
 - Fit a nonlinear model** that appropriately describes the relationship.
 - Model may still be linear, but with multiple parameters.
(Ex: $y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2}^2 + \varepsilon_i$ (quadratic model)).
 - Model might be nonlinear requiring a numerical solution for the parameters.
(Ex: $y_i = Cx^\gamma + \varepsilon_i$).
 - Transform** either Y or X to make the relationship linear (i.e.: maybe $\log Y$ and X have a linear relationship).

- Both of these are visualized below. Which do you prefer? Why?



The log transformation model is:

$$\begin{aligned}\ln(y) &= -73.09616 + 0.03914 \cdot \text{year} \\ y &= \exp(-73.09616 + 0.03914 \cdot \text{year}) = \exp(-73.09616) \cdot \exp(0.03914 \cdot \text{year}) \\ &= 1.797799 \times 10^{-32} \cdot \exp(0.03914 \cdot \text{year})\end{aligned}$$

The model using a quadratic term is:

$$y = 258289.93 + -264.8855477 \cdot \text{year} + 0.067918134373875338472 \cdot \text{year}^2$$

Which is better? It depends on what you are trying to do. According to convention, since much of the inference on the regression parameters hinges on the assumptions of approximate normality and variance homogeneity of the residuals, whichever of the two choices above gives a parsimonious model which meets these assumptions is favorable. In this class, however, we will focus on doing transformations that make the relationship linear.

The first graph was created via the **R** code below.

```
ggplot(cranes, aes(x = year, y = logc)) +
  geom_point() +
  labs(x = "Year", y = "Log(Number of Whooping Cranes)",
       title = "Using a Log Transformation") +
  geom_smooth(method = "lm", se = F, color = "black") +
  theme_bw()
```

Let's go through the transformation using the cranes example. In this case, we begin with transforming the number of cranes to get the model

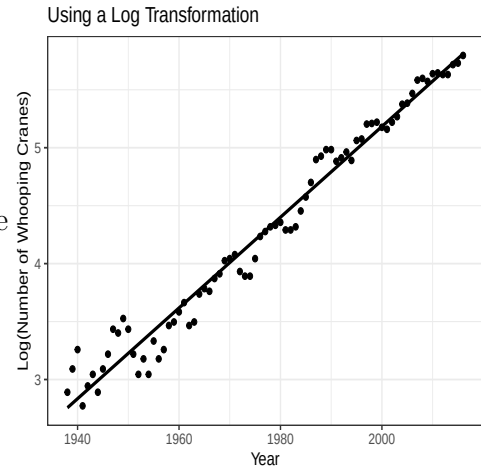
$$\ln(y_i) = \beta_0 + \beta_1 x_i + \varepsilon_i$$

We, of course, are dealing with a sample, so our model will become

$$\widehat{\ln(y_i)} = b_0 + b_1 x_i.$$

it is best to keep the log in there so we know the response variable has been transformed.

- Is the relationship linear? Do the errors appear to be normal? How might we check for normality of the residuals?



The following **R** output summarizes the regression fit for the crane model. Some explanation of the statistics appearing in this output is given.

```
crane_mod <- lm(logc ~ year, data = cranes)
summary(crane_mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-7.310e+01	1.472e+00	-49.67	<2e-16 ***
year	3.914e-02	7.443e-04	52.58	<2e-16 ***

Residual standard error: 0.1509 on 77 degrees of freedom

Multiple R-squared: 0.9729, Adjusted R-squared: 0.9726

F-statistic: 2765 on 1 and 77 DF, p-value: < 2.2e-16

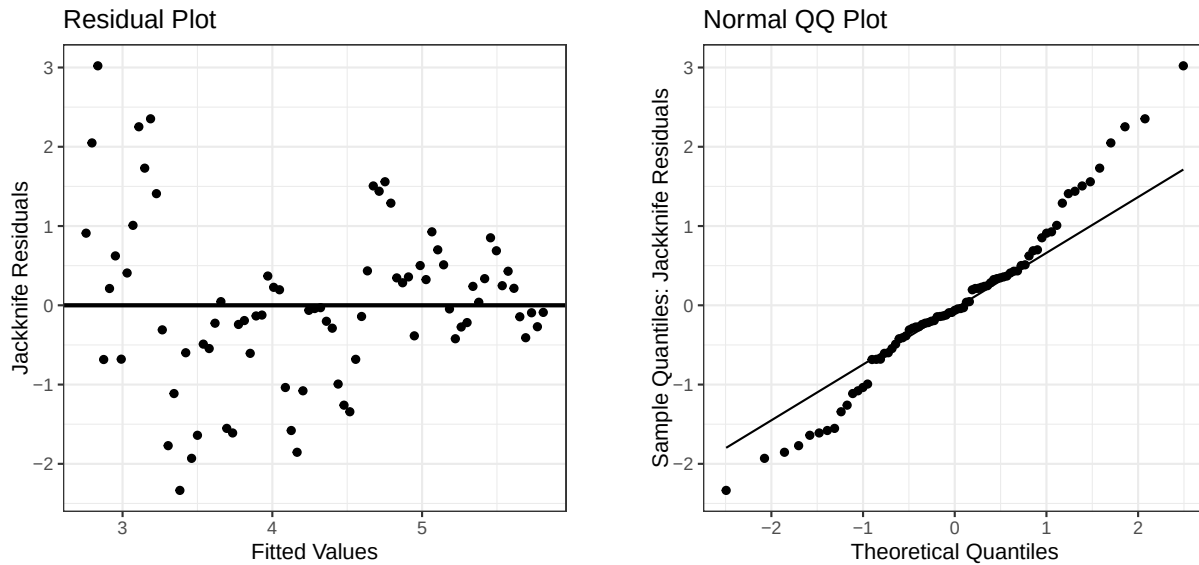
From this output, the resulting regression line is given by:

$$\widehat{\ln(y)} = -73.10 + 0.03914x, \quad 1938 < x < 2016,$$

where Y is the number of cranes, and X is the year. Inference on these parameters will be discussed soon.

- The F-statistic and corresponding p -value that tests whether or not the linear regression of $\log(Y)$ on X was significant indicates that there is strong evidence of a linear relationship between Y and X . How do we tell?
- The value of $R^2 = 0.9729$, indicating an excellent regression fit for these data.
- The value reported as the “residual standard error” is:
 $\sqrt{\text{MSE}} = \sqrt{\text{SSE}/(n-2)} = \sqrt{\text{SSE}/77}.$

- The table of coefficients or parameter estimates also contains a column of standard errors of these coefficients, a t -statistic for testing whether or not the parameters differ from zero, and the corresponding p -value for this test. None of this output (with the exception of the parameter estimates) is valid unless the errors from the model of the log number of cranes on years are approximately normally distributed.



Had we used the quadratic model, the model would be $\hat{y} = 258289.9 - 264.89x + 0.0679x^2$.

```
crane_mod2 <- cranes |>
  mutate(year_sq = year^2) |>
  lm(cranes ~ year + year_sq, data = _) # _ pipes data frame into data argument
summary(crane_mod2)
```

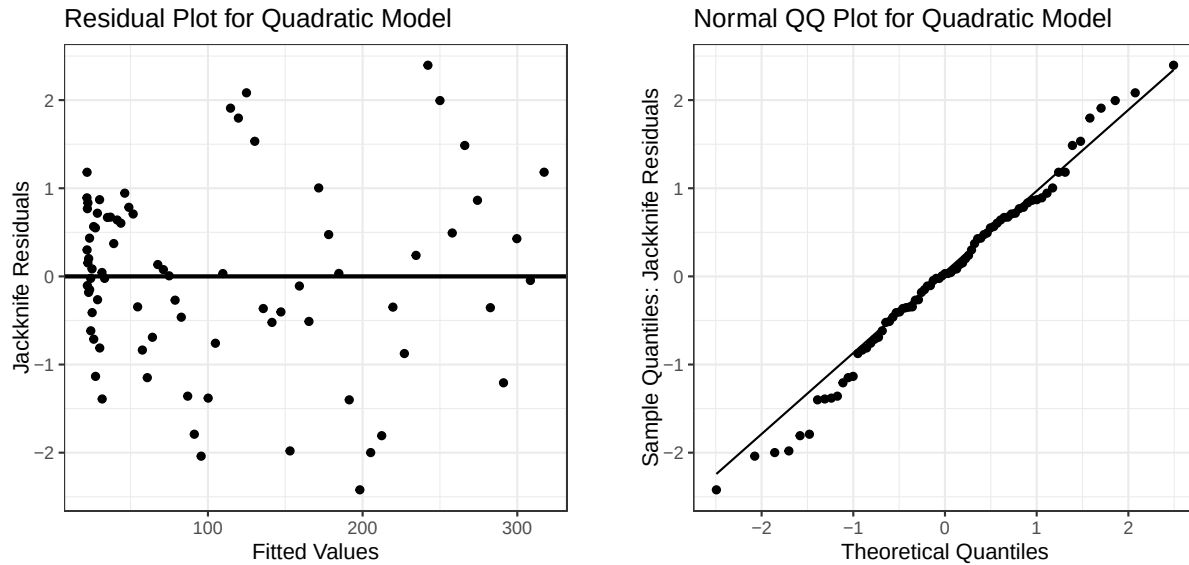
Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.583e+05	9.893e+03	26.11	<2e-16 ***
year	-2.649e+02	1.001e+01	-26.46	<2e-16 ***
year_sq	6.792e-02	2.531e-03	26.83	<2e-16 ***

Residual standard error: 10.46 on 76 degrees of freedom

Multiple R-squared: 0.987, Adjusted R-squared: 0.9866

F-statistic: 2877 on 2 and 76 DF, p-value: < 2.2e-16



Pros and cons of using quadratic model over log transform in general?

Interpretation when Log Transforming

To interpret the slope, first note that we will have to transform back to the original y variable in our model. We began with

$$\widehat{\log(y)} = b_0 + b_1 x$$

We then solve for y :

We can use the `exp()` function in **R** to make the input of that function a power of the number e .

- Suppose it turned out that the square root of the number of cranes had made the relationship with area linear instead of the log. Back transform this model to the original units: do you obtain a convenient interpretation of the coefficients as you do with the log transformation?

Case 1: No transformation (linear-linear model)

If the model has not been transformed, then we have

$$\hat{y} = b_0 + b_1x.$$

Slope interpretation: As the X variable increases by 1 unit, the predicted change in the Y variable is b_1 .

Intercept interpretation: The predicted value of the Y variable is equal to b_0 when the X variable is equal to 0.

Example: If X = the amount of time spent practicing (in hours), Y = points scored in a basketball game, and the linear regression model is

$$\hat{y} = 6.2 + 1.3x,$$

then we can interpret the slope as follows: As we increase practice time by 1 hour, the the number of points score per game is predicted to increase by 1.3.

Intercept: The predicted number of points scored in a basketball game when the person spends no time practicing is 6.2 points.

Case 2: Y is transformed, X is not transformed (log-linear model)

The model here is

$$\widehat{\log_a(y)} = b_0 + b_1x.$$

Slope interpretation: As the X variable increases by 1 unit, the Y variable is predicted to change by a factor of a^{b_1} . (Note that a is the base of the log.)

Intercept interpretation: The predicted value of the Y variable is equal to a^{b_0} when the X variable is equal to 0.

Example If X = the year, Y = number of whooping cranes, and the model is

$$\widehat{\ln(y)} = -73.10 + 0.03914x,$$

then the slope can be interpreted as follows: As the year increases by 1, the number of whooping cranes is predicted to increase by a factor of $e^{0.03914} = 1.03992$ (or increase by 3.992%).

This also works when using a different base.

$$\widehat{\log_{10}(y)} = -31.75 + 0.0170x,$$

then the slope can be interpreted as follows: As the year increases by 1, the number of whooping cranes is predicted to increase by a factor of $10^{0.0170} = 1.03992$ (or increase by 3.992%).

For the intercept: The number of cranes at Aransas in the year 0 is predicted to be equal to $10^{-31.75} = 1.778 \times 10^{-32} \approx 0$.

Case 3: Y is not transformed, X is transformed (linear-log model)

The model here is

$$\hat{y} = b_0 + b_1 \log_a(x).$$

Slope interpretation 1: As the X variable increases by a factor of a , the predicted change in the Y variable is b_1 . (Note that a is the base of the log.)

Slope interpretation 2: As the X variable increases by 1%, the predicted change in the Y variable is $b_1 * \log_a(1.01)$.

Intercept interpretation: The predicted value of the Y variable is equal to b_0 when the X variable is equal to 1 (not 0!).

Example: If X = the displacement of a car, Y = mpg of a car, and the model is

$$\hat{y} = 81.4 - 11.321 \log_3(x),$$

then the slope can be interpreted as follows: As displacement increases by a factor of 3, the predicted decrease in mpg is 11.321 mpg. Or, we can say: As the displacement increases by 1%, the predicted decrease in mpg is $11.321 \cdot \log_3(1.01) = 0.1025$ mpg.

Intercept: The predicted mpg of a car when the displacement is 0 is 81.4 mpg.

Case 4: Y is transformed, X is transformed (log-log model)

The model here is

$$\widehat{\log_a(y)} = b_0 + b_1 \log_a(x).$$

Slope interpretation 1: As the X variable increases by a factor of a , the Y variable is predicted to change by a factor of a^{b_1} . (Note that a is the base of the log.)

Slope interpretation 2: As the X variable increases by 1%, the Y variable is predicted to increase by $[(1.01)^{b_1} - 1] \times 100\%$.

Slope interpretation 3 (Approximation): As the X variable increases by 1%, the Y variable is predicted to change by about $b_1\%$.

Intercept interpretation: The predicted value of the Y variable is equal to a^{b_0} when the X variable is equal to 1 (not 0!).

Example: If X = investment per capita (in dollars), Y = GDP per capita (in dollars), and the model is

$$\widehat{\ln(y)} = 8.41 + 1.143 \ln(x),$$

then the slope can be interpreted as follows: As investment per capita increases by a factor of e , the GDP per capita is predicted to increase by a factor of $e^{1.143} = 3.1361$. Or, we can say: as the investment per capita increases by 1%, the GDP per capita is predicted to increase by $[(1.01)^{1.143} - 1] \times 100\% = 1.144\%$. Or, we can say: as the investment per capita increases by 1%, the GDP per capita is predicted to increase by about 1.143%.

For the intercept: The GDP per capita is predicted to be $e^{8.41} = \$4,491.76$ when the investment per capita is \$1.

Note: While the approximate interpretation is nearly identical to the second interpretation when the slope value is small, the approximation becomes worse as the slope increases. For example, if X = the year, Y = number of whooping cranes, and the model is

$$\widehat{\ln(y)} = -582.74 + 77.349 \ln(x),$$

then the slope can be interpreted as follows: As the year increases by a factor of e , the number of whooping cranes is predicted to increase by a factor of $e^{77.349}$. We could also say that as the year increases by 1%, the number of whooping cranes is predicted to increase by $[(1.01)^{77.349} - 1] \times 100\% = 115.9\%$. The approximate interpretation would be: as the year increases by 1%, the number of whooping cranes is predicted to increase by 77.349%. However, 77.349% is quite different than the correct 115.9%.

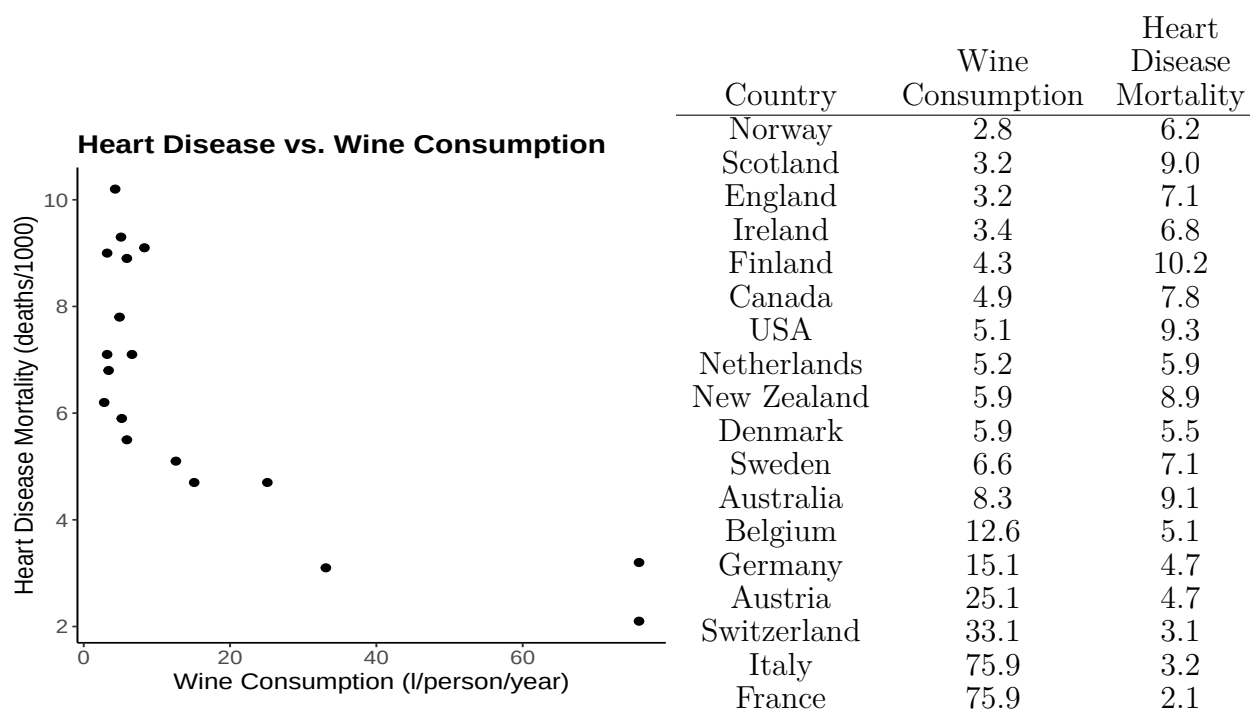
Of course, none of these interpretations are very good. Increasing year by a factor of e , for example, would be like increasing from the year 2000 to the year 5437. Even increasing by 1% would be changing from the year 2000 to the year 2020, which is a quite a leap. It would probably be better to interpret it as follows: as the year increases by 0.1%, the number of whooping cranes is predicted to increase by $[(1.001)^{77.349} - 1] \times 100\% = 8.04\%$ since a 0.1% increase would be about a 2-year increase. For about a one-year increase, we could say: as the year increases by 0.05%, the number of whooping cranes is predicted to increase by $[(1.0005)^{77.349} - 1] \times 100\% = 3.94\%$ (a value that is quite close to the value in the interpretation of the example in Case 2).

Note on intercept interpretation: As we have mentioned before, the interpretation of the intercept term does not always make sense. If the X variable is not theoretically able to be equal to 0 (or 1 in the case of the log transformation) or if that value of X is outside the range of our data, then the interpretations should be taken with a grain of salt.

Heart Wine Example

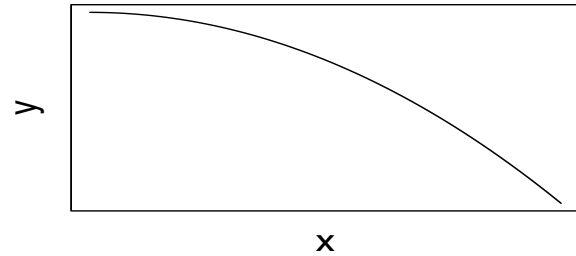
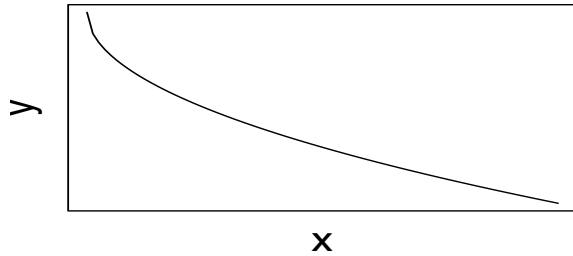
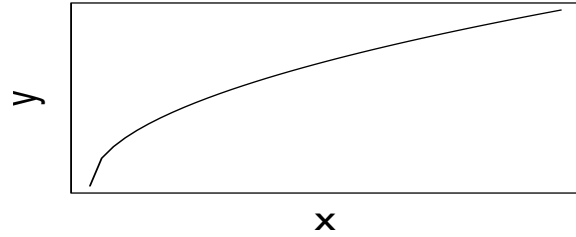
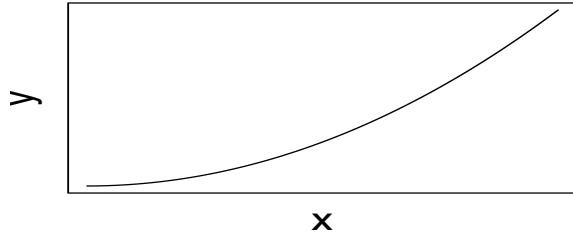
Example 2: The table and scatterplot below give the average wine consumption rates (in liters per person per year) and heart disease deaths (per 1000 men aged 55 to 64 years old) for 18 industrialized countries [Data from A.S. Leger et.al., *Lancet*, 1979]. \ \ Researchers were interested in questions such as:

1. Is heart disease associated with average wine consumption? If so, how can that relationship be described?
2. Do any countries seem to have higher or lower death rates than others with similar wine consumption rates?
- If the association is *very strong*, can we use these data to conclude that wine consumption prevents heart disease?



1. Association?

Can we linearize this relationship? How? What “patterns” lead to certain types of transformations?

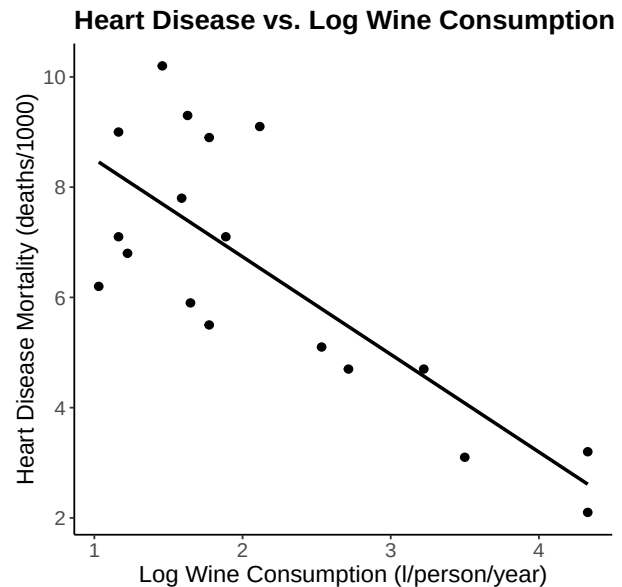
Some Patterns and Remedies:

- For the wine data, consider a log transformation on the wine consumptions (x -values). Transformations on y simply didn't adequately linearize the relationship.

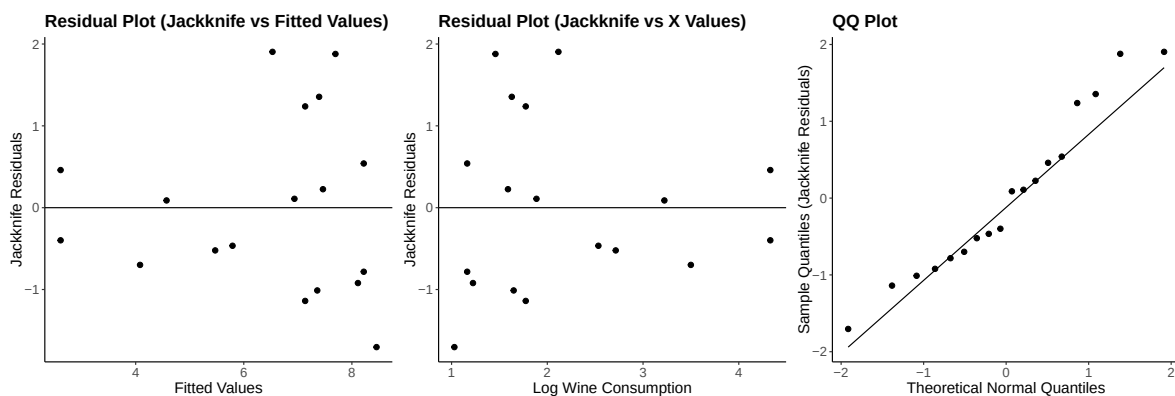
Do things look good now?

- The resulting least squares regression equation was:

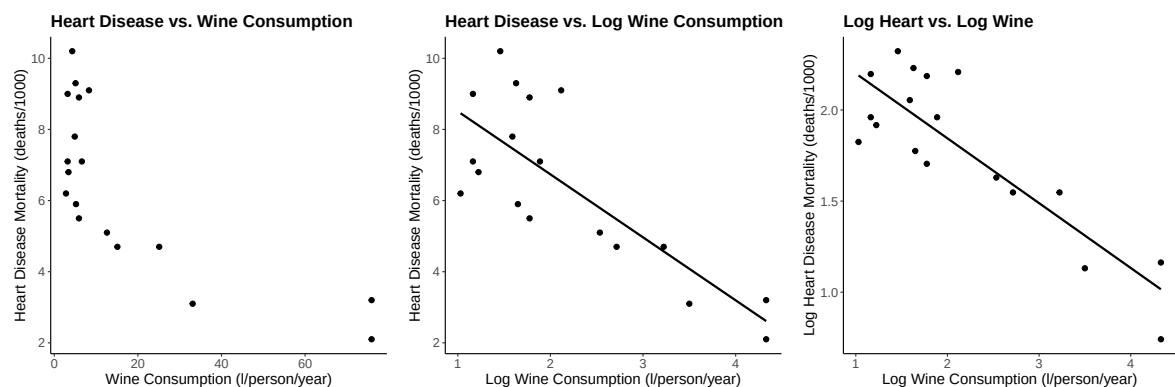
$$\widehat{\text{Heart Disease Mortality}} = 10.2795 - 1.7712 \cdot \ln(\text{Wine}).$$

Notes:

- Transformations sometimes solve several problems at once (and sometimes *cause* some!):
 - the relationship is linearized
 - outliers and influential points may be reduced or eliminated
 - the scatter may have increased.



- Don't use R^2 to choose an appropriate transformation; use plots. There are two reasons for this:
 1. A high R^2 does NOT necessarily mean the relationship is linear.
 2. The values of R^2 for two different models cannot be directly compared if the response variables are different in any way.



- Question: Does it matter whether we transform x or y ?
 - Not in general, and sometimes both x and y are transformed.
 - Note: If there is a quadratic relationship between y and x , we might fit the model: $y_i = \beta_0 + \beta_2 x_i^2 + \varepsilon_i$. Although the relationship may be quadratic, omission of the linear term ($\beta_1 x_i$) from this model places a restriction on the form of the model.
 - Statistical convention is to always include lower order terms in any regression model fit.
- Question: Why transform at all?
 - One could fit a nonlinear or multiple regression model instead.
 - One reason to transform is that it may require fewer parameters in the model. In general, if two models explain the relationship equally well, the better model is the one with fewer parameters.

Inference – Simple Linear Regression

The fitted simple linear regression model is $\hat{y} = b_0 + b_1x$ where b_0 is the estimated intercept and b_1 is the estimated slope.

Interpretation of estimated slope?

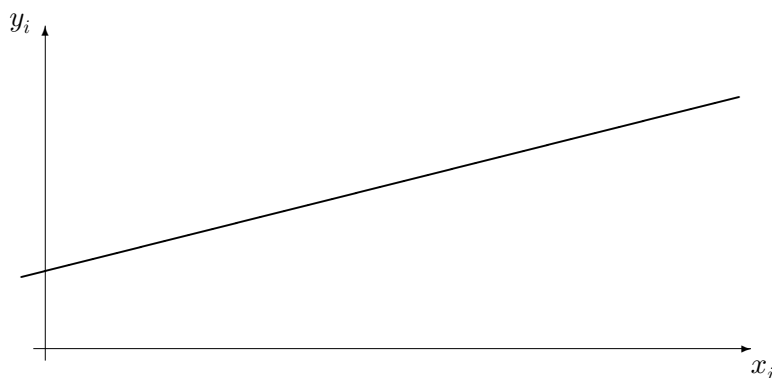
- We might test: $H_0 : \beta_1 = \beta_{1,0}$
vs. $H_a : \beta_1 \neq \beta_{1,0}$
- A point estimate of β_1 is:
- What do we need to conduct these tests?

Inference on β_0 and β_1

- For the purpose of doing least squares regression, we assumed that the average of the errors, $E(\varepsilon_i) = 0$. This was the only assumption required to produce the least squares estimates b_0, b_1 of β_0, β_1 .
- For (normal-based) inference in regression, three additional assumptions are required:
 1. The ε_i 's are **independent** (i.e.: there is no serial correlation or patterns among the residuals).
 2. The variance of all ε at each x_i , namely $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$ for each $i = 1, \dots, n$. This is called the **homogeneity of variance** assumption.
 3. $\varepsilon_i|x_i \sim N(0, \sigma_\varepsilon^2) \forall i$. In other words, we assume the errors are **normally distributed**.

Implications of These Assumptions:

1. Since $y_i = \beta_0 + \beta_1x_i + \varepsilon_i$, then



2. $b_0 \sim$

$$b_1 \sim$$

Aside: $b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$, which is a linear function of the y_i 's. *Any linear function of independent normal random variables is normal.*

Example: Consider the following data relating blood alcohol content (BAC) to the number of beers consumed for several teen volunteers at Ohio State where the number of beers was randomly assigned to a student. The BAC was measured by a police officer 30 minutes after consumption.

BAC (Y)	.10	.03	.19	.12	.04	.095	.07	.06	.02	.05	.07	.10	.085	.09	.01	.05
# Beers (X)	5	2	9	8	3	7	3	5	3	5	4	6	5	7	1	4

The regression line (given by: $\hat{y} = -0.0127 + 0.0180x$) clearly provides a good fit ($R^2 = 0.800$) to these data. Are the resulting residuals normally distributed? To answer this, we construct a normal quantile (or QQ) plot on the residuals, shown below.

Some important summary statistics resulting from the regression fit, which we will need shortly, are:

$$b_0 = -0.0127, b_1 = 0.0180, n = 16,$$

$$\text{SSE} = 0.00585, \sum x_i^2 = 443,$$

$$\sum (x_i - \bar{x})^2 = \sum x_i^2 - n\bar{x}^2 = 72.437!$$

with ANOVA table:

Analysis of Variance Table

Response: BAC

	Df	Sum Sq	Mean Sq
beers	1	0.02338	0.02338
Resid	14	0.00585	0.00042

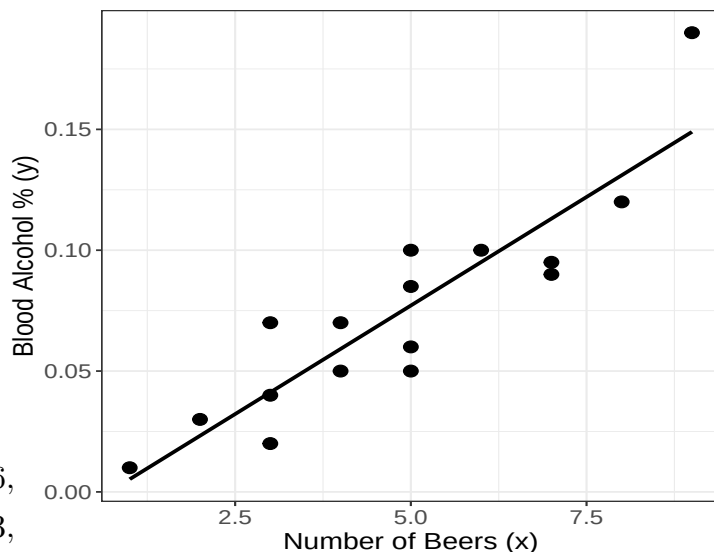
- It was mentioned earlier that to perform inference on the regression parameter β_1 , we need to know the standard deviation of its estimator b_1 , given as

$$\sigma_{b_1} = \sigma_\varepsilon / \sqrt{\sum (x_j - \bar{x})^2}.$$

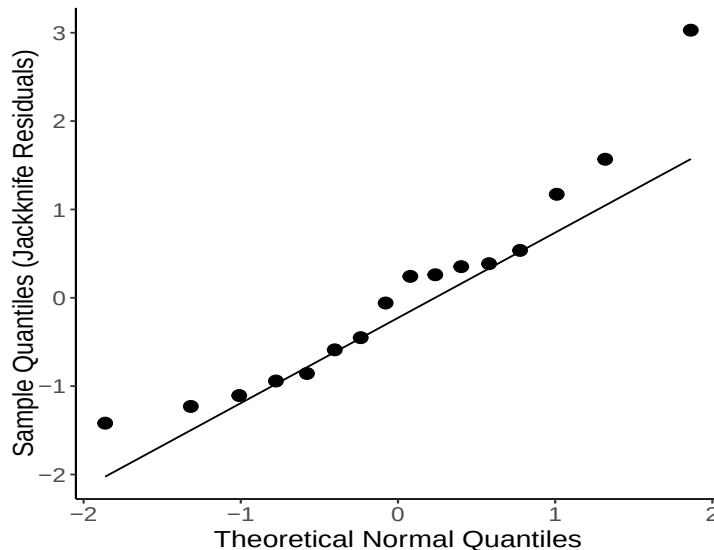
- Do we know σ_ε^2 ? What is σ_ε^2 ?
- How do we estimate σ_ε^2 ?

- This estimate of σ_ε^2 , namely s_ε^2 , is known as the mean squared error (MSE).

BAC vs. Number of Beers



QQ Plot



Back to the Example: $s_\varepsilon^2 = \frac{\text{SSE}}{n-2} = \frac{0.00585}{14} = \underline{0.000418}$

$$\begin{aligned}\Rightarrow \sigma_{b_0} &= \sigma_\varepsilon \sqrt{\frac{\sum x_i^2}{nS_{xx}}} = \sigma_\varepsilon \sqrt{\frac{443}{16(72.4375)}} = .6182\sigma_\varepsilon \quad (\text{True SD of } b_0) \\ \underline{s_{b_0}} &= .6182s_\varepsilon = .6182\sqrt{0.000418} = \underline{0.012638} \quad (\text{Standard error of } b_0) \\ \sigma_{b_1} &= \sigma_\varepsilon / \sqrt{\sum (x_j - \bar{x})^2} = \sigma_\varepsilon / \sqrt{72.4375} = \sigma_\varepsilon / 8.5110 \\ \underline{s_{b_1}} &= s_\varepsilon / 8.5110 = \underline{0.002402} \quad (\text{Standard error of } b_1).\end{aligned}$$

Summary of Regression Fit: When performing a linear regression fit with **R**, there are three primary pieces of output, resulting from the **lm** and **anova** functions respectively, as listed below.

1. A model summary usually including R^2 as a measure of predictive ability, and an F-test of model significance (**lm**).
2. A table of coefficients (parameter estimates), their standard errors, the t-statistic to test for significance of the parameters, and the corresponding p-value for this test (**lm**).
3. An ANOVA table which reports the SSR, SSE, and SST, and the F-statistic which test the goodness of fit of the model (**anova**).

For the BAC data, the following output were obtained from **R**:

```
b_mod = lm(BAC ~ beers, data = beers_df)
summary(b_mod)
```

```
Coefficients:  Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.012701   0.012638  -1.005   0.332
beers        0.017964   0.002402   7.480 2.96e-06
```

```
Residual standard error: 0.02044 on 14 degrees of freedom
Multiple R-squared:  0.7998,    Adjusted R-squared:  0.7855
F-statistic: 55.94 on 1 and 14 DF,  p-value: 2.969e-06
```

```
anova(b_mod)
```

Analysis of Variance Table

Response: BAC

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
beers	1	0.0233753	0.0233753	55.944	2.969e-06
Residuals	14	0.0058497	0.0004178		

Assessing Quality of Fit of the Regression Model

As we saw in Notes 5, In addition to R^2 as an indicator of goodness of fit of the regression model, a formal test of model adequacy based on an **F-test** can be conducted and is often reported in the statistical output.

Result: The statistic $F = \frac{SSR/1}{SSE/(n-2)} = \frac{MSR}{MSE}$ has an F -distribution with 1 numerator degrees of freedom and $n - 2$ denominator d.f. Why? In general, this statistic tests the hypotheses:

$$\begin{aligned} H_0 : & \quad \text{The regression model is not significant.} \\ \text{vs. } H_a : & \quad \text{The model is significant.} \end{aligned}$$

Is the BAC/# Beers simple linear regression model significant?

Recall: There are two basic forms of inference:

1. Confidence Interval Estimation
2. Tests of Significance

Confidence Intervals (CIs) for β_0 and β_1

Under the assumption: $\varepsilon_i \sim N(0, \sigma_\varepsilon^2)$, we know that: $b_0 \sim N(\beta_0, \sigma_{b_0}^2)$ and $b_1 \sim N(\beta_1, \sigma_{b_1}^2)$.

- All CIs (relying directly on normality) have the form:

$$\text{estimate} \pm \left(\begin{array}{c} \text{critical} \\ \text{value} \end{array} \right) \cdot \left(\begin{array}{c} \text{standard error} \\ \text{of estimate} \end{array} \right)$$

- So:

$100(1 - \alpha)\% \text{ CIs for } \beta_0, \beta_1 \text{ have the form:}$
 $b_0 \pm t_{\alpha/2} \cdot s_{b_0} \text{ and } b_1 \pm t_{\alpha/2} \cdot s_{b_1}$
- Why do we use a t -distribution critical value?

Back to BAC Example: A $100(1 - \alpha)\%$ confidence interval for β_0 has the form:

$$b_0 \pm t_{\alpha/2} s_{b_0}$$

where $t_{\alpha/2}$ is the critical value, and
 $P(T > t_{\alpha/2}) = \alpha/2$ for $T \sim t(n - 2)$.

For the BAC data, we found $b_0 = -0.0127$ and $s_{b_0} = 0.0126$. This gives the following set of confidence intervals (depending on the level desired):

Confidence Coefficient	Significance Level (α)	Critical Value ($t_{\alpha/2}$)	Interval (in $\pm$ form)	Confidence Interval
90%	0.10	1.761	$-0.0127 \pm (1.761)(0.0126)$	$(-0.0350, 0.0096)$
95%	0.05	2.145	$-0.0127 \pm (2.145)(0.0126)$	$(-0.0398, 0.0144)$
99%	0.01	2.977	$-0.0127 \pm (2.977)(0.0126)$	$(-0.0503, 0.0249)$

Does the intercept term seem significant?

A $100(1 - \alpha)\%$ confidence interval for β_1 has the form:

$$\boxed{b_1 \pm t_{\alpha/2} s_{b_1}}$$

where $t_{\alpha/2}$ is the critical value, and $P(T > t_{\alpha/2}) = \alpha/2$ for $T \sim t(n - 2)$.

For the BAC data, we found $b_1 = 0.0180$ and $s_{b_1} = 0.0024$. This gives the following set of confidence intervals (depending on the level desired):

Confidence Coefficient	Significance Level (α)	Critical Value ($t_{\alpha/2}$)	Interval (in $\pm$ form)	Confidence Interval
90%	0.10	1.761	$0.0180 \pm (1.761)(0.0024)$	$(0.0137, 0.0222)$
95%	0.05	2.145	$0.0180 \pm (2.145)(0.0024)$	$(0.0128, 0.0231)$
99%	0.01	2.977	$0.0180 \pm (2.977)(0.0024)$	$(0.0108, 0.0251)$

Does the slope term seem significant?

- In **R**, you can produce confidence intervals for the regression model parameters by typing: `confint(b_mod, level = 0.95)`.

Hypothesis Tests for β_0, β_1

Since $b_0 \sim N$ and $b_1 \sim N$, then the test statistics:

$$\frac{b_0}{s_{b_0}} \sim$$

$$\frac{b_1}{s_{b_1}} \sim$$

With the distributions of b_0 and b_1 , tests of significance on β_0 and β_1 can be conducted according to the following recipes:

To test $H_0 : \beta_0 = \beta_{0,0}$ versus H_a, the test statistic is: $t = \frac{b_0 - \beta_{0,0}}{s_{b_0}} \quad (n - 2 \text{ d.f.}).$ For the 3 alternative hypotheses, the P-values are: <table> <tr> <th>Alternative Hypothesis</th> <th>P-value</th> </tr> <tr> <td>$H_a : \beta_0 < \beta_{0,0}$</td> <td>$P(T < t)$</td> </tr> <tr> <td>$H_a : \beta_0 > \beta_{0,0}$</td> <td>$P(T > t)$</td> </tr> <tr> <td>$H_a : \beta_0 \neq \beta_{0,0}$</td> <td>$2P(T > t)$</td> </tr> </table> where $T \sim t(n - 2)$	Alternative Hypothesis	P-value	$H_a : \beta_0 < \beta_{0,0}$	$P(T < t)$	$H_a : \beta_0 > \beta_{0,0}$	$P(T > t)$	$H_a : \beta_0 \neq \beta_{0,0}$	$2P(T > t)$	AND	To test $H_0 : \beta_1 = \beta_{1,0}$ versus H_a, the test statistic is: $t = \frac{b_1 - \beta_{1,0}}{s_{b_1}} \quad (n - 2 \text{ d.f.}).$ For the 3 alternative hypotheses, the P-values are: <table> <tr> <th>Alternative Hypothesis</th> <th>P-value</th> </tr> <tr> <td>$H_a : \beta_1 < \beta_{1,0}$</td> <td>$P(T < t)$</td> </tr> <tr> <td>$H_a : \beta_1 > \beta_{1,0}$</td> <td>$P(T > t)$</td> </tr> <tr> <td>$H_a : \beta_1 \neq \beta_{1,0}$</td> <td>$2P(T > t)$</td> </tr> </table> where $T \sim t(n - 2)$	Alternative Hypothesis	P-value	$H_a : \beta_1 < \beta_{1,0}$	$P(T < t)$	$H_a : \beta_1 > \beta_{1,0}$	$P(T > t)$	$H_a : \beta_1 \neq \beta_{1,0}$	$2P(T > t)$
Alternative Hypothesis	P-value																	
$H_a : \beta_0 < \beta_{0,0}$	$P(T < t)$																	
$H_a : \beta_0 > \beta_{0,0}$	$P(T > t)$																	
$H_a : \beta_0 \neq \beta_{0,0}$	$2P(T > t)$																	
Alternative Hypothesis	P-value																	
$H_a : \beta_1 < \beta_{1,0}$	$P(T < t)$																	
$H_a : \beta_1 > \beta_{1,0}$	$P(T > t)$																	
$H_a : \beta_1 \neq \beta_{1,0}$	$2P(T > t)$																	

- In most cases, the numbers $\beta_{0,0}$ and $\beta_{1,0}$ are set to zero in the above tests. When equal to zero, these tests examine the significance of the intercept (β_0) and slope (β_1).
- Once a regression model has been fit, the most common form of inference is to test for significance of the parameters. Why? If a parameter is found to not be significant (not significantly different from zero), then it may be dropped from the model. This makes the model simpler and allows an extra degree of freedom for other inferences and for estimating the error (SSE) remaining in the model.
- If the slope parameter is found to be insignificant, then the explanatory variable (X) corresponding to it is not a good predictor of the response variable (Y), and may be dropped from the model.

Back to the Example: Suppose we wish to test whether or not the slope parameter is different from zero for the BAC data.

1. We test $H_0 : \beta_1 = 0$ vs. $H_a : \beta_1 \neq 0$.
 2. The test statistic for this test is: $t = \frac{b_1}{s_{b_1}} = \frac{0.0180}{0.0024} = 7.48$, on $n - 2 = 14$ d.f.
 3. The p-value for this test is $\underline{2P(T > |t|)} = 2P(T > 7.48) = 2(0.00000148) = \underline{0.00000296}$ ($T \sim t(14)$).
 4. Conclusion: Since the p-value is very small, we have overwhelming evidence that the slope parameter β_1 is different from zero (in fact greater than zero), and hence that the number of beers is a significant linear predictor of BAC. In fact, with 95% confidence, this slope is between 0.0128 and 0.0231 %/beer.
- The table of coefficients in the regression computer output provides the t-statistic and p-value for the two-sided test of all parameters in the model. Note that the t-statistic and p-value found above matches that of the table.
 - Is the intercept parameter significant? Should we drop it from the model? The convention in statistics is to always leave the intercept term in the model regardless of its significance unless there is some reason to believe that it should be zero theoretically.
 - This idea of leaving a statistically insignificant term in a model pervades statistics. In general, the convention is to leave all lower order terms in the model regardless of significance. For example, if you fit a quadratic model in x , you should retain the linear x term even if it is statistically insignificant.

Confidence and Prediction Intervals

Inference on $E(y|x)$ (Confidence Intervals & Bands)

- One goal of linear regression is to predict the value of the response variable y with the values of the explanatory variable x . Such predictions are made using the regression equation: $\hat{y}_i = b_0 + b_1x_i$ for values of x_i within the range of the collected data.
- This only gives a point estimate of the response at x_i . In general, we would like to give a confidence interval of predicted values, or conduct hypothesis tests on the response.

Recall: For our SLR model $y_i = \beta_0 + \beta_1x_i + \varepsilon_i$, we found that:

$$E(y_i|x_i) = \beta_0 + \beta_1x_i \text{ is the true average value of } y_i \text{ at } x_i \text{ (since } E(\varepsilon_i|x_i) = 0\text{)}.$$

- How do we estimate $E(y_i|x_i)$?
- To perform inferences on $E(y_i|x_i)$, what do we need?

1.

2.

Result:
$$\text{Var}(\hat{y}_i|x_i) = \sigma_\varepsilon^2 \left(\frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum (x_j - \bar{x})^2} \right).$$

- And since $\hat{y}_i = b_0 + b_1x_i$ is a linear function of b_0 & b_1 , which are linear functions of the y_i 's, which are normal, then:

$$\hat{y}_i|x_i \sim N\left(\beta_0 + \beta_1x_i, \text{Var}(\hat{y}_i|x_i)\right)$$

(assuming the ε_i 's are normally distributed).

Confidence Interval for $E(y|x)$: A $100(1 - \alpha)\%$ confidence interval (CI) for $E(y|x_i)$ at $x = x_i$ is given by:

Back to the BAC Example: Recall the simple linear regression model relating blood alcohol content (BAC) (in %) to number of beers consumed. The data and corresponding regression equation ($\hat{y} = -0.0127 + 0.0180x$) are plotted on the following page.

1. Give a 95% CI for $E(y_i|x_i)$ at $x = 5$.

Some summary statistics for these data are:

$$s_\varepsilon = 0.02044, \bar{x} = 4.8125, \text{ and } \sum(x_j - \bar{x})^2 = 72.4375.$$

A point estimate of $E(y|x_i)$ is given by:

The SE of this point estimate is given by:

$$SE(\hat{y}_i|x_i) = s_\varepsilon \sqrt{\frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum(x_j - \bar{x})^2}} = 0.02044 \sqrt{\frac{1}{16} + \frac{(5 - 4.8125)^2}{72.4375}} = 0.00513$$

2. Find a 95% CI for $E(y|x)$ for any x .

We just found that for these data, 95% confidence bands for $E(y|x)$ can be found for different values of x using the equation:

$$\hat{y}_i \pm (2.145)(0.02044) \sqrt{\frac{1}{16} + \frac{(x_i - 4.8125)^2}{72.4375}}.$$

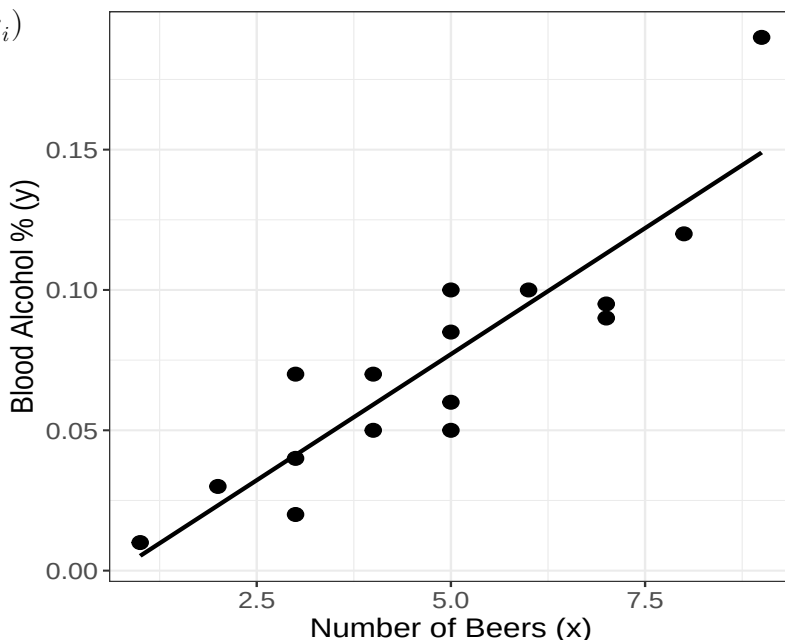
Using this equation for a number of different x -values, the following table of lower and upper confidence limits was generated:

x_i	1	2	3	4	5	6	7	8	9
$\hat{y}_i$	0.0053	0.0232	0.0412	0.0592	0.0771	0.0951	0.1130	0.1310	0.1490
Lower Limit	-0.0172	0.0051	0.0268	0.0474	0.0661	0.0825	0.0973	0.1113	0.1248
Upper Limit	0.0278	0.0414	0.0556	0.0709	0.0881	0.1076	0.1288	0.1508	0.1732
Margin of Error	0.0225	0.0182	0.0144	0.0117	0.0110	0.0126	0.0157	0.0197	0.0242

Plot these lower and upper limits on the scatterplot above. What do you notice about the resulting **confidence bands**?

1. We can estimate $E(y|x_i)$ more precisely for values of x_i in the “center” of the data (closer to $\bar{x}$).
2. Viewing these bands emphasizes the danger of extrapolation: increasing variability. The estimates become less precise the further from $\bar{x}$ one is trying to predict.

BAC vs. Number of Beers



Hypothesis Test: To test $H_0 : E(y|x_i) = \mu_0$ (at some x_i) vs. H_a , the test statistic is:

$$t =$$

Conclusions may be drawn in the usual way:
where α = level of significance desired,
and $T \sim t(n-2)$.

Alternative Hypothesis	P-value
$H_a : E(y) > \mu_0$	$P(T > t)$
$H_a : E(y) < \mu_0$	$P(T < t)$
$H_a : E(y) \neq \mu_0$	$2P(T > t)$

Inference on $y|x$ (Prediction Intervals & Bands)

- Interval estimates on parameters are known as **confidence intervals**.
Interval estimates on variables are known as **prediction intervals**.
- Suppose instead of a CI on $E(y|x_i) = \mu$ (here we estimate an average), we want to predict the value of $y_i|x_i$ at a particular x_i (here, we estimate a random variable Y).

What is a point estimate for $y_i|x_i$?

What else do we need?

Result: A $100(1 - \alpha)\%$ prediction interval (PI) for $y_i|x_i$ at some x_i has the form:

$$\hat{y}_i \pm t_{\alpha/2} s_\varepsilon \sqrt{1 + \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum (x_j - \bar{x})^2}},$$

where $s_\varepsilon = \sqrt{\frac{\text{SSE}}{n-2}}$ and $t_{\alpha/2}$ is the $\alpha/2$ critical value from a $t(n-2)$ -distribution.

Note: The only difference between this interval and the earlier confidence interval for the mean is the “1” inside the square root portion of the standard error.

Back to the BAC Example: Find a 95% PI for $y_i|x_i$ at $x_i = 5$.

- The form of the prediction bands is: $\hat{y}_i \pm (2.145)(0.02044) \sqrt{1 + \frac{1}{16} + \frac{(x_i - 4.8125)^2}{72.4375}}$.
- For $x_i = 5$, $\hat{y}_i = 0.0771$ ($= b_0 + b_1 x_i$) $\Rightarrow 0.0771 \pm 0.0452 \Rightarrow \underline{(0.0319, 0.1223)}$.
- A 95% CI for $E(y|x_i = 5)$ was: $0.0771 \pm 0.0110 \Rightarrow \underline{(0.0661, 0.0881)}$.

Notes:

1. The prediction interval is approximately 4 times wider than the CI at $x_i = 5$. Why?
2. What factors control the width of CI's and PI's?

Sample size increases $\implies$ Width

Confidence level increases $\implies$ Width

$\sum(x_j - \bar{x})^2$ increases $\implies$ Width

- So taking a wider range of x -values decreases the width of the intervals, but only if the model remains valid for all x -values. As an implication of this for design purposes, this says that the range of x -values sampled should be as large as possible as long as the design assumptions are met.
- Above, we found that for these data, 95% prediction bands for y at particular values of x can be found using the equation:

$$\hat{y}_i \pm (2.145)(0.02044)\sqrt{1 + \frac{1}{16} + \frac{(x_i - 5)^2}{72.4375}}.$$

Using this equation for a number of different x -values, the following table of lower and upper prediction limits was generated:

x_i	1	2	3	4	5	6	7	8	9
$\hat{y}_i$	0.0053	0.0232	0.0412	0.0592	0.0771	0.0951	0.1130	0.1310	0.1490
Lower Limit	-0.0440	-0.0242	-0.0050	0.0138	0.0319	0.0495	0.0665	0.0829	0.0989
Upper Limit	0.0545	0.0707	0.0873	0.1045	0.1223	0.1407	0.1596	0.1791	0.1990
Margin of Error	0.0493	0.0475	0.0461	0.0454	0.0452	0.0456	0.0466	0.0481	0.0501

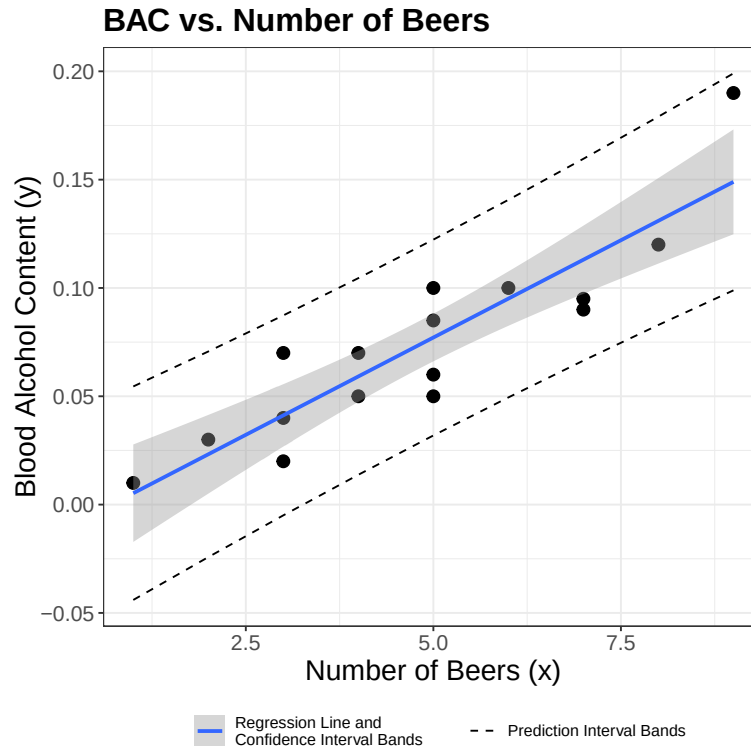
We can get confidence and prediction intervals using the `predict()` function in **R**:

```
predict(b_mod, newdata = data.frame(beers = 5), interval = "confidence")
```

```
      fit      lwr      upr
1 0.07711821 0.06611536 0.08812105
```

```
predict(b_mod, newdata = data.frame(beers = c(5, 8)), interval = "prediction")
```

```
      fit      lwr      upr
1 0.07711821 0.03191712 0.1223193
2 0.13100949 0.08292834 0.1790906
```



What do you notice?

```
xval <- data.frame(beers = 1:9) # Sequence of x-values from 1 to 9 by 1
pred <- predict(b_mod, xval, interval = "prediction") |>
  as_tibble() |>
  mutate(beers = xval$beers)
ggplot(data.frame(beers, BAC), aes(x = beers, y = BAC)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", formula = "y ~ x",
             aes(fill = "Regression Line and\nConfidence Interval Bands")) +
  geom_line(data = pred, aes(x = beers, y = lwr), linetype = "dashed") +
  geom_line(data = pred, linetype = "dashed",
           aes(x = beers, y = upr, color = "Prediction Interval Bands")) +
  labs(x = "Number of Beers (x)", y = "Blood Alcohol Content (y)",
       title = "BAC vs. Number of Beers") +
  theme_bw() +
  theme(axis.title = element_text(size = 16),
        axis.text = element_text(size = 12),
        plot.title = element_text(size = 18, face = "bold"),
        legend.position = "bottom") +
  scale_color_manual("", values = c("black")) +
  scale_fill_manual("", values = "gray60")
```

Notes 7: Multiple Linear Regression

Table of contents

Types of Multiple Linear Regression Models	118
Quantitative vs Quantitative	118
Quantitative vs Mix of Quantitative and Categorical	120
Quantitative vs Categorical	122
Finding Least Squares Estimates using Matrices	122
R^2 in Multiple Linear Regression	123
Multicollinearity	124
Variance Inflation Factor (VIF)	125
Updated Regression Assumptions	126
Inference in Multiple Regression with Example	127
F-Test of Model Significance	129
Inference on the Regression Parameters (the β_i s)	129
Inference on $\mathbf{E}(\mathbf{Y} \mathbf{x}_1, \dots, \mathbf{x}_p)$ and $\mathbf{y} \mathbf{x}_1, \dots, \mathbf{x}_p$	130
Back to the Example	130
Computing Confidence and Prediction Intervals	134

It is often the case that we would like to predict a response variable using more than one explanatory variable. We can do this by using multiple linear regression.

- The general form of a multiple linear regression model, known as the **general linear model (GLM)**, is given by:
- It is worth noting that by “linear” here, we mean “linear in the parameters.”

Examples:

- (i) $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon$ (quadratic model)
- (ii) $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \varepsilon$ (first-order, additive model)
- (iii) $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \varepsilon$ (2nd-order, interaction model)

Interpretation of the β_i 's

- β_0 is still the intercept, i.e.: the expected value of y when $x_1 = \dots = x_p = 0$.
- $\beta_1, \dots, \beta_p$ are known as the partial slopes, where β_i represents the change in Y per unit increase in X_i , holding all other X_j ($j \neq i$) constant or fixed. In this way, β_i describes a partial change in Y .

First-Order Model: $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i$ (2 variables, additive)

- The exclusion of higher-order terms (such as $\beta_3 x_{i1}^2$ or $\beta_4 x_{i2}^2$, etc.) indicates our belief that Y has a linear relationships with both X_1 and X_2 .
- The exclusion of an **interaction term** (cross-product) $x_{i1}x_{i2}$ makes an assumption that the effects of the explanatory variables are **additive**.

(i.e. - no matter what the value of X_2 , β_1 represents the
change in Y per unit increase in X_1 and vice versa.)

Types of Multiple Linear Regression Models

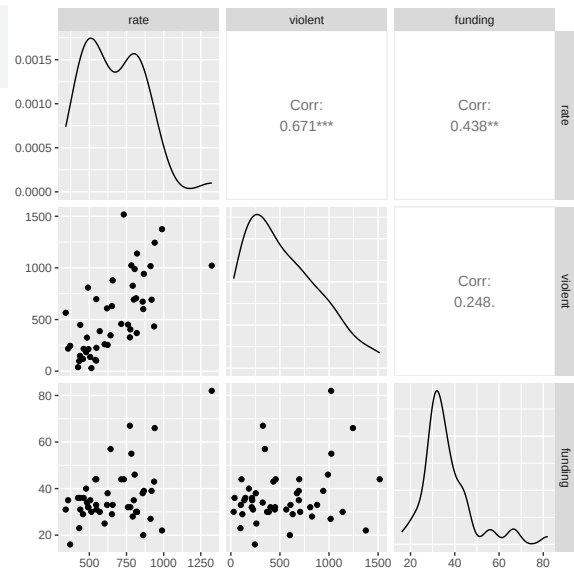
1. Quantitative vs. Quantitative - this is what we have been doing thus far.
2. Quantitative vs Mix of Quantitative and Categorical.
3. Quantitative vs. Categorical - this case occurs when one or more of the explanatory variables is categorical (finite # of levels).

Quantitative vs Quantitative

Consider the example of attempting to estimate the total crime rate (per million residents) using the violent crime rate (per 100,000 residents) and the annual police funding (in \$/resident) for 47 cities in the U.S.

First, observe the relationships between the different variables:

```
library(GGally)
ggpairs(crime) # Make plot
```



Now fit a multiple linear regression model.

```
crime <- math3150package::crime # Load data
crime_mod <- lm(rate ~ violent + funding, data = crime) # Fit the model
summary(crime_mod) # Obtain the model summary
```

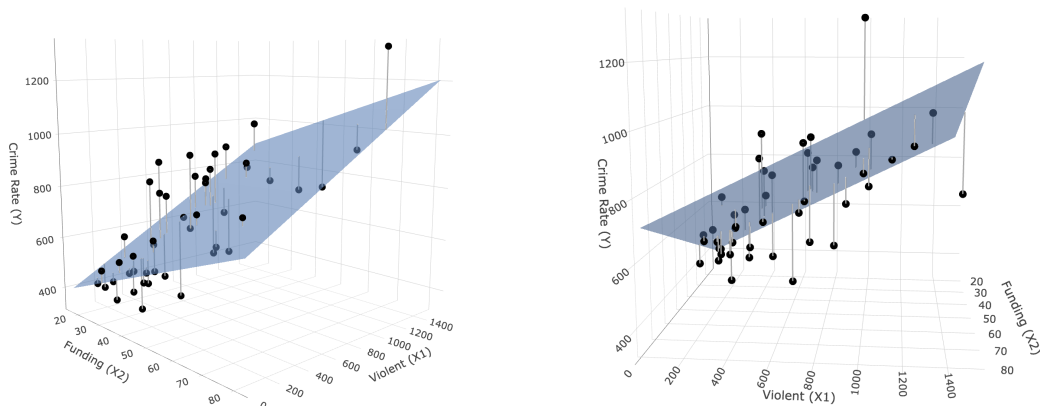
Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	350.8865	79.2530	4.427	5.65e-05 ***
violent	0.3355	0.0548	6.122	1.76e-07 ***
funding	4.2470	2.2748	1.867	0.0681 .

Residual standard error: 189.4 on 47 degrees of freedom

Multiple R-squared: 0.6018, Adjusted R-squared: 0.5849

F-statistic: 35.52 on 2 and 47 DF, p-value: 3.996e-10



So our model is $\hat{y} = b_0 + b_1x_1 + b_2x_2 = 312.13 + 0.3260x_1 + 4.85x_2$.

- We can interpret the intercept, b_0 , as the predicted crime rate per million people when the violent crime and police funding are both zero.
- We can interpret the first partial slope, b_1 :
- We can interpret the second partial slope, b_2 :

Quantitative vs Mix of Quantitative and Categorical

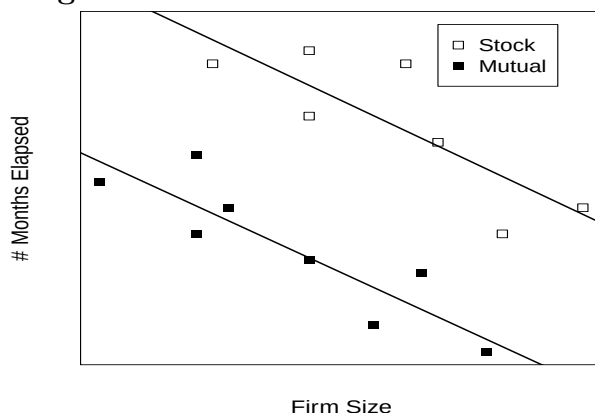
Example: Suppose we have data collected on:

Y = # of months elapsed before an insurance innovation is adopted

X_1 = size of the firm (total assets)

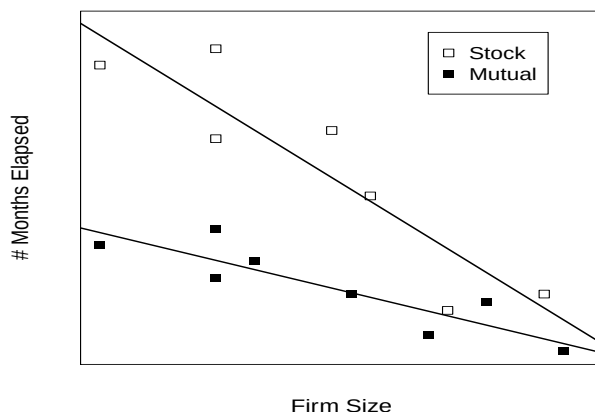
X_2 = type of company (stock or mutual)

- We might code the stock and mutual fund companies as a 0 and 1 respectively.



- The slopes of Y vs. X_1 for each value of X_2 remain the same.

- Suppose instead we have the following scatterplot:
- Here, the partial slope β_1 depends on the level of X_2 (stock or mutual)
 $\Rightarrow$ Interaction between X_1 & X_2 .



- In this interaction case, we need the model:

– If $x_2 = 0$, the slope for x_1 is

, with intercept =

– If $x_2 = 1$, the slope for x_1 is

, with intercept =

- How do we interpret β_2 & β_3 ?

β_2 =

β_3 =

Example: Consider a response variable y = infection risk in a hospital (measured as the average probability of infection). Some candidate explanatory variables are:

1. Age of patient (quantitative)
2. Type of hospital (General, Teaching, Specialty, Community) (categorical)
 - Note that these hospital types cannot be ordered; they are simply four different types. These 4 types comprise 4 treatments or levels of the hospital type variable.

How do we define a GLM for this scenario?: Let:

- Note that we do not need x_5 for the general type. If an observation is from a general hospital, then $x_2 = x_3 = x_4 = 0$ (baseline case). So the fact that each of these indicator variables is zero for general hospitals distinguishes it from the other three types in the model.
- The GLM is then: $E(y|x_{i1}, x_{i2}, x_{i3}, x_{i4}) = E(y|x_i) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4$.
- How do we interpret the β_i 's? We could say something like:

β_3 represents the expected change in infection risk per unit increase of X_3 , holding all other X 's fixed.

How meaningful is this statement? What is meant by “per unit increase in X_3 ?”

- To interpret the parameters in this model, consider the following table:

	Hospital Type			
	General	Teaching	Specialty	Community

$$E(y) =$$

$\Rightarrow$

- So β_2 , β_3 , & β_4 represent differences in the mean infection risks between G & T, between G & S, and between G & C, respectively (given that age is in the model).
- How would we test for a difference in mean infection risk between the General and Community hospital types? Test:

- How would we test for a difference in mean infection risk between the Specialty and Community hospital types? Test:
- What is this type of GLM called?

Quantitative vs Categorical

In this case, we are essentially doing a multi-way ANOVA. The linear model will give us more information about how large the differences in groups are. We can use the same model structure as we just discussed above and the inference we discussed in Notes 4 about the two-way ANOVA.

Finding Least Squares Estimates using Matrices

- Use the computer. What is the computer doing?
- In **R**, you can enter as many explanatory variables as you want using the **lm** function. The model for the infection risk example could have been fit as:

```
reg_mod <- lm(y ~ x1 + x2 + x3 + x4)
```

- Consider the GLM: $y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{i,p} + \varepsilon_i$, $i = 1, \dots, n$. This can be written in matrix form as:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1,p} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{n,p} \end{bmatrix}_{n \times (p+1)} \times \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}_{(p+1) \times 1} + \begin{bmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{bmatrix}_{n \times 1}.$$

- Least squares theory says to find the parameter values $(\beta_0, \beta_1, \dots, \beta_p)$ which minimize $\sum_{i=1}^n \varepsilon_i^2$ (the sum of the squared residuals).
- In matrix form, $\sum \varepsilon_i^2 = \varepsilon^T \varepsilon$, where $\varepsilon^T = [\varepsilon_1, \dots, \varepsilon_n]$, the transpose of ε .
- Since $\varepsilon = y - X\beta$, then we need to minimize $(y - X\beta)^T (y - X\beta)$.
- Via calculus, we obtain: $(X^T X)b = X^T y$ (the **Normal Equations**).

$$\Rightarrow \hat{\beta} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_p \end{bmatrix} = (X^T X)^{-1} X^T y$$

- We can also write out the normal equations (not in matrix form), but they are hard to deal with without a computer.

R^2 in Multiple Linear Regression

In a simple linear regression model, the coefficient of determination R^2 is the percentage of variation in y explained by the linear regression of y on x . This definition generalizes to the multiple linear model:

R^2 is the percentage of variation in Y explained by the linear regression of Y on $X_1, \dots, X_p$ simultaneously. This value is sometimes denoted as: $R^2_{y \cdot x_1, \dots, x_p}$ to indicate precisely which variables were used in the model.

How is R^2 computed?: As with SLR, the value R^2 is computed as: $R^2 = \frac{\text{SSR}}{\text{SST}}$, where now:

$$\begin{aligned} \text{SST} &= \sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n y_i^2 - n\bar{y}^2 \quad (\text{exactly as before}), \\ \text{SSE} &= \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n (y_i - b_0 - b_1 x_{i1} - \dots - b_p x_{ip})^2, \\ \text{SSR} &= \text{SST} - \text{SSE}. \end{aligned}$$

Relationship Between R^2 and correlation r : In SLR, the statistic r represented the correlation between Y and X . In multiple linear regression, we are interested in the individual correlations between Y and each of the X_i 's, $i = 1, \dots, p$. For this, some new notation is required:

$$r_{yx_i} = \text{sample correlation between } Y \text{ and } X_i, \quad i = 1, \dots, p.$$

In SLR, we saw that $\underline{R^2 = r^2}$. Does a similar relationship hold for multiple regression? Two cases:

1. If $x_1, \dots, x_p$ are uncorrelated, then: $R^2 = r_{yx_1}^2 + \dots + r_{yx_p}^2$.
2. If $x_1, \dots, x_p$ are correlated, then: $R^2 < r_{yx_1}^2 + \dots + r_{yx_p}^2$.

Implications:

- If two variables X_1 & X_2 are uncorrelated, and the R^2 value is 0.4 (insignificant) for a SLR of y on x_1 and x_2 individually, then the multiple linear regression model with both x_1 and x_2 included in the model has $R^2 = 0.8$, a good fit.
- So although individual variables may not explain much of the variation in Y alone, the group as a whole may provide a good model for explaining the variation in Y .
- The less correlated the explanatory variables are, the more information toward explaining Y can be extracted from them. i.e.: Two highly correlated explanatory variables should usually not be included together in a model as they provide essentially the same information regarding Y .
- The problems caused by highly correlated explanatory variables is known as the **multi-collinearity** problem, and is investigated through methods such as principal components analysis (PCA), or variance inflation factors (VIF).

Multicollinearity

Consider the `cars99` data set again from Notes 1. Suppose we want to run a regression analysis using highway miles per gallon as our response variable and using the time to accelerate from 0 to 30 and using quarter mile time as our explanatory variables. First, we will fit a model using just `HwyMPG` and `Accel0_30`:

```
cars <- na.omit(math3150package::cars99) # Remove rows with a missing value
car_mod1 <- lm(HwyMPG ~ Accel0_30, data = cars) # Fit the model
summary(car_mod1)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	19.2814	2.8678	6.724	3.76e-09 ***
Accel0_30	2.7315	0.8028	3.403	0.0011 **

Residual standard error: 2.861 on 71 degrees of freedom

Multiple R-squared: 0.1402, Adjusted R-squared: 0.1281

F-statistic: 11.58 on 1 and 71 DF, p-value: 0.0011

Now fit a model using only `QtrMile` as a predictor:

```
car_mod2 <- lm(HwyMPG ~ QtrMile, data = cars) # Fit the second model
summary(car_mod2)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.3351	5.8789	0.907	0.36722
QtrMile	1.3588	0.3374	4.027	0.00014 ***

Residual standard error: 2.784 on 71 degrees of freedom

Multiple R-squared: 0.1859, Adjusted R-squared: 0.1745

F-statistic: 16.22 on 1 and 71 DF, p-value: 0.0001398

Finally, fit the model with both predictor variables:

```
car_mod3 <- lm(HwyMPG ~ Accel0_30 + QtrMile, data = cars)
summary(car_mod3)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.0167	9.5390	0.002	0.9986
Accel0_30	-1.5379	2.1676	-0.710	0.4804
QtrMile	1.9782	0.9364	2.113	0.0382 *

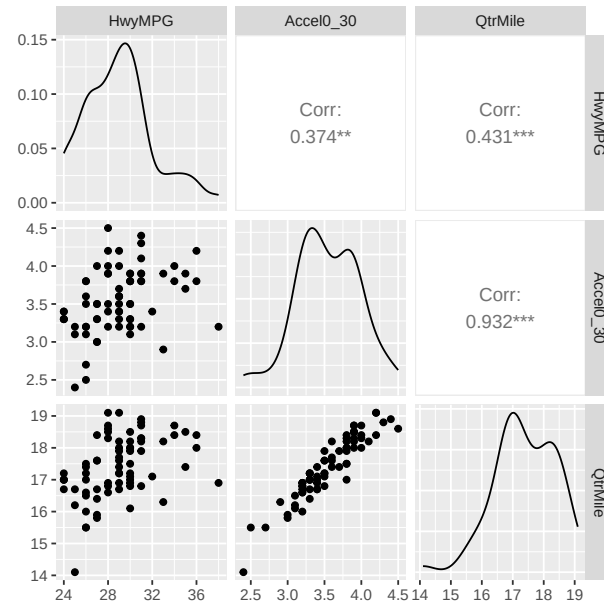
Residual standard error: 2.794 on 70 degrees of freedom

Multiple R-squared: 0.1917, Adjusted R-squared: 0.1686

F-statistic: 8.303 on 2 and 70 DF, p-value: 0.0005811

Both explanatory variables were considered significant with positive slopes when in the model by themselves, but together they are far less significant and the slope of `Acce10_30` changed to negative. Why?

To answer this question, we can view a pairs plot. What do we notice?



When we have variables that are highly correlated with each other in the same model, we have **multicollinearity** and we cannot trust the slope values. Recall how the slope of each variable is interpreted in multiple linear regression:

When two or more variables are highly correlated, it doesn't make sense for one of them to increase and the other to be held constant.

Variance Inflation Factor (VIF)

A common way to check if we have significant multicollinearity in a model is by calculating the variance inflation factor (VIF) for each of the predictor variables. We can calculate the VIF for the i th predictor variable's slope using

$$VIF_i = \frac{1}{1 - R_i^2},$$

where R_i^2 is the coefficient of determination when using all predictor variables except the i th one as explanatory variables in a regression model predicting the i th predictor variable. We can also use the `vif()` function in the `car` package in **R**.

Finding the VIF in this case manually:

```
vif_mod1 <- lm(Accele10_30 ~ QtrMile, data = cars)
summary(vif_mod1)      # R^2 = 0.8692
vif1 <- 1/(1 - 0.8692) # Equal to 7.64526
vif_mod2 <- lm(QtrMile ~ Accele10_30, data = cars)
summary(vif_mod2)      # R^2 = 0.8692
vif2 <- 1/(1 - 0.8692) # Same value: 7.64526
```

Finding VIF using the `vif` function:

```
library(car) # Loads the car package that contains the vif function
vif(car_mod3) # Calculate the variance inflation factor
```

```
Accel10_30    QtrMile
7.646512    7.646512
```

This means that the variance for each slope coefficient in this multiple regression model is about 7.65 times higher than we would expect it to be if collinearity was not present.

VIF Rules of Thumb: In general,

- If the VIF for all variables is less than 5, then multicollinearity is usually not a problem.
- If the VIF for some variables are between 5 and 10, multicollinearity could pose some problems.
- If the VIF for some variables are greater than 10, multicollinearity will cause serious issues in our regression model.

It should be noted that multicollinearity is only a problem with **interpretation**. If your only goal is to use the regression model for predicting y values, then multicollinearity is not a cause for concern.

Addressing multicollinearity:

1. If we are only using the regression model for prediction, we can ignore the multicollinearity.
2. If data come from an experiment, changing the experiment or increasing the sample size may reduce the problem.
3. We can drop one or more of the variables from the regression model.
4. We can use **principal component analysis**.
5. Other methods such as LASSO, ridge regression, or partial least squares regression can be used, but we will not discuss any of these methods in this course.

Updated Regression Assumptions

Now that we have discussed multicollinearity, here are the updated regression assumptions for regression analysis:

1. The errors have a mean of zero: $E(\varepsilon|x_i) = 0$ for all x_i . This reflects that we have a linear association. This is needed for **prediction and inference**.
2. The errors have the same variance across all values of x : $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$ for all x_i . This is needed for **inference**.
3. The errors follow a normal distribution. This is needed for **inference**.
4. The errors are independent of one another. This is needed for **inference**.
5. There is no multicollinearity. This is needed for **inference and interpretation**.

Inference in Multiple Regression with Example

Good news: most of the inference in multiple linear regression is essentially identical to the inference in simple linear regression! Let's walk through an example and talk about the inference procedures here.

Meadowfoam (*Limnanthes alba*) is a small plant which grows in moist meadows in the Pacific Northwest. A nongreasy and highly stable vegetable oil can be extracted from the meadowfoam seed. To study how to maximize meadowfoam production to a profitable crop, researchers set up the following controlled experiment. Using 6 different light intensities at levels of 150, 300, 450, 600, 750, and 900 $\mu\text{mol}/m^2/\text{sec}$, and 2 different times before the onset of the light treatment (at photoperiodic floral induction (PFI) or 24 days before PFI), the average number of flowers per meadowfoam plant from 10 seedlings in each of the 12 light intensity-light onset treatments was recorded. The entire experiment was repeated twice to give some replication, and produced the following data [M. Seddigh and G.D. Joliff, "Light Intensity Effects on Meadowfoam Growth and Flowering," *Crop Science* 34 (1994): 497-503.]:

Average Number of Flowers per Meadowfoam Plant		Light Intensity ($\mu\text{mol}/m^2/\text{sec}$)					
		150	300	450	600	750	900
Timing	at PFI	62.3	55.3	49.6	39.4	31.3	36.8
		77.4	54.2	61.9	45.7	44.9	41.9
	24 days before PFI	77.8	69.1	57.0	62.9	60.3	52.6
		75.6	78.0	71.1	52.2	45.6	44.4

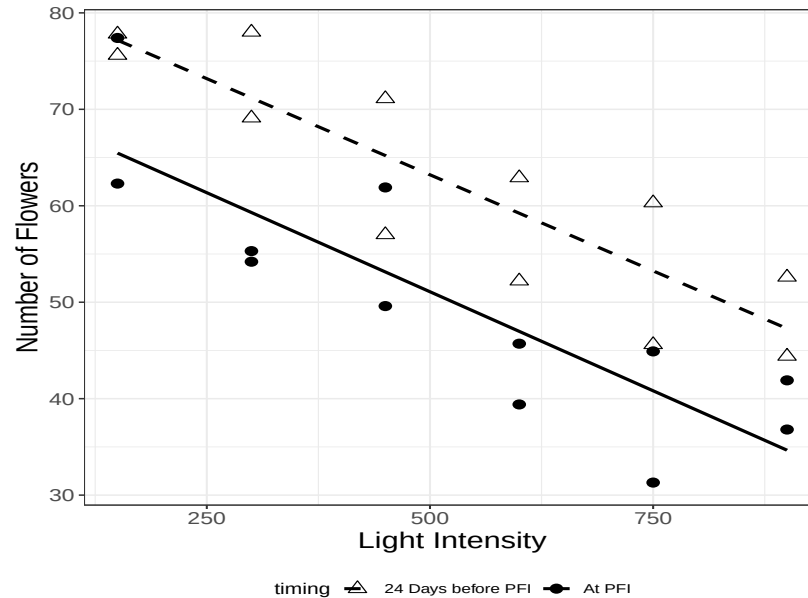
So this experiment has the following variables of interest:

- Y = the average number of flowers on a meadowfoam plant
- X_1 = the light intensity used on the plant ($\mu\text{mol}/m^2/\text{sec}$)
- X_2 = the timing of the light treatment (at PFI, 24 days before PFI)

Some questions sought by the researchers are:

1. What are the effects of the different light intensity levels?
2. What is the effect of timing?
3. Does the effect of intensity on flower production depend on timing?

The first step taken toward analysis of the meadowfoam data is to make a simple scatterplot of our response variable Y versus X_1 , at the two levels of X_2 . We are treating X_1 (light intensity) as a quantitative variable, and X_2 (timing) as a qualitative variable at 2 levels. Under this viewpoint, the following scatterplot with fitted regression lines depicts the relationship between these three variables:



This plot was generated in **R** using code that can be found in the **Notes 7 Script.R** file.

What does this plot tell you? Can you qualitatively answer the three posed questions:

1. What are the effects of the different light intensity levels?
2. What is the effect of timing?
3. Does the effect of intensity depend on timing?

Based on this plot then, what type of model might you suggest to fit? Surely, no quadratic terms are needed, and although there does not appear to be an interaction between intensity and timing, it might be worth testing for this lack of interaction formally. Letting $x_1 =$ intensity value, and

$x_2 = \begin{cases} 1 & \text{if the timing is "at PFI"} \\ 0 & \text{otherwise} \end{cases}$, one suggested model might be:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i1} x_{i2} + \varepsilon_i, \quad i = 1, \dots, n.$$

To include the interaction term, we could to create a variable $x_1 x_2$. To do this in **R**, we can issue the command:

```
(timing2 <- as.numeric(timing=="At PFI")) # Codes timing2 = 1 if "At PFI"
[1] 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0

(x1x2 <- intensity * timing2) # Defines the interaction term
[1] 150 150 0 0 300 300 0 0 450 450 0 0 600 600 0 0 750 750 0
[20] 0 900 900 0 0
```

Before fitting the regression model posed above, we first investigate the answers to the following question:

- Do the F-test statistic of model significance, the tests of significance for the parameter values and the confidence intervals on the parameters, mean of Y , and predicted value of Y have the same interpretations for the multiple linear regression model as for the SLR model?

F-Test of Model Significance

Recall that this tests the hypotheses:

H_0 : The regression model is not significant vs.

H_a : The regression model is significant.

For the multiple linear regression model, this test is exactly the same, except the degrees of freedom are different, and can still be found on the regression output in the ANOVA table. In this case, $df_N = p$ and $df_D = n - p$ so $MSR = SSE/p$ and $MSE = SSE/(n - p)$.

Inference on the Regression Parameters (the β_i s)

As before, to perform inference on β_i , $i = 0, \dots, p$, we need the distribution of the point estimator b_i , and its standard error.

- As before, b_i has a **normal** distribution ($i = 0, \dots, p$) whenever it is reasonable to assume the errors are normally distributed.
- The standard error of b_i has a nasty form, and we will never calculate it by hand in practice. For completeness, the form is given below:

$$s_{b_i} = s_e \sqrt{\frac{1}{\left(\sum_j x_{ji}^2 - n\bar{x}_i^2\right) (1 - R_{x_i \cdot x_1, \dots, x_{i-1} x_{i+1} \dots x_p}^2)}} \quad \text{where:}$$

$$s_e = \sqrt{\frac{SSE}{n - p}}, \quad p \text{ is the number of predictors, and}$$

$R_{x_i \cdot x_1, \dots, x_{i-1} x_{i+1} \dots x_p}^2$ is the coefficient of determination for the multiple regression of x_i on all of the other explanatory variables.

With this information then, it is straightforward to develop a CI for β_i and a corresponding hypothesis test. Both types of inference are summarized in the boxes below.

A $100(1 - \alpha)\%$ confidence interval for β_i , $i = 0, \dots, p$ is given by:

$$b_i \pm t_{\alpha/2} s_{b_i}.$$

For testing $H_0 : \beta_i = \beta_{i,0}$ versus H_a , the test statistic is: $t = \frac{b_i - \beta_{i,0}}{s_{b_i}}.$

For the three alternative hypotheses, the P-values are given by:

Alternative Hypothesis	P-value
$H_a : \beta_i < \beta_{i,0}$	$P(T < t)$
$H_a : \beta_i > \beta_{i,0}$	$P(T > t)$
$H_a : \beta_i \neq \beta_{i,0}$	$2P(T > t)$

where $T \sim t(n - p)$ for p parameters.

- It should be emphasized that inferences on β_i are performed assuming all other factors are in the model.

Inference on $E(Y|x_1, \dots, x_p)$ and $y|x_1, \dots, x_p$

To compute confidence intervals for the mean of Y , $E(Y|x_1, \dots, x_p)$, at particular values of the X -variables, or prediction intervals for $y|x_1, \dots, x_p$ at particular values of the X -variables, we use exactly the same methods as with SLR.

The only new aspect in determining these intervals is computing the standard error. The formulas are horrendous and we will depend on software to do the work for us.

Back to the Example

The multiple regression model with an interaction term, given earlier as:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i1} x_{i2} + \varepsilon_i, \quad i = 1, \dots, n,$$

was fit in **R** using the variables:

y = avg. number of flowers as the response variable,
 x_1 = intensity,
 $x_2 = \begin{cases} 1 & \text{at PFI} \\ 0 & \text{24 days before PFI} \end{cases}$ (timing), and
 $x_1 x_2$ = the interaction term.

```
flower_mod3 <- lm(flowers ~ intensity + timing + x1x2)#Fits interaction model
```

or equivalently:

```
flower_mod3 <- lm(flowers ~ intensity * timing) # Fits interaction model
```

The following output was obtained.

```
summary(flower_mod3) # Regression summary
```

Coefficients:	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	83.146667	4.343305	19.144	2.49e-14 ***
intensity	-0.039867	0.007435	-5.362	3.01e-05 ***
timingAt PFI	-11.523333	6.142360	-1.876	0.0753 .
intensity:timingAt PFI	-0.001210	0.010515	-0.115	0.9096

Residual standard error: 6.598 on 20 degrees of freedom

Multiple R-squared: 0.7993, Adjusted R-squared: 0.7692

F-statistic: 26.55 on 3 and 20 DF, p-value: 3.549e-07

```
anova(flower_mod3) # Regression ANOVA table
```

Analysis of Variance Table (Response: flowers)

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
intensity	1	2579.75	2579.75	59.2597	2.101e-07 ***
timing	1	886.95	886.95	20.3742	0.0002119 ***
intensity:timing	1	0.58	0.58	0.0132	0.9095675
Residuals	20	870.66	43.53		

- Is the regression model significant?
- Is there a significant interaction between light intensity and timing? (i.e.: Does the effect of intensity on the average number of flowers per plant depend on timing?)
- Is timing significant?
- How many degrees of freedom are associated with the SSE in multiple linear regression models?

Since the interaction term is not significant, it was removed from the model, and the multiple linear regression model below was fit:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i, \quad i = 1, \dots, n.$$

In Notes 8, we will discuss methods of variable removal from, and addition to, a model with the goal of finding the “best model” in some sense. Such methods are known as **model selection**.

The output from the first-order model with both x_1 and x_2 appears below:

```
flower_mod4 <- lm(flowers ~ intensity + timing) # Fits first-order model
summary(flower_mod4)                          # Regression summary
```

```
Coefficients:      Estimate Std. Error t value Pr(>|t|)
(Intercept)      83.464167   3.273772  25.495 < 2e-16
intensity         -0.040471   0.005132  -7.886 1.04e-07
timingAt PFI     -12.158333   2.629557  -4.624 0.000146
```

Residual standard error: 6.441 on 21 degrees of freedom

Multiple R-squared: 0.7992, Adjusted R-squared: 0.78

F-statistic: 41.78 on 2 and 21 DF, p-value: 4.786e-08

```
anova(flower_mod4) # Regression ANOVA table
```

Analysis of Variance Table (Response: flowers)

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
intensity	1	2579.75	2579.75	62.181	1.037e-07
timing	1	886.95	886.95	21.379	0.0001464
Residuals	21	871.24	41.49		

- Is the model significant?

- What are the effects of the different light intensity levels? (i.e.: Is there an intensity effect on the number of flowers produced given that timing is in the model?)

- What is the effect of timing?

- Find individual 95% confidence intervals for $\beta_0, \beta_1, \& \beta_2$.

$$\text{For } \beta_0 : \quad b_0 \pm t_{.975}(21) \cdot s_{b_0} = 83.464 \pm 2.080(3.274) = \underline{(76.66, 90.27)}.$$

$$\text{For } \beta_1 : \quad b_1 \pm t_{.975}(21) \cdot s_{b_1} = -0.0405 \pm 2.080(0.0051) = \underline{(-0.0511, -0.0298)}.$$

$$\text{For } \beta_2 : \quad b_2 \pm t_{.975}(21) \cdot s_{b_2} = -12.158 \pm 2.080(2.630) = \underline{(-17.627, -6.690)}.$$

These confidence intervals can be found using `confint(flower_mod4)` in **R**.

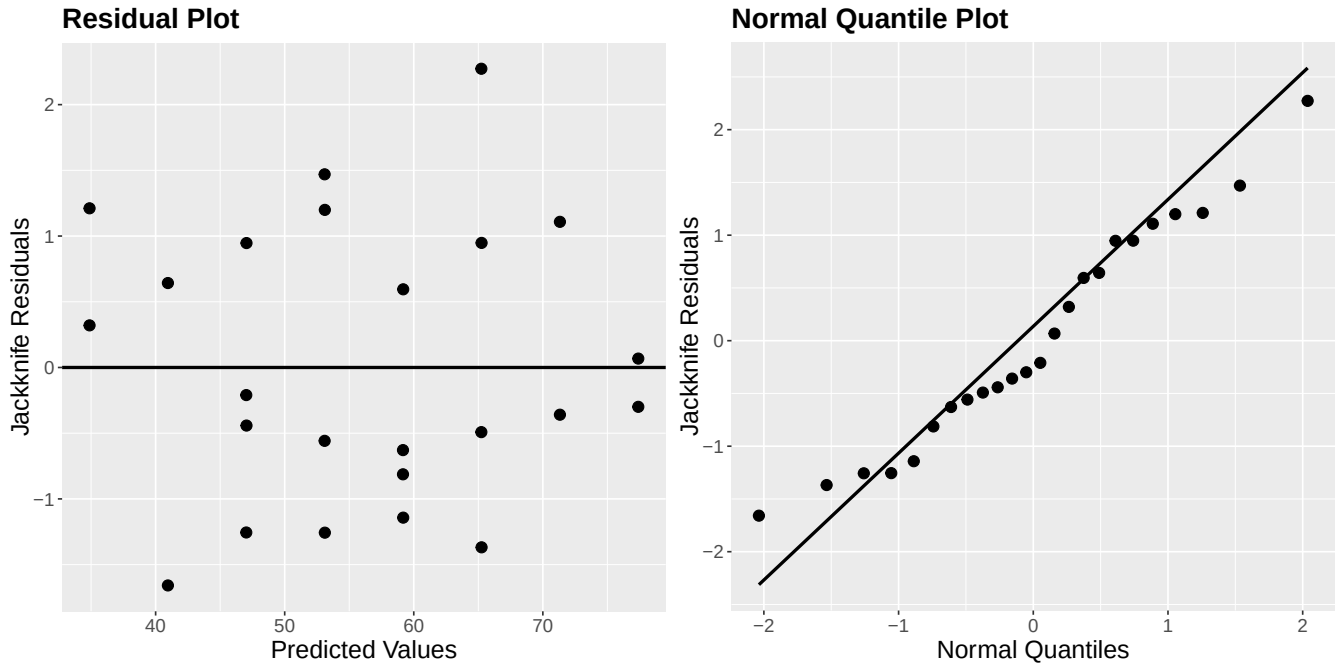
- How much did the R^2 change when the interaction term was dropped from the model? What does this mean?

- How did the SSE, SSR, and SST change between the two models? What do these changes mean?

- Can the inferences made above be trusted? How can we tell?

- If these inferences are valid, can we establish a causal relationship between the average number of flowers per plant and the intensity and timing of the light treatment? Why or why not?

To investigate the assumptions underlying the inferences made above, a residual plot and normal quantile (QQ) plot were created via **R**. Both plots were constructed using the jackknife residuals in the first-order model.



- In viewing the residual plot, there does not appear to be any departure from the homogeneous variance assumption on the errors.
- There is some S-shaped pattern in the normal quantile plot, but nothing to get too excited about with just 24 data values.

Computing Confidence and Prediction Intervals

- To get confidence intervals for the mean number of flowers or prediction intervals for the number of flowers for the intensities and timings used in the data, we can issue the following commands:

```
predict(flower_mod4, interval = "confidence") # CIs for data values
predict(flower_mod4, interval = "prediction") # PIs for data values
```

Confidence Intervals				Prediction Intervals			
	fit	lwr	upr		fit	lwr	upr
1	65.23512	59.66986	70.80038	1	65.23512	50.73006	79.74017
2	65.23512	59.66986	70.80038	2	65.23512	50.73006	79.74017
3	59.16440	54.61256	63.71625	3	59.16440	45.01719	73.31162
4	59.16440	54.61256	63.71625	4	59.16440	45.01719	73.31162
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
23	47.03988	41.47462	52.60514	23	47.03988	32.53483	61.54494
24	47.03988	41.47462	52.60514	24	47.03988	32.53483	61.54494

- Suppose we want to compute a confidence interval or prediction interval for a new data value. For example, suppose we want a confidence and prediction interval for a data value where the light intensity is 400 and the timing is “at PFI” (timing = 1). This can be done using the following **R** code.

```
xval <- data.frame(intensity = 400, timing = "At PFI") # Data frame of x-values
conf <- predict(flower_mod4, xval, interval = "confidence") # CI for (400, 1)
conf # Confidence interval
```

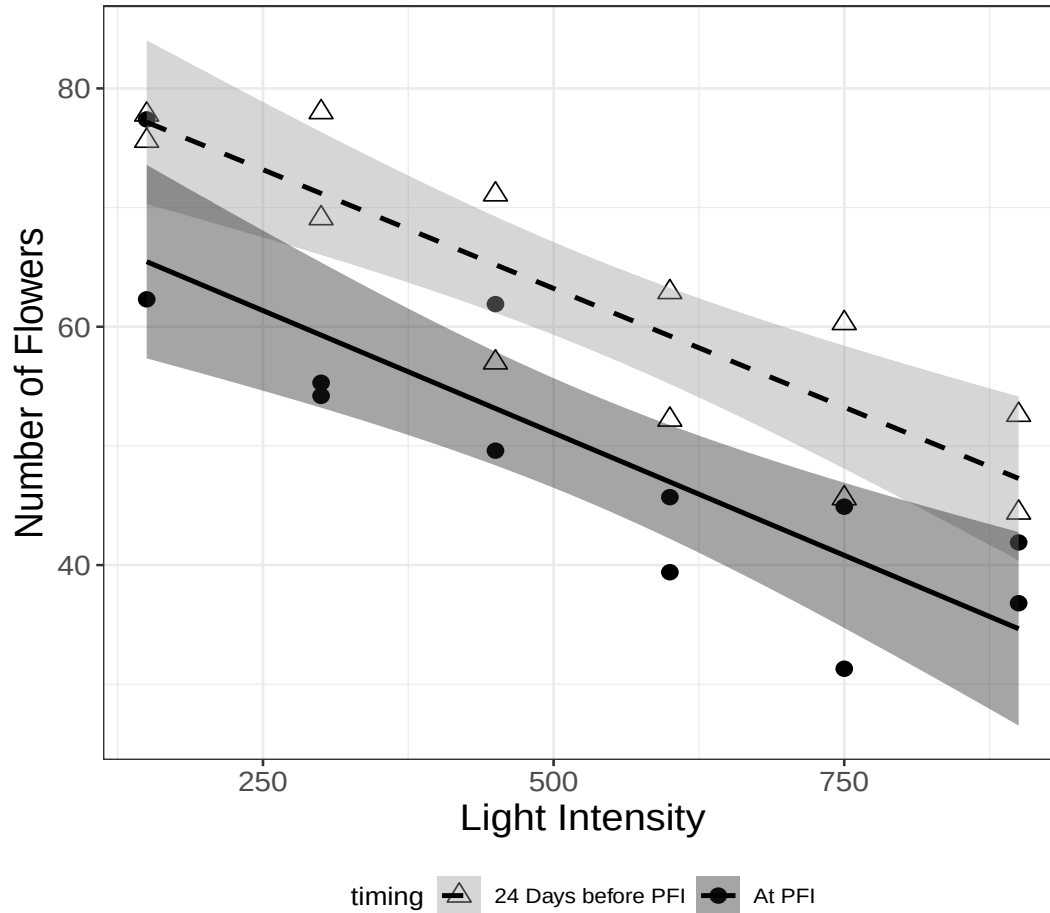
```
      fit      lwr      upr
1 55.11726 51.02678 59.20774
```

```
pred <- predict(flower_mod4, xval, interval = "prediction") # PI for (400, 1)
pred # Prediction interval
```

```
      fit      lwr      upr
1 55.11726 41.11167 69.12285
```

The code for plotting the confidence and prediction intervals bands is very similar to the code for simple linear regression in Notes 6 and can be found in the Notes 7 Script. The prediction bands were not plotted to avoid cluttering the overall picture.

Confidence Bands for Both Timings



Notes 8: Model Selection and Diagnostics

Table of contents

Model Development	137
Variable Selection Criteria	138
1. Smallest MSE (s_e^2)	138
2. Highest R^2	139
3. Highest R_{adjusted}^2	139
4. Smallest Akaike's Information Criterion (AIC)	139
5. Smallest Bayesian Information Criterion (BIC)	139
Fundamental Problem with All Selection Criteria	140
Subset Selection for Regression Models	141
Backward Elimination	141
Forward Selection	141
Stepwise Selection	142
Finding the Best-Fitting Model: An Example	143
Model Diagnostics	148
Diagnostics: Residual Statistics	148
Diagnostics: Influence Statistics	150

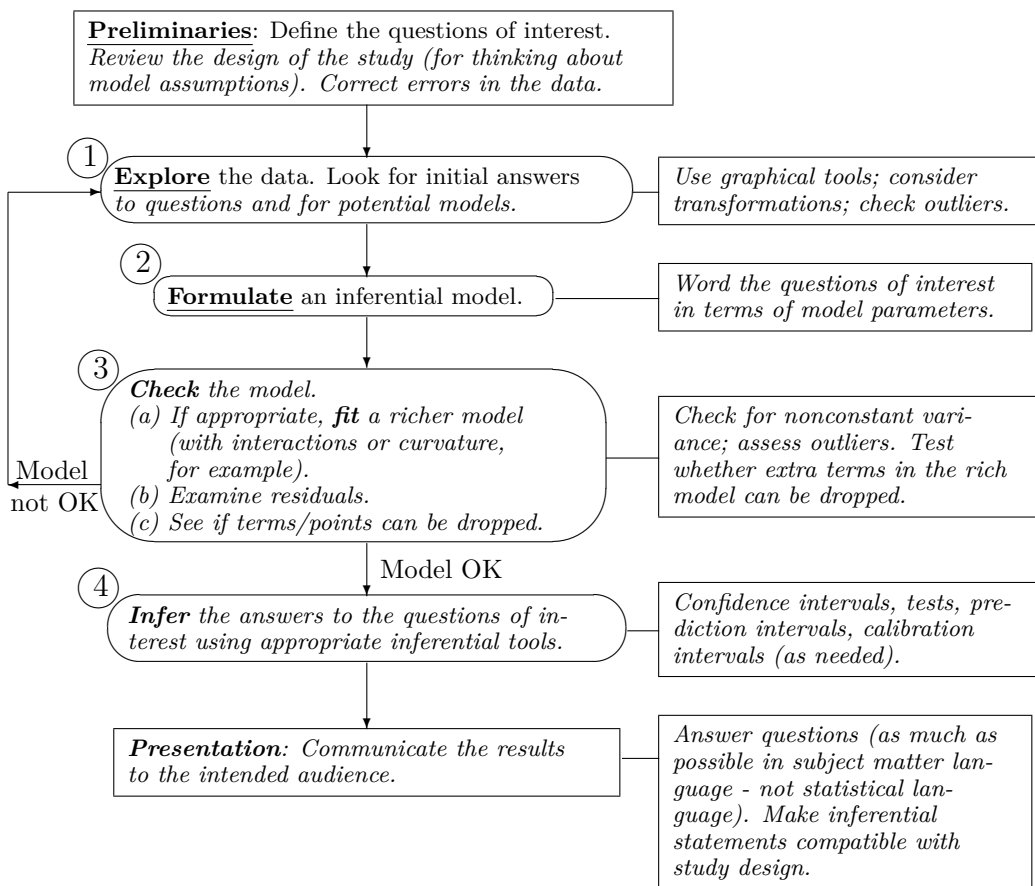
Model Development

- In practice, there is much more to regression than merely fitting a model, looking at the R^2 -value and model F -test, estimating parameters, and making predictions. In general, regression model development can be broken up into three distinct steps:.

3 Steps in Regression Model Development

1. **Variable Selection:** Identifying the response variable y (or several y 's if we want multiple models) and all candidate explanatory variables $x_1, \dots, x_p$, as well as any higher-order terms and interactions involving these variables.
2. **Model Formation:** Fitting and comparing models to determine a “best” model or set of competing “best” models.
3. **Model Diagnostics:** Checking for problems in a model.
 - (a) **Residual Analysis:** Identifying outliers and assessing model assumptions, testing for model lack of fit.
 - (b) **Influence Statistics:** Identifying observations which affect the model greatly in terms of model predictions or estimates of specific parameters.

A Strategy for Data Analysis using Statistical Models



Variable Selection Criteria

Here, we want to clearly identify the response variable y and possibly many candidate explanatory variables (the x 's).

Example: Suppose we are studying differences in the abundance of several bird species in three different forest habitats. Perhaps these “habitats” represent different levels of prescribed burns in some area.

- An experiment is done where the number of birds of each species type heard within a 100-meter radius in a 10-minute period is counted. This is done for many points in a given habitat.
- Response variables?
- Explanatory variables?
- Suppose we decide on 8 candidate explanatory variables $x_1, \dots, x_8$. How many possible 1st-order models are there?
- How many first-order models are possible with 20 variables?
- Even if we have the software to fit so many models, do you see any problem with doing that? Suppose we fit a “lot” of models and find the “best” model according to some criterion. Are we done? Would this model necessarily provide a good fit?

Criteria for Finding a “Best” Model: Below are given *some* commonly used criteria for identifying “best” models among a set of competing linear models. Most of these use some statistic that measures quality of fit, while penalizing the model according to how many variables it has. In other words, these **model selection criteria** attempt to balance the size of the errors in a model with the principle of parsimony, the idea that smaller models that are easier to interpret are better than complicated models that predict better.

1. Smallest MSE (s_e^2)

Select the model with the **smallest** estimated error variance. Since this measures on average how large the error associated with a prediction is, it provides a reasonable criterion by which models can be compared. Does it penalize overparameterized models?

2. Highest R^2

Select the model with the **largest** R^2 . Problem?

3. Highest R^2_{adjusted}

Select the model with the **largest** R^2_{adjusted} .

$$\begin{aligned} \underline{R^2_{\text{adjusted}}} &= 1 - \frac{\text{MSE}}{\text{MST}} = \\ &= \end{aligned}$$

4. Smallest Akaike's Information Criterion (AIC)

Select the model with the **lowest** AIC value.

$\text{AIC} = n \ln(\text{SSE}/n) + 2(p + 1)$, where SSE is the error sum of squares for the model under consideration.

- The model with the smallest AIC is considered “best” among competing models.
- This method has a sound basis in likelihood theory (and thus assumes that the distribution of the errors is known).
- In recent years, AIC as a tool for model selection has become the principal means by which models are compared in many fields, most notably ecology. The use of this tool has bordered on *dogma* and some journals seem only to accept papers that have followed the proper protocol for using AIC as spelled out in the text by Burnham and Anderson (“Model Selection and Multi-Model Inference”, 2002). I would argue against the use of AIC as the *only* tool examined and argue for the use of several different criteria in performing model selection (i.e.: don’t put all your eggs in one basket!).
- The AIC statistic generally is very good at distinguishing between the adequacy of different models, but not always. The bottom line is that AIC is but one tool for model selection (a very good tool), but there are many other tools available as well, some of which are refinements of AIC for specific situations.

5. Smallest Bayesian Information Criterion (BIC)

Select the model with the **lowest** BIC value.

$\text{BIC} = n \ln(\text{SSE}/n) + (p + 1) \ln(n)$.

- As with AIC, this function places a penalty on the first term, but replaces the “2” with $\ln(n)$. Some prefer the use of BIC because it places a heavier penalty on model complexity.

Some Other Criteria: AIC_c , Information Complexity Criterion (ICOMP), etc.

Fundamental Problem with All Selection Criteria

The real goal of assessing model fit is to find the model that is the “best predicting” model for *future* y -values, but we don’t know these future values. All we have are the sample data that were used to construct the model in the first place.

- To address this, consider the following idea. Suppose we fit a model using a subset of the data, then validate the model using the remainder of the data. This technique is known as **model crossvalidation**. The big question is how to split the data into a model development group and a model validation group.
- One technique for performing crossvalidation is called “leave-one-out crossvalidation”, as outlined in what follows. Suppose we remove the i^{th} observation, then fit the model with the remaining $(n - 1)$ observations, and repeat this for all n observations, one at a time, to produce n total model fits. Let:

$$y_i^* = \text{the estimated } i^{th} \text{ observation from model } i.$$

Computing the prediction error $y_i - y_i^*$ for each fit, $i = 1, \dots, n$, the **predicted error sum of squares (PRESS)** statistic is given by:

$$\text{PRESS} = \sum_{i=1}^n (y_i - y_i^*)^2,$$

a measure of the level of agreement between the true and independently predicted y -values in the data. The model(s) with the lowest PRESS statistics are considered best. To assess the actual quality of these predictions, one could compute the correlation between the true and crossvalidated y -values.

- In a 2002 paper by Boyce et al., a method known as **k-Fold Crossvalidation** was introduced. The paper outlined a technique to determine the optimal percentage of cross-validation data to leave out in fitting the model. Under this setup, we might leave out 5% of the data and use the remaining 95% to develop a model. We could then predict the 5% that were removed and validate the predictive quality of the model by comparing these predicted y -values with the actual removed values. In practice, the most common choice of *folding* seems to be a 10%-fold, whereby 10% of the data are omitted, the model is fit with the other 90%, and this is repeated 10 times for a different 10% of the data each time.

R Notes: After running a linear regression model with the `lm()` function and storing the output in an object called `reg_mod`, we can calculate the above statistics with the following functions. All of these will be demonstrated with an example later in this handout.

1. `summary(reg_mod)`: Computes MSE, SSE, R^2 , and R_{adj}^2 .
2. `extractAIC(reg_mod)`: Computes the AIC value.
3. `extractAIC(reg_mod, k = log(n))`: Computes the BIC value. You need to enter n .
4. `PRESS(reg_mod)`: Computes the PRESS statistic (`PRESS()` is in the MPV library).

Subset Selection for Regression Models

As we saw when considering a group of candidate explanatory variables, there are generally too many models to consider even for a relatively small number of variables. The topic of subset selection for multiple linear regression models deals with methods for reducing the number of models to consider. Three of these methods are presented below, and it should be emphasized that none of these guarantees that the “best” model based on some criterion will be found. But each is a practical tool for reducing the number of models examined in the course of searching for the “best” model.

The three most common types of subset selection methods employed (all supported in **R**) are outlined below.

Backward Elimination

- (a) Fit a model with all of the candidate variables $x_1, \dots, x_p$ and any higher order terms which might be important.
- (b) Remove each individual term while all other terms are in the model and calculate the AIC (or BIC) value for each slightly smaller model. Eliminate the term that gives the model with the smallest AIC (or BIC).
- (c) Refit the model without the eliminated term and continue eliminating terms until the AIC (or BIC) no longer gets reduced when removing a variable.
 - This approach to model fitting is known as the “saturated” model approach, where we saturate the model with all terms and remove those that are insignificant relative to the presence of the others.
 - This process is automated in **R** using the `step` function with the option `direction = "backward"`. Unfortunately, **R** doesn’t pay attention to the meaning of terms so may drop a lower-order term in the presence of a higher-order term.

Forward Selection

- (a) Fit the intercept model $y = \beta_0 + \varepsilon$ first.
- (b) Add terms one-by-one to the model according to which candidate term reduces the AIC (or BIC) the most. Again, higher-order terms for which not all of the corresponding lower-order terms are not already in the model should not be considered.
- (c) Continue adding terms to the model until there is no term that can be added that reduces the AIC (or BIC).
 - This method is very difficult to conduct without supporting software, as one would have to perform model fits of every candidate term remaining at each step and examine the significance of that term. Fortunately, **R** will do this for us automatically using the `step()` function with the option `direction = "forward"`. Again, **R** doesn’t pay attention to the meaning of terms so may add a high-order term without the corresponding lower-order terms present.

Stepwise Selection

This method is really a hybrid of the previous two, involving both forward and backward components to the selection.

- (a) Fit the intercept model $y = \beta_0 + \varepsilon$ first.
- (b) Add terms one-by-one to the model as with forward selection, but after the addition of each variable, verify that all variables then in the model should still be in the model. If the AIC (or BIC) gets smaller when removing a variable that was previously added, the variable should be removed before attempting to add other candidate terms. As with the other methods, lower-order terms in the presence of higher-order terms should not be considered for removal.
 - Stepwise selection can be automated in **R** using the `direction = "both"` option in the `step()` function.
 - What might cause a term already in the model to become insignificant (or lead to an increase in the AIC or BIC) with the addition of another term?

Having now discussed several criteria for trying to determine the “best-fitting” regression model for a response variable and a candidate list of explanatory variables, let’s do an example demonstrating the use of these criteria.

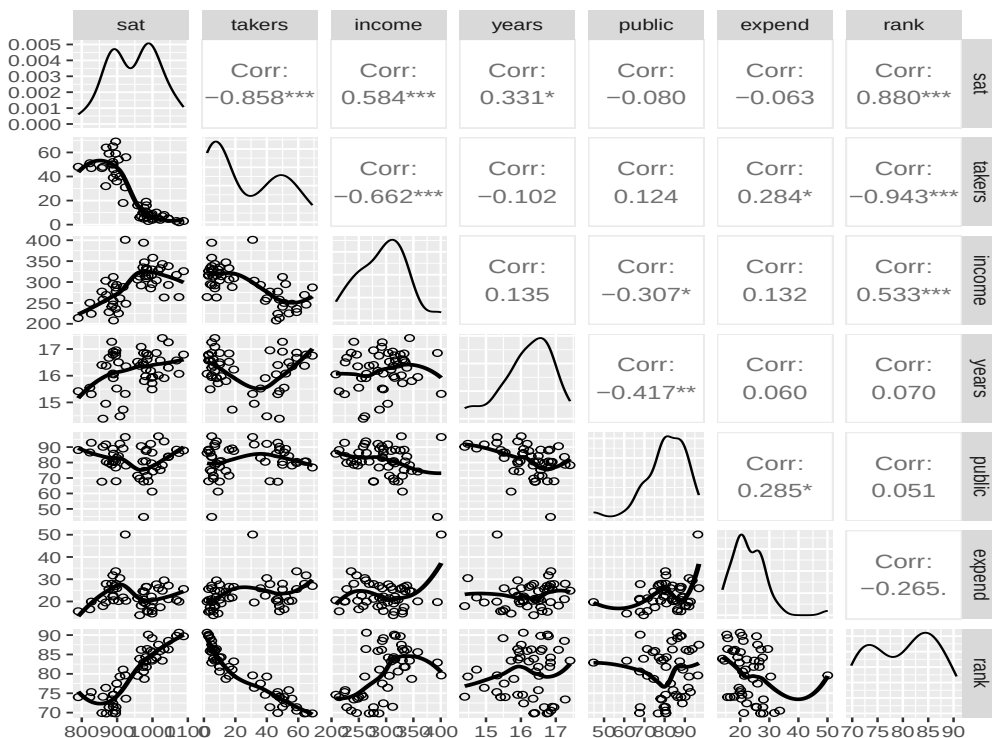
Finding the Best-Fitting Model: An Example

Example: Data were collected for each of the 50 states on the average SAT score and a number of other variables. The reason for collecting the other variables is to help explain the discrepancy between states' SAT averages. For example, many midwestern states and some western ones (including Utah) have much higher SAT scores than other regions. A closer look reveals that this difference is due primarily to the fact that only the better students in these states actually take the SAT exam. Hence it is important to examine what factors affect the average SAT scores for each state. Some of the variables considered as “explanatory” variables were:

1. Percentage of eligible students who took the exam (TAKERS)
2. Median income of families of test-takers (INCOME)
3. Average number of years of study in social science, natural science, and humanities among the test-takers (YEARS)
4. Percentage of test-takers in public schools (PUBLIC)
5. State expenditures in hundreds of dollars per student (EXPEND)
6. Median percentile ranking of test-takers within their schools (RANK).

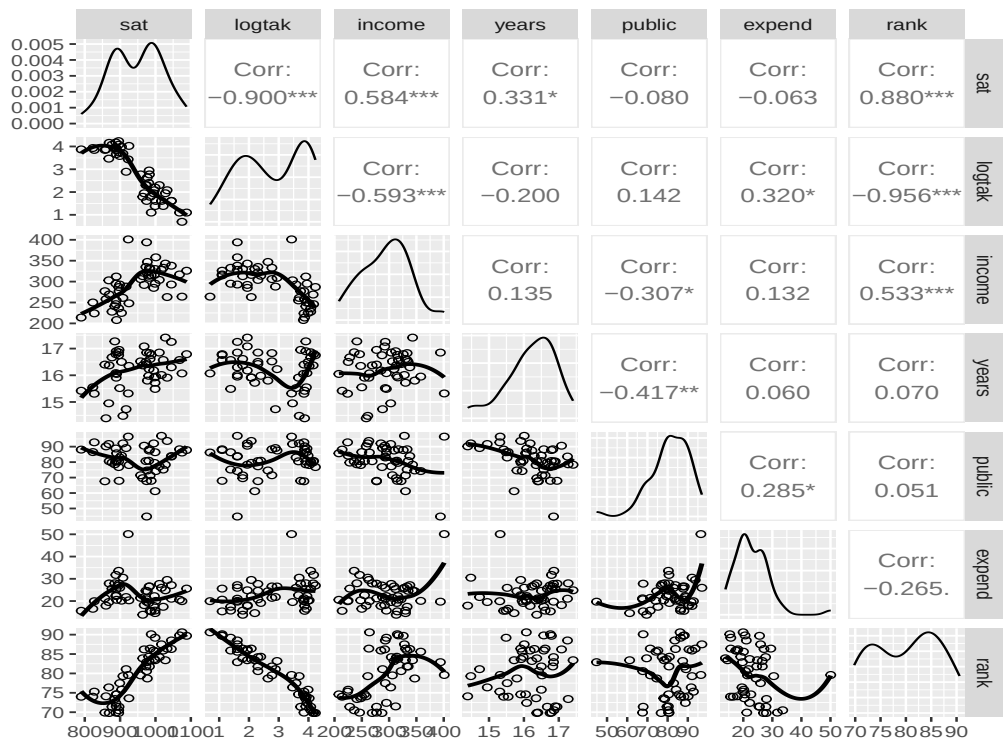
Before fitting any models, it is a good idea to study the relationships between all pairs of variables. A matrix of all pairwise scatterplots along with correlations appears below.

What do you notice?
What variables seem to be related to SAT score?



Because the variable TAKERS appears to have a nonlinear relationship with SAT score, a log transformation on the TAKERS values was performed, giving the following updated matrix of scatterplots and correlations:

What do these plots and corresponding correlations indicate about LOGTAK?



In order to identify the best-fitting model, I will demonstrate the use of the backward elimination selection procedure, where the initial model will contain all of the first-order explanatory variables (TAKERS is now on a log scale) and SAT score as the response variable. The sequence of tables of coefficients with each subsequent model follow. From table to table, I am simply removing that variable that would lead to the largest decrease in AIC (remember that lower AIC is better).

```
sat2 <- math3150package::sat |>
  mutate(takers = log(takers)) |>
  select(!state) |>
  rename(logtak = takers)
sat_full <- lm(sat ~ logtak + income + years + public + expend + rank,
              data = sat2)
summary(sat_full)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	407.53990	282.76325	1.441	0.15675
logtak	-38.43758	15.95214	-2.410	0.02032 *
income	-0.03588	0.13011	-0.276	0.78407
years	17.21811	6.32007	2.724	0.00928 **
public	-0.11301	0.56239	-0.201	0.84168
expend	2.56691	0.80641	3.183	0.00271 **
rank	4.11427	2.50166	1.645	0.10734

```
step(sat_full)
```

Start: AIC=327.8

```
sat ~ logtak + income + years + public + expend + rank
```

	Df	Sum of Sq	RSS	AIC
- public	1	25.0	26610	325.85
- income	1	47.0	26632	325.89
<none>			26585	327.80
- rank	1	1672.2	28257	328.85
- logtak	1	3589.6	30175	332.14
- years	1	4588.8	31174	333.77
- expend	1	6264.4	32850	336.38

Step: AIC=325.85

```
sat ~ logtak + income + years + expend + rank
```

	Df	Sum of Sq	RSS	AIC
- income	1	26.6	26637	323.90
<none>			26610	325.85
- rank	1	1918.1	28528	327.33
- logtak	1	4249.6	30860	331.26
- years	1	5452.8	32063	333.17
- expend	1	7430.3	34040	336.16

Step: AIC=323.9

```
sat ~ logtak + years + expend + rank
```

	Df	Sum of Sq	RSS	AIC
<none>			26637	323.90
- rank	1	2225.4	28862	325.91
- logtak	1	5071.4	31708	330.62
- years	1	5743.5	32380	331.66
- expend	1	9065.8	35703	336.55

```
sat_step_mod <- lm(sat ~ logtak + years + expend + rank, data = sat2)
summary(sat_step_mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	388.425	253.253	1.534	0.132094
logtak	-38.015	12.987	-2.927	0.005350 **
years	17.857	5.733	3.115	0.003198 **
expend	2.423	0.619	3.914	0.000305 ***
rank	4.004	2.065	1.939	0.058793 .

- As we saw along the way, backward elimination would have identified the model with $\log(\text{TAKER})$, EXPEND , YEARS , and RANK as explanatory variables as the best model (using AIC).
- In **R**, the three model selection procedures can be automated using the `step()` function by selecting "both", "backward", or "forward" as the `direction` within the `step()` function. This function can compare models based on the AIC or BIC criterion (by putting `k = log(n)` where `n` is the sample size).
- Some people will use p-values rather than AIC or BIC to pick the best model. That usually results in a similar model chosen by AIC, though it is more obviously p-hacking.

We have looked at several methods for determining the so-called “best” model according to various criteria. The table below reports these statistics for the 6 models fit ($n = 50$ for all models). PRESS values were computed using the `PRESS()` function in the `MPV` library (both `PRESS` and `MPV` are capitalized).

Models	p	MSE	R^2	R^2_{adj}	SSE_p	AIC	BIC	PRESS
P,I,R,Y,E,L	7	618.26	0.892	0.877	26585.2	327.80	341.19	45936.23
I,R,Y,E,L	6	604.78	0.892	0.880	26610.2	325.85	337.32	43274.76
R,Y,E,L	5	591.93	0.892	0.882	26636.8	323.90	333.46	39269.62
Y,E,L	4	627.44	0.883	0.875	28862.2	325.91	333.56	39819.11
E,L	3	693.41	0.868	0.862	32590.1	329.99	335.72	44623.00
L	2	969.89	0.811	0.807	46554.5	345.82	349.64	50754.03

- P = percentage of the test-takers who attended public schools,
- I = median income of the families of test-takers,
- R = median percentile ranking of the test-takers within their classes,
- Y = avg. # years of study in social & pure science, humanities among the test-takers,
- E = state expenditures per student in hundreds of dollars, and
- L = log percentage of test-takers among eligible students.

According to these statistics and these groups of models, which model would you classify as best? Why?

- Note that this is only 6 of the 63 possible first order models. So there are other models we could consider. The backward elimination procedure has the effect of limiting the number of models to examine. But there is no guarantee that we aren't missing the best model in this group.
- **Warning!** When using stepwise model selection like was illustrated here, we should be very careful about the resulting model. The p-values and estimates of the remaining variables will be biased toward being more significant than they really are. That means that inference will be suspect. There are other methods for model selection like LASSO and using domain knowledge. Stepwise is still used in many disciplines (often because they don't know it leads to biased models) and knowledge of it is important.

One aspect of model selection we have ignored in examining these data is that of what questions we are seeking to answer. If we simply want to look for variables which are good predictors of SAT score, then perhaps we have done the right thing.

Suppose the question we are interested in is actually:

After accounting for the percentage of students who took the test ($\text{Log}(\text{TAKERS})$) and the median class rank of the test-takers (RANK), which variables are important predictors of state SAT scores?

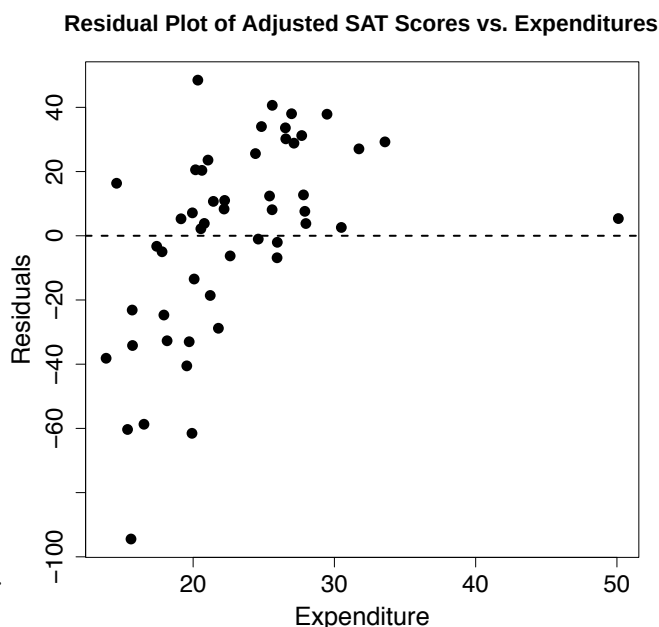
Here, we force the variables TAKERS and RANK to be in the model, and then examine the contributions of the other variables. Suppose instead we want to ask:

After accounting for the percentage of students who took the test ($\text{Log}(\text{TAKERS})$) and the median class rank of the test-takers (RANK), which states performed best for the amount of money they spend?

To answer this question, consider the following idea. First, fit the regression model involving the TAKERS and RANK variables. What do the resulting residuals tell us? The residuals are the difference in the observed SAT scores and those predicted by the variables TAKERS and RANK. A positive residual means the SAT score is higher than predicted and a negative residual means it is lower than predicted based on these 2 variables. Consider the plot below of these residuals versus the expenditure (EXPEND) values, with a regression line fit through these data.

What does this plot tell us? Points falling above the line indicate SAT scores higher than expected for the expenditures for that state.

So we could rank the states according to their residual values from this regression fit.



- These residual plots are sometimes referred to as *partial* residual plots. However, a “true” partial residual plot plots the $(e_i + b_i x_i)$ vs. x_i to examine the effect of the i^{th} variable on the model. We won’t discuss these in this class.
- A second type of diagnostic plot to look at the potential impact of a new variable x_j given that x_i is currently in the model is known as a **partial regression plot**. This plots the residuals from the model with x_i against the residuals resulting from regressing x_j on x_i . That is, it examines the relationship between the response y and x_j after adjusting both variables for the effects of x_i .

Model Diagnostics

The term “model diagnostics” refers to methodology for examining whether or not there are problems in a model. We already know how to use plots to assess the model assumptions. Now we will look at those involving specific observations (outliers & influential points).

In multiple linear regression, there are two basic types of troublesome observations: outliers and influential observations. We have seen that scatterplots, residual plots, and partial residual plots are useful visual tools for identifying outliers. We now examine some numerical means of detecting outliers (residual statistics) and influential observations (influence statistics).

Diagnostics: Residual Statistics

We have been primarily using the Studentized deleted (jackknife) residuals in this class because these residuals, denoted $\hat{r}_i$, follow a Student’s t -distribution with $n - p$ degrees of freedom and are independent of one another.

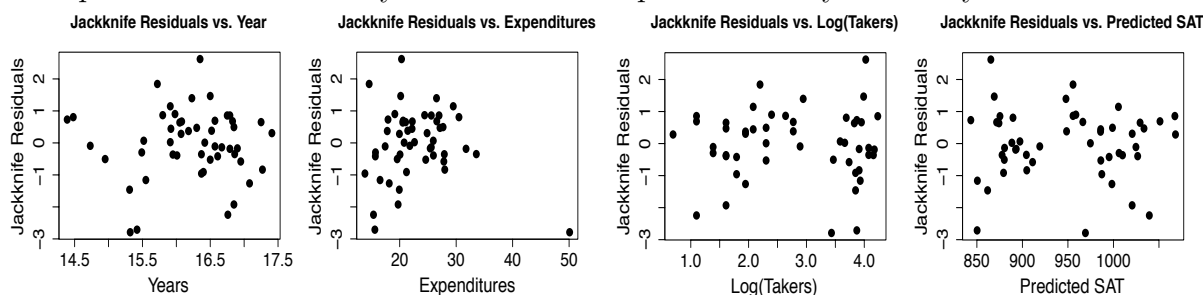
- With a distribution attached to the $\hat{r}_i$ ’s, we can compute the exact probability that a particular observation falls t units from the expected 0 line.
- Keep in mind that with 20 observations, we would expect, simply by chance, that 1 of the 20 observations would be greater than 2 standard deviations from 0 (for approximately normal values).

Example: Recall the data on states’ SAT scores and a number of explanatory variables used to examine differences in the SAT scores between states. Earlier, we found that an adequate linear regression model relating SAT score to the list of candidate explanatory variables was given by:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \varepsilon, \text{ where:}$$

- y = SAT score,
- x_1 = log percentage of eligible students who took the exam (**LOGTAKE**),
- x_2 = average number of years of study in social science, natural science, and humanities among the test-takers (**YEARS**), and
- x_3 = state expenditures in hundreds of dollars per student (**EXPEND**).

Below appear the resulting residual plots, using the Studentized deleted (jackknife) residuals, as computed in **R**. What do you notice? What points would you classify as outliers?



Below is a list of all 50 states and their corresponding jackknife residuals. How might you conduct a test of whether or not Alaska is an outlier?

Obs.	State	y SAT	x_1 YEARS	x_2 EXPEND	x_3 LOGTAK	JRES
1	IO	1088	16.79	25.60	1.10	.859
2	SD	1075	16.07	19.95	.69	.283
3	ND	1068	16.57	20.62	1.10	.692
4	KS	1045	16.30	27.14	1.61	.471
5	NE	1045	17.25	21.05	1.61	.650
6	MT	1033	15.91	29.48	2.08	1.144
7	MN	1028	17.41	24.84	1.95	.306
8	UT	1022	16.57	17.42	1.39	-.108
9	WY	1017	16.01	25.96	1.61	-.393
10	WI	1011	16.85	27.69	2.30	.493
11	OK	1001	15.95	20.07	1.61	-.363
12	AR	999	15.49	15.71	1.39	-.298
13	TN	999	15.72	14.58	2.20	1.841
14	NM	997	15.92	22.19	2.08	.443
15	ID	995	16.18	17.80	1.95	.371
16	MS	988	16.76	15.36	1.10	-2.244
17	KY	985	16.61	15.69	1.79	-.418
18	CO	983	16.83	26.56	2.77	.676
19	WA	982	16.23	26.53	2.94	1.397
20	AZ	981	15.98	19.14	2.40	.902
21	IL	977	15.80	24.41	2.64	.865
22	LA	975	16.85	19.72	1.61	-1.926
23	MO	975	16.42	20.79	2.30	.010
24	MI	973	16.50	24.61	2.30	-.526
25	WV	968	17.08	18.16	1.95	-1.263
26	AL	964	16.37	13.84	1.79	-.958
27	OH	958	16.52	21.43	2.77	.381
28	NH	925	16.35	20.33	4.03	2.619
29	AK	923	15.32	50.10	3.43	-2.793
30	NV	917	14.73	21.79	2.89	-.091
31	OR	908	14.48	30.49	3.69	.805
32	VT	904	16.50	20.17	3.99	1.465
33	CA	899	15.52	25.94	3.58	.067
34	DE	897	16.95	27.81	3.74	-.581
35	CT	896	16.75	26.97	4.23	.855
36	NY	896	16.86	33.58	4.08	-.352
37	ME	890	16.05	20.55	3.83	.638
38	FL	889	15.91	22.62	3.66	.024
39	MD	889	16.90	25.41	3.91	-.171
40	VA	888	16.08	22.23	3.95	.667
41	MA	888	16.79	31.74	4.17	-.187
42	PA	885	17.27	27.98	3.91	-.835
43	RI	877	16.67	25.59	4.08	-.138
44	NJ	869	16.37	27.91	4.16	-.356
45	TX	868	14.95	19.55	3.47	-.504
46	IN	860	14.39	17.93	3.87	.732
47	HI	857	16.40	21.21	3.85	-.912
48	NC	827	15.31	19.92	3.85	-1.462
49	GA	823	15.55	16.52	3.93	-1.162
50	SC	790	15.42	15.60	3.87	-2.711

Diagnostics: Influence Statistics

In simple linear regression, an outlier is a data value which does not agree with the overall pattern of the observed relationship, whereas an influential observation is a data value which greatly affects the slope of the line. In multiple linear regression, the distinction between outliers and influential points is similar, but not quite as simple. Outliers are defined as values with “large” residuals as discussed earlier. Influential observations are defined as those values which greatly influence any of the partial slopes or have a significant effect on the predicted response. This definition seems reasonable, but how do we identify such points?

A number of influence statistics have been developed to help identify potentially influential observations. Most of these operate under the same philosophy: Remove an observation and see how much the regression model changes. These methods differ only in the type of model change detected. Some look at how the predicted response values change and some at how the estimated parameters change. Below is a summary of four influence statistics which are supported by **R**.

1. **Leverage:** We discussed the leverage of a data value earlier in the course. In simple linear regression, a point was said to have high leverage if it is unusual in its x-value. Via algebra, the variance of the i^{th} residual can be shown to have the form:

$$\text{Var}(e_i|x_i) = \sigma_\varepsilon^2[1 - h_i], \text{ where } h_i = \left(\frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum (x_j - \bar{x})^2} \right) = \left(\frac{1}{n} + \frac{(x_i - \bar{x})^2}{(n-1)s_x^2} \right).$$

- The value h_i is known as the leverage of the i^{th} data value. Leverage values are always between 0 and 1.
- In general, we can write the variance of the residual vector as:

$$\text{Var}(\mathbf{e}|\mathbf{X}) = \sigma_\varepsilon^2(\mathbf{I} - \mathbf{H}), \text{ where } \mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

for design matrix $\mathbf{X}$ where the diagonal elements h_i , $i = 1, \dots, n$ of $\mathbf{H}$ are the leverages of the n data values.

- Large leverages indicate points with potentially influential, but not all points with high leverage influence the model much.
- The average leverage in the multiple linear regression model is $(p+1)/n$.
Rule of Thumb: In practice, leverage values above $2(p+1)/n$ (twice the average value) are considered to have high leverage.

2. **Cook's Distance:** Cook's D statistic (Cook, 1977) measures the overall change in the estimated y values when the i^{th} observation is removed from the data set. The Cook's Distance statistic for observation i is given by:

$$D_i = \frac{\sum_{j=1}^n (\hat{y}_j - \hat{y}_{j(i)})^2}{(p+1) \cdot MSE}$$

where $\hat{y}_{j(i)}$ denotes the predicted y value when observation i is removed from the model.

- Each observation $i = 1, \dots, n$ will have a Cook's D statistic corresponding to it.
- Cook's D statistic D_i essentially provides an overall measure of the change in the predicted values, $\hat{y}$, when the i^{th} observation is removed.
- Rule of thumb: If $D_i > 4/n$, the observation is flagged as influential.

3. **DfFits**: The DfFits statistic (Belsley, Kuh, & Welsch, 1980), denoted by DfFits_i , measures the change in the predicted response for observation i when the i^{th} observation is removed from the data set, $i = 1, \dots, n$. In other words, it measures the “standardized difference” between $\hat{y}_i$ and $\hat{y}_{(i)}$.
 - Each observation $i = 1, \dots, n$ will have a DfFits statistic corresponding to it.
 - Rule of Thumb: If $|\text{DfFits}_i| > 2\sqrt{(p+1)/n}$, the observation is flagged as influential.
4. **DfBetas**: The DfBetas statistic (Belsley, Kuh, & Welsch, 1980), denoted by $\text{DfBetas}_{j(i)}$, measures the change in the j^{th} parameter estimate when the i^{th} observation is removed from the data set, $j = 0, 1, \dots, p$, $i = 1, \dots, n$. It measures the “standardized difference” between b_j and $b_{j(i)}$, which is the estimate of β_j with the i^{th} observation removed.
 - Each observation $i = 1, \dots, n$ will have $p + 1$ DfBetas statistics corresponding to it, one for each parameter.
 - Rule of Thumb: If $|\text{DfBetas}_i| > 2/\sqrt{n}$, the observation is flagged as influential.

What do we do with influential points? The entire idea of flagging influential points is for further investigation. We **DO NOT** want to just mindlessly remove them all from the model. Usually many points will be flagged by at least one of the influence statistics. We should look into them to see if there is a reason they are unusual. Occasionally we will have points that are so different from the rest of the data that they should be removed. However, when doing that we have to be aware that the entire scope of our model needs to be altered when removing a point (unless it was simply an error).

To illustrate the use of these influence statistics, consider the regression equation for the SAT data, given by: $\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 = 851.315 + 12.833x_1 + 2.698x_2 - 62.271x_3$, where: y = SAT score, x_1 = average number of years of study in social science, natural science, and humanities among the test-takers, x_2 = state expenditures in hundreds of dollars per student, and x_3 = log percentage of eligible students who took the exam.

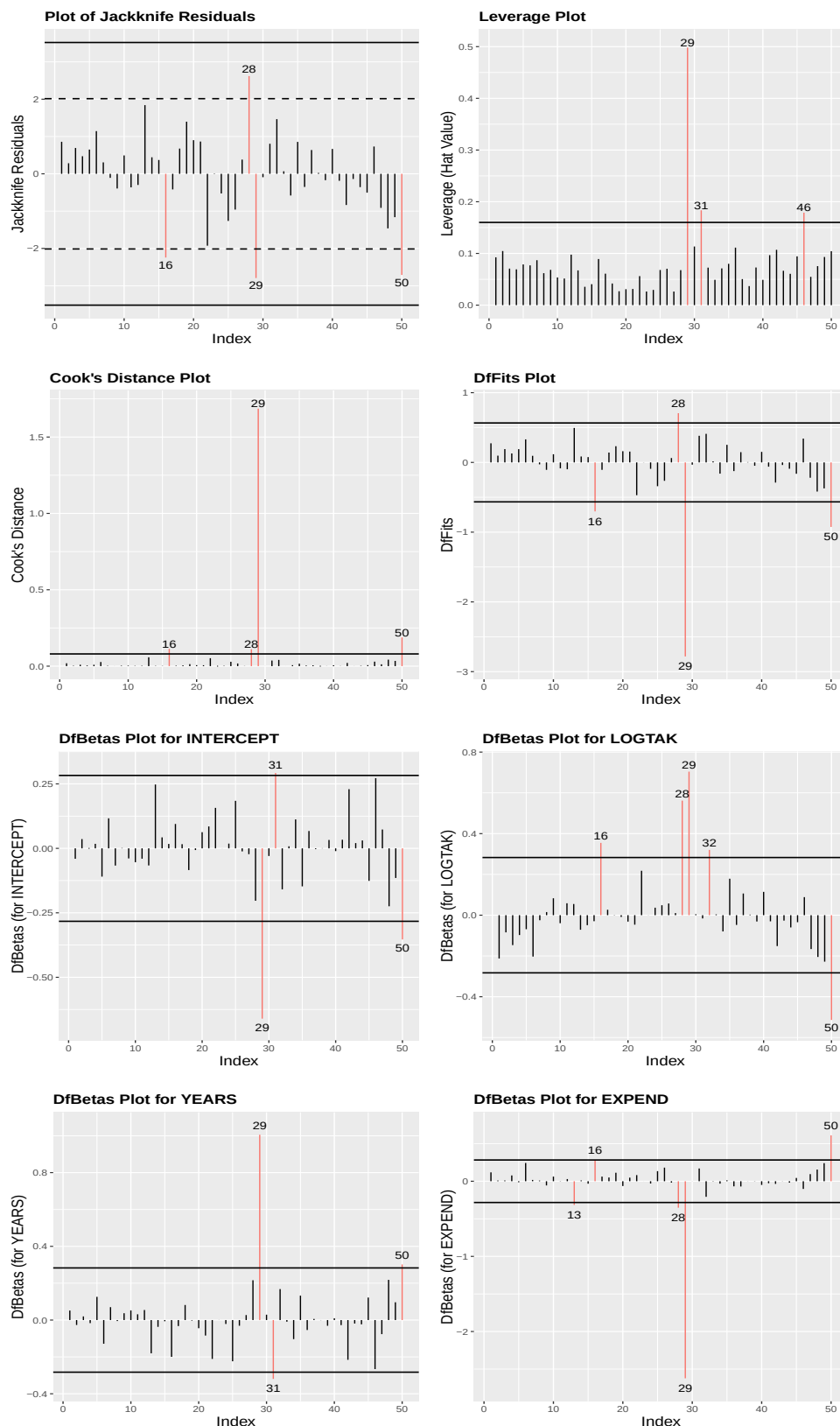
In **R**, the four influence statistics discussed can be requested using the functions `hatvalues()`, `cooks.distance()`, `dffits()`, and `dfbetas()` by typing:

```
hatvalues(model)      # Computes leverages
cooks.distance(model)  # Computes Cook's D values
dffits(model)         # Computes DfFits values
dfbetas(model)        # Computes DfBetas values
```

All of these statistics were computed for this SAT data set and are given on the next page. Using the suggested rules of thumb, which points would you consider as influential? Do these influence statistics agree reasonably well?

Obs.	State	y SAT	x_1 YEAR	x_2 EXPEN	x_3 LTAK	JRES	Lev	Cook's D	DF Fits	DFBetas			
										β_0	β_1	β_2	β_3
1	IO	1088	16.79	25.60	1.10	.859	.092	.019	.274	-.040	.052	.118	-.212
2	SD	1075	16.07	19.95	.69	.283	.105	.002	.097	.036	-.027	.009	-.084
3	ND	1068	16.57	20.62	1.10	.692	.071	.009	.191	.000	.020	.010	-.147
4	KS	1045	16.30	27.14	1.61	.471	.069	.004	.129	.018	-.017	.077	-.097
5	NE	1045	17.25	21.05	1.61	.650	.079	.009	.190	-.110	.126	-.013	-.069
6	MT	1033	15.91	29.48	2.08	1.144	.077	.027	.330	.116	-.129	.243	-.203
7	MN	1028	17.41	24.84	1.95	.306	.087	.002	.094	-.067	.070	.017	-.025
8	UT	1022	16.57	17.42	1.39	-.108	.062	.000	-.028	.001	-.005	.009	.015
9	WY	1017	16.01	25.96	1.61	-.393	.068	.003	-.106	-.039	.037	-.056	.083
10	WI	1011	16.85	27.69	2.30	.493	.054	.003	.117	-.054	.052	.062	-.039
11	OK	1001	15.95	20.07	1.61	-.363	.051	.002	-.085	-.040	.031	.003	.058
12	AR	999	15.49	15.71	1.39	-.298	.098	.002	-.098	-.066	.055	.029	.055
13	TN	999	15.72	14.58	2.20	1.841	.067	.058	.494	.248	-.180	-.315	-.071
14	NM	997	15.92	22.19	2.08	.443	.035	.002	.085	.043	-.037	.010	-.049
15	ID	995	16.18	17.80	1.95	.371	.041	.001	.076	.017	-.006	-.033	-.030
16	MS	988	16.76	15.36	1.10	-2.244	.089	.113	-.702	.095	-.200	.286	.355
17	KY	985	16.61	15.69	1.79	-.418	.061	.003	-.106	.016	-.032	.062	.027
18	CO	983	16.83	26.56	2.77	.676	.042	.005	.141	-.084	.082	.050	-.001
19	WA	982	16.23	26.53	2.94	1.397	.027	.013	.232	-.006	-.003	.113	-.008
20	AZ	981	15.98	19.14	2.40	.902	.031	.006	.161	.063	-.044	-.064	-.031
21	IL	977	15.80	24.41	2.64	.865	.031	.006	.155	.085	-.084	.048	-.047
22	LA	975	16.85	19.72	1.61	-1.926	.056	.052	-.470	.157	-.211	.081	.218
23	MO	975	16.42	20.79	2.30	.010	.026	.000	.002	.000	.000	.000	.000
24	MI	973	16.50	24.61	2.30	-.526	.029	.002	-.092	.018	-.021	-.030	.037
25	WV	968	17.08	18.16	1.95	-1.263	.068	.029	-.341	.184	-.224	.134	.049
26	AL	964	16.37	13.84	1.79	-.958	.070	.017	-.264	-.012	-.031	.181	.057
27	OH	958	16.52	21.43	2.77	.381	.026	.001	.063	-.023	.027	-.018	.010
28	NH	925	16.35	20.33	4.03	2.619	.068	.110	.706	-.203	.216	-.352	.562
29	AK	923	15.32	50.10	3.43	-2.793	.498	1.686	-2.782	-.660	1.005	-2.623	.704
30	NV	917	14.73	21.79	2.89	-.091	.113	.000	-.032	-.030	.029	-.001	.005
31	OR	908	14.48	30.49	3.69	.805	.183	.037	.382	.292	-.319	.170	-.016
32	VT	904	16.50	20.17	3.99	1.465	.073	.041	.410	-.159	.168	-.207	.320
33	CA	899	15.52	25.94	3.58	.067	.049	.000	.015	.008	-.009	.004	.004
34	DE	897	16.95	27.81	3.74	-.581	.071	.007	-.161	.112	-.103	-.032	-.080
35	CT	896	16.75	26.97	4.23	.855	.080	.016	.252	-.147	.133	.012	.179
36	NY	896	16.86	33.58	4.08	-.352	.111	.004	-.124	.067	-.054	-.068	-.048
37	ME	890	16.05	20.55	3.83	.638	.050	.005	.147	-.003	.006	-.070	.106
38	FL	889	15.91	22.62	3.66	.024	.037	.000	.005	.001	-.001	-.001	.003
39	MD	889	16.90	25.41	3.91	-.171	.073	.001	-.048	.032	-.031	.002	-.031
40	VA	888	16.08	22.23	3.95	.667	.049	.006	.151	-.011	.010	-.049	.114
41	MA	888	16.79	31.74	4.17	-.187	.097	.001	-.061	.033	-.027	-.026	-.030
42	PA	885	17.27	27.98	3.91	-.835	.107	.021	-.289	.229	-.216	-.035	-.151
43	RI	877	16.67	25.59	4.08	-.138	.066	.000	-.037	.020	-.019	.002	-.027
44	NJ	869	16.37	27.91	4.16	-.356	.061	.002	-.090	.031	-.023	-.019	-.060
45	TX	868	14.95	19.55	3.47	-.504	.094	.007	-.163	-.126	.122	.044	-.034
46	IN	860	14.39	17.93	3.87	.732	.179	.029	.342	.272	-.266	-.102	.089
47	HI	857	16.40	21.21	3.85	-.912	.055	.012	-.220	.073	-.076	.095	-.166
48	NC	827	15.31	19.92	3.85	-1.462	.075	.043	-.418	-.225	.218	.155	-.205
49	GA	823	15.55	16.52	3.93	-1.162	.093	.034	-.372	-.115	.096	.240	-.228
50	SC	790	15.42	15.60	3.87	-2.711	.104	.188	-.925	-.353	.302	.611	-.514

The graphs for these statistics are shown below. They were obtained using the `influence_plots()` function that is included with our course **R** library.



Observations 29 and 50 come up a lot. 29 especially is everywhere!

```
sat$state[c(29, 50)]
```

```
[1] "Alaska"          "South Carolina"
```

Remove Alaska and refit the model:

```
# Fit model with Alaska
sat_mod <- lm(sat ~ logtak + years + expend, data = sat2) # Model with AK
# Remove AK and refits model
sat_mod2 <- lm(sat ~ logtak + years + expend, data = sat2[-29,])
summary(sat_mod2) # Model without AK
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	904.9478	83.4627	10.843	3.88e-14	***
logtak	-64.6310	3.4580	-18.690	< 2e-16	***
years	7.8923	5.2228	1.511	0.138	
expend	4.2167	0.7942	5.309	3.26e-06	***

Residual standard error: 23.38 on 45 degrees of freedom
Multiple R-squared: 0.8998, Adjusted R-squared: 0.8931
F-statistic: 134.6 on 3 and 45 DF, p-value: < 2.2e-16

```
summary(sat_mod) # Model with Alaska
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	851.3151	87.0220	9.783	8.15e-13	***
logtak	-62.2705	3.5925	-17.333	< 2e-16	***
years	12.8334	5.2649	2.438	0.0187	*
expend	2.6982	0.6203	4.350	7.50e-05	***

Residual standard error: 25.05 on 46 degrees of freedom
Multiple R-squared: 0.8827, Adjusted R-squared: 0.875
F-statistic: 115.4 on 3 and 46 DF, p-value: < 2.2e-16

Notes 9: Logistic Regression and Classification

Table of contents

Odds and Odds Ratio	156
Logistic Regression	158
Differences Between Linear and Logistic Regression	161
How to Fit a Logistic Regression Model - Maximum Likelihood	161
Interpretation of the Logistic Regression Coefficients	164
Logistic Regression Diagnostics	166
Final Comments on Logistic Regression	167
Multinomial Logistic Regression	168
Multinomial Logistic Regression Example	169
Interpreting Coefficients in Multinomial Logistic Regression	172
Obtaining Predictions	172
Multinomial Logistic Regression Diagnostics	173

Odds and Odds Ratio

Logistic regression models are regression models for binary response variables. Before we get into how these work, we should quickly discuss the odds of an event and the odds ratio of two events.

For a probabilistic event, the odds are used to compare the chances of that event happening with the chances of that event not happening.

The odds of an outcome can be found by dividing the number of outcomes in one category to the number of outcomes not in that category. It can also be found by taking the probability of an event and dividing it by 1 minus the probability of that event:

$$\text{odds}(A) = \frac{P(A)}{1 - P(A)} \text{ “to 1”}$$

- If the odds > 1 to 1 then the outcome of interest is more likely than not.
- If the odds < 1 to 1 the outcome of interest is not as likely as everything else.
- odds = 1 to 1 implies an even split between the outcome of interest and every other possible outcome.

We say “to 1”, because the odds compares the likelihood of an event happening to that event not happening. So, if the odds of the Broncos beating the Browns are listed as “4 to 1”, we could say that we would expect the Broncos to beat the Browns 4 times for every time the Browns beat the Broncos.

Example: What are the odds that when flipping two coins, both come up heads?

Translating from odds to probability:

If the odds of an event are odds(A) = a to b , then $P(A) = \frac{a}{a+b}$.

Example: In Pokémon FireRed/LeafGreen, the odds of catching a paralyzed, level 70 Mewtwo with 1 HP if you hurl an Ultra Ball at it is 17 to 583, what is the probability of catching the Mewtwo?

Odds Ratio

The odds ratio is simply a ratio of odds (surprise!). It is usually used to compare two levels of one variable at the same level of another, but is also useful in the interpretation of a logistic regression model.

$$\text{Odds Ratio} = OR = \frac{\text{odds}(A)}{\text{odds}(B)} = \frac{\frac{P(A)}{1 - P(A)}}{\frac{P(B)}{1 - P(B)}}$$

- If the odds ratio > 1 , the odds in the numerator are bigger than the odds in the denominator.

- If the odds ratio < 1 , the odds in the denominator are bigger than the odds in the numerator.
- If the odds ratio $= 1$, the two odds are equal.

One thing to be careful of here is that we don't confuse a ratio of odds with a ratio of probabilities. For instance, an odds ratio of 4 does not mean that one event is 4 times more likely to occur than the other. It means that one event has 4 times the odds of the other.

Example: One influential example of using the odds ratio is a study comparing smoking to lung cancer. This was not an easy relationship to discover; it would be highly unethical to perform a controlled experiment studying this relationship. An alternative study type is a “case-control” study. If we want to see if lung cancer is associated with smoking, we can find a bunch of people with lung cancer (the “cases”) and a bunch of people without lung cancer (the “controls”). Then we can see how many people in each category smoke, and compute the relevant odds ratio. In 1950, epidemiologists Doll and Hill did such a study and the data are presented below.

		Cancer Status		
		Lung Cancer	No Cancer	Total
Smoking Status	Smoker	1350	1296	2646
	Non-Smoker	7	61	68
	Total	1357	1357	2714

- Note that, in this study, it is not meaningful to compute probabilities of people having cancer or not having cancer, because we intentionally sampled the same number of each. We also cannot compute probabilities of being a smoker or not being a smoker, because this is not a genuine random sample: if it is true that smoking causes cancer (spoiler alert: it is), then there will be a greater number of smokers in our sample than there are in the overall population, since we made a point of sampling the same number of “cases” as “controls”.
- This is a big advantage that the odds ratio has over simply calculating probabilities. Probabilities calculated from contingency tables are not valid if the row or column totals for the variable of interest are “fixed”, i.e. determined by a non-random process.
- Odds ratios, however, compare relative probabilities, and even if the individual probabilities are not meaningful on their own, their relative magnitudes will still be meaningful. So, even though odds ratios are more difficult to interpret, they are sometimes the only way to analyze data collected on categorical variables.

Computing and interpreting the odds ratio for cancer:

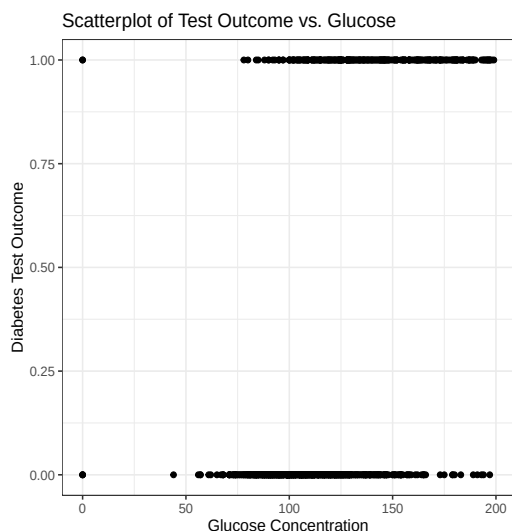
Logistic Regression

As was stated before, the response variable for logistic regression is always either 0 or 1. To see when logistic regression models might be applicable, consider the following example.

Example: The National Institute of Diabetes and Digestive and Kidney Diseases conducted a study on 768 adult female Pima Native Americans living near Phoenix. The purpose of the study was to investigate factors related to diabetes. The response variable (y) in this study (*test*) was whether or not the patient showed signs of diabetes (1 = yes, 0 = no). One explanatory variable (x) considered was the plasma glucose concentration (in mg/dL) at 2 hours in an oral glucose tolerance test (*glucose*). Many other potential explanatory variables were considered as well, but for now, we will focus just on the glucose variable.

- Additionally, we will assume that the subjects used in the study were independently selected. This assumption is critical to the use of logistic regression models. If this assumption is not satisfied (as with time series or spatial data), a class of models known as *autologistic* models may be used for predicting a binary response.
- The goal here is to predict the diabetes status or probability of showing signs of diabetes given the glucose concentration of the subject.

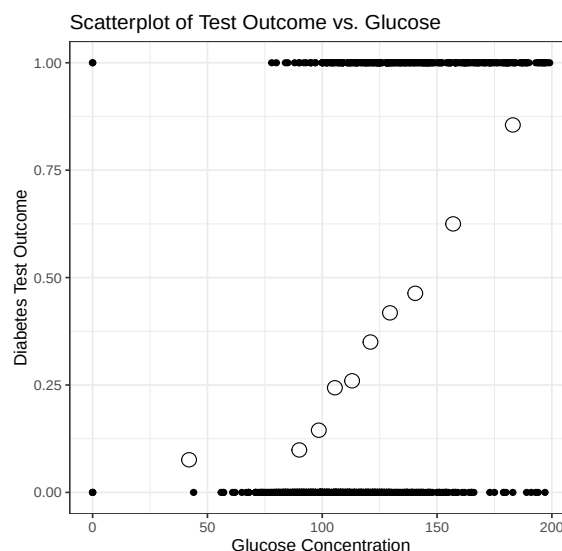
To examine the relationship between the incidence of diabetes and glucose concentration, a scatterplot was constructed below. What does this plot tell you? Anything? Would a linear regression model be appropriate here?



- Better Idea: We could categorize the glucose concentrations into groups and plot the mean diabetes incidence (proportion of subjects in that group testing positive for diabetes) vs. the glucose group. To see how this works, consider the following table and corresponding plot shown below. What do you notice about the shape of the relationship between the proportion of subjects with diabetes and glucose?

Frequency Table of Glucose Group by Diabetes

Glucose Group	n	Diabetes		Proportion with Diabetes
		Absent	Present	
0-85	79	73	6	0.076
86-95	81	73	8	0.099
96-102	76	65	11	0.145
103-109	78	59	19	0.244
110-117	77	57	20	0.260
118-125	80	52	28	0.350
126-134	67	39	28	0.418
135-147	82	44	38	0.463
148-167	72	27	45	0.625
168-200	76	11	65	0.855
Total	768	500	268	0.349



- The association between the proportion of subjects with diabetes and glucose level is somewhat S-shaped. This type of relationship is well-represented by the logistic model:

$$\pi(x) = P(Y = 1|x) = E(Y|x) = \frac{\exp\{\beta_0 + \beta_1 x\}}{1 + \exp\{\beta_0 + \beta_1 x\}}, \text{ where}$$

$\pi(x)$ represents the probability of having diabetes given a glucose concentration of x . The portion of this model in the exponent is just a linear function of the parameters β_0, β_1 . The **R**-code used to create the table and graph is given in the **Notes 9 Script.R** file.

- The logistic regression model is not the only type of function (known as a link function) used for binary data which has this S-shape. Another commonly considered model for this situation is the probit model, a model based on the distribution function of the normal distribution. This will not be discussed here.

Notes:

- $P(Y = 0|x) = \frac{1}{1 + \exp\{\beta_0 + \beta_1 x\}}$. Why?

- Consider the transformation: $\log \left[\frac{P(Y = 1|x)}{1 - P(Y = 1|x)} \right] = \log \left[\frac{P(Y = 1|x)}{P(Y = 0|x)} \right]$.

This is known as the logit transformation, and is interpreted as the log odds that $y = 1$ (diabetes) versus $y = 0$ (no diabetes) given x (glucose). In terms of the example, as glucose concentration (x) increases, we expect the log odds of having diabetes ($y = 1$) to increase as well. This logistic model specifies that this increase occur in the form of an S-shape as specified earlier.

- In addition to the simple interpretation of the logit function as a log odds ratio, this transformation has another useful property. Evaluating the logit function:

$$\begin{aligned}\boxed{\text{logit}[P(Y = 1|x)]} &= \log \left[\frac{P(Y = 1|x)}{P(Y = 0|x)} \right] = \log \left[\frac{\exp\{\beta_0 + \beta_1 x\} / (1 + \exp\{\beta_0 + \beta_1 x\})}{1 / (1 + \exp\{\beta_0 + \beta_1 x\})} \right] \\ &= \log[\exp\{\beta_0 + \beta_1 x\}] = \boxed{\beta_0 + \beta_1 x}.\end{aligned}$$

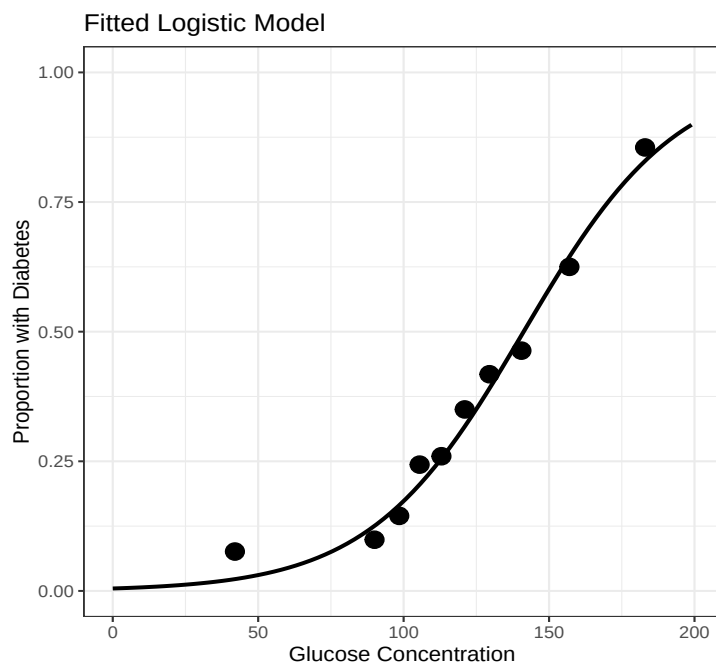
So the logit function is a linear function of the parameters. Why is this important? This means that on the logit scale, we can appeal to what we know about linear regression to interpret the model parameters.

- The equality $\boxed{\text{logit}[P(Y = 1|x)] = \beta_0 + \beta_1 x}$ is known as the logistic regression model.

To see how well this model fits the data, reconsider the categorized table and plot shown earlier on the proportion of diabetes subjects versus glucose concentration:

Frequency Table of Glucose Group by Diabetes

Glucose Group	n	Diabetes		Mean (Proportion)	Glucose	$\hat{\pi}(x)$
		Absent	Present			
0-85	79	73	6	0.076	42.5	0.023
86-95	81	73	8	0.099	90.0	0.125
96-102	76	65	11	0.145	98.5	0.165
103-109	78	59	19	0.244	105.5	0.205
110-117	77	57	20	0.260	113.0	0.255
118-125	80	52	28	0.350	121.0	0.317
126-134	67	39	28	0.418	129.5	0.390
135-147	82	44	38	0.463	140.5	0.493
148-167	72	27	45	0.625	157.0	0.645
168-200	76	11	65	0.855	183.0	0.829
Total	768	500	268	0.349		



$$\hat{\pi}(x) = \frac{\exp\{-5.35 + .0379x\}}{1 + \exp\{-5.35 + .0379x\}}$$

Differences Between Linear and Logistic Regression

1. Range of $E(Y|x)$

- In simple linear regression: $E(Y|x) = \beta_0 + \beta_1 x$. For $-\infty < x < \infty$, $E(Y|x)$ can take on values between $(-\infty, \infty)$.
- In logistic regression: Y is 0 or 1, so $E(Y|x)$ can only take on values between $(0,1)$.

2. Error Assumption

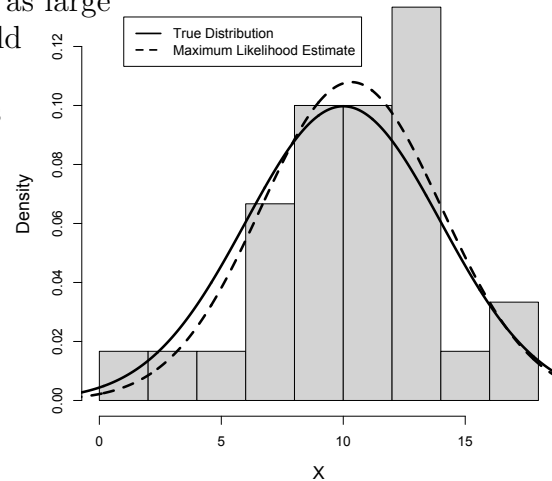
- In simple linear regression: the model is $y = E(Y|x) + \varepsilon$, where we typically assume that $\varepsilon|x \stackrel{iid}{\sim} N(0, \sigma_\varepsilon^2)$.
- In logistic regression: the model is $y = \pi(x) + \varepsilon$. Here, since Y is binary:
 - if $y = 1$, then $\varepsilon = 1 - \pi(x)$ with probability $\pi(x)$, and
 - if $y = 0$, then $\varepsilon = -\pi(x)$ with probability $1 - \pi(x)$.

So $\varepsilon|x \stackrel{iid}{\sim} \text{Bernoulli}(\pi(x))$, but are not identically distributed because the success probability depends on x .

How to Fit a Logistic Regression Model - Maximum Likelihood

- Why not use the method of least squares (as with linear regression)? Many statistical properties that hold for linear regression models do not hold for logistic regression models (e.g.: normality of $\hat{y}$, b_i , etc.).
- Maximum likelihood (ML), and its many variants, is the most popular technique for estimating parameters in statistical models. Roughly, the idea is as follows. Consider the probability of observing a particular response value y_i given some x , represented by: $P(Y = y_i|x)$. Maximum likelihood finds those parameters (say β_0, β_1 in our case) which make this probability of observing y_i as large as possible. This is what we want. We would like our estimated parameters to make the probability of observing our data is large as possible. In this way, these parameters “best” explain the observed data.

Example with a normal distribution. Imagine you have a data set that looks like the histogram on the right. ML estimation would select the parameters μ and σ such that the normal density would fit the data best.



Returning to the Diabetes/Glucose Example: The proposed model for these data is the logistic regression model given by:

$$\pi(x) = \frac{\exp\{\beta_0 + \beta_1 x\}}{1 + \exp\{\beta_0 + \beta_1 x\}} \quad \text{or} \quad \hat{\pi}(x) = \frac{\exp\{b_0 + b_1 x\}}{1 + \exp\{b_0 + b_1 x\}},$$

where x = glucose concentration, and $\pi(x) = E(Y|x)$ = the probability of diabetes at glucose concentration x . To fit the logistic regression model in **R**, we use the `glm()` function with the option `family = binomial` to select binomial (*logistic*) regression. The code to produce the logistic regression fit is given below along with the corresponding output.

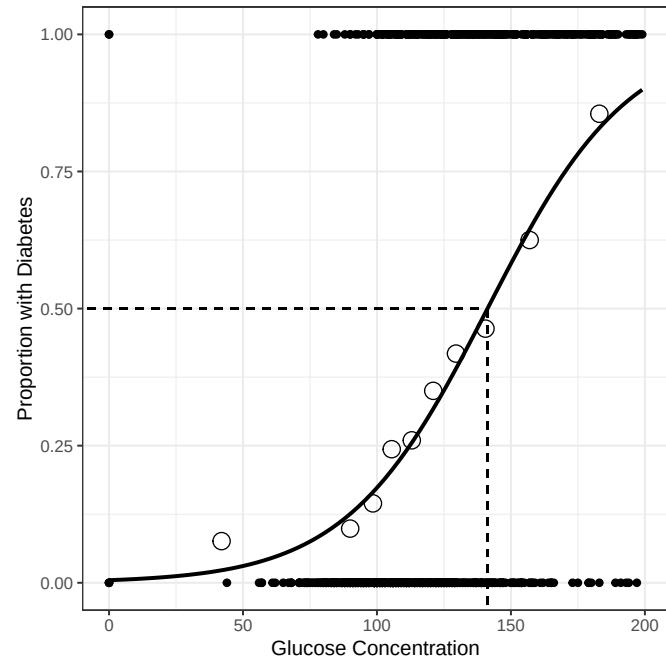
```
# Fits a logistic regression model of diabetes presence vs. glucose
logitmod <- glm(test ~ glucose, data = pima, family = binomial)
summary(logitmod) # Summary of logistic fit
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-5.350080	0.420827	-12.71	<2e-16 ***
glucose	0.037873	0.003252	11.65	<2e-16 ***

Null deviance: 993.48 on 767 degrees of freedom
 Residual deviance: 808.72 on 766 degrees of freedom
 AIC: 812.72

The resulting logistic regression equation is thus: $\hat{\pi}(x) = \frac{\exp\{-5.35 + 0.0379x\}}{1 + \exp\{-5.35 + 0.0379x\}}$. This model is plotted here:



Commonly Desired Quantity:

One quantity commonly computed with logistic regression models is that value of x such that the probability that $y = 1$ is exactly equal to 0.50. This is called the LD_{50} value. In terms of our problem, it represents the glucose concentration at which your probability of having diabetes becomes greater than 50%. At the LD_{50} point, say x^* , we know that $\pi(x^*) = 0.5$. This will occur when $\exp\{\beta_0 + \beta_1 x^*\} = 1$ which is when $\beta_0 + \beta_1 x^* = 0$.

We can estimate this by solving $b_0 + b_1 x^* = 0$

$$-5.35 + 0.0379x^* = 0 \Rightarrow x^* = \underline{141.26}.$$

Prediction: Suppose we want to predict the probability that a member of this population tests positive for diabetes if her glucose concentration level is 100. This can be done with the following **R**-code:

```
# Computes the prediction at x=100 on the logit scale (inside the exp)
logitpred <- predict(logitmod, data.frame(glucose = 100), se.fit=T)
ilogit(logitpred$fit) # Predicted prob. at x=100
```

```
1
0.1732486
```

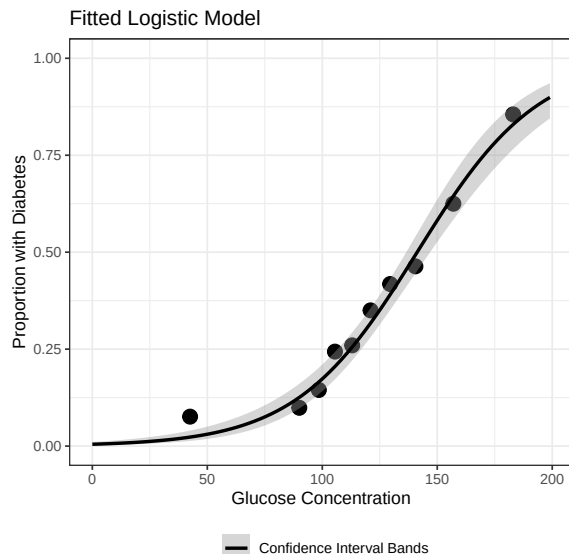
```
se = logitpred$se.fit # SE of prediction
ilogit(c(logitpred$fit-1.96*se, logitpred$fit+1.96*se)) # CI for prediction
```

```
1      1
0.1415424 0.2103170
```

giving 0.173 as the estimated probability of a positive diabetes test with a 95% confidence interval for the probability given by (0.142, 0.210).

We can extend this idea to get confidence intervals for the true probability for any glucose value in the range of our data.

```
ggplot(pima, aes(x = glucose, y = test)) +
  labs(x = "Glucose Concentration", y = "Proportion with Diabetes",
       title = "Fitted Logistic Model") +
  geom_point(data = pima_props, aes(x = midpoints, y = props), size = 4) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"),
             formula = "y ~ x", se = T, color = "black",
             aes(fill = "Confidence Interval Bands")) +
  lims(y = c(0, 1)) +
  theme_bw() +
  scale_fill_manual("", values = "gray60") +
  theme(legend.position = "bottom")
```



How well does the model predict? To address this, we could compute the predicted probabilities of a positive test for all 768 women in the study and then using 0.5 as a cutoff, see how well these predicted probabilities agree with the actual test results. This was done in **R** producing the following contingency table. How well does the model do?

```
# Binary vector set to 1 if the prob. exceeds 0.50 and 0 otherwise
ptest <- as.numeric(logitmod$fitted > 0.5)
table(truth = pima$test, prediction = ptest) # Contingency table of outcomes
```

In this case, the model correctly predicts $(443 + 130)/768 = 74.6\%$ of the cases using 0.5 as a cutoff. Other cutoffs can be used as well. A common visual technique for identifying such a cutoff is the receiver operating characteristic (ROC) curve (not considered here).

	prediction	
truth	0	1
0	443	57
1	138	130

- Since we are using our data to predict the outcomes of our data, this accuracy level is somewhat misleading and will tend to be optimistic. To avoid this, with a sufficient amount of data, it is recommended to first remove a subset of your data, and treat the remaining data as a training set. We then use the training set to classify the removed observations and estimate the classification accuracy for this test data set. In this way, the data used to construct the model are independent of the data used to estimate the model accuracy.
- Cutoff values others than 0.5 can also be used. If it is worse to incorrectly predict a 1 than a 0, it makes sense to increase the cutoff and vise-versa.

Interpretation of the Logistic Regression Coefficients

The estimate of β_1 was $b_1 = 0.0379$. How do we interpret this value?

For the model $\hat{\pi}(x) = \frac{\exp\{b_0 + b_1x\}}{1 + \exp\{b_0 + b_1x\}}$, the “slope” is interpreted in terms of the odds ratio. Namely, $\exp(b_1)$ is the estimated odds of a success at the new value divided by the estimated odds of success at the previous value. So,

$$\widehat{\text{Odds Ratio}} = \widehat{OR} = \frac{\text{odds}_{\text{new}}}{\text{odds}_{\text{prev}}} = \frac{\frac{\hat{\pi}(x+1)}{1-\hat{\pi}(x+1)}}{\frac{\hat{\pi}(x)}{1-\hat{\pi}(x)}} = \exp(b_1).$$

Example: In the diabetes example we saw that $\hat{\pi}(100) \approx \frac{\exp\{-5.35+0.0379(100)\}}{1+\exp\{-5.35+0.0379(100)\}} \approx 0.173249$ and $\hat{\pi}(99) = \frac{\exp\{-5.35+0.0379(99)\}}{1+\exp\{-5.35+0.0379(99)\}} \approx 0.167891$.

So $\text{odds}_{\text{new}} = \frac{0.173249}{1-0.173249} = 0.20955$ and $\text{odds}_{\text{prev}} = \frac{0.167891}{1-0.167891} = 0.20177$ and thus $\widehat{OR} = 0.20955/0.20177 = 1.0386 = \exp(0.0379)$.

When x increases by c units, the predicted change in the odds of a success is $\exp(c \cdot b_1)$.

Interpreting the intercept:

The inverse logit of the intercept of a logistic regression model gives the predicted probability of a success when all predictor variables equal zero. In the diabetes example, b_0 is -5.35.

Taking the inverse logit of the intercept gives $\hat{\pi}(0) = \frac{\exp(-5.35)}{1 + \exp(-5.35)} = 0.0047$. This means

Interpreting the slope of an indicator variable:

Consider the smoking example from the beginning of this set of notes again. Let's fit a logistic regression model using cancer as Y and smoking as X :

```
cancer <- c(rep(1,1357),rep(0,1357))          # Define the cancer vector
smoking <- as.factor(c(rep(1, 1350), rep(0, 7),  # Define the smoking vector
                      rep(1, 1296), rep(0, 61)))
cancer.mod <- glm(cancer ~ smoking, family = binomial) # Fit the model
summary(cancer.mod)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.1650	0.3990	-5.425	5.78e-08 ***
smoking1	2.2058	0.4009	5.502	3.76e-08 ***

As before, when x increases by 1, the predicted change in the odds of a success is $\exp(b_1)$. That means

Alternative Slope Interpretation (Using Derivative)

Recall that $\hat{\pi}(x) = \frac{\exp(b_0 + b_1x)}{1 + \exp(b_0 + b_1x)} = \frac{e^{b_0+b_1x}}{1 + e^{b_0+b_1x}}$. Let's use the derivative to find the rate of change at a given value of x . Using both the quotient rule and chain rule:

$$\begin{aligned} \frac{d\hat{\pi}}{dx} &= \frac{(b_1 e^{b_0+b_1x})(1 + e^{b_0+b_1x}) - (e^{b_0+b_1x})(b_1 e^{b_0+b_1x})}{(1 + e^{b_0+b_1x})^2} \\ &= \frac{b_1 e^{b_0+b_1x} + (b_1 e^{b_0+b_1x})(e^{b_0+b_1x}) - (e^{b_0+b_1x})(b_1 e^{b_0+b_1x})}{(1 + e^{b_0+b_1x})^2} \\ &= \frac{b_1 e^{b_0+b_1x}}{(1 + e^{b_0+b_1x})^2} = b_1 \cdot \frac{e^{b_0+b_1x}}{1 + e^{b_0+b_1x}} \cdot \frac{1}{1 + e^{b_0+b_1x}} \end{aligned}$$

Now, $\frac{e^{b_0+b_1x}}{1 + e^{b_0+b_1x}} = \hat{\pi}(x)$ and $\frac{1}{1 + e^{b_0+b_1x}} = 1 - \hat{\pi}(x)$. That means:

$$\frac{d\hat{\pi}}{dx} = b_1 \cdot \hat{\pi}(x) \cdot (1 - \hat{\pi}(x)).$$

Therefore, the rate of change in the predicted probability of a “success” at x is $b_1 \cdot \hat{\pi}(x) \cdot (1 - \hat{\pi}(x))$.

Logistic Regression Diagnostics

Just like in simple or multiple linear regression, we should look at diagnostics when doing logistic regression. Most of the diagnostics discussed before are still valid here.

Residuals: Before we discuss the plots, we will first discuss residuals for logistic regression. Since each y value is either 0 or 1, all residuals will be either $1 - \hat{\pi}$ or $-\hat{\pi}$. Those will always simply be parallel lines and not as informative as the residuals in linear regression. There are many different residuals that have been developed for GLMs including: deviance, Pearson, working, response, and partial residuals. Each is useful for different purposes. We will focus on one other type of residual: the standardized Pearson residuals.

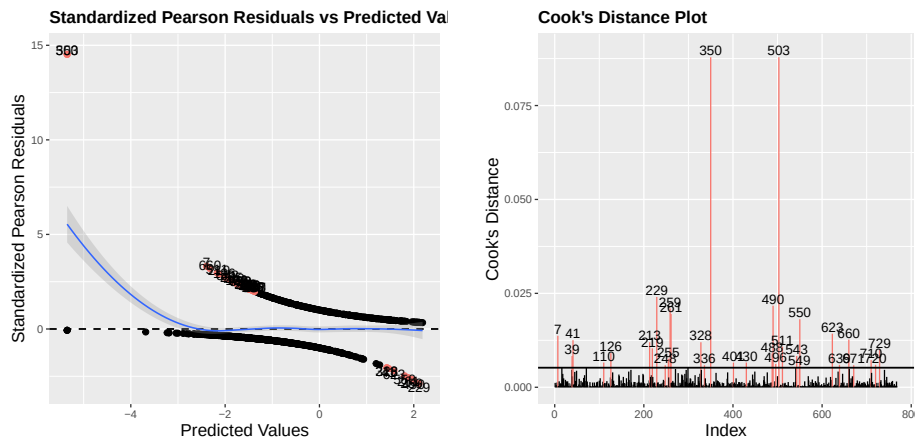
The standardized Pearson residuals (also called the studentized Pearson residuals) are found using

$$r_{SP_i} = \frac{y_i - \hat{\pi}_i}{\sqrt{\hat{\pi}_i(1 - \hat{\pi}_i)(1 - h_i)}},$$

where h_i is the leverage (or hat) value of the i th observation. It can be shown that these residuals approximately follow a standard normal distribution: $N(0, 1)$.

1. Residual plot: in linear regression, we often plotted the jackknife residuals against the fitted (or predicted) values. We used the `fitted.values()` function to obtain these fitted values. Now we will use the standardized Pearson residuals instead of the jackknife residuals. Instead of the fitted values, it is more common in logistic regression to plot the residuals against the linear predictors, $b_0 + b_1 X_1$, not the predicted probabilities. These are obtained using the `predict()` function in **R**.

If the model is adequate, then fitted a lowess smoother to the plot should result in an approximately horizontal line with an intercept of zero. On the course web page, the `influence_plots()` function works with logistic regression models as well. The plot is given on the left below.



2. Cook's Distance: Points in logistic regression can still be highly influential. Plotting Cook's distance can be useful in identifying points that influence all of the predicted values. The plot is on the right above.
3. DfFits: Like before, this measures the change in the i th prediction when the i th observation is removed.
4. DfBetas: The j th DfBetas measures the change in b_j when the i th predictor is removed.

We can see in both plots above that observations 350 and 503 are very abnormal and influential. Upon further investigation, we can see those correspond to two individuals who had a mean glucose concentration of 0 and yet tested positive.

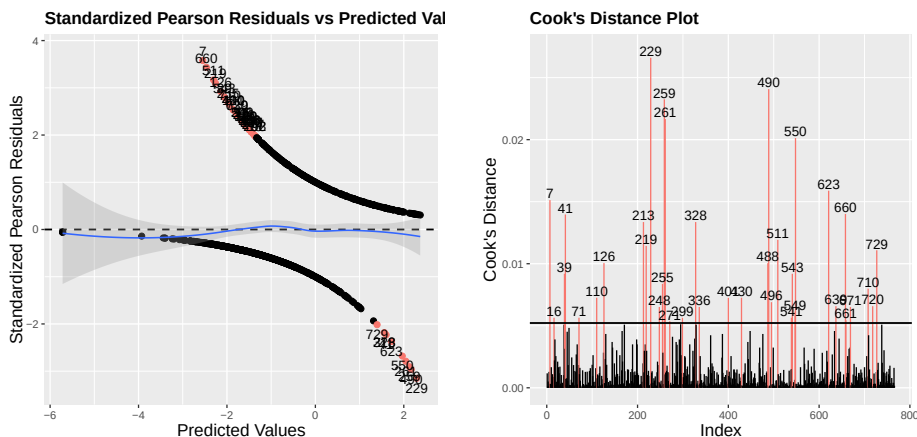
```
pima$glucose[c(350,503)]
```

```
[1] 0 0
```

```
pima$test[c(350,503)]
```

```
[1] 1 1
```

Removing these observations and refitting the model yields the following plots with the model



Final Comments on Logistic Regression

1. In **R**, we can obtain the linear predictors, $b_0 + b_1X_1$, using the `predict` function. Example: `predict(logitmod)`. We can obtain the predicted probabilities, $\hat{\pi}$ using the `fitted.values` function or the `$fitted` command for the model. So `fitted.values(logitmod)` gives the same result as `logitmod$fitted` and `ilogit(predict(logitmod))`. Then we can get our predictions (0 or 1) using `as.numeric(logitmod$fitted > 0.5)`.
2. In general, as many variables as desired may be used as explanatory factors in a logistic regression model. This can be done in the following way:

$$\hat{\pi}(x) = \frac{\exp \{b_0 + b_1x_1 + \cdots + b_px_p\}}{1 + \exp \{b_0 + b_1x_1 + \cdots + b_px_p\}}, \quad \text{for } p \text{ predictors } x_1, \dots, x_p.$$

Like linear regression models, logistic regression models can still suffer from multicollinearity (discussed in more detail later) as each slope is interpreted holding the others constant. Namely, $\exp(b_i)$ is the predicted change in the odds of a success when X_i increases by 1, holding all other X_j $j \neq i$ constant.

3. In the coefficients table given by **R**, asymptotic standard errors, a z -test statistic for examining partial significance of the parameters, and the corresponding p -value are given. Without going into the details, the use of the z -statistic is known as the Wald test; it is one of three favored tests for examining parameter significance in this model. In the example, both parameters are highly significant as their p -values are both $< 2 \times 10^{-16}$.

In practice, the recommendation is to not use this Wald test, but rather to use what is known as the **likelihood ratio test**. This can be accomplished by typing

```
anova(logitmod, test = "Chisq")
```

Analysis of Deviance Table

Model: binomial, link: logit

Response: test

Terms added sequentially (first to last)

	Df	Deviance	Resid. Df	Resid. Dev	Pr(>Chi)
NULL			767	993.48391	
glucose	1	184.76427	766	808.71964	< 2.22e-16 ***

This is testing the hypotheses $H_0 : \beta_1 = 0$ vs. $H_a : \beta_1 \neq 0$, the likelihood ratio test statistic is 184.76, which follows a χ^2 distribution (with 1 df in this case). This p -value is also close to zero, indicating that glucose concentration is a highly significant predictor of diabetes incidence. If more variables are in the model, then each will get its own χ^2 test stat and p -value. These are tested sequentially, so order matters.

4. Logistic regression (and multinomial logistic regression) are **classification methods** since they assign observations to groups.

Multinomial Logistic Regression

Logistic regression works well for predicting binary variables – those with only two levels. However, it cannot be used if there are more than two categories. A generalization of logistic regression to work with J different categories is **multinomial logistic regression**. This is also known as polytomous or multcategory logistic regression. This is best used for nominal categorical variables. There are other methods used for ordinal variables.

In regular logistic regression, we found a model for $\log \left[\frac{P(Y = 1|x)}{1 - P(Y = 1|x)} \right] = \log \left[\frac{P(Y = 1|x)}{P(Y = 0|x)} \right]$. Calling $Y = 0$ category 1 and $Y = 1$ category 2, we can write $P(Y = 0|x) = \pi_1(x)$ and $P(Y = 1|x) = \pi_2(x)$ and we found $\log \left[\frac{\pi_2(x)}{\pi_1(x)} \right] = \beta_0 + \beta_1 x$.

For multinomial logistic regression, we have more categories to consider. For example, suppose we have three groups: $Y = 1$, $Y = 2$ and $Y = 3$. We should obtain three models:

$$\begin{aligned} \log \left[\frac{\pi_2(x_i)}{\pi_1(x_i)} \right] &= \beta_{0,12} + \beta_{1,12}x_i \implies \frac{\pi_2(x_i)}{\pi_1(x_i)} = \exp(\beta_{0,12} + \beta_{1,12}x_i) \\ \log \left[\frac{\pi_3(x_i)}{\pi_1(x_i)} \right] &= \beta_{0,13} + \beta_{1,13}x_i \implies \frac{\pi_3(x_i)}{\pi_1(x_i)} = \exp(\beta_{0,13} + \beta_{1,13}x_i) \\ \log \left[\frac{\pi_3(x_i)}{\pi_2(x_i)} \right] &= \beta_{0,23} + \beta_{1,23}x_i \implies \frac{\pi_3(x_i)}{\pi_2(x_i)} = \exp(\beta_{0,23} + \beta_{1,23}x_i) \end{aligned}$$

We don't have to actually obtain all three models, because we know that $\pi_1(x_i) + \pi_2(x_i) + \pi_3(x_i) = 1$. So, we can ignore that third model and just use the two that compare π_2 and π_3 to π_1 as the baseline category. Now it's algebra time!

Set up the equations:

$$\frac{\pi_2}{\pi_1} = \exp(\beta_{0,12} + \beta_{1,12}x_i) \quad \text{and} \quad \frac{\pi_3}{\pi_1} = \exp(\beta_{0,13} + \beta_{1,13}x_i)$$

then

$$\pi_2 = \pi_1 \exp(\beta_{0,12} + \beta_{1,12}x_i) \quad \text{and} \quad \pi_3 = \pi_1 \exp(\beta_{0,13} + \beta_{1,13}x_i)$$

Using the fact that $\pi_1 + \pi_2 + \pi_3 = 1$ so $\pi_1 = 1 - \pi_2 - \pi_3$:

$$\pi_2 = (1 - \pi_2 - \pi_3) \exp(\beta_{0,12} + \beta_{1,12}x_i) \quad \text{and} \quad \pi_3 = (1 - \pi_2 - \pi_3) \exp(\beta_{0,13} + \beta_{1,13}x_i)$$

Now this is a system of two equations and two unknowns. Solving for π_2 and π_3 , and then putting hats were appropriate, gives us:

$$\begin{aligned} \hat{\pi}_2(x_i) &= \frac{\exp(b_{0,12} + b_{1,12}x_i)}{1 + \exp(b_{0,12} + b_{1,12}x_i) + \exp(b_{0,13} + b_{1,13}x_i)}, \\ \hat{\pi}_3(x_i) &= \frac{\exp(b_{0,13} + b_{1,13}x_i)}{1 + \exp(b_{0,12} + b_{1,12}x_i) + \exp(b_{0,13} + b_{1,13}x_i)}, \\ \hat{\pi}_1(x_i) &= 1 - \hat{\pi}_2(x_i) - \hat{\pi}_3(x_i). \end{aligned}$$

So, in general, if we have J categories,

$$\hat{\pi}_j(x_i) = \frac{\exp(b_{0,1j} + b_{1,1j}x_i)}{1 + \sum_{k=2}^J \exp(b_{0,1k} + b_{1,1k}x_i)}, \quad \hat{\pi}_1(x_i) = 1 - \sum_{j=2}^J \hat{\pi}_j(x_i)$$

We will obtain each of the different logistic models using the `multinom()` function from the `nnet` library. By default, it has the “baseline” category (the one we compare the others to) as the first level of a factor variable, the first alphabetically if it is a character, or the smallest number if it is stored as numeric.

Multinomial Logistic Regression Example

A study was undertaken to determine the strength of association between several risk factors and the duration of pregnancies. The risk factors considered were mother’s age, nutritional status, history of tobacco use, and history of alcohol use. The response of interest, pregnancy duration, is a three-category variable that was coded as follows: $Y = 1$ indicates preterm (less than 36 weeks), $Y = 2$ indicates intermediate term (36 to 37 weeks), and $Y = 3$ indicates full term (38 weeks or greater). Relevant data for 102 women who had recently given birth at a large metropolitan hospital were obtained. The data can be found in the `pregnancy.txt` file on Canvas.

The response variable, pregnancy duration (y), is in the first column. Nutritional status, is an index of nutritional status (higher score denotes better nutritional status). The predictor variable for age was categorized into different groups: `age1` represented those who are less than 20 years of age and `age2` are those who were greater than 30 years of age. Everyone else is between 21 and 30 years of age. Alcohol and smoking history were also qualitative

predictors; the categories were Yes" (coded 1) and No" (coded 0) for each. We will treat this response variable as nominal even though there is a natural ordering. There is another method we can use for ordinal variables that we will not discuss here.

```
library(nnet) # Load the nnet library
preg <- math3150package::pregnancy # Rename the data
preg_mods <- multinom(duration ~ nutrition + age1 + age2 + alcohol + smoking,
                      data = preg) # Fit the multinomial regression model
summary(preg_mods) # Model summary
```

Coefficients:

	(Intercept)	nutrition	age1	age2	alcohol	smoking
2	-1.517009	0.01897219	-0.04355881	-0.1721573	-0.9758857	-0.2218629
3	-5.475353	0.06542052	-2.95693397	-2.0596355	-2.0428964	-2.4522828

Std. Errors:

	(Intercept)	nutrition	age1	age2	alcohol	smoking
2	2.047984	0.01631570	0.7373261	0.6943946	0.5869073	0.5856117
3	2.271691	0.01823932	0.9644844	0.8947673	0.7097457	0.7315046

Residual Deviance: 168.6754

AIC: 192.6754

We can obtain the p -values for each of our predictors in both models using the fact that $z = b_i/s_{b_i}$ is approximately $N(0,1)$. Using the `multinom_summary()` function in the `math3150package` will give these to us.

```
multinom_summary(preg_mods)
```

\$`2 vs 1 (baseline)`

	term	estimate	std.error	z.stat	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	-1.52	2.05	-0.741	0.459
2	nutrition	0.0190	0.0163	1.16	0.245
3	age1	-0.0436	0.737	-0.0591	0.953
4	age2	-0.172	0.694	-0.248	0.804
5	alcohol	-0.976	0.587	-1.66	0.0964
6	smoking	-0.222	0.586	-0.379	0.705

\$`3 vs 1 (baseline)`

	term	estimate	std.error	z.stat	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	-5.48	2.27	-2.41	0.0159
2	nutrition	0.0654	0.0182	3.59	0.000335
3	age1	-2.96	0.964	-3.07	0.00217
4	age2	-2.06	0.895	-2.30	0.0213
5	alcohol	-2.04	0.710	-2.88	0.00400
6	smoking	-2.45	0.732	-3.35	0.000801

Doing a χ^2 test for overall model significance:

```
int_mods <- multinom(duration ~ 1, data = preg)
anova(int_mods, preg_mods) # Compare full model to intercept model
```

```
Response: duration
```

	Model	Resid. df	Resid. Dev	Test	Df	LR stat.	Pr(Chi)
1	1	202	220.6862				
2	full model	192	168.6754	1 vs 2	10	52.01072	1.13591e-07

Of course, there is nothing particularly special about using group 1 as the baseline. We can always change that. If we use group 3 as the reference instead of group 1:

```
duration2 <- factor(preg$duration, levels = c(3, 2, 1))
preg_mods2 <- multinom(duration2 ~ nutrition + age1 + age2 +
                        alcohol + smoking,
                        data = preg, trace = FALSE)
# Putting trace = FALSE silences a message about weights in the model
multinom_summary(preg_mods2)
```

```
$`2 vs 3 (baseline)`
# A tibble: 6 x 5
```

term	estimate	std.error	z.stat	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	3.96	1.94	2.04	0.0414
2 nutrition	-0.0464	0.0149	-3.12	0.00181
3 age1	2.91	0.858	3.40	0.000680
4 age2	1.89	0.809	2.33	0.0196
5 alcohol	1.07	0.650	1.64	0.100
6 smoking	2.23	0.668	3.34	0.000844

```
$`1 vs 3 (baseline)`
# A tibble: 6 x 5
```

term	estimate	std.error	z.stat	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	5.48	2.27	2.41	0.0159
2 nutrition	-0.0654	0.0182	-3.59	0.000335
3 age1	2.96	0.964	3.07	0.00217
4 age2	2.06	0.895	2.30	0.0213
5 alcohol	2.04	0.710	2.88	0.00400
6 smoking	2.45	0.732	3.35	0.000801

```
$AIC
[1] 192.6754
```

```
$deviance
[1] 168.6754
```

Interpreting Coefficients in Multinomial Logistic Regression

We can interpret the model coefficients in terms of relative risk instead of odds ratios. In general, relative risk compares the probability of an event occurring in one group compared to the probability of an event occurring in the other group. In this case, we are comparing the predicted probability of belonging to group j for $j = 2, \dots, J$ to belonging to group 1. In particular, as x increases by 1, the relative risk of belonging to group j over group 1 increases by a factor of $\exp(b_{1j})$.

In our example using group 3 as the reference group the slope of nutrition for group 2 is -0.04645 and can be defined as:

Smoking slope: the relative risk of being in the intermediate term pregnancy group compared to the full term group increases by $(\exp(2.2305) - 1) \cdot 100\% = 830\%$ for women with a smoking history.

Remember that does not mean that the women is 8.3 times more likely to have an intermediate term pregnancy compared to before. It just means the ratio in probabilities increases by 8.3 times. For example, it could have been a ratio of $0.3/0.3$ for intermediate to full and change to $0.8/0.1$. The probability in this case of being in the intermediate group increased by 1.67 times while the probability of being in the full term group fell $2/3$ ths.

Obtaining Predictions

The predicted group will be the one that has the highest probability. We can use the `fitted.values()` function to obtain all of the predicted probabilities for each group for each observation and we can use the `predict()` function to obtain the predictions. The `predict()` function behaves differently for this multinomial object than it did in logistic regression.

```
predtest <- predict(preg_mods2) # Obtain model predictions
(pred_tab <- table(truth = duration2, prediction = predtest)) # Contingency table
```

```
      prediction
truth 3  2  1
    3 32  5  4
    2  8 23  4
    1  4 10 12
```

```
sum(diag(pred_tab)) / sum(pred_tab) # Proportion of correct predictions
```

```
[1] 0.6568627
```

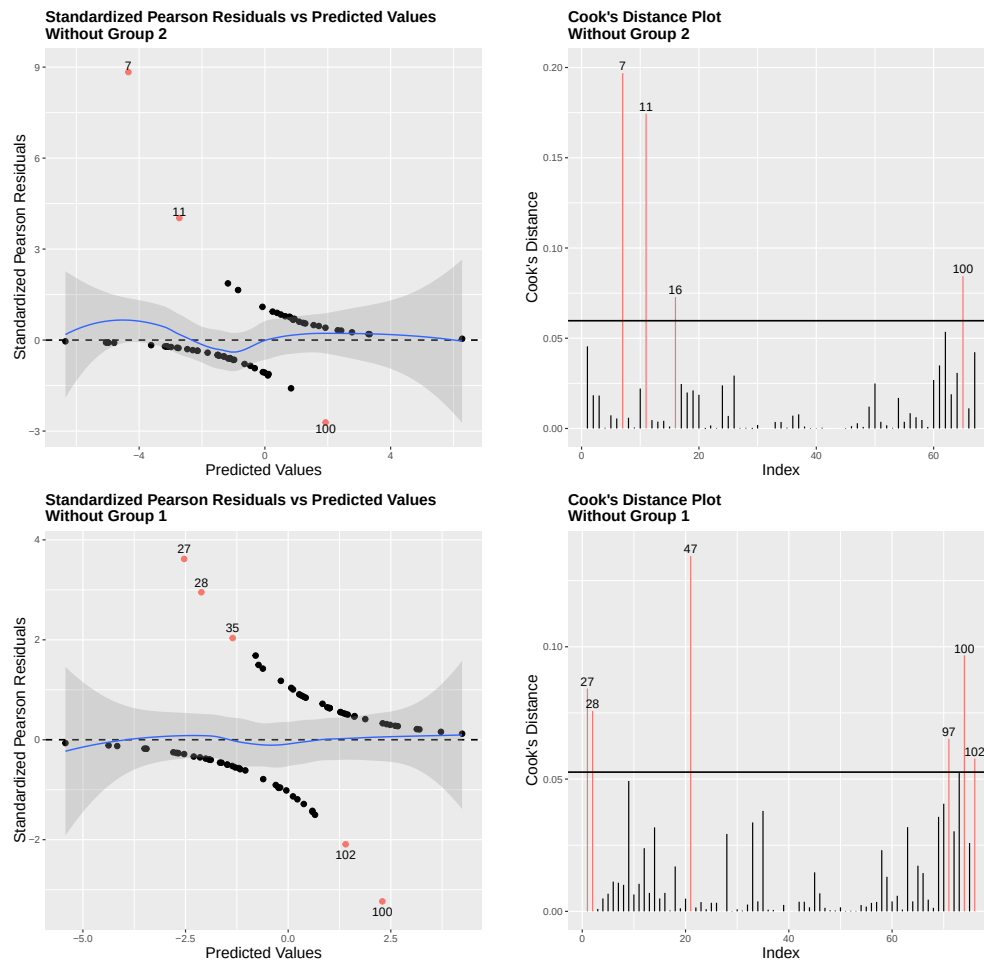
Multinomial Logistic Regression Diagnostics

The best way to check diagnostics for multinomial logistic regression is to look at the $J - 1$ individual binary logistic regression plots. In our example above, using group 3 as the baseline, that would be equivalent to fitting two logistic regression models individually: one using the data where only group 2 and group 3 are in the model and the other where only group 1 and group 3 are in the model. Then we can apply the same diagnostics as in binary logistic regression. The function `logistic_plots()` in the `math3150package` can be used for this. This function also relies on the `influence_plots()` function. The input for `logistic_plots()` is a logistic regression model (either a binary or multinomial model).

Example: Returning to the pregnancy example:

```
logistic_plots(preg_mods2)
```

A few of the plots are shown below:



The top graphs are without group 2. The bottom graphs are without group 1. Note that the point number listed is in the original data set. For instance, there are not 100 points in the model when group 2 is removed, but point 100 is flagged since that corresponds to the observation in row 100 in the original data frame.

Notes 10: Nonparametric Tests and Bootstrapping

Table of contents

Power of Hypothesis Tests	175
Calculating Power of Hypothesis Tests	176
What affects the power of a test?	177
Simulation for Power	178
Inference for the Median	179
Confidence Interval for the Population Median M	180
Hypothesis Test for the Population Median M	181
The Wilcoxon Rank Sum Test	183
Wilcoxon Rank Sum Test Steps	184
A Second Wilcoxon Rank Sum Test Example	186
Kruskal-Wallis Test	188
The Method of Bootstrapping	190
Bootstrap Example Using R	193
Bootstrapping in Regression	196
Bootstrapping the residuals	196
Bootstrapping the cases	198

Nearly all of the hypothesis tests we have seen in this class have been conducted with the assumption that either the population or the random variable (test statistic) follows some specified distribution, like a normal, t , χ^2 , or F . In order to perform those tests, the original data need follow a named distribution and/or certain assumptions must hold. In the case where the necessary conditions are not met, we can use **nonparametric tests** where no population distribution need be assumed. We will run through some popular nonparametric tests. First, however, let's quickly discuss the power of hypothesis tests.

Power of Hypothesis Tests

Since nonparametric tests can be used in lieu of the traditional parametric tests that we are used to but require fewer assumptions, a natural question is why use parametric tests at all? The answer to that comes down to **power**.

Remember that in any hypothesis test, there are two outcomes if a fixed α -level is used. We either reject H_0 or fail to reject H_0 . Because there are only two possible outcomes, there are only two types of errors which can be made with the conclusion. These two errors are known as Type I and Type II errors.

1. If we reject the null hypothesis when it is actually true, we have committed a **Type I error**.
2. If we fail to reject the null hypothesis when it is actually false, we have committed a **Type II error**.

The set of all possible decisions and corresponding probabilities are shown in the table.

- A “good” test is one with both a small Type I error probability α and a small Type II error probability β .

Decision	Null Hypothesis	
	True	False
Reject H_0	Type I Error α	Correct Decision $1 - \beta$
Fail to Reject H_0	Correct Decision $1 - \alpha$	Type II Error β

- Unfortunately, we cannot simultaneously minimize both types of errors, as decreasing one type of error will cause an increase in the other.
- The common practice is to fix the probability of a Type I error α , and then try to minimize the probability of a Type II error β . This is done to protect the null hypothesis from being rejected when it is actually true (the “conservative” approach).
- The value $1 - \beta$ for some hypothesis test is known as the **power** of the test. It measures the power or ability one has to detect differences between hypothesized means and some actual mean.
- In general, for a fixed α level, if the assumptions are met, parametric tests have higher power than nonparametric tests and nonparametric tests usually have higher power than parametric tests if the assumptions for the parametric do not hold.

Calculating Power of Hypothesis Tests

Power cannot be exactly calculated for every test, but we will focus on one case where it can. Simulations can be done for other cases.

Example: Suppose we have a random sample of size 50 from a population and we are testing the following hypotheses at the 0.05 significance level.

$$H_0: \mu = 20$$

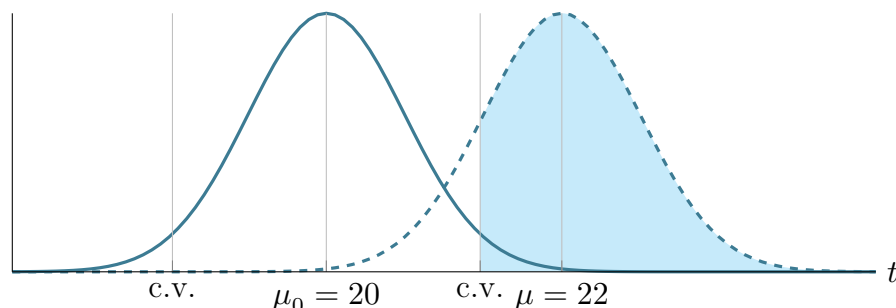
$$H_A: \mu \neq 20$$

Using the usual convention, we would say $\mu_0 = 20$ in this example. Now, suppose that we know $\mu = 22$ and $\sigma = 4.5$. Of course, we would not ordinarily know this, but we cannot calculate the power without knowing it.

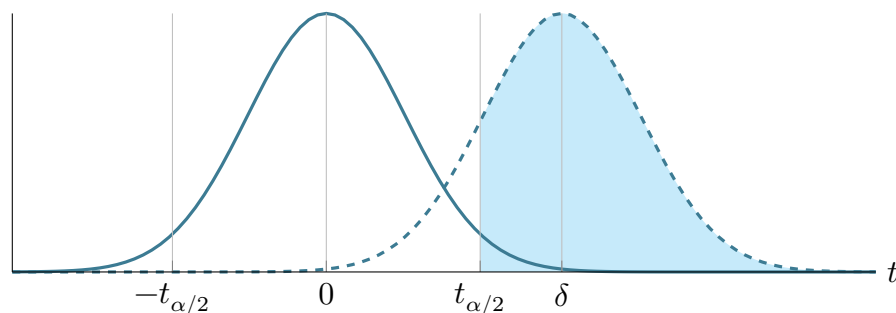
We know that for a fixed α ,

$$\text{Power} = 1 - \beta = 1 - P(\text{Fail to reject } H_0 | H_0 \text{ is false}) = P(\text{Reject } H_0 | H_0 \text{ is false})$$

To calculate this, it is common to draw two distributions: one of the sampling distribution under H_0 (assuming $\mu = \mu_0$) and one of the true sampling distribution (with the correct μ). That would lead to this picture:



The shaded area above represents the power of the test. To calculate this probability, we need to convert both of these distributions to t distributions. The first will be centered at zero and the other will have a noncentrality parameter, called δ .



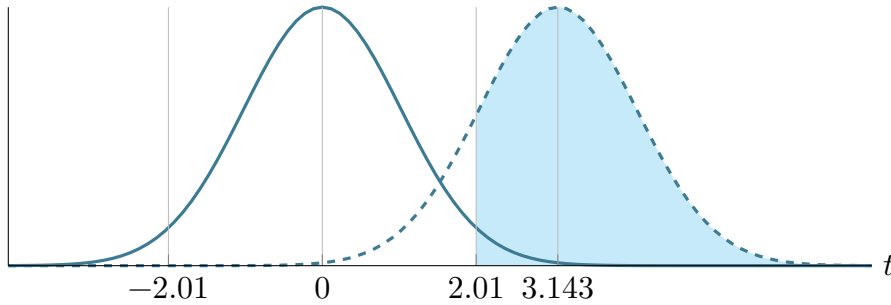
We can compute δ by using the following expression:

$$\delta = \frac{\mu - \mu_0}{\sigma/\sqrt{n}}$$

In this example,

$$\delta = \frac{22 - 20}{4.5/\sqrt{50}} = 3.143$$

and our critical value is $t_{0.05/2} = t_{0.025} = 2.01$ with $n - 1 = 50 - 1 = 49$ degrees of freedom. So, our updated picture looks like this:



Finally, we can compute the power using

$$\begin{aligned} \text{Power} &= P(T_{nc} > t_{\alpha/2}; n - 1; \delta) + P(T_{nc} < -t_{\alpha/2}; n - 1; \delta) \\ &= P(T_{nc} > 2.01; 49; 3.143) + P(T_{nc} < -2.01; 49; 3.143) \end{aligned}$$

where T_{nc} is a value from a noncentral t distribution with noncentrality parameter δ and $n - 1$ degrees of freedom. We can compute this value in **R** using the `pt()` function.

$$\begin{aligned} \text{Power} &= P(T_{nc} > 2.01; 49; 3.143) + P(T_{nc} < -2.01; 49; 3.143) \\ &= 1 - \text{pt}(2.01, 50 - 1, 3.143) + \text{pt}(-2.01, 50 - 1, 3.143) = 0.8687. \end{aligned}$$

Therefore, the power of this test is 0.8687. This means the probability of a type two error for this test is $1 - 0.8687 = 0.1313$.

What affects the power of a test?

- As the effect size, $d = \frac{\mu - \mu_0}{\sigma}$ increases, the power will
- As the sample size increases, the power will
- If the assumptions of the test are not met, the power will

In this example, if the true mean was 25 instead of 22, the power would be:

In this example, if the true mean was 22, but the sample size was 100, the power would be:

Simulation for Power

The following code will simulate an estimation of the power.

```
set.seed(3150)
mu = 22          # Set the true mean
sigma = 4.5      # Set the true SD
alpha = 0.05     # Set the significance level
n = 50           # Set the sample size
mu0 = 20         # Set the hypothesized mean for testing H_a: mu != mu0
error = rep(0, 10000) # Initialize a vector for storing when we have an error
for(i in 1:10000) {
  x = rnorm(n, mu, sigma)          # Randomly generate a sample
  t_stat = (mean(x) - mu0)/(sd(x)/sqrt(n)) # Compute the test stat
  p_val = 2 * (1 - pt(abs(t_stat), n - 1)) # Compute the p-value
  if(p_val > alpha){
    error[i] = 1                  # Mark it as an error if we fail to reject H_0
  }
}
beta_sim = mean(error)          # Estimated probability of Type II error
power_sim = 1 - beta_sim        # Estimated power
power_sim
```

```
[1] 0.8712
```

In this example, we calculated the power for a two-tailed test. You can compute it for a one-tailed test as well. Each of the following are listed below:

$$\text{Power for two-tailed test} = P(T_{nc} > t_{\alpha/2}; n - 1; \delta) + P(T_{nc} < -t_{\alpha/2}; n - 1; \delta)$$

$$\text{Power for left-tailed test} = P(T_{nc} < t_{\alpha}; n - 1; \delta)$$

$$\text{Power for right-tailed test} = P(T_{nc} > t_{1-\alpha}; n - 1; \delta)$$

Below is a table that shows the power for different sample sizes and different hypothesized mean (μ_0) values for the example when testing $H_0 : \mu = \mu_0$ with $\mu = 22$ and $\sigma = 4.5$.

Sample size:	5	10	20	30	50	100	200	1000
$\mu_0 = 20$	0.1213	0.2421	0.4709	0.6529	0.8687	0.9927	0.99999	1
$\mu_0 = 21$	0.0676	0.0967	0.1567	0.2177	0.3377	0.5951	0.8785	1
$\mu_0 = 22$	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.05
$\mu_0 = 23$	0.0676	0.0967	0.1567	0.2177	0.3377	0.5951	0.8785	1
$\mu_0 = 24$	0.1213	0.2421	0.4709	0.6529	0.8687	0.9927	0.99999	1
$\mu_0 = 25$	0.2113	0.4692	0.8073	0.9416	0.9961	1	1	1

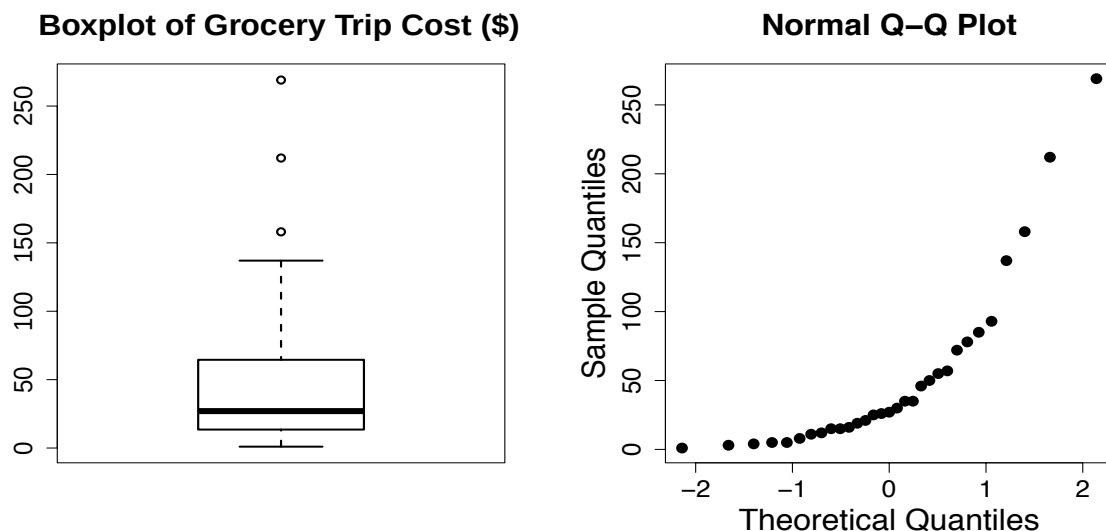
Inference for the Median

We have discussed how to perform statistical inference for a population mean in the form of a confidence interval or test of significance. These inferences depended critically on an assumption that the underlying population from which the random sample was drawn was normal. When the population distribution is heavy-tailed (with outliers) or highly skewed, the mean is not an appropriate measure of center and we often use the median to describe the center of a distribution. In such cases, we will want to perform inferences on the median rather than the mean, so we need to consider how a confidence interval and significance test can be carried out for the median. These procedures will be illustrated by way of the following example.

Example: The amount of money spent on trips to a grocery store can vary considerably depending on the purpose of the trip. Suppose a local grocery store agrees to provide me a sample of the amount spent by 31 randomly selected customer visits over the past month. The amount spent (rounded to the nearest dollar) for each of these 31 customer visits is given (ordered from smallest to largest) in the table below.

Grocery Prices															
1	3	4	5	5	8	11	12	15	15	16	19	21	25	26	27
30	35	35	46	50	55	57	72	78	85	93	137	158	212	269	

First, consider the boxplot and normal quantile plot of these data below. What do these indicate about the data?



- Since the data are skewed with some extreme outliers, the median is a more appropriate measure of center than the mean for these data. Since $n = 31$ is odd, the median is the $(n + 1)/2 = 16^{th}$ value in the ordered list: $m = \$27$.
- Note that the mean is \$52.42, a value such that 21 of the 31 amounts fell below this value. This makes the mean a poor measure of center.

Confidence Interval for the Population Median M

The form of a confidence interval for M is based on the binomial distribution and will not be derived here. Recall that the order statistics are just the ordered data, denoted by $x_{(1)}, x_{(2)}, \dots, x_{(n)}$. A confidence interval for M with confidence level of at least $100(1 - \alpha)\%$ has the following form:

$$(M_L, M_U) = (x_{(L_{\alpha/2})}, x_{(U_{\alpha/2})}),$$

where $L_{\alpha/2} = C_{1-\alpha/2, n}$, $U_{\alpha/2} = C_{\alpha/2, n} + 1$, and $C_{\alpha/2, n}$ and $C_{1-\alpha/2, n}$ are percentile values from a binomial distribution with $p = 0.5$.

- We can obtain $C_{1-\alpha/2, n}$ using `qbinom(alpha/2, n, 0.5)` and $C_{\alpha/2, n}$ using `qbinom(1 - alpha/2, n, 0.5)`.
- We can also find the confidence interval using the **SignTest()** function in the **DescTools** library in **R**. This function conducts this exact binomial test, reporting the confidence interval and the true coverage (true confidence coefficient).
- Find a confidence interval for the median amount spent by a customer to this grocery store with confidence level at least 95%.

```
grocery <- c(
  1, 3, 4, 5, 5, 8, 11, 12, 15, 15, 16, 19, 21, 25, 26, 27, 30, 35, 35, 46,
  50, 55, 57, 72, 78, 85, 93, 137, 158, 212, 269
)
lower <- sort(grocery)[qbinom(0.05/2, 31, 0.5)]      # Obtain the lower
upper <- sort(grocery)[qbinom(1 - 0.05/2, 31, 0.5) + 1] # and upper bounds
lower; upper
```

```
[1] 15
[1] 55
```

```
library(DescTools) # Loads "DescTools" library
# Constructs the interval
SignTest(grocery, mu = 20, conf.level = 0.95)$conf.int
```

```
lwr.ci upr.ci
    15     55
attr(,"conf.level")
[1] 0.971
```

- Interpretation?

Hypothesis Test for the Population Median M

Suppose we had wanted to test whether or not the median amount of money spent on groceries at the store is more than \$20 per visit. We would like to test the hypotheses:

$$H_0 : M = 20 \text{ (or } M \leq 20) \text{ vs. } H_a : M > 20.$$

- When we conducted a test on the population mean μ , we appealed to the Central Limit Theorem or a normal population to argue that $\bar{x}$ followed a normal model and developed a test based on the normal (or t) distribution. How do we conduct a hypothesis test on the median?

Sign Test: In general, for hypotheses on M of the form: $\left\{ \begin{array}{l} H_0 : M = M_0 \text{ vs. } H_a : M < M_0 \\ H_a : M > M_0 \\ H_a : M \neq M_0 \end{array} \right\}$.

The steps for conducting a sign test are as follows.

- For a test on M with null hypothesis value M_0 , define $W_i = x_i - M_0$. Some of these values will be positive and some negative depending on whether or not they are above or below the hypothesized median.
 - The **sign test statistic** is defined as B = the number of positive W_i -values. In other words, B is the number of data values greater than the hypothesized median.
 - Compute the p-value for this test by computing the exact probability of obtaining B or something more extreme if H_0 is true.
- We will use the `SignTest()` function in the **R** library `DescTools` to conduct this test, but the mechanics of how the test statistic itself is calculated is demonstrated with the grocery data below.

Back to the Example: In the grocery cost data, we wanted to test the hypotheses:

$$H_0 : M \leq 20 \text{ vs. } H_a : M > 20.$$

The 31 grocery visit costs and their differences from the hypothesized median value of \$20 are given in the table below (where $W_i = x_i - M_0$):

x_i	W_i	Sign	x_i	W_i	Sign	x_i	W_i	Sign	x_i	W_i	Sign	x_i	W_i	Sign
1	-19	(-)	12	-8	(-)	25	5	(+)	46	26	(+)	85	65	(+)
3	-17	(-)	15	-5	(-)	26	6	(+)	50	30	(+)	93	73	(+)
4	-16	(-)	15	-5	(-)	27	7	(+)	55	35	(+)	137	117	(+)
5	-15	(-)	16	-4	(-)	30	10	(+)	57	37	(+)	158	138	(+)
5	-15	(-)	19	-1	(-)	35	15	(+)	72	52	(+)	212	192	(+)
8	-12	(-)	21	1	(+)	35	15	(+)	78	58	(+)	269	249	(+)
11	-9	(-)												

- Scanning this table, we see that the number of positive values is $B = 19$. How likely would it be to see at least 19 values (of the 31) above the true median just by chance? Again the binomial distribution can be used to compute this exactly; we use the `SignTest()` function to actually carry out the test as follows:

```
library(DescTools) # Loads "DescTools" library
SignTest(grocery, mu = 20, alternative = "greater")
```

One-sample Sign-Test

```
data:  grocery
S = 19, number of differences = 31, p-value = 0.1405
alternative hypothesis: true median is greater than 20
96.5 percent confidence interval:
 16 Inf
sample estimates:
median of the differences
      27
```

- With our test statistic of $B = 19$ values above the hypothesized median of \$20 and a p-value of 0.1405, we have insufficient evidence that the median is greater than \$20.
- If we had tested $H_0 : M \leq 15$ vs. $H_a : M > 15$, what would you expect the p-value to have been smaller or larger?
- Simulation studies have demonstrated that the sign test for the median is much more powerful than a t-test for detecting differences from a hypothesized median when the distribution is severely skewed or heavy tailed (outliers) at small sample sizes. As the sample size increases, the t-test and sign test tend to have more similar power for detecting differences of a given size.
- The t-test is more powerful when the population distribution is fairly symmetric and mound-shaped for any sample size, as the assumption of normality is stronger (when correct) than making no distributional assumption (as with the sign test).
- Recall that with z-based and t-based inferences on the population mean μ , we relied on knowing or estimating the standard deviation of the sample mean $\bar{y}$, in the form $\sigma/\sqrt{n}$ or $s/\sqrt{n}$. Inferences for the median utilize the actual data values to construct a CI or conduct a hypothesis test without calculating the variation in the sample median m . Why?
- An alternative computational method that allows us to estimate the standard deviation in the median from the data and then use this estimated SD to perform statistical inference is known as **bootstrapping**. This technique can actually be used in a wide variety of applications as seen shortly.

The Wilcoxon Rank Sum Test

We know how to use two-sample t-procedures for comparing the population means from two distributions. The use of t-tests in comparing two population means hinges critically on an assumption of normal populations, in addition to having independent samples.

- If the two samples are reasonably large (say $n_1, n_2 \geq 30$), then the use of t-tests is valid when the original populations are not normal, as long as there is not severe skewness or outliers present (due to **robustness** of t-procedures).
- What do we do if we want to test whether two populations are different when we have small sample sizes AND/OR the data have severe skewness or heavy tails? In this case, we cannot use a t-procedure and must consider its nonparametric equivalent: the **Wilcoxon rank sum test**. This test is equivalent to the Mann-Whitney U-test.

Basic Setup: Consider taking a sample of size n_1 from population 1, and a sample of size n_2 from population 2, and suppose these two samples are independent of one another.

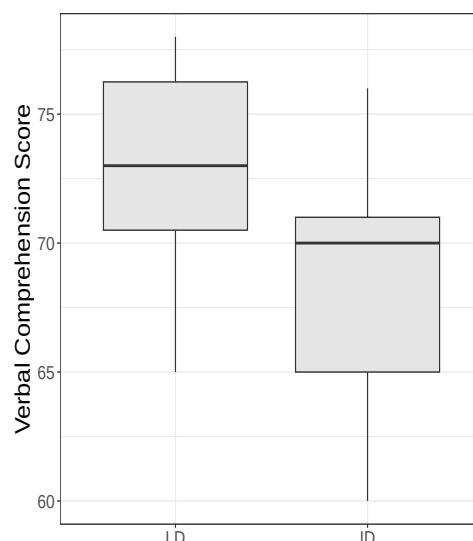
- We want to test whether or not the two populations are identical or whether one distribution is “shifted” to the right or left of the other.
- If the two populations are identical, then of course the means will be equal as well.

Example: Suppose a verbal comprehension test is given to independent samples of children who have a learning disability (LD) (population 1) and to those with an intellectual disability (ID) (population 2). The scores from the test are given in the table below along with a box plot comparing the two distributions.

Learning Disability (LD)	77	78	70	72	65	74		$(n_1 = 6)$
Intellectual Disability (ID)	60	62	70	76	68	72	70	$(n_2 = 7)$

- Since the sample sizes (n_1, n_2) are so small, we don’t really know if the populations from which these data arose are normal and we cannot use a *t*-test since the CLT does not apply.
- If we decide not to use a *t*-test to compare the mean scores from these two populations of children, we might use the Wilcoxon rank sum test to test the hypotheses:

H_0 : The population distributions of
 scores are identical
 vs. H_a : The population distributions of
 scores are not identical (2-sided test).



- We could also conduct a one-sided test where the alternative hypothesis might indicate that one distribution is to the left or right of the other. This will be shown in the next example.

Wilcoxon Rank Sum Test Steps

- Rank all $n_1 + n_2$ observations (smallest to largest) from 1 to $n_1 + n_2$.
 - In the example, the data are ordered below with the LD sample data boxed.

Data	60	62	65	68	70	70	70	72	72	74	76	77	78
Rank													

- Compute the sum of the ranks, T , for (either) one of the samples.
 - For the LD group, $T = 3 + 6 + 8.5 + 10 + 12 + 13 = \underline{52.5}$. What is it for the ID group?
 - If the two populations were truly identical, the sum of the ranks should be slightly smaller for the LD group since it had a slightly smaller sample size. More precisely, this sum should be about $\left(\frac{n_1}{n_1 + n_2}\right)$ times the total sum of the ranks, which is: $[1 + 2 + \dots + (n_1 + n_2)]$. Hence, we expect:

$$\begin{aligned}
 T &= \left(\frac{n_1}{n_1 + n_2}\right) [1 + 2 + \dots + (n_1 + n_2)] \\
 &= \left(\frac{n_1}{n_1 + n_2}\right) \cdot \frac{(n_1 + n_2)(n_1 + n_2 + 1)}{2} \quad \text{Why?} \\
 \Rightarrow T &= \frac{n_1(n_1 + n_2 + 1)}{2} \quad \text{IF the populations are identical } (H_0).
 \end{aligned}$$

- In fact, the statistic T has mean $\mu_T = \frac{n_1(n_1 + n_2 + 1)}{2}$ and variance given by $\sigma_T^2 = \frac{n_1 n_2}{12} (n_1 + n_2 + 1)$. Some books provide a slight adjustment to the variance σ_T^2 for the situation of tied ranks. This is only necessary if there are many ties.

In the example: $\mu_T = \frac{6(6 + 7 + 1)}{2} = \frac{84}{2} = \underline{42}$, so we expect the sum of the ranks to be 42 under H_0 . Does this make sense?

- If T is much smaller (larger) than μ_T , then sample 1 has smaller (larger) ranks $\Rightarrow$ smaller (larger) data values $\Rightarrow$ the populations are not identical.
 - When is T too large or too small?
- Compute the p-value corresponding to the test statistic T . p-values and rejection regions are found using the Wilcoxon distribution of the sum of the ranks, but we will rely on the `wilcox.test()` function in **R** to determine a p-value for the test. This can be done with the following **R** code:

```

vscore <- c(77, 78, 70, 72, 65, 74, 60, 62, 70, 76, 68, 72, 70)
group <- factor(c(rep("LD", 6), rep("ID", 7)), levels = c("LD", "ID"))
dat <- data.frame(vscore = vscore, group = group)
wilcox.test(vscore ~ group, data = dat, alternative = "two.sided")

```

$W = 31.5$, $p\text{-value} = 0.1445$

alternative hypothesis: true location shift is not equal to 0

4. State a conclusion. With a p -value of 0.1445, there is insufficient evidence of a difference in the verbal comprehension score distributions for the two populations of children.

How do we estimate the size of the difference?

A Confidence Interval for the Median Difference:

Let Δ = the size of the shift (or difference) between the two populations. To place a confidence interval on Δ , we need to estimate Δ .

- Since the data are not paired, to estimate the median difference in the verbal comprehension scores, we must consider all possible differences between values. Letting x_i be the i^{th} data value for the 1st sample and y_j be the j^{th} value for the 2nd sample, we estimate Δ as: $\widehat{\Delta} = \text{median}[(x_i - y_j) : i = 1, \dots, n_1, j = 1, \dots, n_2]$, where there are $n_1 \cdot n_2$ possible such differences.
- To place a confidence interval on Δ , we again rely on the Wilcoxon distribution of the sum of the ranks considered earlier. As before, we will utilize the `wilcox.test()` function in **R** to find the confidence limits for a given confidence level.
- To see how this works, reconsider the verbal comprehension data with the two groups (LR, ID) of children. Suppose we want to find a 95% confidence interval for the median difference in the verbal comprehension scores between the two groups. First, we consider all $n_1 n_2 = 6 \cdot 7 = 42$ possible differences between the two distributions of scores, given below.

	-11	-7	-6	-5	-5	-4	-3	-2	-2	0	0	0	1	2
Differences	2	2	2	2	3	4	4	4	5	5	6	6	7	7
	8	8	8	9	10	10	10	12	12	14	15	16	17	18

The median of these 42 differences is the average of the 21st and 22nd ordered difference, namely $\widehat{\Delta} = (4 + 4)/2 = 4$.

- To find the differences that correspond to the 95% confidence limits for these data, we use the `wilcox.test()` function in **R** as follows:

```
wilcox.test(vscore ~ group, data = dat, alternative = "two.sided",
            conf.level = 0.95, conf.int = T)
```

$W = 31.5$, $p\text{-value} = 0.1445$

alternative hypothesis: true location shift is not equal to 0

95.3 percent confidence interval: -2 10

sample estimates: difference in location 4

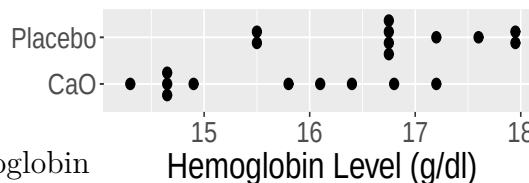
- Hence, we are 95% confident that the true median verbal comprehension score for the LD group is between 3 points less and 12 points greater than that of the ID group.
- Since this confidence interval is not exact (due to the ties), how else might we calculate this interval?
- If $n_1, n_2 \geq 10$, then there is a large-sample approximation yielding: $\frac{T - \mu_T}{\sigma_T} \overset{\circ}{\sim} N(0, 1)$, and we can use normal distribution theory to perform the test.

A Second Wilcoxon Rank Sum Test Example

To study the effects of prolonged inhalation of cadmium, researchers exposed 10 dogs to cadmium oxide while 10 dogs serving as controls were not exposed to this substance. At the end of the experiment, they determined levels of hemoglobin of the 20 dogs, as shown in the table and graph below.

Hemoglobin Determinations (grams/dl) in 20 Dogs										
Exposed to Cadmium Oxide	14.6	15.8	16.4	14.6	14.9	14.3	14.7	17.2	16.8	16.1
Controls	15.5	17.9	15.5	16.7	17.6	16.8	16.7	16.8	17.2	18.0

Suppose we test whether the levels of hemoglobin are the same for the population of dogs exposed to CdO (population 1) and the population of dogs which were not (population 2), or whether the distribution of hemoglobin levels for population 1 is to the left of that of population 2. How should we proceed?



With just 10 data values from each population, a determination of normality would be guesswork at best. If we are unwilling to simply assume normality of the population distributions, we might use the Wilcoxon Rank Sum Test. Using this test then, we wish to test:

H_0 : The populations are identical.

H_a : Population 1 is to the left of Population 2 (one-sided test).

- Ranking all $n_1 + n_2 = 10 + 10 = 20$ observations, we have:

Obs.	14.3	14.6	14.6	14.7	14.9	15.5	15.5	15.8	16.1	16.4
Rank	1	2.5	2.5	4	5	6.5	6.5	8	9	10
Obs.	16.7	16.7	16.8	16.8	16.8	17.2	17.2	17.6	17.9	18.0
Rank	11.5	11.5	14	14	14	16.5	16.5	18	19	20

- With the data values for the Cadmium oxide group boxed, the sum of the ranks T , for this group is: $T = 1 + 2.5 + 2.5 + 4 + 5 + 8 + 9 + 10 + 14 + 16.5 = 72.5$

If the populations were truly identical, we would expect the sum of these ranks to be:

$$\mu_T = \frac{n_1(n_1 + n_2 + 1)}{2} = \frac{(10)(21)}{2} = 105.$$

- Since $n_1, n_2 \geq 10$, then the statistic $\frac{T - \mu_T}{\sigma_T} \stackrel{\circ}{\sim} N(0, 1)$, where here:

$$\sigma_T = \sqrt{\frac{n_1 n_2}{12} (n_1 + n_2 + 1)} = \sqrt{\frac{100}{12} (21)} = \sqrt{175} = 13.229. \text{ Units?}$$

Computing then: $z = \frac{72.5 - 105}{13.229} = -2.456769$. Hence the p-value for this test is:

$$P(Z < -2.456769) = \text{pnorm}(-2.456769) = 0.0070.$$

In **R**, this Wilcoxon rank sum test can be run using the following code.

```
hemo1 <- c(14.6, 15.8, 16.4, 14.6, 14.9, 14.3, 14.7, 17.2, 16.8, 16.1)
hemo2 <- c(15.5, 17.9, 15.5, 16.7, 17.6, 16.8, 16.7, 16.8, 17.2, 18.0)
wilcox.test(hemo1, hemo2, alternative = "less")
```

W = 17.5, p-value = 0.00604

alternative hypothesis: true location shift is less than 0

4. Conclusion: Since this p-value is small, we have strong evidence that the distribution of hemoglobin levels for the population of dogs exposed to cadmium oxide is to the left of that of the population not exposed. In fact, with 95% confidence, the median level is between 0.4 and 2.3 g/dl lower for the cadmium oxide distribution (via `wilcox.test`).
 - Note that there was no mention of the *mean* hemoglobin level anywhere in this example. It is worth emphasizing here that a nonparametric procedure is given this name because no parameters are used in this test!
-

Suppose we had used a t-distribution to test whether or not the true mean hemoglobin level for dogs in the cadmium oxide exposure population is smaller than that of the control population. Then, letting μ_i = the true mean hemoglobin level for population i , $i = 1, 2$, we would test: $H_0 : \mu_1 - \mu_2 = 0$ vs. $H_a : \mu_1 - \mu_2 < 0$.

1. Computing, we find that: $\bar{y}_1 = 15.54$, $\bar{y}_2 = 16.87$, $s_1^2 = 1.0982$, $s_2^2 = 0.7557$. This gives the test statistic for this test:

$$t = \frac{\bar{y}_1 - \bar{y}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} = \frac{15.54 - 16.87}{\sqrt{\frac{1.0982}{10} + \frac{0.7557}{10}}} = \underline{-3.089}.$$

2. The P-value for this test is: $P(T < -3.089)$ where $T \sim t(17.406)$ where the degrees of freedom were calculated using a Satterthwaite approximation. Computing via **R**, the P-value is: `pt(-3.089, 17.406) = 0.0033`.
3. Note that this p-value is only about half the size as that found under the Wilcoxon Rank Sum test procedure. What does this mean?

Summary: If the population distributions really are normal, then the t-procedure is better at detecting differences (more powerful) between two populations through their means. If the population distributions are not normal, the t-test is invalid (except under large sample sizes), and a procedure such as the Wilcoxon Rank Sum test is better at detecting differences between two populations, because it does not depend on normality.

The book *Statistical Methods for Data Analysis* by Ott & Longnecker presents a nice simulation study on page 324 illustrating that the t-test is more powerful when the population distributions are normal, about equally powerful when they are skewed, and less powerful than the Wilcoxon rank sum test when there are heavy tails (outliers).

Kruskal-Wallis Test

The Kruskal-Wallis test is a nonparametric alternative to the 1-way ANOVA procedure for comparing distributions for multiple populations.

- It is used primarily when the normality or variance homogeneity assumption is violated.
- This test is nothing more than an extension of the Wilcoxon rank sum test to the case of more than two populations. Recall that the Wilcoxon test was a nonparametric alternative to the two-sample t-test when either of these same assumptions was violated, which compared the size of the ranks between two populations.

The Kruskal-Wallis test tests the following hypotheses:

H_0 : The k populations have identical distributions.

H_a : The populations have different distributions.

- Note that there is no mention of means anywhere in these hypotheses.

- Test Statistic: $H = \left[\frac{12}{n_T(n_T + 1)} \sum_{i=1}^k \frac{T_i^2}{n_i} \right] - 3(n_T + 1)$, where:

T_i = sum of the ranks for the sample i observations, with all obs. ranked, $i = 1, \dots, t$.

- Note that if the ranks (T_i) are large for some i relative to others, then H will be large, indicating differences in the populations.
- Under H_0 , $H \stackrel{\circ}{\sim} \chi^2(k-1)$ if the n_i 's are not too small. A common recommendation made is that each $n_i \geq 10$.

Recall: For the Wilcoxon rank sum test, $z = \frac{T - \mu_T}{\sigma_T} \stackrel{\circ}{\sim} N(0, 1)$ if $n_1, n_2 \geq 10$.

We reject H_0 for large values of H .

- If there are MANY ties, H should be adjusted to account for this.

Example: Suppose a company is looking at the time between computer failures in 3 different locations. The data are given below. The differences in the sum of the ranks likely indicates differences in the three locations.

Data (Ranks)							Sum of Ranks
Location	1	105 (11)	3 (2)	90 (10)	217 (14)	22 (4)	$T_1 = 41$
	2	76 (8.5)	43 (7)	1 (1)	37 (5)	14 (3)	$T_2 = 24.5$
	3	183 (13)	144 (12)	219 (15)	76 (8.5)	39 (6)	$T_3 = 54.5$

- For example, the first rank sum is computed as: $T_1 = 11 + 2 + 10 + 14 + 4 = 41$.
- Waiting-time data are often very skewed and hence nonnormal. Sometimes, a log transformation on these data will stabilize the variance as well as “normalize” the data; however, sometimes it will not. In the case where such a transformation fails to meet the ANOVA assumptions, a Kruskal-Wallis test may be used. Here:

$$H = \left[\frac{12}{15(16)} \left(\frac{41^2}{5} + \frac{24.5^2}{5} + \frac{54.5^2}{5} \right) \right] - 3(16) = \underline{4.515} \text{ with } k-1 = 2 \text{ d.f.}$$

Do you see any problems here? Since the sample size in each group is only 5, this violates the rule of thumb given for using a chi-square procedure, so the test may not be appropriate. Fortunately, **R** will rely on exact (not approximate) p-values to perform the test if the sample sizes are too small.

Ignoring these issues here, since $1 - \text{pchisq}(4.515, 2) = 0.1046$, there is suggestive evidence of differences in the computer failure time distributions.

- In **R**, a Kruskal-Wallis test can be run using the `kruskal.test` function using the **R**-code shown below.

```
# Define the vector of failure times
time <- c(105, 3, 90, 217, 22, 76, 43, 1, 37, 14, 183, 144, 219, 76, 39)
location <- as.factor(rep(1:3, each = 5)) # Vector of location levels
kruskal.test(time ~ location) # Conducts Kruskal-Wallis test
```

```
Kruskal-Wallis rank sum test
data: time by location
Kruskal-Wallis chi-squared = 4.5231, df = 2, p-value = 0.1042
```

- Note that this p-value (0.1042) agrees with what we found above, doing the test by hand and using the approximate chi-square distribution.
- If a 1-way ANOVA was performed on these data (as shown below), the p-value for $H_0 : \mu_1 = \mu_2 = \mu_3$ was $p = .1094$. As this nearly matches that of the Kruskal-Wallis test, it didn't seem to matter which test was used in this case.

```
reg <- lm(time ~ location) # 1-way ANOVA of time on location
summary(reg) # Regression summary
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 87.40 29.99 2.915 0.013
location2 -53.20 42.41 -1.254 0.234
location3 44.80 42.41 1.056 0.312
Residual standard error: 67.05 on 12 degrees of freedom
Multiple R-squared: 0.3085, Adjusted R-squared: 0.1932
F-statistic: 2.677 on 2 and 12 DF, p-value: 0.1094
```

```
anova(reg) # ANOVA summary
```

```
Analysis of Variance Table (Response: time)
      Df Sum Sq Mean Sq F value Pr(>F)
location  2  24069 12034.4   2.6766 0.1094
Residuals 12  53955  4496.2
```

- Note the p-values for locations 2 & 3 in the regression summary table above. These large p-values indicate that there appear to be no differences between the mean computer failure times for locations 2 and 1 (the baseline) ($p = 0.234$), and between the mean computer failure times for locations 3 and 1 ($p = 0.312$).

- In general, if the ANOVA assumptions are met, a 1-way ANOVA is a stronger test as it takes advantage of these stronger model assumptions.
- However, if ever one is unsure of which test to use, use both! Here, the fact that the ANOVA-based and Kruskal-Wallis tests agree in terms of the degree of evidence against the distributions being the same lends additional support to the conclusions drawn.
- ANOVA and the K-W test are similar when the data are symmetric (since ANOVA is fairly robust to nonnormality). They can give very different results for skewed data.

The Method of Bootstrapping

Suppose $x_1, \dots, x_n$ is a random sample from some large population and we wish to estimate some population parameter θ . If $\hat{\theta}$ is an estimate of θ with some estimated standard error $SE(\hat{\theta})$, then under certain conditions a $100(1 - \alpha)\%$ confidence interval for θ can be formed as:

$$\hat{\theta} \pm t_{\alpha/2} SE(\hat{\theta}) \quad (z_{\alpha/2} \text{ may sometimes be used instead of } t_{\alpha/2}).$$

- For example, if $\theta = \mu$ is the population mean, then a $100(1 - \alpha)\%$ confidence interval for μ is given by:

$$\bar{x}_n \pm t_{\alpha/2} \cdot \frac{s}{\sqrt{n}} \Leftrightarrow (\text{estimate}) \pm \left(\begin{array}{c} \text{critical} \\ \text{value} \end{array} \right) \cdot \left(\begin{array}{c} \text{standard error} \\ \text{of the estimate} \end{array} \right).$$

The validity and accuracy of this CI depends on three basic assumptions:

1. $SE(\hat{\theta})$ is a good estimate of the standard deviation of the sampling distribution of $\hat{\theta}$.
2. $\hat{\theta}$ is an unbiased or nearly unbiased estimator for the parameter θ .
3. The sampling distribution of $\hat{\theta}$ is approximately normal. (The Central Limit Theorem can often be used here.)

The method of bootstrapping can address all of these issues:

1. It can provide an estimate of $SE(\hat{\theta})$ when no theoretical closed-form expression for $SE(\hat{\theta})$ exists, or provide an alternative if we are uncertain about the accuracy of an existing estimate. Can you think of some examples where the SE cannot be estimated or must be approximated?
2. It can provide an estimate of the bias of $\hat{\theta}$ as an estimator of θ .
3. It can provide information on the shape of the sampling distribution of $\hat{\theta}$. This can be used to calculate improved confidence intervals over the normal-based ones if the sampling distribution is not normal.

So what is bootstrapping? The theory behind bootstrapping can best be explained with a simple example.

- Suppose we have a simple random sample (SRS) and wish to estimate the standard error of the sample 10% trimmed mean, say $\bar{x}_{TM}$, as an estimate of the population mean, say μ . Unlike the sample mean, whose variance depends only on the population variance ($\sigma/\sqrt{n}$), $\text{Var}(\bar{x}_{TM})$ depends on the exact population distribution, which of course we don't know.
- The idea behind the (nonparametric) bootstrap is this: if we knew the x -values for the entire population ($x_1, \dots, x_N$), then we could estimate the sampling distribution of $\bar{x}_{TM}$ by simulation. How?
 - 1.
 - 2.
 - 3.

We could estimate the $\text{SE}(\bar{x}_{TM})$ from the standard deviation of the several thousand trimmed means.

- Unfortunately, we do not know the x -values for the population; we only know the x -values for our sample. Bootstrapping says to assume that the population looks exactly like our sample, only many times larger. This is really our best nonparametric guess as to what the population looks like.
- For example, suppose the following SRS of x -values ($n = 20$) represents guesses of my age when I taught a stats class in 2017:

26 25 26 29 29 28 27 29 28 26 41 33 29 34 27 33 27 30 29 30

then we assume that 5% of the population x -values are 26, 5% are 25, etc. In this way, this “bootstrap population” represents our best guess as to what the actual population looks like.

- To perform bootstrapping then, we simulate drawing samples of size $n = 20$ from this “bootstrap population.” If the population size N is large relative to n , this is equivalent to drawing random samples of size 20 with replacement from the original sample of size 20. Since we are sampling with replacement, we will not necessarily get the same sample every time. For example, if `ages` is the vector of 20 age guesses, issuing the **R** command `sample(ages, 20, replace = T)` two times gave the following two samples:

29 27 34 25 27 29 33 28 25 28 26 29 26 26 27 26 27 28 29 28

28 29 33 27 26 27 26 29 33 33 29 29 26 30 26 30 30 27 25 26

- Finally, generate a large number of these bootstrap samples (say 10,000 or more), calculate the sample 10% trimmed mean for each sample, and then calculate the sample standard deviation of the 10,000 sample trimmed means as an estimate of $\text{SE}(\bar{x}_{TM})$.

Example: Finding the $SE(\bar{x}_{TM})$ using **R:** Suppose as discussed above that an SRS of size 20 yielded the age guesses 26, 25, ..., 30, and we would like to estimate $SE(\bar{x}_{TM})$. To do this in **R**, we could issue the commands below, and consider the output which follows. There is a command in the `math3150package` called `bootstrapping()` that will do this automatically, but to understand how to use it, it's useful to see how we might write our own program to do bootstrapping in **R**. To generate one bootstrap sample, we could do the following:

```
set.seed(3150)
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
(i <- sample(1:20, 20, replace = T))

[1] 5 18 17 20 9 16 9 4 7 20 15 11 14 10 1 14 15 6 16 19

ages[i]

[1] 29 30 27 30 28 33 28 29 27 30 27 41 34 26 26 34 27 28 33 29

mean(ages[i], trim = 0.10)

[1] 29.3125
```

We would want to store the value of the trimmed mean for the bootstrap sample (the “29.3125”) and then repeat the process (with a loop) several thousand times, getting a new vector `i` each time we called “sample”. Notice that rather than sampling with replacement from the data vector (e.g., `sample(ages, 20, replace = T)`), I sampled from the integers 1 to n and then got the data values for the bootstrap sample by using the expression `x[i]`. There is no difference between the two ways of doing it, but this way illustrates how the `bootstrapping()` function does it in our `math3150package` and will help in understanding how to use `bootstrapping`. The commands to carry out the bootstrapping look like this.

```
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
trmean <- function(x,i) {mean(x[i], trim = 0.10)}
(b1 = bootstrapping(ages, trmean, 10000))

# A tibble: 1 x 5
  statistic sample_average original_value    bias std.error
  <chr>          <dbl>          <dbl>   <dbl>    <dbl>
1 stat1          28.8          28.8 0.0596    0.731
```

- From this output, we have the average of the 10,000 bootstrapped trimmed means is 28.8096, the trimmed mean value from our original sample is 28.75, so the estimated bias = $\text{sample_average} - \text{original_value} = 0.0596$. Then the bootstrap estimated standard error: $SE_B(\bar{x}_{TM}) = 0.739$.
- The estimated bias is thus the difference between the average of the 10,000 bootstrap trimmed means and the actual sample trimmed mean.
- **If** (not usually the case) the sampling distribution of the sample trimmed mean is approximately normal, we could establish an approximate 95% confidence interval for μ_{TM} as:

$$\bar{x}_{TM} \pm z_{.025} \cdot SE(\bar{x}_{TM}) = 28.75 \pm 1.96(0.739) = 28.75 \pm 1.45 = (27.3, 30.2).$$

In the bootstrap procedure outlined above, it is important to note the following assumptions and related issues:

1. It assumes an infinite population since it uses sampling with replacement from the observed sample. This is not a problem if the population size N is in fact large relative to the sample size n . However, if N is small, it is unclear what to do!
2. It assumes simple random sampling. Since the bootstrap samples are generated by simple random sampling (from an infinite population), it is estimating the sampling distribution of $\hat{\theta}$ for simple random sampling from an infinite population.

If the original sample was acquired based on some other type of sampling scheme, such as stratified random sampling, then we want to estimate the sampling distribution of $\hat{\theta}$ for a *stratified random sample*. Hence, the bootstrap samples must be taken in exactly the same way as the original sample. It is sometimes very tricky to figure out how to do this correctly and is still an active area of research for many types of sampling.

Some Background on Bootstrapping

- When did bootstrapping originate? The first paper on the theory of bootstrapping is attributed to Bradley Efron (1979), but it wasn't until the early 1990's with the *Science* paper by Efron & Tibshirani (1991), their subsequent book (1993), and advances in modern computing that the method gained widespread attention and use. It is now one of the most common methods of obtaining measures of uncertainty in numerous statistical applications where no closed-form expressions and/or approximations are available.
- The idea of resampling your data to generate more data is akin to “pulling yourself up by your bootstraps” to do analysis without extra outside help. Statisticians were very suspicious of bootstrapping when it first gained attention, but are now convinced of its power. If it seems like you are getting something for nothing (or downright cheating) with bootstrapping, I tend to agree, but the method has proven useful in an incredible variety of problems (censored data, survival data, time series, classification trees, goodness of fit statistics, linear and nonlinear regression, etc.).

Bootstrap Example Using R

Reconsider the data on grocery costs at a particular grocery, given below. In class, we learned how to develop a confidence interval for M using percentiles from a binomial distribution with the `SignTest()` function in **R**. Here, we use bootstrapping to estimate the SE of the sample median $m = \widehat{M}$, find a confidence interval for M , and examine the sampling distribution of m .

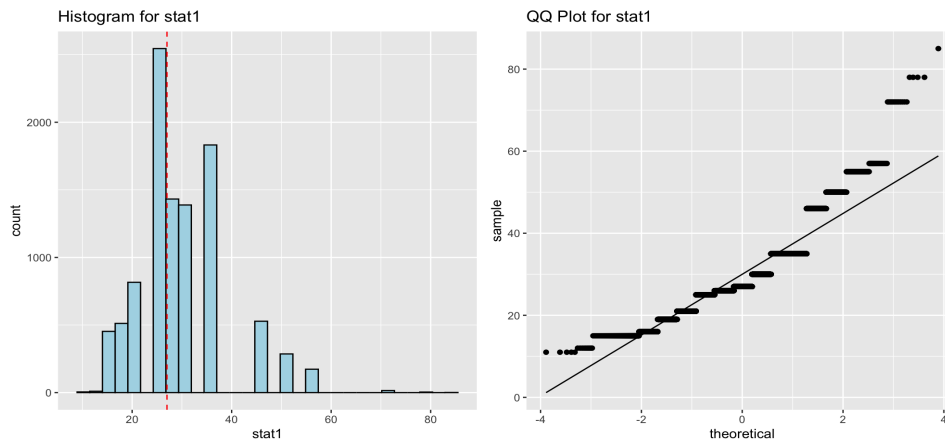
Grocery Prices															
1	3	4	5	5	8	11	12	15	15	16	19	21	25	26	27
30	35	35	46	50	55	57	72	78	85	93	137	158	212	269	

Taking 10,000 bootstrap samples from our sample of 31 costs is performed as follows:

```
spent <- c(1,3,4,5,5,8,11,12,15,15,16,19, # Defines the 31 grocery amounts
          21,25,26,27,30,35,35,46,50,55, # spent, as given in the notes
          57,72,78,85,93,137,158,212,269)
med <- function(x,i) median(x[i])      # Defines "med" as the median function
(b2 <- bootstrapping(spent, med, 10000)) # Bootstraps the median of spent
```

```
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>         <dbl>         <dbl> <dbl>    <dbl>
1 stat1         29.3           27  2.26    8.50
```

- From this output, we have the sample median $m = 27$, the estimated bias = 2.26, the mean of the bootstrap medians is 29.3, and the bootstrap estimated standard error: $SE_B(m) = 8.50$.
- To use a normal-based confidence interval, we need to assess whether the sampling distribution of the sample median is approximately normal. We can do this by examining the distribution of the 10,000 bootstrap medians, by issuing the command `plot(b2)`. This command creates the following two displays. Do the medians appear normal?



Instead of relying on the approximate normality of the bootstrap median distribution to construct a 95% confidence interval, we could compute the 2.5 and 97.5 percentiles for the bootstrap sample, and use these as the limits of the CI. This can adjust for any bias or skewness that might be present in the distribution. One of the methods built into **R** is the “bias-corrected & adjusted” (BCa) method, whereby relevant percentiles of the bootstrap sample are estimated. To illustrate this:

```
boot_ci(b2)      # Constructs bootstrap CIs for M
```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS

Based on 10000 bootstrap replicates

```
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>         <dbl> <dbl> <dbl>
1 normal         0.95  8.09  41.4
2 basic          0.95   4     38
3 percent        0.95  16     50
4 bca            0.95  15     46
```

Calculations and Intervals on Original Scale

Some BCa intervals may be unstable

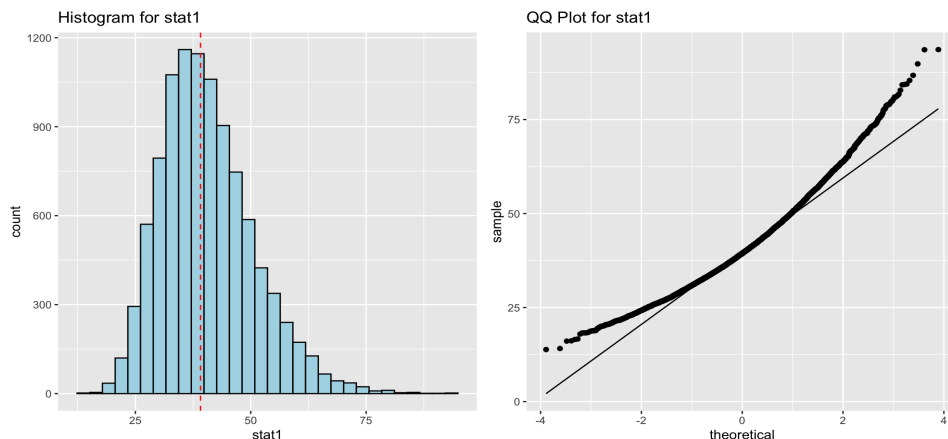
Hence, a 95% nonparametric CI based on this method is (\$15, \$46), which is asymmetric about the estimate of $m = \$27$. One can change the confidence level using the optional argument `conf=` for `boot_ci()`; see the **R** help menu. The BCa method is based on the actual distribution of the bootstrap estimates (rather than just their standard deviation) and requires a large number of bootstrap replications to be reliable (say 5000 or more for a 95% CI). In general, more bootstrap replications are needed to reliably estimate percentiles further in the tails (for a 99% CI, for example, where one needs the 0.5% and 99.5% percentiles).

- How does this confidence interval compare to that found on page 184 of the notes, namely (15, 55)?
- What if we wanted a 95% confidence interval for the 10% trimmed mean grocery amount? We would carry out the same steps, but would bootstrap the 10% trimmed mean rather than the median. [A 10% trimmed mean in this case drops the three highest and three lowest values, thus avoiding the three outliers.] Code for doing this appears below with the output following.

```
tpct <- 10
trmean <- function(x, i, trimpct) {mean(x[i], trim = trimpct/100)}
(b3 <- bootstrapping(spent, trmean, 10000, trimpct = tpct))
```

```
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          40.7          39.1  1.60    10.2
```

- How would you interpret this output?
- A histogram of the bootstrap trimmed means and corresponding normal quantile plot are shown below. Also, a 95% confidence interval for the true 10% trimmed mean grocery cost is reported. These are found by typing `plot(b3)`



```
boot_ci(b3)
```

	interval_type	conf_level	lower	upper
	<chr>	<dbl>	<dbl>	<dbl>
1	normal	0.95	17.6	57.4
2	basic	0.95	14.5	54.1
3	percent	0.95	24.2	63.7
4	bca	0.95	24.5	64.8

Bootstrapping in Regression

We discussed in Notes 6 that in order to perform inference on the slopes and predicted values in regression, the errors need to be normal and have a constant variance. That is true when doing parametric inference. If those assumptions are violated, however, we can rely on bootstrapping to obtain estimates of the standard errors.

We will discuss two types of bootstrapping for regression:

1. Bootstrapping the residuals.
 - Used when the x values are fixed, the model provides an adequate fit, and the errors have constant variance.
2. Bootstrapping the cases.
 - Less sensitive to violations of any of the regression assumptions but problems can arise when there are outliers in the model.

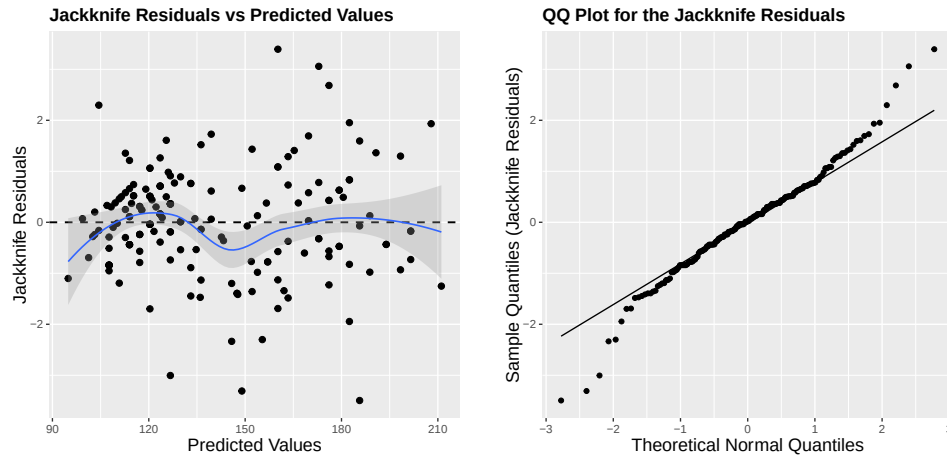
Bootstrapping the residuals

If we are reasonably assured that the model provides an adequate fit and the errors have constant variance, we can bootstrap the residuals to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Fit a model to the data to obtain $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{i,p}$, $i = 1, \dots, n$.
2. Obtain the residuals for the model: $\hat{\epsilon}_i = y_i - \hat{y}_i$, $i = 1, \dots, n$.
3. Randomly sample the set of residuals with replacement to obtain $\hat{\epsilon}_i^*$, $i = 1, \dots, n$.
4. Create a set of pseudo-responses: $y_i^* = \hat{y}_i + \hat{\epsilon}_i^*$.
5. Fit a model to the pseudo-responses to obtain $\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{i,p}$, $i = 1, \dots, n$.
6. Repeat steps 3-5 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0, \dots, p$.

Example:

Recall the data set that contains actual and ideal weights for males and females. We are attempting to predict ideal weight using sex and actual weight. Below are some plots for this model.



```
wtmod <- lm(ideal ~ sex + actual, data = idealwt)
summary(wtmod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.64973	4.14729	9.078	<2e-16 ***
sexMale	23.93657	1.94410	12.312	<2e-16 ***
actual	0.63621	0.03059	20.797	<2e-16 ***

```
confint(wtmod)
```

	2.5 %	97.5 %
(Intercept)	29.4658574	45.8336081
sexMale	20.1002659	27.7728749
actual	0.5758395	0.6965713

```
bootresids <- function(resids, i) {
  lm(I(wtmod$fitted + resids[i]) ~ sex + actual, data = idealwt)$coef
}
b4 <- bootstrapping(residuals(wtmod), bootresids, 5000)
boot_ci(b4)
```

term	sample_average	norm_lower	norm_upper	bca_lower	bca_upper
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	37.7	29.47	45.75	29.42	45.78
2 sexMale	24.0	20.09	27.73	20.17	27.85
3 actual	0.636	0.577	0.696	0.575	0.696

Bootstrapping the cases

If we are unsure that the model assumptions hold, we can bootstrap the cases to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Sample n rows from the data frame with replacement.
2. Fit a model to the data using the sampled rows of the data frame to obtain

$$\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{ip}, \quad i = 1, \dots, n.$$
3. Repeat steps 1-2 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0 \dots, p$.

Example:

```
bootcases <- function(resids, i) {
  lm(ideal ~ sex + actual, data = idealwt[i,])$coef
}
b5 <- bootstrapping(residuals(wtmod), bootcases, 5000)
boot_ci(b5)
```

term	sample_average	norm_lower	norm_upper	bca_lower	bca_upper
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	37.7	47.01	45.75	28.64	47.43
2 sexMale	23.9	19.18	28.67	19.53	29.10
3 actual	0.636	0.563	0.710	0.575	0.708

We can plot these bootstrap distributions using `plot(b5)`.

R Reference Sheet

Tips:

1. When in the console, the up arrow retrieves your previous command.
2. You can auto-complete by hitting the tab key.
3. We always need to specify the path to a file unless it is in the working directory. We can also specify the path from the working directory to the file.
4. Typing **Control+Enter** on Windows or **Command+Return** on Mac will run the line of code your cursor is on.
5. Typing **Option+minus** will enter the arrow operator `<-` with spaces around it.
6. Typing **Control+Shift+M** on Windows or **Command+Shift+M** gives the pipe (`|>` or `%>%`).
7. In **R**, the comment symbol is `#`.
8. Type a question mark in front of a function or data set to get documentation about it. Example: `?rep`.

Important Base R Functions

<u>Function</u>	<u>Explanation</u>
<code>c()</code>	Creates a vector.
<code>data.frame()</code>	Creates a data frame.
<code>install.packages("")</code>	Installs the indicated package. The name must be in quotes. Only need to use once per package.
<code>library()</code>	Loads a library/package to give access to its functions and data sets.
<code>read.csv("")</code>	Reads in a comma separated value file. Put file name in quotes.
<code>read.table("")</code>	Reads in a text file. Put file name in quotes.
<code>factor()</code>	Turns a vector into a factor (categorical variable).
<code>length()</code>	Counts the number of elements in a vector.
<code>rep()</code>	Repeat a value, string, or vector.
<code>mean()</code>	Computes the average of a vector.
<code>sd()</code>	Computes the standard deviation of a vector.
<code>median()</code>	Computes the median of a vector.
<code>quantile()</code>	Computes a quantile. Put 0.25 as second entry for Q_1 , 0.75 for Q_3 .
<code>min()</code>	Finds the minimum of a vector.
<code>max()</code>	Finds the maximum of a vector.
<code>sum()</code>	Adds up all values in a vector.
<code>table()</code>	Counts how many of each category or value for a vector.
<code>summary()</code>	Computes the 5-number summary (and the mean) of a vector.
<code>round()</code>	Rounds a number or vector to the desired number of decimal places.
<code>data()</code>	Loads a data set into memory.
<code>class()</code>	Find the class an object belongs to, like vector, factor, data frame, etc.
<code>exp()</code>	Compute the number e raised to a power.
<code>log()</code>	Compute the log of a number (natural log by default).
<code>unique()</code>	Returns all of the different (unique) values in a vector.

<code>cbind()</code>	Binds columns.
<code>rbind()</code>	Binds rows.
<code>nrow()</code>	Gives number of rows in data frame.
<code>ncol()</code>	Gives number of columns in data frame.
<code>dim()</code>	Gives number of rows and columns in data frame.
<code>rm()</code>	Removes a variable from the environment.
<code>as.factor()</code>	Changes the type of a vector to a factor.
<code>as.character()</code>	Changes the type of a vector to a character.
<code>as.numeric()</code>	Changes the type of a vector to numeric.
<code>na.omit()</code>	Removes all NA values from a vector or all NA rows from a data frame.
<code>function(){ }</code>	Creates a function. The function body should be between { and }.

Assigning and Subsetting Vectors

<u>Command</u>	<u>Result</u>	<u>Explanation</u>
<code>x <- c(2, 4, 6, 8)</code>	2 4 6 8	Join elements into a vector.
<code>x = c(2, 4, 6, 8)</code>	2 4 6 8	Alternative way to join elements into a vector.
<code>2:6</code>	2 3 4 5 6	An integer sequence.
<code>seq(2, 3, by = 0.5)</code>	2.0 2.5 3.0	A more complicated sequence.
<code>rep(1:2, times = 3)</code>	1 2 1 2 1 2	Repeat a vector.
<code>rep(1:2, each = 3)</code>	1 1 1 2 2 2	Repeat elements of a vector.
<code>x[4]</code>	8	Take the fourth element of <code>x</code> .
<code>x[-4]</code>	2 4 6	Take all but the fourth element of <code>x</code> .
<code>x[2:4]</code>	4 6 8	Take elements two through four of <code>x</code> .
<code>x[c(1, 3)]</code>	2 6	Take elements one and three of <code>x</code> .
<code>x[-c(2, 3)]</code>	2 8	Take all but the second and third elements of <code>x</code> .
<code>x[x == 6]</code>	6	Take elements that are equal to 10 in <code>x</code> .
<code>x[x < 5]</code>	2 4	Take elements that are less than 5 in <code>x</code> .
<code>x[x %in% c(1, 2, 3, 4)]</code>	2 4	Take elements in the set 1, 2, 3, 4 in <code>x</code> .
<code>x["apple"]</code>		Take elements in <code>x</code> named "apple".

Assigning and Subsetting Data Frames or Tibbles

<u>Command</u>	<u>Explanation</u>
<code>df <- data.frame(x = 1:3, y = c("a", "b", "c"))</code>	Creates data frame with columns <code>x</code> and <code>y</code> . An equal sign can also be used instead of the arrow.
<code>df\$x</code>	Takes the <code>x</code> column in the data frame.
<code>df\$x[1:3]</code>	Takes the 1st through 3rd element the <code>x</code> column.
<code>df[, 2]</code>	Takes everything in the 2nd column of the data frame.
<code>df[3,]</code>	Takes everything in the 3rd row of the data frame.
<code>df[2:3, 1]</code>	Takes the 2nd and 3rd rows in the first column.
<code>df[[1]]</code>	Takes the first column.

Important Tidyverse Functions

Type `library(tidyverse)` after you've installed it with `install.packages("tidyverse")` to load the tidyverse and have access to these functions.

<u>Function</u>	<u>Explanation</u>
<code>tibble()</code>	Creates a tibble, which is a more modern data frame.
<code>mutate(data, variable)</code>	Creates a new variable in the given data frame.
<code>filter(data, condition)</code>	Subsets data frame based on the condition. Selects rows.
<code>select(data, variables)</code>	Selects variables based on their name. Selects columns.
<code>summarize(data, functions)</code>	Computes the function value for a variable in the data frame.
<code>arrange(data, variable)</code>	Changes the ordering of the rows. The <code>desc()</code> function is useful.
<code>group_by(data, variable)</code>	allows you to perform any operation "by group". Useful before using <code>summarize()</code> .
<code>pivot_longer(data, cols)</code>	Changes data format from not-tidy to tidy.
<code>read_csv("")</code>	Reads in a csv file as a tibble. Put file name in quotes.
<code>read_table("")</code>	Reads in a text file as a tibble. Put file name in quotes.
<code>read_excel("")</code>	Read in excel file. Put file name in quotes. Requires <code>readxl</code> package.

Important ggplot2 Functions

`ggplot2` is part of the tidyverse, so typing `library(tidyverse)` will also load `ggplot2`.

Every `ggplot` needs three components:

1. The data in tidy format (one observation/data value per row with variables in each column).
2. An aesthetic mapping using the `aes()` function, which usually goes in the `ggplot()` function.
3. A geometry using a `geom_something()` function.

<u>Function</u>	<u>Explanation</u>
<code>ggplot(data, aes())</code>	The core plotting function. Needs a data frame/tibble entered and often has the <code>aes()</code> function if the arguments of <code>aes()</code> apply to all.
<code>aes()</code>	The aesthetic map. Typical arguments include <code>x</code> , <code>y</code> , <code>fill</code> , <code>color</code> , <code>shape</code> .
<code>geom_point()</code>	Creates a scatter plot.
<code>geom_bar()</code>	Creates a bar plot.
<code>geom_boxplot()</code>	Creates a box plot.
<code>geom_histogram()</code>	Creates a histogram.
<code>geom_density()</code>	Creates a density plot.
<code>geom_abline()</code>	Adds a line to the plot with the <code>slope</code> and <code>intercept</code> arguments.
<code>geom_qq()</code>	Creates a quantile-quantile (QQ) plot.
<code>geom_qq_line()</code>	Put a QQ line on a QQ plot.
<code>labs()</code>	Changes the labels for <code>x</code> , <code>y</code> , <code>title</code> , <code>fill</code> , <code>color</code> , <code>shape</code> , etc.
<code>lims()</code>	Changes the limits on <code>x</code> , or <code>y</code> or both.
<code>theme()</code>	Allows for the adjustments of many options on the graph, like text size.
<code>theme_bw()</code>	One of several <code>theme_</code> functions that changes the plot style. Put this function before the <code>theme()</code> function if both are used.

Each of the above functions, with the exception of `ggplot()` and `aes()`, is added on to the previous function with a plus sign (+). An example of some ggplot code is given below:

```
ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 2) +
  labs(x = "Weight", y = "MPG", color = "Cylinders", title = "ggplot!") +
  theme(axis.title = element_text(size = 14),
        plot.title = element_text(size = 18, face = "bold"),
        legend.position = "bottom")
```

Probability Functions

<u>Function</u>	<u>Explanation</u>
<code>dbinom()</code>	Finds $P(X = x)$ for a binomial.
<code>pbinom()</code>	Finds $P(X \leq x)$ for a binomial.
<code>rbinom()</code>	Randomly generates values from a binomial distribution.
<code>dpois()</code>	Finds $P(X = x)$ for a Poisson.
<code>ppois()</code>	Finds $P(X \leq x)$ for a Poisson.
<code>rpois()</code>	Randomly generates values from a Poisson distribution.
<code>pnorm()</code>	Finds $P(X \leq x) = P(X < x)$ for a normal.
<code>qnorm()</code>	Finds a percentile for a normal.
<code>rnorm()</code>	Randomly generates values from a normal distribution.

R also has functions that begin with **p**, **q**, and **r** for **t**, **chisq**, **f** that work the same way as the `pnorm()`, `qnorm()`, and `rnorm()` functions.

More Advanced Statistics Functions

<u>Function</u>	<u>Explanation</u>
<code>cor(x, y)</code>	Computes the correlation between two vectors.
<code>t.test()</code>	Performs a t test for one or two means.
<code>prop.test()</code>	Performs a test for one or two proportions.
<code>chisq.test()</code>	Performs a goodness-of-fit test or test for independence.
<code>cor.test()</code>	Performs a correlation test.
<code>lm(y ~ x, data = df)</code>	Creates a linear model.
<code>glm(y ~ x, data, family)</code>	Creates a generalized linear model.
<code>aov(y ~ x, data = df)</code>	Creates an ANOVA object.
<code>anova(model)</code>	Outputs an ANOVA table for a given model.
<code>predict()</code>	Obtain predictions, confidence intervals, and/or prediction intervals.
<code>prcomp()</code>	Used for principal component analysis.
<code>wilcox.test()</code>	Performs a Wilcoxon rank sum test.
<code>kruskal.test()</code>	Performs a Kruskal-Wallis test.